Principia Softwarica: The Plan 9 Core Libraries

version 0.1

Yoann Padioleau
yoann.padioleau@gmail.com

January 14, 2026

The text and figures are Copyright © 2014–2026 Yoann Padioleau.
All rights reserved.

The source code is Copyright © 2021 Plan 9 Foundation.
MIT license.

Contents

1 Introduction **13**
1.1 Motivations . . . 13
1.2 The Plan 9 core libraries: `libc`, `libregexp`, `libflate`, and `libthread` . . . 13
1.3 Other core libraries . . . 13
1.4 Getting started . . . 13
1.5 Requirements . . . 13
1.6 About this document . . . 13
1.7 Copyright . . . 14
1.8 Acknowledgments . . . 14

2 Overview **15**
2.1 Principles . . . 15
2.1.1 Core versus non-core . . . 15
2.1.2 User/kernel bridge, system calls . . . 15
2.1.3 Memory allocator . . . 15
2.1.4 Zero-cost abstractions . . . 15
2.2 `libc.h` interface . . . 15
2.3 Code organization . . . 15
2.4 Software architecture . . . 15
2.5 Book structure . . . 15

3 Core Types and Data Structures **16**
3.1 Basic types . . . 16
3.1.1 Booleans . . . 16
3.1.2 Integers . . . 16
3.1.3 Floats . . . 16
3.1.4 Strings and characters . . . 17
3.2 Complex types . . . 18
3.2.1 Pointers . . . 18
3.2.2 Arrays . . . 19
3.2.3 Structures . . . 21
3.2.4 Functions . . . 21
3.3 Regular expressions . . . 21
3.4 Files . . . 21
3.5 Time . . . 22
3.6 Concurrency . . . 22
3.7 `Fcall` . . . 22
3.8 Collections . . . 22

4 _main() 23
4.1 Command-line arguments: ARGBEGIN(), ARGEND() . . . 23
4.2 Syscall: _exits() . . . 23
4.3 exits() . . . 23

5 User/Kernel Bridge 25

6 Memory Area Operations 26

7 Memory Management 28
7.1 Syscall: brk() . . . 28
7.2 sbrk() . . . 28
7.3 Data structures . . . 28
7.3.1 Pool . . . 28
7.3.2 sbrkmem . . . 29
7.3.3 XXX . . . 30
7.4 malloc() . . . 32
7.5 free() . . . 32
7.6 Debugging support . . . 32
7.7 Advanced topics . . . 32

8 Strings 33
8.1 Plain C strings . . . 33
8.2 Runes . . . 35
8.2.1 Conversion . . . 36
8.2.2 Basics . . . 38
8.3 UTF . . . 40
8.4 Formatted Output . . . 42
8.5 Quoted strings . . . 42
8.6 Extensible strings . . . 42

9 Regular Expressions 43
9.1 Data structures . . . 43
9.2 Advanced topics . . . 43

10 File IO 44
10.1 Data structures . . . 44
10.2 Syscalls: open(), pread(), etc . . . 45
10.3 read(), write() . . . 45
10.4 readn(), writen() . . . 45

11 Directories and File Paths 46
11.1 Data structures: Dir . . . 46
11.2 Syscalls: create(), chdir(), etc . . . 47
11.3 xxxfstat() . . . 47
11.4 xxxstat() . . . 48
11.5 getwd() . . . 49
11.6 The deprecated mktemp() . . . 49
11.7 The safe mkstemp() . . . 50
11.8 Normalizing a filename: cleanname() . . . 50
11.9 access() . . . 51

11.10 Path manipulation . . . 52

12 Namespace **53**
12.1 Syscalls: `bind()`, `mount()`, etc . . . 53

13 Time **54**
13.1 Syscalls: `sleep()`, `alarm()`, etc . . . 54
13.2 Time and day . . . 54
13.3 Time zones and `localtime()` . . . 54
13.4 `time()` . . . 55
13.5 Conversions . . . 56

14 Fork, Exec, and Wait **58**
14.1 Syscalls: `rfork()`, `exec()`, `await()`, etc . . . 58
14.2 `fork()` . . . 58
14.3 `execl()` . . . 58
14.4 `wait()` . . . 59
14.5 `getpid()` . . . 59

15 Concurrency **60**
15.1 Syscalls: `rendezvous()`, etc . . . 60
15.2 Locks . . . 60
15.3 Atomic operations: `ainc()` (ARM) . . . 61
15.4 Waiting queues . . . 61
15.5 Read-write locks . . . 62
15.6 Rendez-vous . . . 65
15.7 Coroutines . . . 67

16 IPC **68**
16.1 Syscalls: `pipe()`, `notify()`, `await()`, `segattach()` . . . 68
16.2 Helpers . . . 68
16.3 Parent-child IPC . . . 69
16.3.1 Wait message . . . 69
16.3.2 Environment . . . 70
16.4 Notes . . . 71
16.4.1 Posting . . . 71
16.4.2 Notified . . . 71
16.5 Pipes . . . 73
16.6 Shared segments . . . 73
16.7 Fileservers . . . 73
16.7.1 `Fcall` . . . 73
16.7.2 Dumping . . . 74
16.7.3 Marshalling macros . . . 77
16.7.4 `readp9msg()` . . . 78
16.7.5 Parsing . . . 78

17 Channels and Cooperative Threads **94**
17.1 Overview . . . 94
17.1.1 Code organization . . . 94
17.1.2 Software architecture . . . 94
17.2 A toy example . . . 94

17.3 Core data structures . . . 94
17.3.1 Concurrency buiding blocks . . . 94
17.3.2 `Channel` . . . 95
17.3.3 `Proc` and `Pqueue` . . . 98
17.3.4 `Thread` and `Tqueue` . . . 100
17.3.5 `Alt` . . . 105
17.4 `main()` . . . 105
17.5 Threads scheduler . . . 106
17.6 `alt()` . . . 110
17.7 Send/Receive . . . 116
17.7.1 `runop` . . . 116
17.7.2 `send()/recv()` . . . 117
17.7.3 Non blocking operations: `nbxxx()` . . . 117
17.7.4 Typed send and receive . . . 117
17.8 IO processus . . . 119
17.8.1 `Ioproc` . . . 119
17.8.2 `xiproc()` . . . 120
17.8.3 `iocall()` . . . 121
17.8.4 IO wrappers . . . 121
17.9 Thread-aware libc . . . 123
17.9.1 Memory . . . 123
17.9.2 Exit . . . 124
17.9.3 Fork . . . 125
17.9.4 Exec . . . 125
17.9.5 Interruptions . . . 128
17.9.6 Notes . . . 128
17.9.7 Dialing . . . 131
17.10 Error management . . . 135
17.11 Debugging . . . 135
17.12 Advanced topics . . . 136
17.12.1 Per thread private data . . . 137
17.12.2 Thread groups . . . 138
17.12.3 Thread kills . . . 138
17.12.4 Rendez vous . . . 140
17.12.5 `chanprint()` . . . 142

18 Network **143**
18.1 Example: an echo server . . . 143
18.2 `netmkaddr()` . . . 143
18.3 `announce()`, `accept()`, `listen()`, `reject()` . . . 144
18.4 `dial()` . . . 147

19 Mathematics **149**
19.1 Basics . . . 149
19.2 Integers . . . 150
19.2.1 Parsing . . . 150
19.3 Floats . . . 152
19.3.1 Special numbers . . . 152
19.3.2 Parsing . . . 154
19.3.3 `sqrt()` . . . 163

19.3.4 modf() . . . 164
19.3.5 fmod() . . . 164
19.3.6 pow() . . . 165
19.4 Trigonometry . . . 167
19.5 Logarithms and exponentials . . . 170
19.5.1 exp() . . . 170
19.5.2 ldexp() . . . 171
19.5.3 frexp() . . . 172
19.5.4 log() . . . 173
19.6 Random numbers . . . 174
19.7 muldiv . . . 175

20 Compression **177**

21 Error Management **178**
21.1 Syscall: errstr() . . . 178
21.2 rerrstr(), werrstr() . . . 178
21.3 perror() . . . 178
21.4 Error return values . . . 179
21.5 assert() . . . 179
21.6 sysfatal() . . . 180

22 Debugging Support **181**
22.1 Logging . . . 181

23 Profiling Support **184**
23.1 _profin() . . . 185
23.2 _profout() . . . 186
23.3 _profinit() . . . 186
23.4 _profmain() . . . 186
23.5 cputime() . . . 187

24 Security **188**
24.1 getuser() . . . 188
24.2 Cryptography . . . 188
24.3 SSL/TLS . . . 189

25 Advanced Topics **192**
25.1 Arbitrary precision arithmetics . . . 192
25.2 IO chunks . . . 192
25.3 Extra functions . . . 192
25.3.1 Strings . . . 192

26 Conclusion **193**

A Extra Code **194**
A.1 include/ . . . 194
A.1.1 include/arm/u.h . . . 194
A.1.2 include/core/libc.h . . . 194
A.1.3 include/core/ctype.h . . . 204
A.1.4 include/core/syscall.h . . . 205

A.1.5 include/regexp.h . . . 207
A.1.6 include/ipc/thread.h . . . 208
A.1.7 include/core/internals/pool.h . . . 210
A.1.8 include/fcall.h . . . 211
A.2 lib_core/libc/port . . . 212
A.2.1 libc/port/_assert.c . . . 212
A.2.2 libc/port/abs.c . . . 212
A.2.3 libc/port/asin.c . . . 212
A.2.4 libc/port/atan.c . . . 212
A.2.5 libc/port/atan2.c . . . 214
A.2.6 libc/port/atexit.c . . . 215
A.2.7 libc/port/atnotify.c . . . 215
A.2.8 libc/port/atof.c . . . 215
A.2.9 libc/port/atol.c . . . 215
A.2.10 libc/port/atoll.c . . . 216
A.2.11 libc/port/charstod.c . . . 216
A.2.12 libc/port/cistrcmp.c . . . 217
A.2.13 libc/port/cistrncmp.c . . . 218
A.2.14 libc/port/cistrstr.c . . . 219
A.2.15 libc/port/cleanname.c . . . 219
A.2.16 libc/port/crypt.c . . . 219
A.2.17 libc/port/ctype.c . . . 220
A.2.18 libc/port/encodefmt.c . . . 220
A.2.19 libc/port/execl.c . . . 221
A.2.20 libc/port/exp.c . . . 221
A.2.21 libc/port/fabs.c . . . 222
A.2.22 libc/port/floor.c . . . 222
A.2.23 libc/port/fmod.c . . . 222
A.2.24 libc/port/frand.c . . . 222
A.2.25 libc/port/frexp.c . . . 223
A.2.26 libc/port/getcallerpc.c . . . 223
A.2.27 libc/port/getfields.c . . . 223
A.2.28 libc/port/getuser.c . . . 223
A.2.29 libc/port/hangup.c . . . 224
A.2.30 libc/port/hypot.c . . . 224
A.2.31 libc/port/lnrand.c . . . 225
A.2.32 libc/port/lock.c . . . 225
A.2.33 libc/port/log.c . . . 225
A.2.34 libc/port/lrand.c . . . 226
A.2.35 libc/port/malloc.c . . . 227
A.2.36 libc/port/memccpy.c . . . 234
A.2.37 libc/port/memchr.c . . . 234
A.2.38 libc/port/memcmp.c . . . 234
A.2.39 libc/port/memmove.c . . . 234
A.2.40 libc/port/memset.c . . . 234
A.2.41 libc/port/mktemp.c . . . 234
A.2.42 libc/port/muldiv.c . . . 235
A.2.43 libc/port/nan.c . . . 235
A.2.44 libc/port/needsrcquote.c . . . 235

A.2.45 libc/port/netcrypt.c . . . 235
A.2.46 libc/port/netmkaddr.c . . . 236
A.2.47 libc/port/nrand.c . . . 236
A.2.48 libc/port/ntruerand.c . . . 236
A.2.49 libc/port/perror.c . . . 237
A.2.50 libc/port/pool.c . . . 237
A.2.51 libc/port/pow.c . . . 263
A.2.52 libc/port/pow10.c . . . 263
A.2.53 libc/port/profile.c . . . 264
A.2.54 libc/port/qsort.c . . . 266
A.2.55 libc/port/quote.c . . . 266
A.2.56 libc/port/rand.c . . . 269
A.2.57 libc/port/readn.c . . . 269
A.2.58 libc/port/rune.c . . . 269
A.2.59 libc/port/runebase.c . . . 271
A.2.60 libc/port/runebsearch.c . . . 271
A.2.61 libc/port/runestrcat.c . . . 271
A.2.62 libc/port/runestrchr.c . . . 271
A.2.63 libc/port/runestrcmp.c . . . 271
A.2.64 libc/port/runestrcpy.c . . . 271
A.2.65 libc/port/runestrdup.c . . . 271
A.2.66 libc/port/runestrecpy.c . . . 272
A.2.67 libc/port/runestrlen.c . . . 272
A.2.68 libc/port/runestrncat.c . . . 272
A.2.69 libc/port/runestrncmp.c . . . 273
A.2.70 libc/port/runestrncpy.c . . . 273
A.2.71 libc/port/runestrrchr.c . . . 273
A.2.72 libc/port/runestrstr.c . . . 274
A.2.73 libc/port/runetype.c and runetypebody-6.2.0.h . . . 274
A.2.74 libc/port/sin.c . . . 274
A.2.75 libc/port/sinh.c . . . 274
A.2.76 libc/port/sqrt.c . . . 276
A.2.77 libc/port/strcat.c . . . 276
A.2.78 libc/port/strchr.c . . . 276
A.2.79 libc/port/strcmp.c . . . 277
A.2.80 libc/port/strcpy.c . . . 277
A.2.81 libc/port/strcspn.c . . . 277
A.2.82 libc/port/strdup.c . . . 277
A.2.83 libc/port/strecpy.c . . . 278
A.2.84 libc/port/strlen.c . . . 278
A.2.85 libc/port/strncat.c . . . 278
A.2.86 libc/port/strncmp.c . . . 279
A.2.87 libc/port/strncpy.c . . . 279
A.2.88 libc/port/strpbrk.c . . . 279
A.2.89 libc/port/strrchr.c . . . 280
A.2.90 libc/port/strspn.c . . . 280
A.2.91 libc/port/strstr.c . . . 281
A.2.92 libc/port/strtod.c . . . 281
A.2.93 libc/port/strtok.c . . . 282

A.2.94 libc/port/strtol.c . . . 282
A.2.95 libc/port/strtoll.c . . . 282
A.2.96 libc/port/strtoul.c . . . 284
A.2.97 libc/port/strtoull.c . . . 286
A.2.98 libc/port/tan.c . . . 288
A.2.99 libc/port/tanh.c . . . 288
A.2.100 libc/port/tokenize.c . . . 289
A.2.101 libc/port/toupper.c . . . 291
A.2.102 libc/port/u16.c . . . 291
A.2.103 libc/port/u32.c . . . 292
A.2.104 libc/port/u64.c . . . 294
A.2.105 libc/port/utfecpy.c . . . 297
A.2.106 libc/port/utflen.c . . . 297
A.2.107 libc/port/utfnlen.c . . . 297
A.2.108 libc/port/utfrrune.c . . . 298
A.2.109 libc/port/utfrune.c . . . 298
A.2.110 libc/port/utfutf.c . . . 298
A.3 lib_core/libc/9sys/ . . . 298
A.3.1 libc/9sys/abort.c . . . 298
A.3.2 libc/9sys/access.c . . . 299
A.3.3 libc/9sys/announce.c . . . 299
A.3.4 libc/9sys/convD2M.c . . . 301
A.3.5 libc/9sys/convM2D.c . . . 302
A.3.6 libc/9sys/convM2S.c . . . 302
A.3.7 libc/9sys/convS2M.c . . . 303
A.3.8 libc/9sys/cputime.c . . . 303
A.3.9 libc/9sys/ctime.c . . . 303
A.3.10 libc/9sys/dial.c . . . 308
A.3.11 libc/9sys/dirfstat.c . . . 317
A.3.12 libc/9sys/dirfwstat.c . . . 317
A.3.13 libc/9sys/dirmodefmt.c . . . 318
A.3.14 libc/9sys/dirread.c . . . 319
A.3.15 libc/9sys/dirstat.c . . . 320
A.3.16 libc/9sys/dirwstat.c . . . 321
A.3.17 libc/9sys/fcallfmt.c . . . 321
A.3.18 libc/9sys/fork.c . . . 322
A.3.19 libc/9sys/getenv.c . . . 323
A.3.20 libc/9sys/getnetconninfo.c . . . 323
A.3.21 libc/9sys/getpid.c . . . 325
A.3.22 libc/9sys/getppid.c . . . 325
A.3.23 libc/9sys/getwd.c . . . 326
A.3.24 libc/9sys/iounit.c . . . 326
A.3.25 libc/9sys/nsec.c . . . 326
A.3.26 libc/9sys/nulldir.c . . . 328
A.3.27 libc/9sys/postnote.c . . . 328
A.3.28 libc/9sys/privalloc.c . . . 328
A.3.29 libc/9sys/pushssl.c . . . 329
A.3.30 libc/9sys/pushtls.c . . . 329
A.3.31 libc/9sys/putenv.c . . . 330

A.3.32 libc/9sys/qlock.c . . . 330
A.3.33 libc/9sys/read.c . . . 332
A.3.34 libc/9sys/read9pmsg.c . . . 332
A.3.35 libc/9sys/readv.c . . . 332
A.3.36 libc/9sys/rerrstr.c . . . 333
A.3.37 libc/9sys/sbrk.c . . . 333
A.3.38 libc/9sys/setnetmtpt.c . . . 334
A.3.39 libc/9sys/sysfatal.c . . . 334
A.3.40 libc/9sys/syslog.c . . . 334
A.3.41 libc/9sys/sysname.c . . . 335
A.3.42 libc/9sys/time.c . . . 335
A.3.43 libc/9sys/times.c . . . 335
A.3.44 libc/9sys/tm2sec.c . . . 336
A.3.45 libc/9sys/truerand.c . . . 339
A.3.46 libc/9sys/wait.c . . . 339
A.3.47 libc/9sys/waitpid.c . . . 340
A.3.48 libc/9sys/werrstr.c . . . 340
A.3.49 libc/9sys/write.c . . . 340
A.3.50 libc/9sys/writev.c . . . 340
A.4 lib_core/libc/fmt/ . . . 341
A.4.1 libc/fmt/fmtdef.h . . . 341
A.4.2 libc/fmt/dofmt.c . . . 343
A.4.3 libc/fmt/dorfmt.c . . . 352
A.4.4 libc/fmt/errfmt.c . . . 353
A.4.5 libc/fmt/fltfmt.c . . . 354
A.4.6 libc/fmt/fmt.c . . . 359
A.4.7 libc/fmt/fmtfd.c . . . 363
A.4.8 libc/fmt/fmtlock.c . . . 364
A.4.9 libc/fmt/fmtprint.c . . . 365
A.4.10 libc/fmt/fmtquote.c . . . 365
A.4.11 libc/fmt/fmtrune.c . . . 370
A.4.12 libc/fmt/fmtstr.c . . . 370
A.4.13 libc/fmt/fmtvprint.c . . . 371
A.4.14 libc/fmt/fprint.c . . . 371
A.4.15 libc/fmt/print.c . . . 372
A.4.16 libc/fmt/runefmtstr.c . . . 372
A.4.17 libc/fmt/runeseprint.c . . . 372
A.4.18 libc/fmt/runesmprint.c . . . 373
A.4.19 libc/fmt/runesnprint.c . . . 373
A.4.20 libc/fmt/runesprint.c . . . 373
A.4.21 libc/fmt/runevseprint.c . . . 374
A.4.22 libc/fmt/runevsmprint.c . . . 374
A.4.23 libc/fmt/runevsnprint.c . . . 376
A.4.24 libc/fmt/seprint.c . . . 376
A.4.25 libc/fmt/smprint.c . . . 377
A.4.26 libc/fmt/snprint.c . . . 377
A.4.27 libc/fmt/sprint.c . . . 377
A.4.28 libc/fmt/vfprint.c . . . 378
A.4.29 libc/fmt/vseprint.c . . . 378

A.4.30 libc/fmt/vsmprint.c . . . 379
A.4.31 libc/fmt/vsnprint.c . . . 380
A.5 lib_core/libc/arm/ . . . 381
A.5.1 libc/arm/cycles.c . . . 381
A.5.2 libc/arm/notejmp.c . . . 381
A.5.3 libc/arm/vlrt.c . . . 382
A.6 libstring/ . . . 396
A.7 libflate/ . . . 396
A.8 libregexp/ . . . 396
A.8.1 libregexp/regcomp.h . . . 396
A.8.2 libregexp/regaux.c . . . 397
A.8.3 libregexp/regcomp.c . . . 399
A.8.4 libregexp/regerror.c . . . 411
A.8.5 libregexp/regexec.c . . . 412
A.8.6 libregexp/regsub.c . . . 416
A.8.7 libregexp/rregexec.c . . . 417
A.8.8 libregexp/rregsub.c . . . 421
A.9 lib_core/libthread/arm . . . 422
A.10 lib_core/libthread/x86 . . . 423
A.10.1 libthread/386.c . . . 423
A.11 lib_core/libthread/ . . . 423
A.11.1 libthread/ref.c . . . 423
A.11.2 libthread/threadimpl.h . . . 423
A.11.3 libthread/xincarm.c . . . 425
A.11.4 libthread/xincport.h . . . 425
A.11.5 libthread/globals.c . . . 425
A.11.6 libthread/debug.c . . . 426
A.11.7 libthread/id.c . . . 426
A.11.8 libthread/exit.c . . . 426
A.11.9 libthread/lib.c . . . 427
A.11.10 libthread/note.c . . . 427
A.11.11 libthread/rendez.c . . . 428
A.11.12 libthread/sched.c . . . 428
A.11.13 libthread/create.c . . . 428
A.11.14 libthread/channel.c . . . 429
A.11.15 libthread/main.c . . . 430
A.11.16 libthread/kill.c . . . 431
A.11.17 libthread/ioproc.c . . . 431
A.11.18 libthread/iocall.c . . . 432
A.11.19 libthread/chanprint.c . . . 432
A.11.20 libthread/ioclose.c . . . 432
A.11.21 libthread/iodial.c . . . 432
A.11.22 libthread/ioopen.c . . . 433
A.11.23 libthread/ioread.c . . . 433
A.11.24 libthread/ioreadn.c . . . 433
A.11.25 libthread/iosleep.c . . . 433
A.11.26 libthread/iowrite.c . . . 433
A.11.27 libthread/dial.c . . . 434
A.11.28 libthread/exec.c . . . 434

Glossary **435**

Index **436**

References **436**

Chapter 1

Introduction

The goal of this book is to explain with full details the source code of a few core libraries.

1.1 Motivations

Why core libraries? Because I think you are a better programmer if you fully understand how things work under the hood.

Here are a few questions I hope this book will answer:

- Is there any code executed before `main()` in a C program?
- Which program does memory allocation? The kernel? The C library? How is `malloc()` implemented? How does it relate to the `sbrk()` system call?
- How does a thread scheduler work? How is implemented `alt()` without `select()` in the kernel?

1.2 The Plan 9 core libraries: `libc`, `libregexp`, `libflate`, and `libthread`

1.3 Other core libraries

Here are a few core libraries that I considered for this book, but which I ultimately discarded:

- GNU Libc
- dietlibc
- microlibc

1.4 Getting started

1.5 Requirements

1.6 About this document

This document is a *literate program* [Knu92]. It derives from a set of files processed by a tool, `syncweb` [Pad09], generating either this book or the actual source code of the program. The code and its documentation are thus strongly connected.

1.7 Copyright

Most of this document is made of source code from Plan 9, so those parts are copyright by Plan 9 Foundation. The prose is mine and is licensed under the All Rights Reserved License.

1.8 Acknowledgments

Chapter 2

Overview

2.1 Principles

2.1.1 Core versus non-core

2.1.2 User/kernel bridge, system calls

2.1.3 Memory allocator

2.1.4 Zero-cost abstractions

2.2 `libc.h` interface

2.3 Code organization

2.4 Software architecture

2.5 Book structure

Chapter 3

Core Types and Data Structures

3.1 Basic types

3.1.1 Booleans

⟨*type* bool 16a⟩≡ (194b)

```
typedef int bool;
enum _bool {
  false = 0,
  true = 1
};
```

3.1.2 Integers

⟨*type* uxxx 16b⟩≡ (194a)

```
typedef unsigned short  ushort;
typedef unsigned char   uchar;
typedef unsigned long   ulong;
typedef unsigned int    uint;
```

⟨*type* uxxxint 16c⟩≡ (194a)

```
typedef unsigned char u8int;
typedef unsigned short u16int;
typedef unsigned int    u32int;
typedef unsigned long long u64int;
```

⟨*type* xxxvlong 16d⟩≡ (194a)

```
typedef long long   vlong;
typedef unsigned long long uvlong;
```

⟨*type* usize 16e⟩≡ (194a)

```
typedef unsigned long   usize;
```

3.1.3 Floats

⟨*type* FPdbleword 16f⟩≡ (194a)

```
union FPdbleword
{
    double  x;

    struct {    /* little endian */
        ulong lo;
        ulong hi;
    };
};
```

3.1.4 Strings and characters

ASCII characters

⟨*type* Ctype_flag 17a⟩≡ (204)

```
#define _U 01 // upper
#define _L 02 // lower
#define _N 04 // number
#define _S 010 // space
#define _P 020 // punctuation
#define _C 040 // ctlr
#define _B 0100 // ??
#define _X 0200 // ??
```

⟨*global* _ctype 17b⟩≡ (220a)

```
uchar   _ctype[256] =
{
/*   0   1   2   3   4   5   6   7  */

/*  0*/ _C, _C, _C, _C, _C, _C, _C, _C,
/* 10*/ _C, _S|_C,  _S|_C,  _S|_C,  _S|_C,  _S|_C,  _C, _C,
/* 20*/ _C, _C, _C, _C, _C, _C, _C, _C,
/* 30*/ _C, _C, _C, _C, _C, _C, _C, _C,
/* 40*/ _S|_B,  _P, _P, _P, _P, _P, _P, _P,
/* 50*/ _P, _P, _P, _P, _P, _P, _P, _P,
/* 60*/ _N|_X,  _N|_X,  _N|_X,  _N|_X,  _N|_X,  _N|_X,  _N|_X,  _N|_X,
/* 70*/ _N|_X,  _N|_X,  _P, _P, _P, _P, _P, _P,
/*100*/ _P, _U|_X,  _U|_X,  _U|_X,  _U|_X,  _U|_X,  _U|_X,  _U,
/*110*/ _U, _U, _U, _U, _U, _U, _U, _U,
/*120*/ _U, _U, _U, _U, _U, _U, _U, _U,
/*130*/ _U, _U, _U, _P, _P, _P, _P, _P,
/*140*/ _P, _L|_X,  _L|_X,  _L|_X,  _L|_X,  _L|_X,  _L|_X,  _L,
/*150*/ _L, _L, _L, _L, _L, _L, _L, _L,
/*160*/ _L, _L, _L, _L, _L, _L, _L, _L,
/*170*/ _L, _L, _L, _P, _P, _P, _P, _C,
};
```

⟨*macros isxxx* 17c⟩≡ (204)

```
#define isalpha(c) (_ctype[(unsigned char)(c)]&(_U|_L))
#define isupper(c) (_ctype[(unsigned char)(c)]&_U)
#define islower(c) (_ctype[(unsigned char)(c)]&_L)
#define isdigit(c) (_ctype[(unsigned char)(c)]&_N)
#define isxdigit(c) (_ctype[(unsigned char)(c)]&_X)
#define isspace(c) (_ctype[(unsigned char)(c)]&_S)
#define ispunct(c) (_ctype[(unsigned char)(c)]&_P)
#define isalnum(c) (_ctype[(unsigned char)(c)]&(_U|_L|_N))
#define isprint(c) (_ctype[(unsigned char)(c)]&(_P|_U|_L|_N|_B))
#define isgraph(c) (_ctype[(unsigned char)(c)]&(_P|_U|_L|_N))
#define iscntrl(c) (_ctype[(unsigned char)(c)]&_C)
#define isascii(c) ((unsigned char)(c)<=0177)
```

⟨*function* toupper 17d⟩≡ (291b)

```
int
toupper(int c)
{

    if(c < 'a' || c > 'z')
        return c;
    return _toupper(c);
}
```

⟨*macro* _toupper 18a⟩≡ (204)
```
#define _toupper(c) ((c)-'a'+'A')
```

⟨*function* tolower 18b⟩≡ (291b)
```
int
tolower(int c)
{

    if(c < 'A' || c > 'Z')
        return c;
    return _tolower(c);
}
```

⟨*macro* _tolower 18c⟩≡ (204)
```
#define _tolower(c) ((c)-'A'+'a')
```

⟨*macro* toascii 18d⟩≡ (204)
```
#define toascii(c) ((c)&0177)
```

Runes

⟨*type* Rune 18e⟩≡ (194a)
```
typedef uint        Rune;
```

⟨*constant* Runemax 18f⟩≡ (194b)
```
Runemax     = 0x10FFFF, /* 21-bit rune */
```

⟨*constant* Runemask 18g⟩≡ (194b)
```
Runemask    = 0x1FFFFF, /* bits used by runes (see grep) */
```

UTF8

⟨*constant* UTFmax 18h⟩≡ (194b)
```
UTFmax      = 4,        /* maximum bytes per rune */
```

⟨*constant* Runesync 18i⟩≡ (194b)
```
Runesync    = 0x80,     /* cannot represent part of a UTF sequence (<) */
```

⟨*constant* Runeself 18j⟩≡ (194b)
```
Runeself    = 0x80,     /* rune and UTF sequences are the same (<) */
```

⟨*constant* Runeerror 18k⟩≡ (194b)
```
Runeerror   = 0xFFFD,   /* decoding error in UTF */
```

3.2 Complex types

3.2.1 Pointers

⟨*constant* nil 18l⟩≡ (194a)
```
#define nil     ((void*)0)
```

⟨*type* uintptr 18m⟩≡ (194a)
```
typedef unsigned long   uintptr;
```

⟨*type* byte 18n⟩≡ (194b)
```
typedef uchar byte;
```

3.2.2 Arrays

⟨*function* nelem 19a⟩≡ (194b)

```
#define nelem(x)    (sizeof(x)/sizeof((x)[0]))
```

⟨*type* Sort 19b⟩≡ (266a)

```
typedef
struct
{
    int (*cmp)(void*, void*);
    void    (*swap)(char*, char*, long);
    long    es;
} Sort;
```

⟨*function* qsort 19c⟩≡ (266a)

```
void
qsort(void *va, long n, long es, int (*cmp)(void*, void*))
{
    Sort s;

    s.cmp = cmp;
    s.es = es;
    s.swap = swapi;
    if(((uintptr)va | es) % sizeof(long))
        s.swap = swapb;
    qsorts((char*)va, n, &s);
}
```

⟨*function* swapb 19d⟩≡ (266a)

```
static  void
swapb(char *i, char *j, long es)
{
    char c;

    do {
        c = *i;
        *i++ = *j;
        *j++ = c;
        es--;
    } while(es != 0);

}
```

⟨*function* swapi 19e⟩≡ (266a)

```
static  void
swapi(char *ii, char *ij, long es)
{
    long *i, *j, c;

    i = (long*)ii;
    j = (long*)ij;
    do {
        c = *i;
        *i++ = *j;
        *j++ = c;
        es -= sizeof(long);
    } while(es != 0);
}
```

⟨*function* pivot 20a⟩≡ (266a)

```
static  char*
pivot(char *a, long n, Sort *p)
{
    long j;
    char *pi, *pj, *pk;

    j = n/6 * p->es;
    pi = a + j; /* 1/6 */
    j += j;
    pj = pi + j;    /* 1/2 */
    pk = pj + j;    /* 5/6 */
    if(p->cmp(pi, pj) < 0) {
        if(p->cmp(pi, pk) < 0) {
            if(p->cmp(pj, pk) < 0)
                return pj;
            return pk;
        }
        return pi;
    }
    if(p->cmp(pj, pk) < 0) {
        if(p->cmp(pi, pk) < 0)
            return pi;
        return pk;
    }
    return pj;
}
```

⟨*function* qsorts 20b⟩≡ (266a)

```
static  void
qsorts(char *a, long n, Sort *p)
{
    long j, es;
    char *pi, *pj, *pn;

    es = p->es;
    while(n > 1) {
        if(n > 10) {
            pi = pivot(a, n, p);
        } else
            pi = a + (n>>1)*es;

        p->swap(a, pi, es);
        pi = a;
        pn = a + n*es;
        pj = pn;
        for(;;) {
            do
                pi += es;
            while(pi < pn && p->cmp(pi, a) < 0);
            do
                pj -= es;
            while(pj > a && p->cmp(pj, a) > 0);
            if(pj < pi)
                break;
            p->swap(pi, pj, es);
        }
        p->swap(a, pj, es);
        j = (pj - a) / es;
```

```
        n = n-j-1;
        if(j >= n) {
            qsorts(a, j, p);
            a += (j+1)*es;
        } else {
            qsorts(a + (j+1)*es, n, p);
            n = j;
        }
    }
}
```

3.2.3 Structures

⟨*function* `offsetof` 21a⟩≡ (194b)

```
#define offsetof(s, m)  (ulong)(&(((s*)nil)->m))
```

3.2.4 Functions

⟨*type* `va_list` 21b⟩≡ (194a)

```
typedef char*   va_list;
```

⟨*macro* `va_start` 21c⟩≡ (194a)

```
#define va_start(list, start) list =\
    (sizeof(start) < 4?\
        (char*)((int*)&(start)+1):\
        (char*)(&(start)+1))
```

⟨*macro* `va_end` 21d⟩≡ (194a)

```
#define va_end(list)\
    USED(list)
```

⟨*macro* `va_arg` 21e⟩≡ (194a)

```
#define va_arg(list, mode)\
    ((sizeof(mode) == 1)?\
        ((list += 4), (mode*)list)[-4]:\
    (sizeof(mode) == 2)?\
        ((list += 4), (mode*)list)[-2]:\
        ((list += sizeof(mode)), (mode*)list)[-1])
```

3.3 Regular expressions

3.4 Files

⟨*type* `fdt` 21f⟩≡ (194b)

```
typedef int fdt; // file descriptor type
```

⟨*constant* `STDxxx` 21g⟩≡ (194b)

```
#define STDIN 0
#define STDOUT 1
#define STDERR 2
```

3.5 Time

⟨*type* Tm 22⟩≡ (194b)

```
struct Tm {
    int sec;
    int min;
    int hour;

    int mday;
    int mon;
    int year;
    int wday;
    int yday;

    char    zone[4];
    int     tzoff;
};
```

3.6 Concurrency

3.7 Fcall

3.8 Collections

Chapter 4

_main()

4.1 Command-line arguments: ARGBEGIN(), ARGEND()

⟨*signature global argv0* 23a⟩≡ (194b)
```
extern char *argv0;
```

⟨*macro* ARGBEGIN 23b⟩≡ (194b)
```
#define ARGBEGIN    for((argv0||(argv0=*argv)),argv++,argc--;\
                argv[0] && argv[0][0]=='-' && argv[0][1];\
                argc--, argv++) {\
                char *_args, *_argt;\
                Rune _argc;\
                _args = &argv[0][1];\
                if(_args[0]=='-' && _args[1]==0){\
                    argc--; argv++; break;\
                }\
                _argc = 0;\
                while(*_args && (_args += chartorune(&_argc, _args)))\
                switch(_argc)
```

⟨*macro* ARGEND 23c⟩≡ (194b)
```
#define ARGEND      SET(_argt);USED(_argt,_argc,_args);}USED(argv, argc);
```

⟨*macro* ARGF 23d⟩≡ (194b)
```
#define ARGF()      (_argt=_args, _args="",\
                (*_argt? _argt: argv[1]? (argc--, *++argv): 0))
```

⟨*macro* EARGF 23e⟩≡ (194b)
```
#define EARGF(x)    (_argt=_args, _args="",\
                (*_argt? _argt: argv[1]? (argc--, *++argv): ((x), abort(), (char*)0)))
```

⟨*macro* ARGC 23f⟩≡ (194b)
```
#define ARGC()      _argc
```

4.2 Syscall: _exits()

4.3 exits()

⟨*function* exits 23g⟩≡ (215b)
```
void
exits(char *s)
{
    int i, pid;
```

```
    void (*f)(void);

    pid = getpid();
    for(i = NEXIT-1; i >= 0; i--)
        if((f = onex[i].f) && pid == onex[i].pid) {
            onex[i].f = nil;
            (*f)();
        }
    _exits(s);
}
```

⟨*global* onex 24a⟩≡ (215b)

```
Onex onex[NEXIT];
```

⟨*constant* NEXIT 24b⟩≡ (215b)

```
#define NEXIT   33
```

⟨*struct* Onex 24c⟩≡ (215b)

```
struct Onex{
    void    (*f)(void);
    int pid;
};
```

⟨*global* onexlock 24d⟩≡ (215b)

```
static Lock onexlock;
```

⟨*function* atexit 24e⟩≡ (215b)

```
int
atexit(void (*f)(void))
{
    int i;

    lock(&onexlock);
    for(i=0; i<NEXIT; i++)
        if(onex[i].f == nil) {
            onex[i].pid = getpid();
            onex[i].f = f;
            unlock(&onexlock);
            return 1;
        }
    unlock(&onexlock);
    return 0;
}
```

Chapter 5

User/Kernel Bridge

Chapter 6

Memory Area Operations

⟨*function* memset 26a⟩≡ (234e)

```
void*
memset(void *ap, int c, ulong n)
{
    char *p;

    p = ap;
    while(n > 0) {
        *p++ = c;
        n--;
    }
    return ap;
}
```

⟨*function* memcpy 26b⟩≡ (234d)

```
void*
memcpy(void *a1, void *a2, ulong n)
{
    return memmove(a1, a2, n);
}
```

⟨*function* memmove 26c⟩≡ (234d)

```
void*
memmove(void *a1, void *a2, ulong n)
{
    char *s1, *s2;

    if((long)n < 0)
        abort();
    s1 = a1;
    s2 = a2;
    if((s2 < s1) && (s2+n > s1))
        goto back;
    while(n > 0) {
        *s1++ = *s2++;
        n--;
    }
    return a1;

back:
    s1 += n;
    s2 += n;
    while(n > 0) {
        *--s1 = *--s2;
        n--;
```

```
    }
    return a1;
}
```

⟨*function* memcmp 27a⟩≡ (234c)

```
int
memcmp(void *a1, void *a2, ulong n)
{
    uchar *s1, *s2;
    uint c1, c2;

    s1 = a1;
    s2 = a2;
    while(n > 0) {
        c1 = *s1++;
        c2 = *s2++;
        if(c1 != c2) {
            if(c1 > c2)
                return 1;
            return -1;
        }
        n--;
    }
    return 0;
}
```

⟨*function* memchr 27b⟩≡ (234b)

```
void*
memchr(void *ap, int c, ulong n)
{
    uchar *sp;

    sp = ap;
    c &= 0xFF;
    while(n > 0) {
        if(*sp++ == c)
            return sp-1;
        n--;
    }
    return nil;
}
```

Chapter 7

Memory Management

7.1 Syscall: brk()

7.2 sbrk()

⟨*global* bloc 28a⟩≡ (333e)

```
// starting point for the heap, after the text, data, and bss.
static  char    *bloc = { end };
```

⟨*function* sbrk 28b⟩≡ (333e)

```
void*
sbrk(ulong n)
{
    uintptr bl;

    bl = ((uintptr)bloc + Round) & ~Round;
    if(brk((void*)(bl+n)) < 0)
        return (void*)-1;
    bloc = (char*)bl + n;
    return (void*)bl;
}
```

⟨*enum* _anon_ (9sys/sbrk.c) 28c⟩≡ (333e)

```
enum
{
    Round   = 7
};
```

7.3 Data structures

7.3.1 Pool

⟨*type* Pool 28d⟩≡ (210)

```
struct Pool {
 char* name;
 ulong maxsize;

 ulong cursize;
 ulong curfree;
 ulong curalloc;

 ulong minarena; /* smallest size of new arena */
```

```
  ulong quantum; /* allocated blocks should be multiple of */
  ulong minblock; /* smallest newly allocated block */

  void* freeroot; /* actually Free* */
  void* arenalist; /* actually Arena* */

  void* (*alloc)(ulong);
  int    (*merge)(void*, void*);
  void (*move)(void* from, void* to);

  int flags;
  int nfree;
  int lastcompact;

  void (*lock)(Pool*);
  void (*unlock)(Pool*);
  void (*print)(Pool*, char*, ...);
  void (*panic)(Pool*, char*, ...);
  void (*logstack)(Pool*);

  void* private;
 };
```

⟨*type* Pool_flag 29a⟩≡ (210)

```
 enum Pool_flag { /* flags */
  POOL_ANTAGONISM = 1<<0,
  POOL_PARANOIA = 1<<1,
  POOL_VERBOSITY = 1<<2,
  POOL_DEBUGGING = 1<<3,
  POOL_LOGGING = 1<<4,
  POOL_TOLERANCE = 1<<5,
  POOL_NOREUSE = 1<<6,
 };
```

⟨*struct* Private 29b⟩≡ (232d)

```
 struct Private {
     Lock    lk;
     int     pid;
     int     printfd;    /* gets debugging output if set */
 };
```

⟨*global* sbrkmempriv 29c⟩≡ (232d)

```
 Private sbrkmempriv;
```

7.3.2 sbrkmem

⟨*global* sbrkmem 29d⟩≡ (232d)

```
 static Pool sbrkmem = {
     .name=      "sbrkmem",
     .maxsize=   (3840UL-1)*1024*1024,   /* up to ~0xf0000000 */
     .minarena=  4*1024,
     .quantum=   32,
     .alloc=     sbrkalloc,
     .merge=     sbrkmerge,
     .flags=     0,

     .lock=      plock,
     .unlock=        punlock,
     .print=     pprint,
```

```
    .panic=     ppanic,
    .private=       &sbrkmempriv,
};
```

⟨*global* mainmem 30a⟩≡ (232d)

```
Pool *mainmem = &sbrkmem;
```

⟨*global* imagmem 30b⟩≡ (232d)

```
Pool *imagmem = &sbrkmem;
```

7.3.3 XXX

⟨*struct* Bhdr 30c⟩≡ (258d)

```
struct Bhdr {
    ulong   magic;
    ulong   size;
};
```

⟨*enum* _anon_ (port/pool.c) 30d⟩≡ (258d)

```
enum {
    NOT_MAGIC = 0xdeadfa11,
    DEAD_MAGIC = 0xdeaddead,
};
```

⟨*macro* B2NB 30e⟩≡ (258d)

```
#define B2NB(b) ((Bhdr*)((uchar*)(b)+(b)->size))
```

⟨*macro* SHORT 30f⟩≡ (258d)

```
#define SHORT(x) (((x)[0] << 8) | (x)[1])
```

⟨*macro* PSHORT 30g⟩≡ (258d)

```
#define PSHORT(p, x) \
    (((uchar*)(p))[0] = ((x)>>8)&0xFF, \
    ((uchar*)(p))[1] = (x)&0xFF)
```

⟨*enum* _anon_ (port/pool.c)*2* 30h⟩≡ (258d)

```
enum {
    TAIL_MAGIC0 = 0xBE,
    TAIL_MAGIC1 = 0xEF
};
```

⟨*struct* Btail 30i⟩≡ (258d)

```
struct Btail {
    uchar   magic0;
    uchar   datasize[2];
    uchar   magic1;
    ulong   size;   /* same as Bhdr->size */
};
```

⟨*macro* B2T 30j⟩≡ (258d)

```
#define B2T(b)  ((Btail*)((uchar*)(b)+(b)->size-sizeof(Btail)))
```

⟨*macro* B2PT 30k⟩≡ (258d)

```
#define B2PT(b) ((Btail*)((uchar*)(b)-sizeof(Btail)))
```

⟨*macro* T2HDR 30l⟩≡ (258d)

```
#define T2HDR(t) ((Bhdr*)((uchar*)(t)+sizeof(Btail)-(t)->size))
```

⟨*struct* Free 31a⟩≡ (258d)

```
struct Free {
            Bhdr;
    Free*   left;
    Free*   right;
    Free*   next;
    Free*   prev;
};
```

⟨*enum* _anon_ (port/pool.c)*3* 31b⟩≡ (258d)

```
enum {
    FREE_MAGIC = 0xBA5EBA11,
};
```

⟨*struct* Alloc 31c⟩≡ (258d)

```
/*
 * the point of the notused fields is to make 8c differentiate
 * between Bhdr and Allocblk, and between Kempt and Unkempt.
 */
struct Alloc {
            Bhdr;
};
```

⟨*enum* _anon_ (port/pool.c)*4* 31d⟩≡ (258d)

```
enum {
    ALLOC_MAGIC = 0x0A110C09,
    UNALLOC_MAGIC = 0xCAB00D1E+1,
};
```

⟨*struct* Arena 31e⟩≡ (258d)

```
struct Arena {
            Bhdr;
    Arena*  aup;
    Arena*  down;
    ulong   asize;
    ulong   pad;    /* to a multiple of 8 bytes */
};
```

⟨*enum* _anon_ (port/pool.c)*5* 31f⟩≡ (258d)

```
enum {
    ARENA_MAGIC = 0xC0A1E5CE+1,
    ARENATAIL_MAGIC = 0xEC5E1A0C+1,
};
```

⟨*macro* A2TB 31g⟩≡ (258d)

```
#define A2TB(a) ((Bhdr*)((uchar*)(a)+(a)->asize-sizeof(Bhdr)))
```

⟨*macro* A2B 31h⟩≡ (258d)

```
#define A2B(a)  B2NB(a)
```

⟨*enum* _anon_ (port/pool.c)*6* 31i⟩≡ (258d)

```
enum {
    ALIGN_MAGIC = 0xA1F1D1C1,
};
```

⟨*enum* _anon_ (port/pool.c)*7* 31j⟩≡ (258d)

```
enum {
    MINBLOCKSIZE = sizeof(Free)+sizeof(Btail)
};
```

⟨*global* `datamagic` 32a⟩≡ (258d)

```
static uchar datamagic[] = { 0xFE, 0xF1, 0xF0, 0xFA };
```

⟨*constant* `Poison` 32b⟩≡ (258d)

```
#define Poison  (void*)0xCafeBabe
```

⟨*macro* `_B2D` 32c⟩≡ (258d)

```
#define _B2D(a) ((void*)((uchar*)a+sizeof(Bhdr)))
```

⟨*macro* `_D2B` 32d⟩≡ (258d)

```
#define _D2B(v) ((Alloc*)((uchar*)v-sizeof(Bhdr)))
```

7.4 malloc()

⟨*function* `malloc` 32e⟩≡ (232d)

```
void*
malloc(ulong size)
{
    void *v;

    v = poolalloc(mainmem, size+Npadlong*sizeof(ulong));
    if(Npadlong && v != nil) {
        v = (ulong*)v+Npadlong;
        setmalloctag(v, getcallerpc(&size));
        setrealloctag(v, 0);
    }
    return v;
}
```

7.5 free()

⟨*function* `free` 32f⟩≡ (232d)

```
void
free(void *v)
{
    if(v != nil)
        poolfree(mainmem, (ulong*)v-Npadlong);
}
```

7.6 Debugging support

7.7 Advanced topics

Chapter 8

Strings

8.1 Plain C strings

⟨*function* strlen 33a⟩≡ (278c)

```
long
strlen(char *s)
{

    return strchr(s, '\0') - s;
}
```

⟨*function* strcat 33b⟩≡ (276d)

```
char*
strcat(char *s1, char *s2)
{

    strcpy(strchr(s1, '\0'), s2);
    return s1;
}
```

⟨*function* strchr 33c⟩≡ (276e)

```
char*
strchr(char *s, int c)
{
    char c0 = c;
    char c1;

    if(c == '\0') {
        while(*s++)
            ;
        return s-1;
    }

    while(c1 = *s++)
        if(c1 == c0)
            return s-1;
    return nil;
}
```

⟨*function* strrchr 33d⟩≡ (280c)

```
char*
strrchr(char *s, int c)
{
    char *r;
```

```
    if(c == '\0')
        return strchr(s, '\0');

    r = 0;
    while(s = strchr(s, c))
        r = s++;
    return r;
}
```

⟨*function* strcmp 34a⟩≡ (277a)

```
int
strcmp(char *s1, char *s2)
{
    unsigned c1, c2;

    for(;;) {
        c1 = *s1++;
        c2 = *s2++;
        if(c1 != c2) {
            if(c1 > c2)
                return 1;
            return -1;
        }
        if(c1 == 0)
            return 0;
    }
}
```

⟨*constant* N 34b⟩≡ (277b)

```
#define N   10000
```

⟨*function* strcpy 34c⟩≡ (277b)

```
char*
strcpy(char *s1, char *s2)
{
    char *os1;

    os1 = s1;
    while(!memccpy(s1, s2, 0, N)) {
        s1 += N;
        s2 += N;
    }
    return os1;
}
```

⟨*function* memccpy 34d⟩≡ (234a)

```
void*
memccpy(void *a1, void *a2, int c, ulong n)
{
    uchar *s1, *s2;

    s1 = a1;
    s2 = a2;
    c &= 0xFF;
    while(n > 0) {
        if((*s1++ = *s2++) == c)
            return s1;
        n--;
    }
    return nil;
}
```

⟨*function* strdup 35a⟩≡ (277f)

```
char*
strdup(char *s)
{
    char *ns;

    ns = malloc(strlen(s) + 1);
    if(ns == nil)
        return nil;
    setmalloctag(ns, getcallerpc(&s));

    return strcpy(ns, s);
}
```

⟨*function* strstr 35b⟩≡ (281b)

```
/*
 * Return pointer to first occurrence of s2 in s1,
 * 0 if none
 */
char*
strstr(char *s1, char *s2)
{
    char *p, *pa, *pb;
    int c0, c;

    c0 = *s2;
    if(c0 == 0)
        return s1;
    s2++;
    for(p=strchr(s1, c0); p; p=strchr(p+1, c0)) {
        pa = p;
        for(pb=s2;; pb++) {
            c = *pb;
            if(c == 0)
                return p;
            if(c != *++pa)
                break;
        }
    }
    return 0;
}
```

8.2 Runes

⟨*function* tobaserune 35c⟩≡ (? 0—1)

```
Rune
tobaserune(Rune c)
{
    Rune *p;

    p = _runebsearch(c, _base2, nelem(_base2)/2, 2);
    if(p && c == p[0])
        c = p[1];
    return c;
}
```

⟨*function* isbaserune 35d⟩≡ (? 0—1)

```
int
```

```
isbaserune(Rune c)
{
    return tobaserune(c) == c;
}
```

⟨*function* _runebsearch 36a⟩≡ (271a)

```
Rune*
_runebsearch(Rune c, Rune *t, int n, int ne)
{
    Rune *p;
    int m;

    while(n > 1) {
        m = n/2;
        p = t + m*ne;
        if(c >= p[0]) {
            t = p;
            n = n-m;
        } else
            n = m;
    }
    if(n && c >= t[0])
        return t;
    return 0;
}
```

8.2.1 Conversion

⟨*macro* Bit 36b⟩≡ (270c)

```
#define Bit(i) (7-(i))
```

⟨*macro* T 36c⟩≡ (270c)

```
/* N 0's preceded by i 1's, T(Bit(2)) is 1100 0000 */
#define T(i) (((1 << (Bit(i)+1))-1) ^ 0xFF)
```

⟨*macro* RuneX 36d⟩≡ (270c)

```
/* 0000 0000 0000 0111 1111 1111 */
#define RuneX(i) ((1 << (Bit(i) + ((i)-1)*Bitx))-1)
```

⟨*function* chartorune 36e⟩≡ (270c)

```
int
chartorune(Rune *rune, char *str)
{
    int c[UTFmax], i;
    Rune l;

    /*
     * N character sequence
     *  00000-0007F => T1
     *  00080-007FF => T2 Tx
     *  00800-0FFFF => T3 Tx Tx
     *  10000-10FFFF => T4 Tx Tx Tx
     */

    c[0] = *(uchar*)(str);
    if(c[0] < Tx){
        *rune = c[0];
        return 1;
    }
```

```
    l = c[0];

    for(i = 1; i < UTFmax; i++) {
        c[i] = *(uchar*)(str+i);
        c[i] ^= Tx;
        if(c[i] & Testx)
            goto bad;
        l = (l << Bitx) | c[i];
        if(c[0] < T(i + 2)) {
            l &= RuneX(i + 1);
            if(i == 1) {
                if(c[0] < T(2) || l <= Rune1)
                    goto bad;
            } else if(l <= RuneX(i) || l > Runemax)
                goto bad;
            if (i == 2 && SurrogateMin <= l && l <= SurrogateMax)
                goto bad;
            *rune = l;
            return i + 1;
        }
    }

    /*
     * bad decoding
     */
bad:
    *rune = Bad;
    return 1;
}
```

⟨*function* runetochar 37⟩≡ (270c)

```
int
runetochar(char *str, Rune *rune)
{
    int i, j;
    Rune c;

    c = *rune;
    if(c <= Rune1) {
        str[0] = c;
        return 1;
    }

    /*
     * one character sequence
     *  00000-0007F => 00-7F
     * two character sequence
     *  0080-07FF => T2 Tx
     * three character sequence
     *  0800-FFFF => T3 Tx Tx
     * four character sequence (21-bit value)
     *     10000-1FFFFF => T4 Tx Tx Tx
     * If the Rune is out of range or a surrogate half,
     * convert it to the error rune.
     * Do this test when i==3 because the error rune encodes to three bytes.
     * Doing it earlier would duplicate work, since an out of range
     * Rune wouldn't have fit in one or two bytes.
     */
    for(i = 2; i < UTFmax + 1; i++){
        if(i == 3){
```

```
            if(c > Runemax)
                c = Runeerror;
            if(SurrogateMin <= c && c <= SurrogateMax)
                c = Runeerror;
        }
        if (c <= RuneX(i) || i == UTFmax ) {
            str[0] = T(i) |  (c >> (i - 1)*Bitx);
            for(j = 1; j < i; j++)
                str[j] = Tx | ((c >> (i - j - 1)*Bitx) & Maskx);
            return i;
        }
    }
    return UTFmax;
}
```

8.2.2 Basics

⟨*function* runestrlen 38a⟩≡ (272c)

```
long
runestrlen(Rune *s)
{

    return runestrchr(s, 0) - s;
}
```

⟨*function* runestrcpy 38b⟩≡ (271e)

```
Rune*
runestrcpy(Rune *s1, Rune *s2)
{
    Rune *os1;

    os1 = s1;
    while(*s1++ = *s2++)
        ;
    return os1;
}
```

⟨*function* runestrcmp 38c⟩≡ (271d)

```
int
runestrcmp(Rune *s1, Rune *s2)
{
    Rune c1, c2;

    for(;;) {
        c1 = *s1++;
        c2 = *s2++;
        if(c1 != c2) {
            if(c1 > c2)
                return 1;
            return -1;
        }
        if(c1 == 0)
            return 0;
    }
}
```

⟨*function* runestrchr 39a⟩≡ (271c)

```
Rune*
runestrchr(Rune *s, Rune c)
{
    Rune c0 = c;
    Rune c1;

    if(c == 0) {
        while(*s++)
            ;
        return s-1;
    }

    while(c1 = *s++)
        if(c1 == c0)
            return s-1;
    return nil;
}
```

⟨*function* runestrrchr 39b⟩≡ (273e)

```
Rune*
runestrrchr(Rune *s, Rune c)
{
    Rune *r;

    if(c == 0)
        return runestrchr(s, 0);
    r = 0;
    while(s = runestrchr(s, c))
        r = s++;
    return r;
}
```

⟨*function* runestrdup 39c⟩≡ (271f)

```
Rune*
runestrdup(Rune *s)
{
    Rune *ns;

    ns = malloc(sizeof(Rune)*(runestrlen(s) + 1));
    if(ns == nil)
        return nil;
    setmalloctag(ns, getcallerpc(&s));

    return runestrcpy(ns, s);
}
```

⟨*function* runestrcat 39d⟩≡ (271b)

```
Rune*
runestrcat(Rune *s1, Rune *s2)
{

    runestrcpy(runestrchr(s1, 0), s2);
    return s1;
}
```

⟨*function* runestrstr 39e⟩≡ (274a)

```
/*
 * Return pointer to first occurrence of s2 in s1,
 * 0 if none
```

```
 */
Rune*
runestrstr(Rune *s1, Rune *s2)
{
    Rune *p, *pa, *pb;
    int c0, c;

    c0 = *s2;
    if(c0 == 0)
        return s1;
    s2++;
    for(p=runestrchr(s1, c0); p; p=runestrchr(p+1, c0)) {
        pa = p;
        for(pb=s2;; pb++) {
            c = *pb;
            if(c == 0)
                return p;
            if(c != *++pa)
                break;
        }
    }
    return nil;
}
```

8.3 UTF

⟨*function* utflen 40a⟩≡ (297c)

```
int
utflen(char *s)
{
    int c;
    long n;
    Rune rune;

    n = 0;
    for(;;) {
        c = *(uchar*)s;
        if(c < Runeself) {
            if(c == '\0')
                return n;
            s++;
        } else
            s += chartorune(&rune, s);
        n++;
    }
}
```

⟨*function* utfrune 40b⟩≡ (298c)

```
char*
utfrune(char *s, long c)
{
    long c1;
    Rune r;
    int n;

    if(c < Runesync)        /* not part of utf sequence */
        return strchr(s, c);
```

```
    for(;;) {
        c1 = *(uchar*)s;
        if(c1 < Runeself) { /* one byte rune */
            if(c1 == 0)
                return nil;
            if(c1 == c)
                return s;
            s++;
            continue;
        }
        n = chartorune(&r, s);
        if(r == c)
            return s;
        s += n;
    }
}
```

⟨*function* utfrrune 41a⟩≡ (298b)

```
char*
utfrrune(char *s, long c)
{
    long c1;
    Rune r;
    char *s1;

    if(c < Runesync)        /* not part of utf sequence */
        return strrchr(s, c);

    s1 = 0;
    for(;;) {
        c1 = *(uchar*)s;
        if(c1 < Runeself) { /* one byte rune */
            if(c1 == 0)
                return s1;
            if(c1 == c)
                s1 = s;
            s++;
            continue;
        }
        c1 = chartorune(&r, s);
        if(r == c)
            s1 = s;
        s += c1;
    }
}
```

⟨*function* utfutf 41b⟩≡ (298d)

```
/*
 * Return pointer to first occurrence of s2 in s1,
 * 0 if none
 */
char*
utfutf(char *s1, char *s2)
{
    char *p;
    long f, n1, n2;
    Rune r;

    n1 = chartorune(&r, s2);
    f = r;
```

```
    if(f <= Runesync)       /* represents self */
        return strstr(s1, s2);

    n2 = strlen(s2);
    for(p=s1; p=utfrune(p, f); p+=n1)
        if(strncmp(p, s2, n2) == 0)
            return p;
    return 0;
}
```

8.4 Formatted Output

⟨*type* Fmt 42a⟩≡ (194b)

```
struct Fmt {
    uchar   runes;          /* output buffer is runes or chars? */

    void    *start;         /* of buffer */
    void    *to;            /* current place in the buffer */
    void    *stop;          /* end of the buffer; overwritten if flush fails */

    int     (*flush)(Fmt *);    /* called when to == stop */
    void    *farg;          /* to make flush a closure */
    int     nfmt;           /* num chars formatted so far */

    va_list args;           /* args passed to dofmt */

    int     r;          /* % format Rune */
    int     width;
    int     prec;
    ulong   flags;
};
```

⟨*type* Fmt_flag 42b⟩≡ (194b)

```
enum Fmt_flag {
    FmtWidth    = 1,
    FmtLeft     = FmtWidth << 1,
    FmtPrec     = FmtLeft << 1,
    FmtSharp    = FmtPrec << 1,
    FmtSpace    = FmtSharp << 1,
    FmtSign     = FmtSpace << 1,
    FmtZero     = FmtSign << 1,
    FmtUnsigned = FmtZero << 1,
    FmtShort    = FmtUnsigned << 1,
    FmtLong     = FmtShort << 1,
    FmtVLong    = FmtLong << 1,
    FmtComma    = FmtVLong << 1,
    FmtByte     = FmtComma << 1,

    FmtFlag     = FmtByte << 1
};
```

8.5 Quoted strings

8.6 Extensible strings

Chapter 9

Regular Expressions

9.1 Data structures

9.2 Advanced topics

Chapter 10

File IO

10.1 Data structures

⟨*type* Open_flag 44a⟩≡ (205)

```
// enum Open_flag, open parameter
#define OREAD 0 /* open for read */
#define OWRITE 1 /* write */
#define ORDWR 2 /* read and write */
#define OEXEC 3 /* execute, == read but check execute permission */
// advanced stuff (no O_APPEND, O_CREATE, O_NONBLOCK as in Unix though)
#define OTRUNC 16 /* or'ed in (except for exec), truncate file first */
#define OCEXEC 32 /* or'ed in, close on exec */
#define ORCLOSE 64 /* or'ed in, remove on close */
#define OEXCL 0x1000 /* or'ed in, exclusive use (create only) */
```

⟨*type* Access_flag 44b⟩≡ (205)

```
// enum Access_flag
#define AEXIST 0 /* accessible: exists */
#define AEXEC 1 /* execute access */
#define AWRITE 2 /* write access */
#define AREAD 4 /* read access */
```

⟨*type* Qid 44c⟩≡ (205)

```
// Qid as in uniQue id
struct Qid {
 uvlong path;
 ulong vers;
    // bitset<Qidtype>
 uchar type;
};
```

⟨*type* Qid_type 44d⟩≡ (205)

```
/* bits in Qid.type */
#define QTFILE  0x00  /* plain file */
#define QTDIR  0x80  /* type bit for directories */
// advanced stuff
#define QTAPPEND 0x40  /* type bit for append only files */
#define QTEXCL  0x20  /* type bit for exclusive use files */
#define QTMOUNT  0x10  /* type bit for mounted channel */
#define QTAUTH  0x08  /* type bit for authentication file */
#define QTTMP  0x04  /* type bit for not-backed-up file */
```

⟨*type* Seek_cursor 45a⟩≡ (205)

```
// pad's stuff (but it is actually also in stdio.h)
enum Seek_cursor {
    SEEK__START = 0,
    SEEK__CUR = 1,
    SEEK__END = 2,
};
```

10.2 Syscalls: open(), pread(), etc

10.3 read(), write()

⟨*function* read 45b⟩≡ (332a)

```
long
read(fdt fd, void *buf, long n)
{
    return pread(fd, buf, n, -1LL);
}
```

⟨*function* write 45c⟩≡ (340c)

```
long
write(fdt fd, void *buf, long n)
{
    return pwrite(fd, buf, n, -1LL);
}
```

10.4 readn(), writen()

⟨*function* readn 45d⟩≡ (269b)

```
long
readn(fdt f, void *av, long n)
{
    char *a;
    long m, t;

    a = av;
    t = 0;
    while(t < n){
        m = read(f, a+t, n-t);
        if(m <= 0){
            if(t == 0)
                return m;
            break;
        }
        t += m;
    }
    return t;
}
```

Chapter 11

Directories and File Paths

11.1 Data structures: Dir

⟨*type* DirEntry 46a⟩≡ (205)

```
// a similar structure is defined in the kernel!
struct Dir {
 /* system-modified data */
 ushort type; /* server type */
 uint dev; /* server subtype */

 /* file data */
 Qid qid; /* unique id from server */

 ulong mode; /* permissions */
 ulong atime; /* last read time */
 ulong mtime; /* last write time */
 vlong length; /* file length */
 char *name; /* last element of path */

 char *uid; /* owner name */
 char *gid; /* group name */
 char *muid; /* last modifier name */
};
```

⟨*type* Dir_mode 46b⟩≡ (205)

```
/* bits in Dir.mode */
#define DMDIR  0x80000000 /* mode bit for directories */
#define DMREAD  0x4  /* mode bit for read permission */
#define DMWRITE  0x2  /* mode bit for write permission */
#define DMEXEC  0x1  /* mode bit for execute permission */
// advanced stuff
#define DMAPPEND 0x40000000 /* mode bit for append only files */
#define DMEXCL  0x20000000 /* mode bit for exclusive use files */
#define DMMOUNT  0x10000000 /* mode bit for mounted channel */
#define DMAUTH  0x08000000 /* mode bit for authentication file */
#define DMTMP  0x04000000 /* mode bit for non-backed-up files */
```

⟨*constant* DIRMAX 46c⟩≡ (205)

```
#define DIRMAX (sizeof(Dir)+STATMAX) /* max length of Dir structure */
```

⟨*constant* STATMAX 46d⟩≡ (205)

```
#define STATMAX 65535U /* max length of machine-independent stat structure */
```

⟨*function* nulldir 47a⟩≡ (328b)

```
void
nulldir(Dir *d)
{
    memset(d, ~0, sizeof(Dir));
    d->name = d->uid = d->gid = d->muid = "";
}
```

11.2 Syscalls: create(), chdir(), etc

11.3 xxxfstat()

⟨*enum* _anon_ (9sys/dirfstat.c) 47b⟩≡ (317a)

```
enum
{
    DIRSIZE = STATFIXLEN + 16 * 4       /* enough for encoded stat buf + some reasonable strings */
};
```

⟨*function* dirfstat 47c⟩≡ (317a)

```
Dir*
dirfstat(int fd)
{
    Dir *d;
    uchar *buf;
    int n, nd, i;

    nd = DIRSIZE;
    for(i=0; i<2; i++){ /* should work by the second try */
        d = malloc(sizeof(Dir) + BIT16SZ + nd);
        if(d == nil)
            return nil;
        buf = (uchar*)&d[1];
        n = fstat(fd, buf, BIT16SZ+nd);
        if(n < BIT16SZ){
            free(d);
            return nil;
        }
        nd = GBIT16(buf);   /* upper bound on size of Dir + strings */
        if(nd <= n){
            convM2D(buf, n, d, (char*)&d[1]);
            return d;
        }
        /* else sizeof(Dir)+BIT16SZ+nd is plenty */
        free(d);
    }
    return nil;
}
```

⟨*function* dirfwstat 47d⟩≡ (317b)

```
int
dirfwstat(int fd, Dir *d)
{
    uchar *buf;
    int r;

    r = sizeD2M(d);
    buf = malloc(r);
```

```
    if(buf == nil)
        return -1;
    convD2M(d, buf, r);
    r = fwstat(fd, buf, r);
    free(buf);
    return r;
}
```

11.4 xxxstat()

⟨*enum* _anon_ (9sys/dirstat.c) 48a⟩≡ (320c)

```
enum
{
    DIRSIZE = STATFIXLEN + 16 * 4       /* enough for encoded stat buf + some reasonable strings */
};
```

⟨*function* dirstat 48b⟩≡ (320c)

```
Dir*
dirstat(char *name)
{
    Dir *d;
    uchar *buf;
    int n, nd, i;

    nd = DIRSIZE;
    for(i=0; i<2; i++){ /* should work by the second try */
        d = malloc(sizeof(Dir) + BIT16SZ + nd);
        if(d == nil)
            return nil;
        buf = (uchar*)&d[1];
        n = stat(name, buf, BIT16SZ+nd);
        if(n < BIT16SZ){
            free(d);
            return nil;
        }
        nd = GBIT16((uchar*)buf);   /* upper bound on size of Dir + strings */
        if(nd <= n){
            convM2D(buf, n, d, (char*)&d[1]);
            return d;
        }
        /* else sizeof(Dir)+BIT16SZ+nd is plenty */
        free(d);
    }
    return nil;
}
```

⟨*function* dirwstat 48c⟩≡ (321a)

```
int
dirwstat(char *name, Dir *d)
{
    uchar *buf;
    int r;

    r = sizeD2M(d);
    buf = malloc(r);
    if(buf == nil)
        return -1;
    convD2M(d, buf, r);
```

```
    r = wstat(name, buf, r);
    free(buf);
    return r;
}
```

11.5 getwd()

⟨*function* getwd 49a⟩≡ (326a)

```
char*
getwd(char *buf, int nbuf)
{
    int n;
    fdt fd;

    fd = open(".", OREAD);
    if(fd < 0)
        return nil;
    n = fd2path(fd, buf, nbuf);
    close(fd);
    if(n < 0)
        return nil;
    return buf;
}
```

11.6 The deprecated mktemp()

⟨*function* mktemp 49b⟩≡ (234f)

```
char*
mktemp(char *as)
{
    char *s;
    unsigned pid;
    int i;
    char err[ERRMAX];

    pid = getpid();
    s = as;
    while(*s++)
        ;
    s--;
    while(*--s == 'X') {
        *s = pid % 10 + '0';
        pid = pid/10;
    }
    s++;
    i = 'a';
    while(access(as, 0) != -1) {
        if (i == 'z')
            return "/";
        *s = i++;
    }
    err[0] = '\0';
    errstr(err, sizeof err);    /* clear the error */
    return as;
}
```

11.7 The safe mkstemp()

11.8 Normalizing a filename: cleanname()

⟨*macro* SEP 50a⟩≡ (219c)

```
#define SEP(x)  ((x)=='/' || (x) == '\0')
```

⟨*function* cleanname 50b⟩≡ (219c)

```
/*
 * In place, rewrite name to compress multiple /, eliminate ., and process ..
 */
char*
cleanname(char *name)
{
    char *s;    /* source of copy */
    char *d;    /* destination of copy */
    char *d0;   /* start of path afer the root name */
    Rune r;
    bool rooted;

    if(name[0] == '\0')
        return strcpy(name, ".");

    rooted = false;
    d0 = name;
    if(d0[0] == '#'){
        if(d0[1] == '\0')
            return d0;
        d0  += 1 + chartorune(&r, d0+1); /* ignore slash: #/ */
        while(!SEP(*d0))
            d0 += chartorune(&r, d0);
        if(d0 == '\0')
            return name;
        d0++;   /* keep / after #<name> */
        rooted = true;
    }else if(d0[0] == '/'){
        rooted = true;
        d0++;
    }

    s = d0;
    if(rooted){
        /* skip extra '/' at root name */
        for(; *s == '/'; s++)
            ;
    }
    /* remove dup slashes */
    for(d = d0; *s != '\0'; s++){
        *d++ = *s;
        if(*s == '/')
            while(s[1] == '/')
                s++;
    }
    *d = '\0';

    d = d0;
    s = d0;
    while(*s != '\0'){
        if(s[0] == '.' && SEP(s[1])){
```

```
            if(s[1] == '\0')
                break;
            s+= 2;
            continue;
        }
        if(s[0] == '.' && s[1] == '.' && SEP(s[2])){
            if(d == d0){
                if(rooted){
                    /* /../x -> /x */
                    if(s[2] == '\0')
                        break;
                    s += 3;
                    continue;
                }else{
                    /* ../x -> ../x; and never collect ../ */
                    d0 += 3;
                }
            }
            if(d > d0){
                /* a/../x -> x */
                assert(d-2 >= d0 && d[-1] == '/');
                for(d -= 2; d > d0 && d[-1] != '/'; d--)
                        ;
                if(s[2] == '\0')
                    break;
                s += 3;
                continue;
            }
        }
        while(!SEP(*s))
            *d++ = *s++;
        if(*s == '\0')
            break;

        *d++ = *s++;
    }
    *d = '\0';
    if(d-1 > name && d[-1] == '/')  /* thanks to #/ */
        *--d = '\0';
    if(name[0] == '\0')
        strcpy(name, ".");
    return name;
}
```

11.9 access()

⟨*function* access 51⟩≡ (299a)

```
int
access(char *name, int mode)
{
    fdt fd;
    Dir *db;
    static char omode[] = {
        0,
        OEXEC,
        OWRITE,
        ORDWR,
        OREAD,
```

```
        OEXEC,  /* only approximate */
        ORDWR,
        ORDWR   /* only approximate */
    };

    if(mode == AEXIST){
        db = dirstat(name);
        free(db);
        if(db != nil)
            return 0;
        return -1;
    }
    fd = open(name, omode[mode&7]);
    if(fd >= 0){
        close(fd);
        return 0;
    }
    return -1;
}
```

11.10 Path manipulation

Chapter 12

Namespace

⟨*type* Namespace_flag 53⟩≡ (205)

```
// enum Namespace_flag, mount/bind parameter
#define MREPL 0x0000 /* mount replaces object */
#define MBEFORE 0x0001 /* mount goes before others in union directory */
#define MAFTER 0x0002 /* mount goes after others in union directory */

#define MCREATE 0x0004 /* permit creation in mounted directory */
#define MCACHE 0x0010 /* cache some data */
// bitset<Namespace_flag>
#define MORDER 0x0003 /* mask for bits defining order of mounting */

#define MMASK 0x0017 /* all bits on */
```

12.1 Syscalls: bind(), mount(), etc

Chapter 13

Time

13.1 Syscalls: sleep(), alarm(), etc

13.2 Time and day

13.3 Time zones and localtime()

⟨*function* localtime 54a⟩≡ (307)

```
Tm*
localtime(long tim)
{
    Tm *ct;
    long t, *p;
    int dlflag;

    if(timezone.stname[0] == 0)
        readtimezone();
    t = tim + timezone.stdiff;
    dlflag = 0;
    for(p = timezone.dlpairs; *p; p += 2)
        if(t >= p[0])
        if(t < p[1]) {
            t = tim + timezone.dldiff;
            dlflag++;
            break;
        }
    ct = gmtime(t);
    if(dlflag){
        strcpy(ct->zone, timezone.dlname);
        ct->tzoff = timezone.dldiff;
    } else {
        strcpy(ct->zone, timezone.stname);
        ct->tzoff = timezone.stdiff;
    }
    return ct;
}
```

⟨*function* asctime 54b⟩≡ (307)

```
char*
asctime(Tm *t)
{
    char *ncp;
    static char cbuf[30];
```

```
    strcpy(cbuf, "Thu Jan 01 00:00:00 GMT 1970\n");
    ncp = &"SunMonTueWedThuFriSat"[t->wday*3];
    cbuf[0] = *ncp++;
    cbuf[1] = *ncp++;
    cbuf[2] = *ncp;
    ncp = &"JanFebMarAprMayJunJulAugSepOctNovDec"[t->mon*3];
    cbuf[4] = *ncp++;
    cbuf[5] = *ncp++;
    cbuf[6] = *ncp;
    ct_numb(cbuf+8, t->mday);
    ct_numb(cbuf+11, t->hour+100);
    ct_numb(cbuf+14, t->min+100);
    ct_numb(cbuf+17, t->sec+100);
    ncp = t->zone;
    cbuf[20] = *ncp++;
    cbuf[21] = *ncp++;
    cbuf[22] = *ncp;
    if(t->year >= 100) {
        cbuf[24] = '2';
        cbuf[25] = '0';
    }
    ct_numb(cbuf+26, t->year+100);
    return cbuf;
}
```

13.4 time()

⟨*function* time 55a⟩≡ (335c)

```
long
time(long *tp)
{
    vlong t;

    t = nsec()/1000000000LL;
    if(t == 0)
        t = oldtime(0);
    if(tp != nil)
        *tp = t;
    return t;
}
```

⟨*function* oldtime 55b⟩≡ (335c)

```
/*
 *  After a fork with fd's copied, both fd's are pointing to
 *  the same Chan structure.  Since the offset is kept in the Chan
 *  structure, the seek's and read's in the two processes can
 *  compete at moving the offset around.  Hence the unusual loop
 *  in the middle of this routine.
 */
static long
oldtime(long *tp)
{
    char b[20];
    static int f = -1;
    int i, retries;
    long t;
```

```
    memset(b, 0, sizeof(b));
    for(retries = 0; retries < 100; retries++){
        if(f < 0)
            f = open("/dev/time", OREAD|OCEXEC);
        if(f < 0)
            break;
        if(seek(f, 0, 0) < 0 || (i = read(f, b, sizeof(b))) < 0){
            close(f);
            f = -1;
        } else {
            if(i != 0)
                break;
        }
    }
    t = atol(b);
    if(tp)
        *tp = t;
    return t;
}
```

13.5 Conversions

⟨*function* tm2sec 56⟩≡ (339a)

```
/*
 * compute seconds since Jan 1 1970 GMT
 * and convert to our timezone.
 */
long
tm2sec(Tm *tm)
{
    long secs;
    int i, yday, year, *d2m;

    if(strcmp(tm->zone, "GMT") != 0 && timezone.stname[0] == 0)
        readtimezone();
    secs = 0;

    /*
     *  seconds per year
     */
    year = tm->year + 1900;
    for(i = 1970; i < year; i++){
        d2m = yrsize(i);
        secs += d2m[0] * SEC2DAY;
    }

    /*
     *  if mday is set, use mon and mday to compute yday
     */
    if(tm->mday){
        yday = 0;
        d2m = yrsize(year);
        for(i=0; i<tm->mon; i++)
            yday += d2m[i+1];
        yday += tm->mday-1;
    }else{
        yday = tm->yday;
    }
```

```
    secs += yday * SEC2DAY;

    /*
     * hours, minutes, seconds
     */
    secs += tm->hour * SEC2HOUR;
    secs += tm->min * SEC2MIN;
    secs += tm->sec;

    /*
     * Only handles zones mentioned in /env/timezone,
     * but things get too ambiguous otherwise.
     */
    if(strcmp(tm->zone, timezone.stname) == 0)
        secs -= timezone.stdiff;
    else if(strcmp(tm->zone, timezone.dlname) == 0)
        secs -= timezone.dldiff;
    if(secs < 0)
        secs = 0;
    return secs;
}
```

Chapter 14

Fork, Exec, and Wait

⟨*type* Rfork_flag 58a⟩≡ (205)

```
/* rfork */
enum Rfork_flags
{
    RFNAMEG  = (1<<0),
    RFENVG   = (1<<1),
    RFFDG    = (1<<2),
    RFNOTEG  = (1<<3),
    RFPROC   = (1<<4),
    RFMEM    = (1<<5),
    RFNOWAIT = (1<<6),
    RFCNAMEG = (1<<10),
    RFCENVG  = (1<<11),
    RFCFDG   = (1<<12),
    RFREND   = (1<<13),
    RFNOMNT  = (1<<14)
};
```

14.1 Syscalls: rfork(), exec(), await(), etc

14.2 fork()

⟨*function* fork 58b⟩≡ (322b)

```
int
fork(void)
{
    return rfork(RFPROC|RFFDG|RFREND);
}
```

14.3 execl()

⟨*function* execl 58c⟩≡ (221b)

```
int
execl(char *f, ...)
{

    return exec(f, &f+1);
}
```

14.4 wait()

14.5 getpid()

⟨*function* getpid 59a⟩≡ (325d)

```
int
getpid(void)
{
    char b[20];
    fdt f;

    memset(b, 0, sizeof(b));
    f = open("#c/pid", OREAD);
    if(f >= 0) {
        read(f, b, sizeof(b));
        close(f);
    }
    return atol(b);
}
```

⟨*function* getppid 59b⟩≡ (325e)

```
int
getppid(void)
{
    char b[20];
    int f;

    memset(b, 0, sizeof(b));
    f = open("/dev/ppid", OREAD);
    if(f >= 0) {
        read(f, b, sizeof(b));
        close(f);
    }
    return atol(b);
}
```

Chapter 15

Concurrency

15.1 Syscalls: `rendezvous()`, etc

15.2 Locks

⟨*type* `Lock` 60a⟩≡ (194b)

```
struct Lock {
    long    key;
    long    sem;
};
```

⟨*function* `lock` 60b⟩≡ (225d)

```
void
lock(Lock *l)
{
    if(ainc(&l->key) == 1)
        return; /* changed from 0 -> 1: we hold lock */

    /* otherwise wait in kernel */
    while(semacquire(&l->sem, 1) < 0){
        /* interrupted; try again */
    }
}
```

⟨*function* `unlock` 60c⟩≡ (225d)

```
void
unlock(Lock *l)
{
    if(adec(&l->key) == 0)
        return; /* changed from 1 -> 0: no contention */
    semrelease(&l->sem, 1);
}
```

⟨*function* `canlock` 60d⟩≡ (225d)

```
bool
canlock(Lock *l)
{
    if(ainc(&l->key) == 1)
        return true;   /* changed from 0 -> 1: success */

    /* Undo increment (but don't miss wakeup) */
    if(adec(&l->key) == 0)
        return false;   /* changed from 1 -> 0: no contention */
    semrelease(&l->sem, 1);
```

```
    return false;
}
```

15.3 Atomic operations: ainc() (ARM)

15.4 Waiting queues

⟨*type* QLock 61a⟩≡ (194b)

```
struct QLock {
    Lock    lock;
    int locked;

    QLp *head;
    QLp     *tail;
};
```

⟨*type* QLp 61b⟩≡ (194b)

```
struct QLp {
    char    state;
    int inuse;
    // Extra
    QLp *next;
};
```

⟨*function* qlock 61c⟩≡ (331e)

```
void
qlock(QLock *q)
{
    QLp *p, *mp;

    lock(&q->lock);
    if(!q->locked){
        q->locked = 1;
        unlock(&q->lock);
        return;
    }


    /* chain into waiting list */
    mp = getqlp();
    p = q->tail;
    if(p == nil)
        q->head = mp;
    else
        p->next = mp;
    q->tail = mp;
    mp->state = Queuing;
    unlock(&q->lock);

    /* wait */
    while((*_rendezvousp)(mp, (void*)1) == (void*)~0)
        ;
    mp->inuse = 0;
}
```

⟨*function* qunlock 62a⟩≡ (331e)

```
void
qunlock(QLock *q)
{
    QLp *p;

    lock(&q->lock);
    if (q->locked == 0)
        fprint(2, "qunlock called with qlock not held, from %#p\n",
            getcallerpc(&q));
    p = q->head;
    if(p != nil){
        /* wakeup head waiting process */
        q->head = p->next;
        if(q->head == nil)
            q->tail = nil;
        unlock(&q->lock);
        while((*_rendezvousp)(p, (void*)0x12345) == (void*)~0)
            ;
        return;
    }
    q->locked = 0;
    unlock(&q->lock);
}
```

⟨*function* canqlock 62b⟩≡ (331e)

```
int
canqlock(QLock *q)
{
    if(!canlock(&q->lock))
        return 0;
    if(!q->locked){
        q->locked = 1;
        unlock(&q->lock);
        return 1;
    }
    unlock(&q->lock);
    return 0;
}
```

15.5 Read-write locks

⟨*type* RWLock 62c⟩≡ (194b)

```
struct RWLock {
    Lock    lock;
    int readers;    /* number of readers */
    int writer;     /* number of writers */

    QLp *head;      /* list of waiting processes */
    QLp *tail;
};
```

⟨*function* rlock 62d⟩≡ (331e)

```
void
rlock(RWLock *q)
{
    QLp *p, *mp;
```

```
    lock(&q->lock);
    if(q->writer == 0 && q->head == nil){
        /* no writer, go for it */
        q->readers++;
        unlock(&q->lock);
        return;
    }

    mp = getqlp();
    p = q->tail;
    if(p == 0)
        q->head = mp;
    else
        p->next = mp;
    q->tail = mp;
    mp->next = nil;
    mp->state = QueuingR;
    unlock(&q->lock);

    /* wait in kernel */
    while((*_rendezvousp)(mp, (void*)1) == (void*)~0)
        ;
    mp->inuse = 0;
}
```

⟨*function* canrlock 63a⟩≡ (331e)

```
int
canrlock(RWLock *q)
{
    lock(&q->lock);
    if (q->writer == 0 && q->head == nil) {
        /* no writer; go for it */
        q->readers++;
        unlock(&q->lock);
        return 1;
    }
    unlock(&q->lock);
    return 0;
}
```

⟨*function* runlock 63b⟩≡ (331e)

```
void
runlock(RWLock *q)
{
    QLp *p;

    lock(&q->lock);
    if(q->readers <= 0)
        abort();
    p = q->head;
    if(--(q->readers) > 0 || p == nil){
        unlock(&q->lock);
        return;
    }

    /* start waiting writer */
    if(p->state != QueuingW)
        abort();
    q->head = p->next;
    if(q->head == 0)
```

```
        q->tail = 0;
    q->writer = 1;
    unlock(&q->lock);

    /* wakeup waiter */
    while((*_rendezvousp)(p, 0) == (void*)~0)
        ;
}
```

⟨*function* wlock 64a⟩≡ (331e)

```
void
wlock(RWLock *q)
{
    QLp *p, *mp;

    lock(&q->lock);
    if(q->readers == 0 && q->writer == 0){
        /* noone waiting, go for it */
        q->writer = 1;
        unlock(&q->lock);
        return;
    }

    /* wait */
    p = q->tail;
    mp = getqlp();
    if(p == nil)
        q->head = mp;
    else
        p->next = mp;
    q->tail = mp;
    mp->next = nil;
    mp->state = QueuingW;
    unlock(&q->lock);

    /* wait in kernel */
    while((*_rendezvousp)(mp, (void*)1) == (void*)~0)
        ;
    mp->inuse = 0;
}
```

⟨*function* canwlock 64b⟩≡ (331e)

```
int
canwlock(RWLock *q)
{
    lock(&q->lock);
    if (q->readers == 0 && q->writer == 0) {
        /* no one waiting; go for it */
        q->writer = 1;
        unlock(&q->lock);
        return 1;
    }
    unlock(&q->lock);
    return 0;
}
```

⟨*function* wunlock 64c⟩≡ (331e)

```
void
wunlock(RWLock *q)
{
```

```
    QLp *p;

    lock(&q->lock);
    if(q->writer == 0)
        abort();
    p = q->head;
    if(p == nil){
        q->writer = 0;
        unlock(&q->lock);
        return;
    }
    if(p->state == QueuingW){
        /* start waiting writer */
        q->head = p->next;
        if(q->head == nil)
            q->tail = nil;
        unlock(&q->lock);
        while((*_rendezvousp)(p, 0) == (void*)~0)
            ;
        return;
    }

    if(p->state != QueuingR)
        abort();

    /* wake waiting readers */
    while(q->head != nil && q->head->state == QueuingR){
        p = q->head;
        q->head = p->next;
        q->readers++;
        while((*_rendezvousp)(p, 0) == (void*)~0)
            ;
    }
    if(q->head == nil)
        q->tail = nil;
    q->writer = 0;
    unlock(&q->lock);
}
```

15.6 Rendez-vous

⟨*type* Rendez 65a⟩≡ (194b)

```
struct Rendez {
    QLock   *l;

    QLp *head;
    QLp *tail;
};
```

⟨*function* rsleep 65b⟩≡ (331e)

```
void
rsleep(Rendez *r)
{
    QLp *t, *me;

    if(!r->l)
        abort();
    lock(&r->l->lock);
```

```
    /* we should hold the qlock */
    if(!r->l->locked)
        abort();

    /* add ourselves to the wait list */
    me = getqlp();
    me->state = Sleeping;
    if(r->head == nil)
        r->head = me;
    else
        r->tail->next = me;
    me->next = nil;
    r->tail = me;

    /* pass the qlock to the next guy */
    t = r->l->head;
    if(t){
        r->l->head = t->next;
        if(r->l->head == nil)
            r->l->tail = nil;
        unlock(&r->l->lock);
        while((*_rendezvousp)(t, (void*)0x12345) == (void*)~0)
            ;
    }else{
        r->l->locked = 0;
        unlock(&r->l->lock);
    }

    /* wait for a wakeup */
    while((*_rendezvousp)(me, (void*)1) == (void*)~0)
        ;
    me->inuse = 0;
}
```

⟨*function* rwakeup 66⟩≡ (331e)

```
int
rwakeup(Rendez *r)
{
    QLp *t;

    /*
     * take off wait and put on front of queue
     * put on front so guys that have been waiting will not get starved
     */

    if(!r->l)
        abort();
    lock(&r->l->lock);
    if(!r->l->locked)
        abort();

    t = r->head;
    if(t == nil){
        unlock(&r->l->lock);
        return 0;
    }

    r->head = t->next;
    if(r->head == nil)
        r->tail = nil;
```

```
    t->next = r->l->head;
    r->l->head = t;
    if(r->l->tail == nil)
        r->l->tail = t;

    t->state = Queuing;
    unlock(&r->l->lock);
    return 1;
}
```

⟨*function* rwakeupall 67a⟩≡ (331e)

```
int
rwakeupall(Rendez *r)
{
    int i;

    for(i=0; rwakeup(r); i++)
        ;
    return i;
}
```

15.7 Coroutines

⟨*type* jmp_buf 67b⟩≡ (194a)

```
typedef long    jmp_buf[2];
```

⟨*type* jmpbufxxx 67c⟩≡ (194a)

```
#define JMPBUFSP    0
#define JMPBUFPC    1
#define JMPBUFDPC   0
```

Chapter 16

IPC

16.1 Syscalls: pipe(), notify(), await(), segattach()

16.2 Helpers

⟨*function* getfields 68⟩≡ (223e)

```
int
getfields(char *str, char **args, int max, int mflag, char *set)
{
    Rune r;
    int nr, intok, narg;

    if(max <= 0)
        return 0;

    narg = 0;
    args[narg] = str;
    if(!mflag)
        narg++;
    intok = 0;
    for(;; str += nr) {
        nr = chartorune(&r, str);
        if(r == 0)
            break;
        if(utfrune(set, r)) {
            if(narg >= max)
                break;
            *str = 0;
            intok = 0;
            args[narg] = str + nr;
            if(!mflag)
                narg++;
        } else {
            if(!intok && mflag)
                narg++;
            intok = 1;
        }
    }
    return narg;
}
```

16.3 Parent-child IPC

16.3.1 Wait message

⟨*type* Waitmsg 69a⟩≡ (205)

```
/* keep /sys/src/ape/lib/ap/plan9/sys9.h in sync with this -rsc */
struct Waitmsg {
 int pid;  /* of loved one */
 ulong time[3]; /* of loved one & descendants */
 // ref_own?<string>?
 char *msg;
};
```

⟨*function* wait 69b⟩≡ (339c)

```
Waitmsg*
wait(void)
{
    int n, l;
    char buf[512];
    char *fld[5];
    Waitmsg *w;

    n = await(buf, sizeof buf-1);
    if(n < 0)
        return nil;
    buf[n] = '\0';
    if(tokenize(buf, fld, nelem(fld)) != nelem(fld)){
        werrstr("couldn't parse wait message");
        return nil;
    }
    l = strlen(fld[4])+1;
    w = malloc(sizeof(Waitmsg)+l);
    if(w == nil)
        return nil;
    w->pid = atoi(fld[0]);
    w->time[0] = atoi(fld[1]);
    w->time[1] = atoi(fld[2]);
    w->time[2] = atoi(fld[3]);
    w->msg = (char*)&w[1];
    memmove(w->msg, fld[4], l);
    return w;
}
```

⟨*function* waitpid 69c⟩≡ (340a)

```
int
waitpid(void)
{
    int n;
    char buf[512];
    char *fld[5];

    n = await(buf, sizeof buf-1);
    if(n <= 0)
        return -1;
    buf[n] = '\0';
    if(tokenize(buf, fld, nelem(fld)) != nelem(fld)){
        werrstr("couldn't parse wait message");
        return -1;
    }
    return atoi(fld[0]);
}
```

16.3.2 Environment

⟨*function* getenv 70a⟩≡ (323a)

```
char*
getenv(char *name)
{
    int r;
    fdt f;
    long s;
    char *ans;
    char *p, *ep, ename[100];

    if(strchr(name, '/') != nil)
        return nil;
    snprint(ename, sizeof ename, "/env/%s", name);
    if(strcmp(ename+5, name) != 0)
        return nil;
    f = open(ename, OREAD);
    if(f < 0)
        return 0;
    s = seek(f, 0, 2);
    ans = malloc(s+1);
    if(ans) {
        setmalloctag(ans, getcallerpc(&name));
        seek(f, 0, 0);
        r = read(f, ans, s);
        if(r >= 0) {
            ep = ans + s - 1;
            for(p = ans; p < ep; p++)
                if(*p == '\0')
                    *p = ' ';
            ans[s] = '\0';
        }
    }
    close(f);
    return ans;
}
```

⟨*function* putenv 70b⟩≡ (330c)

```
int
putenv(char *name, char *val)
{
    fdt f;
    char ename[100];
    long s;

    if(strchr(name, '/') != nil)
        return -1;
    snprint(ename, sizeof ename, "/env/%s", name);
    if(strcmp(ename+5, name) != 0)
        return -1;
    f = create(ename, OWRITE, 0664);
    if(f < 0)
        return -1;
    s = strlen(val);
    if(write(f, val, s) != s){
        close(f);
        return -1;
    }
    close(f);
```

```
    return 0;
}
```

16.4 Notes

⟨*type* Note_flag 71a⟩≡ (205)

```
#define NCONT 0 /* continue after note */
#define NDFLT 1 /* terminate after note */
#define NSAVE 2 /* clear note but hold state */
#define NRSTR 3 /* restore saved state */
```

16.4.1 Posting

⟨*type* PostnoteKind 71b⟩≡ (194b)

```
enum
{
    PNPROC      = 1,
    PNGROUP     = 2,
};
```

⟨*function* postnote 71c⟩≡ (328c)

```
int
postnote(int group, int pid, char *note)
{
    char file[128];
    fdt f;
    int r;

    switch(group) {
    case PNPROC:
        sprint(file, "/proc/%d/note", pid);
        break;
    case PNGROUP:
        sprint(file, "/proc/%d/notepg", pid);
        break;
    default:
        return -1;
    }

    f = open(file, OWRITE);
    if(f < 0)
        return -1;

    r = strlen(note);
    if(write(f, note, r) != r) {
        close(f);
        return -1;
    }
    close(f);
    return 0;
}
```

16.4.2 Notified

⟨*function* atnotify 71d⟩≡ (215c)

```
int
```

```
atnotify(int (*f)(void*, char*), int in)
{
    int i, n, ret;
    static bool init;

    if(!init){
        notify(notifier);
        init = true;       /* assign = */
    }
    ret = 0;
    lock(&onnotlock);
    if(in){
        for(i=0; i<NFN; i++)
            if(onnot[i] == 0) {
                onnot[i] = f;
                ret = 1;
                break;
            }
    }else{
        n = 0;
        for(i=0; i<NFN; i++)
            if(onnot[i]){
                if(ret==0 && onnot[i]==f){
                    onnot[i] = nil;
                    ret = 1;
                }else
                    n++;
            }
        if(n == 0){
            init = false;
            notify(0);
        }
    }
    unlock(&onnotlock);
    return ret;
}
```

⟨*constant NFN (port/atnotify.c)* 72a⟩≡ (215c)

```
#define NFN 33
```

⟨*global* onnot 72b⟩≡ (215c)

```
static  int (*onnot[NFN])(void*, char*);
```

⟨*global* onnotlock 72c⟩≡ (215c)

```
static  Lock    onnotlock;
```

⟨*function* notifier 72d⟩≡ (215c)

```
static
void
notifier(void *v, char *s)
{
    int i;

    for(i=0; i<NFN; i++)
        if(onnot[i] && ((*onnot[i])(v, s))){
            noted(NCONT);
            return;
        }
    noted(NDFLT);
}
```

16.5 Pipes

16.6 Shared segments

⟨*type* Segattach_flag 73a⟩≡ (205)

```
/* Segattch */
#define SG_RONLY 0040 /* read only */
#define SG_CEXEC 0100 /* detach on exec */
```

16.7 Fileservers

16.7.1 Fcall

⟨*type* Fcall 73b⟩≡ (211)

```
struct  Fcall
{
    // enum<FcallType>
    uchar   type;

    u32int  fid;
    ushort  tag;

    union {
        struct {
            u32int  msize;      /* Tversion, Rversion */
            char    *version;   /* Tversion, Rversion */
        };
        struct {
            ushort  oldtag;     /* Tflush */
        };
        struct {
            char    *ename;     /* Rerror */
        };
        struct {
            Qid qid;             /* Rattach, Ropen, Rcreate */
            u32int  iounit;     /* Ropen, Rcreate */
        };
        struct {
            Qid aqid;       /* Rauth */
        };
        struct {
            u32int  afid;       /* Tauth, Tattach */
            char    *uname;     /* Tauth, Tattach */
            char    *aname;     /* Tauth, Tattach */
        };
        struct {
            u32int  perm;       /* Tcreate */
            char    *name;      /* Tcreate */
            uchar   mode;       /* Tcreate, Topen */
        };
        struct {
            u32int  newfid;     /* Twalk */
            ushort  nwname;     /* Twalk */
            char    *wname[MAXWELEM];   /* Twalk */
        };
        struct {
            ushort  nwqid;      /* Rwalk */
```

```
            Qid wqid[MAXWELEM];     /* Rwalk */
        };
        struct {
            vlong   offset;     /* Tread, Twrite */
            u32int  count;      /* Tread, Twrite, Rread */
            char    *data;      /* Twrite, Rread */
        };
        struct {
            ushort  nstat;      /* Twstat, Rstat */
            uchar   *stat;      /* Twstat, Rstat */
        };
    };
};
```

⟨*type* FcallType 74a⟩≡ (211)

```
enum FcallType
{
    Tversion =  100,
    Rversion,
    Tauth =     102,
    Rauth,
    Tattach =   104,
    Rattach,
    Terror =    106,    /* illegal */
    Rerror,
    Tflush =    108,
    Rflush,
    Twalk =     110,
    Rwalk,
    Topen =     112,
    Ropen,
    Tcreate =   114,
    Rcreate,
    Tread =     116,
    Rread,
    Twrite =    118,
    Rwrite,
    Tclunk =    120,
    Rclunk,
    Tremove =   122,
    Rremove,
    Tstat =     124,
    Rstat,
    Twstat =    126,
    Rwstat,

    Tmax,
};
```

⟨*constant* MAXWELEM 74b⟩≡ (211)

```
#define MAXWELEM    16
```

16.7.2 Dumping

⟨*constant* QIDFMT 74c⟩≡ (322a)

```
#define QIDFMT  "(%.16llux %lud %s)"
```

⟨*function* fcallfmt 75⟩≡ (322a)

```
int
fcallfmt(Fmt *fmt)
{
    Fcall *f;
    int fid, type, tag, i;
    char buf[512], tmp[200];
    char *p, *e;
    Dir *d;
    Qid *q;

    e = buf+sizeof(buf);
    f = va_arg(fmt->args, Fcall*);

    type = f->type;
    fid = f->fid;
    tag = f->tag;

    switch(type){
    case Tversion:  /* 100 */
        seprint(buf, e, "Tversion tag %ud msize %ud version '%s'", tag, f->msize, f->version);
        break;
    case Rversion:
        seprint(buf, e, "Rversion tag %ud msize %ud version '%s'", tag, f->msize, f->version);
        break;
    case Tauth: /* 102 */
        seprint(buf, e, "Tauth tag %ud afid %d uname %s aname %s", tag,
            f->afid, f->uname, f->aname);
        break;
    case Rauth:
        seprint(buf, e, "Rauth tag %ud qid " QIDFMT, tag,
            f->aqid.path, f->aqid.vers, qidtype(tmp, f->aqid.type));
        break;
    case Tattach:   /* 104 */
        seprint(buf, e, "Tattach tag %ud fid %d afid %d uname %s aname %s", tag,
            fid, f->afid, f->uname, f->aname);
        break;
    case Rattach:
        seprint(buf, e, "Rattach tag %ud qid " QIDFMT, tag,
            f->qid.path, f->qid.vers, qidtype(tmp, f->qid.type));
        break;
    case Rerror:    /* 107; 106 (Terror) illegal */
        seprint(buf, e, "Rerror tag %ud ename %s", tag, f->ename);
        break;
    case Tflush:    /* 108 */
        seprint(buf, e, "Tflush tag %ud oldtag %ud", tag, f->oldtag);
        break;
    case Rflush:
        seprint(buf, e, "Rflush tag %ud", tag);
        break;
    case Twalk: /* 110 */
        p = seprint(buf, e, "Twalk tag %ud fid %d newfid %d nwname %d ", tag, fid, f->newfid, f->nwname);
        if(f->nwname <= MAXWELEM)
            for(i=0; i<f->nwname; i++)
                p = seprint(p, e, "%d:%s ", i, f->wname[i]);
        break;
    case Rwalk:
        p = seprint(buf, e, "Rwalk tag %ud nwqid %ud ", tag, f->nwqid);
        if(f->nwqid <= MAXWELEM)
            for(i=0; i<f->nwqid; i++){
```

```
            q = &f->wqid[i];
            p = seprint(p, e, "%d:" QIDFMT " ", i,
                q->path, q->vers, qidtype(tmp, q->type));
        }
    break;
case Topen: /* 112 */
    seprint(buf, e, "Topen tag %ud fid %ud mode %d", tag, fid, f->mode);
    break;
case Ropen:
    seprint(buf, e, "Ropen tag %ud qid " QIDFMT " iounit %ud ", tag,
        f->qid.path, f->qid.vers, qidtype(tmp, f->qid.type), f->iounit);
    break;
case Tcreate:   /* 114 */
    seprint(buf, e, "Tcreate tag %ud fid %ud name %s perm %M mode %d", tag, fid, f->name, (ulong)f->perm, f
    break;
case Rcreate:
    seprint(buf, e, "Rcreate tag %ud qid " QIDFMT " iounit %ud ", tag,
        f->qid.path, f->qid.vers, qidtype(tmp, f->qid.type), f->iounit);
    break;
case Tread: /* 116 */
    seprint(buf, e, "Tread tag %ud fid %d offset %lld count %ud",
        tag, fid, f->offset, f->count);
    break;
case Rread:
    p = seprint(buf, e, "Rread tag %ud count %ud ", tag, f->count);
        dumpsome(p, e, f->data, f->count);
    break;
case Twrite:    /* 118 */
    p = seprint(buf, e, "Twrite tag %ud fid %d offset %lld count %ud ",
        tag, fid, f->offset, f->count);
    dumpsome(p, e, f->data, f->count);
    break;
case Rwrite:
    seprint(buf, e, "Rwrite tag %ud count %ud", tag, f->count);
    break;
case Tclunk:    /* 120 */
    seprint(buf, e, "Tclunk tag %ud fid %ud", tag, fid);
    break;
case Rclunk:
    seprint(buf, e, "Rclunk tag %ud", tag);
    break;
case Tremove:   /* 122 */
    seprint(buf, e, "Tremove tag %ud fid %ud", tag, fid);
    break;
case Rremove:
    seprint(buf, e, "Rremove tag %ud", tag);
    break;
case Tstat: /* 124 */
    seprint(buf, e, "Tstat tag %ud fid %ud", tag, fid);
    break;
case Rstat:
    p = seprint(buf, e, "Rstat tag %ud ", tag);
    if(f->nstat > sizeof tmp)
        seprint(p, e, " stat(%d bytes)", f->nstat);
    else{
        d = (Dir*)tmp;
        convM2D(f->stat, f->nstat, d, (char*)(d+1));
        seprint(p, e, " stat ");
        fdirconv(p+6, e, d);
    }
```

```
        break;
    case Twstat:    /* 126 */
        p = seprint(buf, e, "Twstat tag %ud fid %ud", tag, fid);
        if(f->nstat > sizeof tmp)
            seprint(p, e, " stat(%d bytes)", f->nstat);
        else{
            d = (Dir*)tmp;
            convM2D(f->stat, f->nstat, d, (char*)(d+1));
            seprint(p, e, " stat ");
            fdirconv(p+6, e, d);
        }
        break;
    case Rwstat:
        seprint(buf, e, "Rwstat tag %ud", tag);
        break;
    default:
        seprint(buf, e,  "unknown type %d", type);
    }
    return fmtstrcpy(fmt, buf);
}
```

⟨*function* qidtype 77a⟩≡ (322a)

```
static char*
qidtype(char *s, uchar t)
{
    char *p;

    p = s;
    if(t & QTDIR)
        *p++ = 'd';

    if(t & QTAPPEND)
        *p++ = 'a';
    if(t & QTEXCL)
        *p++ = 'l';
    if(t & QTAUTH)
        *p++ = 'A';

    *p = '\0';
    return s;
}
```

16.7.3 Marshalling macros

⟨*macros GBITxxx* 77b⟩≡ (211)

```
#define GBIT8(p)    ((p)[0])
#define GBIT16(p)   ((p)[0]|((p)[1]<<8))
#define GBIT32(p)   ((p)[0]|((p)[1]<<8)|((p)[2]<<16)|((p)[3]<<24))
#define GBIT64(p)   ((u32int)((p)[0]|((p)[1]<<8)|((p)[2]<<16)|((p)[3]<<24)) |\
                ((vlong)((p)[4]|((p)[5]<<8)|((p)[6]<<16)|((p)[7]<<24)) << 32))
```

⟨*macros PBITxxx* 77c⟩≡ (211)

```
#define PBIT8(p,v)  (p)[0]=(v)
#define PBIT16(p,v) do{(p)[0]=(v);(p)[1]=(v)>>8;}while(0)
#define PBIT32(p,v) do{(p)[0]=(v);(p)[1]=(v)>>8;(p)[2]=(v)>>16;(p)[3]=(v)>>24;}while(0)
#define PBIT64(p,v) do{(p)[0]=(v);(p)[1]=(v)>>8;(p)[2]=(v)>>16;(p)[3]=(v)>>24;\
            (p)[4]=(v)>>32;(p)[5]=(v)>>40;(p)[6]=(v)>>48;(p)[7]=(v)>>56;}while(0)
```

⟨*macros BITxxx* 78a⟩≡ (211)

```
#define BIT8SZ      1
#define BIT16SZ     2
#define BIT32SZ     4
#define BIT64SZ     8
```

⟨*constant* IOHDRSZ 78b⟩≡ (211)

```
#define IOHDRSZ     24  /* ample room for Twrite/Rread header (iounit) */
```

16.7.4 readp9msg()

⟨*function* read9pmsg 78c⟩≡ (332b)

```
error0
read9pmsg(fdt fd, void *abuf, uint n)
{
    int m, len;
    uchar *buf;

    buf = abuf;

    /* read count */
    m = readn(fd, buf, BIT32SZ);
    if(m != BIT32SZ){
        if(m < 0)
            return ERROR_NEG1;
        return ERROR_0;
    }

    len = GBIT32(buf);
    if(len <= BIT32SZ || len > n){
        werrstr("bad length in 9P2000 message header");
        return ERROR_NEG1;
    }
    len -= BIT32SZ;
    m = readn(fd, buf+BIT32SZ, len);
    if(m < len)
        return ERROR_0;
    return BIT32SZ+m;
}
```

16.7.5 Parsing

String to message, convM2S()

⟨*function* convM2S 78d⟩≡ (302d)

```
/*
 * no syntactic checks.
 * three causes for error:
 *  1. message size field is incorrect
 *  2. input buffer too short for its own data (counts too long, etc.)
 *  3. too many names or qids
 * gqid() and gstring() return nil if they would reach beyond buffer.
 * main switch statement checks range and also can fall through
 * to test at end of routine.
 */
error0
convM2S(uchar *ap, uint nap, Fcall *f)
{
```

```
uchar *p, *ep;
uint i, size;

p = ap;
ep = p + nap;

if(p+BIT32SZ+BIT8SZ+BIT16SZ > ep)
    return ERROR_0;
// redo work done in read9pmsg
size = GBIT32(p);
p += BIT32SZ;

if(size < BIT32SZ+BIT8SZ+BIT16SZ)
    return ERROR_0;

f->type = GBIT8(p);
p += BIT8SZ;
f->tag = GBIT16(p);
p += BIT16SZ;

switch(f->type)
{
case Tversion:
    if(p+BIT32SZ > ep)
        return ERROR_0;
    f->msize = GBIT32(p);
    p += BIT32SZ;
    p = gstring(p, ep, &f->version);
    break;

case Tflush:
    if(p+BIT16SZ > ep)
        return ERROR_0;
    f->oldtag = GBIT16(p);
    p += BIT16SZ;
    break;

case Tauth:
    if(p+BIT32SZ > ep)
        return ERROR_0;
    f->afid = GBIT32(p);
    p += BIT32SZ;
    p = gstring(p, ep, &f->uname);
    if(p == nil)
        break;
    p = gstring(p, ep, &f->aname);
    if(p == nil)
        break;
    break;

case Tattach:
    if(p+BIT32SZ > ep)
        return ERROR_0;
    f->fid = GBIT32(p);
    p += BIT32SZ;
    if(p+BIT32SZ > ep)
        return ERROR_0;
    f->afid = GBIT32(p);
    p += BIT32SZ;
    p = gstring(p, ep, &f->uname);
```

```
    if(p == nil)
        break;
    p = gstring(p, ep, &f->aname);
    if(p == nil)
        break;
    break;

case Twalk:
    if(p+BIT32SZ+BIT32SZ+BIT16SZ > ep)
        return ERROR_0;
    f->fid = GBIT32(p);
    p += BIT32SZ;
    f->newfid = GBIT32(p);
    p += BIT32SZ;
    f->nwname = GBIT16(p);
    p += BIT16SZ;
    if(f->nwname > MAXWELEM)
        return ERROR_0;
    for(i=0; i<f->nwname; i++){
        p = gstring(p, ep, &f->wname[i]);
        if(p == nil)
            break;
    }
    break;

case Topen:
    if(p+BIT32SZ+BIT8SZ > ep)
        return ERROR_0;
    f->fid = GBIT32(p);
    p += BIT32SZ;
    f->mode = GBIT8(p);
    p += BIT8SZ;
    break;

case Tcreate:
    if(p+BIT32SZ > ep)
        return ERROR_0;
    f->fid = GBIT32(p);
    p += BIT32SZ;
    p = gstring(p, ep, &f->name);
    if(p == nil)
        break;
    if(p+BIT32SZ+BIT8SZ > ep)
        return ERROR_0;
    f->perm = GBIT32(p);
    p += BIT32SZ;
    f->mode = GBIT8(p);
    p += BIT8SZ;
    break;

case Tread:
    if(p+BIT32SZ+BIT64SZ+BIT32SZ > ep)
        return ERROR_0;
    f->fid = GBIT32(p);
    p += BIT32SZ;
    f->offset = GBIT64(p);
    p += BIT64SZ;
    f->count = GBIT32(p);
    p += BIT32SZ;
    break;
```

```
    case Twrite:
        if(p+BIT32SZ+BIT64SZ+BIT32SZ > ep)
            return ERROR_0;
        f->fid = GBIT32(p);
        p += BIT32SZ;
        f->offset = GBIT64(p);
        p += BIT64SZ;
        f->count = GBIT32(p);
        p += BIT32SZ;
        if(p+f->count > ep)
            return ERROR_0;
        f->data = (char*)p;
        p += f->count;
        break;

    case Tclunk:
    case Tremove:
        if(p+BIT32SZ > ep)
            return ERROR_0;
        f->fid = GBIT32(p);
        p += BIT32SZ;
        break;

    case Tstat:
        if(p+BIT32SZ > ep)
            return ERROR_0;
        f->fid = GBIT32(p);
        p += BIT32SZ;
        break;

    case Twstat:
        if(p+BIT32SZ+BIT16SZ > ep)
            return ERROR_0;
        f->fid = GBIT32(p);
        p += BIT32SZ;
        f->nstat = GBIT16(p);
        p += BIT16SZ;
        if(p+f->nstat > ep)
            return ERROR_0;
        f->stat = p;
        p += f->nstat;
        break;

/*
 */
    case Rversion:
        if(p+BIT32SZ > ep)
            return ERROR_0;
        f->msize = GBIT32(p);
        p += BIT32SZ;
        p = gstring(p, ep, &f->version);
        break;

    case Rerror:
        p = gstring(p, ep, &f->ename);
        break;

    case Rflush:
        break;
```

```
case Rauth:
    p = gqid(p, ep, &f->aqid);
    if(p == nil)
        break;
    break;

case Rattach:
    p = gqid(p, ep, &f->qid);
    if(p == nil)
        break;
    break;

case Rwalk:
    if(p+BIT16SZ > ep)
        return ERROR_0;
    f->nwqid = GBIT16(p);
    p += BIT16SZ;
    if(f->nwqid > MAXWELEM)
        return ERROR_0;
    for(i=0; i<f->nwqid; i++){
        p = gqid(p, ep, &f->wqid[i]);
        if(p == nil)
            break;
    }
    break;

case Ropen:
case Rcreate:
    p = gqid(p, ep, &f->qid);
    if(p == nil)
        break;
    if(p+BIT32SZ > ep)
        return ERROR_0;
    f->iounit = GBIT32(p);
    p += BIT32SZ;
    break;

case Rread:
    if(p+BIT32SZ > ep)
        return ERROR_0;
    f->count = GBIT32(p);
    p += BIT32SZ;
    if(p+f->count > ep)
        return ERROR_0;
    f->data = (char*)p;
    p += f->count;
    break;

case Rwrite:
    if(p+BIT32SZ > ep)
        return ERROR_0;
    f->count = GBIT32(p);
    p += BIT32SZ;
    break;

case Rclunk:
case Rremove:
    break;
```

```
    case Rstat:
        if(p+BIT16SZ > ep)
            return ERROR_0;
        f->nstat = GBIT16(p);
        p += BIT16SZ;
        if(p+f->nstat > ep)
            return ERROR_0;
        f->stat = p;
        p += f->nstat;
        break;

    case Rwstat:
        break;
    default:
        return ERROR_0;
    }

    if(p==nil || p>ep)
        return ERROR_0;
    if(ap+size == p)
        return size;
    return ERROR_0;
}
```

⟨*function* gstring 83a⟩≡ (302d)

```
static
uchar*
gstring(uchar *p, uchar *ep, char **s)
{
    uint n;

    if(p+BIT16SZ > ep)
        return nil;
    n = GBIT16(p);
    p += BIT16SZ - 1;
    if(p+n+1 > ep)
        return nil;
    /* move it down, on top of count, to make room for '\0' */
    memmove(p, p + 1, n);
    p[n] = '\0';
    *s = (char*)p;
    p += n+1;
    return p;
}
```

⟨*function* gqid 83b⟩≡ (302d)

```
static
uchar*
gqid(uchar *p, uchar *ep, Qid *q)
{
    if(p+QIDSZ > ep)
        return nil;
    q->type = GBIT8(p);
    p += BIT8SZ;
    q->vers = GBIT32(p);
    p += BIT32SZ;
    q->path = GBIT64(p);
    p += BIT64SZ;
    return p;
}
```

Message to string, convS2M()

⟨*function* convS2M 84⟩≡ (303a)

```
error0
convS2M(Fcall *f, uchar *ap, uint nap)
{
    uchar *p;
    uint i, size;

    size = sizeS2M(f);
    if(size == 0)
        return ERROR_0;
    if(size > nap)
        return ERROR_0;

    p = (uchar*)ap;

    PBIT32(p, size);
    p += BIT32SZ;

    PBIT8(p, f->type);
    p += BIT8SZ;
    PBIT16(p, f->tag);
    p += BIT16SZ;

    switch(f->type)
    {
    case Tversion:
        PBIT32(p, f->msize);
        p += BIT32SZ;
        p = pstring(p, f->version);
        break;

    case Tflush:
        PBIT16(p, f->oldtag);
        p += BIT16SZ;
        break;

    case Tauth:
        PBIT32(p, f->afid);
        p += BIT32SZ;
        p  = pstring(p, f->uname);
        p  = pstring(p, f->aname);
        break;

    case Tattach:
        PBIT32(p, f->fid);
        p += BIT32SZ;
        PBIT32(p, f->afid);
        p += BIT32SZ;
        p  = pstring(p, f->uname);
        p  = pstring(p, f->aname);
        break;

    case Twalk:
        PBIT32(p, f->fid);
        p += BIT32SZ;
        PBIT32(p, f->newfid);
        p += BIT32SZ;
        PBIT16(p, f->nwname);
```

```
    p += BIT16SZ;
    if(f->nwname > MAXWELEM)
        return ERROR_0;
    for(i=0; i<f->nwname; i++)
        p = pstring(p, f->wname[i]);
    break;

case Topen:
    PBIT32(p, f->fid);
    p += BIT32SZ;
    PBIT8(p, f->mode);
    p += BIT8SZ;
    break;

case Tcreate:
    PBIT32(p, f->fid);
    p += BIT32SZ;
    p = pstring(p, f->name);
    PBIT32(p, f->perm);
    p += BIT32SZ;
    PBIT8(p, f->mode);
    p += BIT8SZ;
    break;

case Tread:
    PBIT32(p, f->fid);
    p += BIT32SZ;
    PBIT64(p, f->offset);
    p += BIT64SZ;
    PBIT32(p, f->count);
    p += BIT32SZ;
    break;

case Twrite:
    PBIT32(p, f->fid);
    p += BIT32SZ;
    PBIT64(p, f->offset);
    p += BIT64SZ;
    PBIT32(p, f->count);
    p += BIT32SZ;
    memmove(p, f->data, f->count);
    p += f->count;
    break;

case Tclunk:
case Tremove:
    PBIT32(p, f->fid);
    p += BIT32SZ;
    break;

case Tstat:
    PBIT32(p, f->fid);
    p += BIT32SZ;
    break;

case Twstat:
    PBIT32(p, f->fid);
    p += BIT32SZ;
    PBIT16(p, f->nstat);
    p += BIT16SZ;
```

```
        memmove(p, f->stat, f->nstat);
        p += f->nstat;
        break;
/*
 */

    case Rversion:
        PBIT32(p, f->msize);
        p += BIT32SZ;
        p = pstring(p, f->version);
        break;

    case Rerror:
        p = pstring(p, f->ename);
        break;

    case Rflush:
        break;

    case Rauth:
        p = pqid(p, &f->aqid);
        break;

    case Rattach:
        p = pqid(p, &f->qid);
        break;

    case Rwalk:
        PBIT16(p, f->nwqid);
        p += BIT16SZ;
        if(f->nwqid > MAXWELEM)
            return ERROR_0;
        for(i=0; i<f->nwqid; i++)
            p = pqid(p, &f->wqid[i]);
        break;

    case Ropen:
    case Rcreate:
        p = pqid(p, &f->qid);
        PBIT32(p, f->iounit);
        p += BIT32SZ;
        break;

    case Rread:
        PBIT32(p, f->count);
        p += BIT32SZ;
        memmove(p, f->data, f->count);
        p += f->count;
        break;

    case Rwrite:
        PBIT32(p, f->count);
        p += BIT32SZ;
        break;

    case Rclunk:
        break;

    case Rremove:
        break;
```

```
    case Rstat:
        PBIT16(p, f->nstat);
        p += BIT16SZ;
        memmove(p, f->stat, f->nstat);
        p += f->nstat;
        break;

    case Rwstat:
        break;
    default:
        return ERROR_0;

    }
    if(size != p-ap)
        return ERROR_0;
    return size;
}
```

⟨*function* pstring 87a⟩≡ (303a)

```
static
uchar*
pstring(uchar *p, char *s)
{
    uint n;

    if(s == nil){
        PBIT16(p, 0);
        p += BIT16SZ;
        return p;
    }

    n = strlen(s);
    /*
     * We are moving the string before the length,
     * so you can S2M a struct into an existing message
     */
    memmove(p + BIT16SZ, s, n);
    PBIT16(p, n);
    p += n + BIT16SZ;
    return p;
}
```

⟨*function* pqid 87b⟩≡ (303a)

```
static
uchar*
pqid(uchar *p, Qid *q)
{
    PBIT8(p, q->type);
    p += BIT8SZ;
    PBIT32(p, q->vers);
    p += BIT32SZ;
    PBIT64(p, q->path);
    p += BIT64SZ;
    return p;
}
```

⟨*function* sizeS2M 87c⟩≡ (303a)

```
uint
sizeS2M(Fcall *f)
```

```
{
    uint n;
    int i;

    n = 0;
    n += BIT32SZ;   /* size */
    n += BIT8SZ;    /* type */
    n += BIT16SZ;   /* tag */

    switch(f->type)
    {
    case Tversion:
        n += BIT32SZ;
        n += stringsz(f->version);
        break;

    case Tflush:
        n += BIT16SZ;
        break;

    case Tauth:
        n += BIT32SZ;
        n += stringsz(f->uname);
        n += stringsz(f->aname);
        break;

    case Tattach:
        n += BIT32SZ;
        n += BIT32SZ;
        n += stringsz(f->uname);
        n += stringsz(f->aname);
        break;

    case Twalk:
        n += BIT32SZ;
        n += BIT32SZ;
        n += BIT16SZ;
        for(i=0; i<f->nwname; i++)
            n += stringsz(f->wname[i]);
        break;

    case Topen:
        n += BIT32SZ;
        n += BIT8SZ;
        break;

    case Tcreate:
        n += BIT32SZ;
        n += stringsz(f->name);
        n += BIT32SZ;
        n += BIT8SZ;
        break;

    case Tread:
        n += BIT32SZ;
        n += BIT64SZ;
        n += BIT32SZ;
        break;

    case Twrite:
```

```
        n += BIT32SZ;
        n += BIT64SZ;
        n += BIT32SZ;
        n += f->count;
        break;

    case Tclunk:
    case Tremove:
        n += BIT32SZ;
        break;

    case Tstat:
        n += BIT32SZ;
        break;

    case Twstat:
        n += BIT32SZ;
        n += BIT16SZ;
        n += f->nstat;
        break;
/*
 */

    case Rversion:
        n += BIT32SZ;
        n += stringsz(f->version);
        break;

    case Rerror:
        n += stringsz(f->ename);
        break;

    case Rflush:
        break;

    case Rauth:
        n += QIDSZ;
        break;

    case Rattach:
        n += QIDSZ;
        break;

    case Rwalk:
        n += BIT16SZ;
        n += f->nwqid*QIDSZ;
        break;

    case Ropen:
    case Rcreate:
        n += QIDSZ;
        n += BIT32SZ;
        break;

    case Rread:
        n += BIT32SZ;
        n += f->count;
        break;

    case Rwrite:
```

```
        n += BIT32SZ;
        break;

    case Rclunk:
        break;

    case Rremove:
        break;

    case Rstat:
        n += BIT16SZ;
        n += f->nstat;
        break;

    case Rwstat:
        break;
    default:
        return 0;

    }
    return n;
}
```

⟨*function* stringsz 90a⟩≡ (303a)

```
static
uint
stringsz(char *s)
{
    if(s == nil)
        return BIT16SZ;

    return BIT16SZ+strlen(s);
}
```

String to directory entry, convM2D()

⟨*constant* QIDSZ 90b⟩≡ (211)

```
#define QIDSZ   (BIT8SZ+BIT32SZ+BIT64SZ)
```

⟨*constant* STATFIXLEN 90c⟩≡ (211)

```
/* STATFIXLEN includes leading 16-bit count */
/* The count, however, excludes itself; total size is BIT16SZ+count */
#define STATFIXLEN  (BIT16SZ+QIDSZ+5*BIT16SZ+4*BIT32SZ+1*BIT64SZ)   /* amount of fixed length data in a stat bu
```

⟨*function* convM2D 90d⟩≡ (302c)

```
uint
convM2D(uchar *buf, uint nbuf, Dir *d, char *strs)
{
    uchar *p, *ebuf;
    char *sv[4];
    int i, ns;

    if(nbuf < STATFIXLEN)
        return 0;

    p = buf;
    ebuf = buf + nbuf;

    p += BIT16SZ;   /* ignore size */
```

```
    d->type = GBIT16(p);
    p += BIT16SZ;
    d->dev = GBIT32(p);
    p += BIT32SZ;
    d->qid.type = GBIT8(p);
    p += BIT8SZ;
    d->qid.vers = GBIT32(p);
    p += BIT32SZ;
    d->qid.path = GBIT64(p);
    p += BIT64SZ;
    d->mode = GBIT32(p);
    p += BIT32SZ;
    d->atime = GBIT32(p);
    p += BIT32SZ;
    d->mtime = GBIT32(p);
    p += BIT32SZ;
    d->length = GBIT64(p);
    p += BIT64SZ;

    for(i = 0; i < 4; i++){
        if(p + BIT16SZ > ebuf)
            return 0;
        ns = GBIT16(p);
        p += BIT16SZ;
        if(p + ns > ebuf)
            return 0;
        if(strs){
            sv[i] = strs;
            memmove(strs, p, ns);
            strs += ns;
            *strs++ = '\0';
        }
        p += ns;
    }

    if(strs){
        d->name = sv[0];
        d->uid = sv[1];
        d->gid = sv[2];
        d->muid = sv[3];
    }else{
        d->name = nullstring;
        d->uid = nullstring;
        d->gid = nullstring;
        d->muid = nullstring;
    }

    return p - buf;
}
```

Directory entry to string, convD2M()

⟨*function* convD2M 91⟩≡ (301c)

```
uint
convD2M(Dir *d, uchar *buf, uint nbuf)
{
    uchar *p, *ebuf;
    char *sv[4];
    int i, ns, nsv[4], ss;
```

```
if(nbuf < BIT16SZ)
    return 0;

p = buf;
ebuf = buf + nbuf;

sv[0] = d->name;
sv[1] = d->uid;
sv[2] = d->gid;
sv[3] = d->muid;

ns = 0;
for(i = 0; i < 4; i++){
    if(sv[i])
        nsv[i] = strlen(sv[i]);
    else
        nsv[i] = 0;
    ns += nsv[i];
}

ss = STATFIXLEN + ns;

/* set size before erroring, so user can know how much is needed */
/* note that length excludes count field itself */
PBIT16(p, ss-BIT16SZ);
p += BIT16SZ;

if(ss > nbuf)
    return BIT16SZ; // ugly error code

PBIT16(p, d->type);
p += BIT16SZ;
PBIT32(p, d->dev);
p += BIT32SZ;

PBIT8(p, d->qid.type);
p += BIT8SZ;
PBIT32(p, d->qid.vers);
p += BIT32SZ;
PBIT64(p, d->qid.path);
p += BIT64SZ;

PBIT32(p, d->mode);
p += BIT32SZ;
PBIT32(p, d->atime);
p += BIT32SZ;
PBIT32(p, d->mtime);
p += BIT32SZ;

PBIT64(p, d->length);
p += BIT64SZ;

for(i = 0; i < 4; i++){
    ns = nsv[i];
    if(p + ns + BIT16SZ > ebuf)
        return 0;
    PBIT16(p, ns);
    p += BIT16SZ;
    if(ns)
```

```
            memmove(p, sv[i], ns);
        p += ns;
    }

    if(ss != p - buf)
        return 0;

    return p - buf;
}
```

Chapter 17

Channels and Cooperative Threads

17.1 Overview

17.1.1 Code organization

17.1.2 Software architecture

17.2 A toy example

17.3 Core data structures

17.3.1 Concurrency buiding blocks

⟨*struct* Ref 94a⟩≡ (208)

```
struct Ref {
    long ref;
};
```

⟨*function* incref 94b⟩≡ (423b)

```
void
incref(Ref *r)
{
    ainc(&r->ref);
}
```

⟨*function* decref 94c⟩≡ (423b)

```
long
decref(Ref *r)
{
    return adec(&r->ref);
}
```

⟨*global* xincport_lock 94d⟩≡ (425b)

```
static Lock xincport_lock;
```

⟨*function* _xinc 94e⟩≡ (425b)

```
void
_xinc(long *p)
{

    lock(&xincport_lock);
    (*p)++;
    unlock(&xincport_lock);
}
```

⟨*function* _xdec 95a⟩≡ (425b)

```
long
_xdec(long *p)
{
    long r;

    lock(&xincport_lock);
    r = --(*p);
    unlock(&xincport_lock);
    return r;
}
```

17.3.2 Channel

⟨*struct* Channel 95b⟩≡ (208)

```
/*
 * Channel structure.  s is the size of the buffer.  For unbuffered channels
 * s is zero.  v is an array of s values.  If s is zero, v is unused.
 * f and n represent the state of the queue pointed to by v.
 */
struct Channel {
    int s;  /* Size of the channel (may be zero) */

    uint f;  /* Extraction point (insertion pt: (f+n) % s) */
    uint n;  /* Number of values in the channel */
    int e;  /* Element size */
    int freed;  /* Set when channel is being deleted */

    volatile Alt **qentry; /* Receivers/senders waiting (malloc) */
    volatile int nentry; /* # of entries malloc-ed */
    volatile int closed; /* channel is closed */

    // must be at the end of the struct!
    byte v[1];  /* Array of s values in the channel */
};
```

⟨*global* chanlock 95c⟩≡ (429)

```
static Lock chanlock;  /* central channel access lock */
```

⟨*function* chancreate 95d⟩≡ (429)

```
Channel*
chancreate(int elemsize, int elemcnt)
{
    Channel *c;

    if(elemcnt < 0 || elemsize <= 0)
        return nil;
    c = _threadmalloc(sizeof(Channel) + elemsize*elemcnt, 1);
    c->e = elemsize;
    c->s = elemcnt;
    _threaddebug(DBGCHAN, "chancreate %p", c);
    return c;
}
```

⟨*function* chaninit 95e⟩≡ (429)

```
int
chaninit(Channel *c, int elemsize, int elemcnt)
{
    if(elemcnt < 0 || elemsize <= 0 || c == nil)
```

```
        return -1;
    c->f = 0;
    c->n = 0;
    c->closed = 0;
    c->freed = 0;
    c->e = elemsize;
    c->s = elemcnt;
    _threaddebug(DBGCHAN, "chaninit %p", c);
    return 1;
}
```

⟨*function* chanfree 96a⟩≡ (429)

```
void
chanfree(Channel *c)
{
    lock(&chanlock);
    _chanfree(c);
    unlock(&chanlock);
}
```

⟨*function* _chanfree 96b⟩≡ (429)

```
static void
_chanfree(Channel *c)
{
    int i, inuse;

    if(c->closed == 1)   /* chanclose is ongoing */
        inuse = 1;
    else{
        inuse = 0;
        for(i = 0; i < c->nentry; i++) /* alt ongoing */
            if(c->qentry[i])
                inuse = 1;
    }
    if(inuse)
        c->freed = 1;
    else{
        if(c->qentry)
            free(c->qentry);
        free(c);
    }
}
```

⟨*enum* _anon_ (libthread/channel.c) 96c⟩≡ (429)

```
/* Value to indicate the channel is closed */
enum {
    CHANCLOSD = 0xc105ed,
};
```

⟨*constant* Closed 96d⟩≡ (429)

```
#define Closed ((void*)CHANCLOSD)
```

⟨*function* chanclose 96e⟩≡ (429)

```
int
chanclose(Channel *c)
{
    Alt *a;
    int i, s;

    s = _procsplhi(); /* note handlers; see :/^alt */
```

```
    lock(&chanlock);
    if(c->closed){
        /* Already close; we fail but it's ok. don't print */
        unlock(&chanlock);
        _procsplx(s);
        return -1;
    }
    c->closed = 1;  /* Being closed */
    /*
     * Locate entries that will fail due to close
     * (send, and receive if nothing buffered) and wake them up.
     * the situation cannot change because all queries
     * should be committed by now and new ones will find the channel
     * closed.  We still need to take the lock during the iteration
     * because we can wake threads on qentrys we have not seen yet
     * as in alt and there would be a race in the access to *a.
     */
    for(i = 0; i < c->nentry; i++){
        if((a = c->qentry[i]) == nil || *a->tag != nil)
            continue;

        if(a->op != CHANSND && (a->op != CHANRCV || c->n != 0))
            continue;
        *a->tag = c;
        unlock(&chanlock);
        _procsplx(s);
        while(_threadrendezvous(a->tag, Closed) == Intred)
            ;
        s = _procsplhi();
        lock(&chanlock);
    }

    c->closed = 2;  /* Fully closed */
    if(c->freed)
        _chanfree(c);
    unlock(&chanlock);
    _procsplx(s);
    return 0;
}
```

⟨*function* chanclosing 97⟩≡ (429)

```
int
chanclosing(Channel *c)
{
    int n, s;

    s = _procsplhi(); /* note handlers; see :/^alt */
    lock(&chanlock);
    if(c->closed == 0)
        n = -1;
    else
        n = c->n;
    unlock(&chanlock);
    _procsplx(s);
    return n;
}
```

17.3.3 Proc and Pqueue

⟨*struct* Proc 98a⟩≡ (423d)

```
struct Proc
{
    int  pid;   /* process id */

    jmp_buf sched;   /* for context switches */

    Thread *thread;  /* running thread */




    bool  splhi;   /* delay notes */

    int  needexec;
    Execargs exec;   /* exec argument */
    Proc *newproc;  /* fork argument */
    char exitstr[ERRMAX]; /* exit status */

    int  rforkflag;

    Tqueue threads;  /* All threads of this proc */
    int  nthreads;

    Tqueue ready;   /* Runnable threads */
    Lock readylock;

    char printbuf[Printsize];
    int  blocked;  /* In a rendezvous */
    int  pending;  /* delayed note pending */
    int  nonotes;  /* delay notes */
    uint nextID;   /* ID of most recently created thread */


    void *arg;   /* passed between shared and unshared stk */
    char str[ERRMAX];  /* used by threadexits to avoid malloc */

    void* wdata;   /* Lib(worker) per-proc data pointer */
    void* udata;   /* User per-proc data pointer */
    char threadint;  /* tag for threadexitsall() */

    // Extra
    Proc *next;   /* linked list of Procs */

    Lock lock;
};
```

⟨*constant* Printsize 98b⟩≡ (423c)

```
Printsize = 2048,
```

⟨*global* procp 98c⟩≡ (425c)

```
// used to be in main.c
static Proc **procp;
```

⟨*function* _threadgetproc 98d⟩≡ (425c)

```
Proc*
_threadgetproc(void)
{
    return *procp;
}
```

⟨*function* _threadsetproc 99a⟩≡ (425c)

```
void
_threadsetproc(Proc *p)
{
    *procp = p;
}
```

⟨*global* _threadpq 99b⟩≡ (425c)

```
// used to be in create.c
Pqueue _threadpq;
```

⟨*struct* Pqueue 99c⟩≡ (423d)

```
struct Pqueue {  /* Proc queue */
    Proc  *head;
    Proc  **tail;

    // Extra
    Lock  lock;
};
```

⟨*function* proccreate 99d⟩≡ (428c)

```
int
proccreate(void (*f)(void*), void *arg, uint stacksize)
{
    return procrfork(f, arg, stacksize, 0);
}
```

⟨*function* procrfork 99e⟩≡ (428c)

```
int
procrfork(void (*f)(void *), void *arg, uint stacksize, int rforkflag)
{
    Proc *p;
    int id;

    p = _threadgetproc();
    assert(p->newproc == nil);
    p->newproc = _newproc(f, arg, stacksize, nil, p->thread->grp, rforkflag);
    id = p->newproc->threads.head->id;
    _sched();
    return id;
}
```

⟨*function* _newproc 99f⟩≡ (428c)

```
/*
 * Create and initialize a new Proc structure with a single Thread
 * running inside it.  Add the Proc to the global process list.
 */
Proc*
_newproc(void (*f)(void *arg), void *arg, uint stacksize, char *name, int grp, int rforkflag)
{
    Proc *p;

    p = _threadmalloc(sizeof *p, 1);
    p->pid = -1;
    p->rforkflag = rforkflag;
    newthread(p, f, arg, stacksize, name, grp);

    lock(&_threadpq.lock);
    if(_threadpq.head == nil)
        _threadpq.head = p;
```

```
    else
        *_threadpq.tail = p;
    _threadpq.tail = &p->next;
    unlock(&_threadpq.lock);

    return p;
}
```

⟨*function* _freeproc 100a⟩≡ (428c)

```
void
_freeproc(Proc *p)
{
    Thread *t, *nextt;

    for(t = p->threads.head; t; t = nextt){
        if(t->cmdname)
            free(t->cmdname);
        assert(t->stk != nil);
        free(t->stk);
        nextt = t->nextt;
        free(t);
    }
    free(p);
}
```

⟨*struct* Execargs 100b⟩≡ (423d)

```
struct Execargs
{
    char *prog;
    char **args;
    int  fd[2];
};
```

17.3.4 Thread and Tqueue

⟨*struct* Thread 100c⟩≡ (423d)

```
struct Thread
{
    int  id;  /* thread id */
    char *cmdname; /* ptr to name of thread */

    jmp_buf sched;  /* for context switches */

    Proc *proc;  /* proc of this thread */

    uint stksize; /* stack size */
    uchar *stk;  /* top of stack (lowest address of stack) */

    State state;  /* run state */
    State nextstate; /* next run state */

    Chanstate chan;  /* which channel operation is current */
    Alt  *alt;  /* pointer to current alt structure (debugging) */



    int  grp;  /* thread group */
    int  moribund; /* thread needs to die */
```

```
    int  ret;  /* return value for Exec, Fork */


    int  inrendez;
    Thread *rendhash; /* Trgrp linked list */
    void* rendtag; /* rendezvous tag */
    void* rendval; /* rendezvous value */
    int  rendbreak; /* rendezvous has been taken */


    void* udata[NPRIV]; /* User per-thread data pointer */

    // Extra
    Lock lock;  /* protects thread data structure */

    Thread *next;  /* next on ready queue */

    Thread *nextt;  /* next on list of threads in this proc*/


};
```

⟨*constant* NPRIV 101a⟩≡ (423c)

```
NPRIV = 8,
```

⟨*function* threadid 101b⟩≡ (426b)

```
int
threadid(void)
{
    return _threadgetproc()->thread->id;
}
```

⟨*function* nextID 101c⟩≡ (428c)

```
static int
nextID(void)
{
    static Lock l;
    static int id;
    int i;

    lock(&l);
    i = ++id;
    unlock(&l);
    return i;
}
```

⟨*function* threadcreate 101d⟩≡ (428c)

```
/*
 * Create a new thread and schedule it to run.
 * The thread grp is inherited from the currently running thread.
 */
int
threadcreate(void (*f)(void *arg), void *arg, uint stacksize)
{
    return newthread(_threadgetproc(), f, arg, stacksize, nil, threadgetgrp());
}
```

⟨*function* newthread 102a⟩≡ (428c)

```
/*
 * Create and initialize a new Thread structure attached to a given proc.
 */
static int
newthread(Proc *p, void (*f)(void *arg), void *arg, uint stacksize, char *name, int grp)
{
    int id;
    Thread *t;

    if(stacksize < 32)
        sysfatal("bad stacksize %d", stacksize);
    t = _threadmalloc(sizeof(Thread), 1);

    t->stksize = stacksize;
    t->stk = _threadmalloc(stacksize, 0);
    memset(t->stk, 0xFE, stacksize);
    _threadinitstack(t, f, arg);

    t->grp = grp;
    if(name)
        t->cmdname = strdup(name);
    t->id = nextID();
    id = t->id;
    t->next = (Thread*)~0;
    t->proc = p;
    _threaddebug(DBGSCHED, "create thread %d.%d name %s", p->pid, t->id, name);

    lock(&p->lock);
    p->nthreads++;
    if(p->threads.head == nil)
        p->threads.head = t;
    else
        *p->threads.tail = t;
    p->threads.tail = &t->nextt;

    t->nextt = nil;
    t->state = Ready;
    _threadready(t);
    unlock(&p->lock);

    return id;
}
```

⟨*function* launcher386 102b⟩≡ (423a)

```
static void
launcher386(void (*f)(void *arg), void *arg)
{
    (*f)(arg);
    threadexits(nil);
}
```

⟨*function* _threadinitstack 102c⟩≡ (423a)

```
void
_threadinitstack(Thread *t, void (*f)(void*), void *arg)
{
    ulong *tos;

    tos = (ulong*)&t->stk[t->stksize&~7];
    *--tos = (ulong)arg;
```

```
    *--tos = (ulong)f;
    t->sched[JMPBUFPC] = (ulong)launcher386+JMPBUFDPC;
    t->sched[JMPBUFSP] = (ulong)tos - 8;  /* old PC and new PC */
}
```

⟨*function* _freethread 103a⟩≡ (428c)

```
void
_freethread(Thread *t)
{
    Proc *p;
    Thread **l;

    p = t->proc;
    lock(&p->lock);
    for(l=&p->threads.head; *l; l=&(*l)->nextt){
        if(*l == t){
            *l = t->nextt;
            if(*l == nil)
                p->threads.tail = l;
            break;
        }
    }
    unlock(&p->lock);
    if (t->cmdname)
        free(t->cmdname);
    assert(t->stk != nil);
    free(t->stk);
    free(t);
}
```

⟨*function* threadpid 103b⟩≡ (426b)

```
int
threadpid(int id)
{
    int pid;
    Proc *p;
    Thread *t;

    if (id < 0)
        return -1;
    if (id == 0)
        return _threadgetproc()->pid;
    lock(&_threadpq.lock);
    for (p = _threadpq.head; p; p = p->next){
        lock(&p->lock);
        for (t = p->threads.head; t; t = t->nextt)
            if (t->id == id){
                pid = p->pid;
                unlock(&p->lock);
                unlock(&_threadpq.lock);
                return pid;
            }
        unlock(&p->lock);
    }
    unlock(&_threadpq.lock);
    return -1;
}
```

⟨*struct* Tqueue 103c⟩≡ (423d)

```
struct Tqueue  /* Thread queue */
```

```
{
    Thread *head;
    Thread **tail;

    int  asleep;
};
```

⟨*enum* state 104a⟩≡ (423d)

```
enum state
{
    Dead,
    Running,
    Ready,
    Rendezvous,
};
```

⟨*enum* chanstate 104b⟩≡ (423d)

```
enum chanstate
{
    Channone,

    Chansend,
    Chanrecv,

    Chanalt,
};
```

⟨*function* threadsetname 104c⟩≡ (426b)

```
void
threadsetname(char *fmt, ...)
{
    int fd;
    char buf[128];
    va_list arg;
    Proc *p;
    Thread *t;

    p = _threadgetproc();
    t = p->thread;
    if (t->cmdname)
        free(t->cmdname);
    va_start(arg, fmt);
    t->cmdname = vsmprint(fmt, arg);
    va_end(arg);
    if(t->cmdname && p->nthreads == 1){
        snprint(buf, sizeof buf, "#p/%lud/args", _tos->pid); //getpid());
        if((fd = open(buf, OWRITE)) >= 0){
            write(fd, t->cmdname, strlen(t->cmdname)+1);
            close(fd);
        }
    }
}
```

⟨*function* threadgetname 104d⟩≡ (426b)

```
char*
threadgetname(void)
{
    Proc *p;

    if((p = _threadgetproc()) && p->thread)
```

```
        return p->thread->cmdname;
    return nil;
}
```

17.3.5 Alt

⟨*enum* chanop 105a⟩≡ (208)

```
/* Channel operations for alt: */
enum chanop {
    CHANEND,

    CHANSND,
    CHANRCV,
    CHANNOP,

    CHANNOBLK,
};
```

⟨*struct* Alt 105b⟩≡ (208)

```
struct Alt {
    Channel *c;  /* channel */
    void *v;  /* pointer to value */
    ChanOp op;  /* operation */

    char *err;  /* did the op fail? */
    /*
     * the next variables are used internally to alt
     * they need not be initialized
     */
    Channel **tag;  /* pointer to rendez-vous tag */
    int entryno; /* entry number */
};
```

17.4 main()

⟨*global* mainp 105c⟩≡ (430)

```
// ref<ref<Proc>
static Proc **mainp;
```

⟨*global* _mainjmp 105d⟩≡ (430)

```
static jmp_buf _mainjmp;
```

⟨*global* mainstacksize 105e⟩≡ (430)

```
int mainstacksize;
```

⟨*function* main 105f⟩≡ (430)

```
void
main(int argc, char **argv)
{
    Mainarg *a;
    Proc *p;

    rfork(RFREND);
    mainp = &p;

    if(setjmp(_mainjmp))
        _schedinit(p);
```

```
    //_threaddebuglevel = (DBGSCHED|DBGCHAN|DBGREND)^~0;
    _systhreadinit();
    _qlockinit(_threadrendezvous);

    _sysfatal = _threadsysfatal;
    _dial     = _threaddial;
    __assert  = _threadassert;

    notify(_threadnote);

    if(mainstacksize == 0)
        mainstacksize = 8*1024;

    a = _threadmalloc(sizeof *a, 1);
    a->argc = argc;
    a->argv = argv;

    p = _newproc(mainlauncher, a, mainstacksize, "threadmain", 0, 0);
    _schedinit(p);

    abort(); /* not reached */
}
```

⟨*function* _systhreadinit 106a⟩≡ (425c)

```
void
_systhreadinit(void)
{
    procp = privalloc();
}
```

⟨*struct* Mainarg 106b⟩≡ (430)

```
struct Mainarg
{
    int  argc;
    char **argv;
};
```

⟨*function* mainlauncher 106c⟩≡ (430)

```
static void
mainlauncher(void *arg)
{
    Mainarg *a;

    a = arg;

    // user defined threadmain()!!
    threadmain(a->argc, a->argv);

    threadexits("threadmain");
}
```

17.5 Threads scheduler

⟨*global* _psstate 106d⟩≡ (428b)

```
static char *_psstate[] = {
    "Moribund",
    "Dead",
```

```
    "Exec",
    "Fork",
    "Running",
    "Ready",
    "Rendezvous",
};
```

⟨*function* psstate 107a⟩≡ (428b)

```
static char*
psstate(int s)
{
    if(s < 0 || s >= nelem(_psstate))
        return "unknown";
    return _psstate[s];
}
```

⟨*function* _schedinit 107b⟩≡ (428b)

```
void
_schedinit(void *arg)
{
    Proc *p;
    Thread *t, **l;

    p = arg;
    _threadsetproc(p);
    p->pid = _tos->pid; //getpid();

    while(setjmp(p->sched))
        ;

    _threaddebug(DBGSCHED, "top of schedinit, _threadexitsallstatus=%p",
                 _threadexitsallstatus);
    if(_threadexitsallstatus)
        exits(_threadexitsallstatus);

    lock(&p->lock);
    t = p->thread;
    if(t != nil){
        p->thread = nil;

        if(t->moribund){
            t->state = Dead;
            for(l=&p->threads.head; *l; l=&(*l)->nextt)
                if(*l == t){
                    *l = t->nextt;
                    if(*l==nil)
                        p->threads.tail = l;
                    p->nthreads--;
                    break;
                }
            unlock(&p->lock);
            if(t->inrendez){
                _threadflagrendez(t);
                _threadbreakrendez();
            }
            free(t->stk);
            free(t->cmdname);
            free(t); /* XXX how do we know there are no references? */
            t = nil;
            _sched();
```

```
        }
        if(p->needexec){
            t->ret = _schedexec(&p->exec);
            p->needexec = 0;
        }
        if(p->newproc){
            t->ret = _schedfork(p->newproc);
            p->newproc = nil;
        }
        t->state = t->nextstate;
        if(t->state == Ready)
            _threadready(t);
    }
    unlock(&p->lock);
    _sched();
}
```

⟨*function* needstack 108a⟩≡ (428b)

```
void
needstack(int n)
{
    int x;
    Proc *p;
    Thread *t;

    p = _threadgetproc();
    t = p->thread;

    if((uchar*)&x - n < (uchar*)t->stk){
        fprint(2, "%s %lud: &x=%p n=%d t->stk=%p\n",
            argv0, _tos->pid, &x, n, t->stk);
        fprint(2, "%s %lud: stack overflow\n", argv0, _tos->pid);
        abort();
    }
}
```

⟨*function* _sched 108b⟩≡ (428b)

```
void
_sched(void)
{
    Proc *p;
    Thread *t;

Resched:
    p = _threadgetproc();
    t = p->thread;
    if(t != nil){
        needstack(128);
        _threaddebug(DBGSCHED, "pausing, state=%s", psstate(t->state));
        if(setjmp(t->sched)==0)
            longjmp(p->sched, 1);
        return;
    }else{
        t = runthread(p);
        if(t == nil){
            _threaddebug(DBGSCHED, "all threads gone; exiting");
            _schedexit(p);
        }
        _threaddebug(DBGSCHED, "running %d.%d", t->proc->pid, t->id);
        p->thread = t;
```

```
        if(t->moribund){
            _threaddebug(DBGSCHED, "%d.%d marked to die");
            goto Resched;
        }
        t->state = Running;
        t->nextstate = Ready;
        longjmp(t->sched, 1);
    }
}
```

⟨*function* `runthread` 109a⟩≡ (428b)

```
static Thread*
runthread(Proc *p)
{
    Thread *t;
    Tqueue *q;

    if(p->nthreads==0)
        return nil;
    q = &p->ready;
    lock(&p->readylock);
    if(q->head == nil){
        q->asleep = 1;
        _threaddebug(DBGSCHED, "sleeping for more work");
        unlock(&p->readylock);
        while(rendezvous(q, 0) == (void*)~0){
            if(_threadexitsallstatus)
                exits(_threadexitsallstatus);
        }
        /* lock picked up from _threadready */
    }
    t = q->head;
    q->head = t->next;
    unlock(&p->readylock);
    return t;
}
```

⟨*function* `_threadready` 109b⟩≡ (428b)

```
void
_threadready(Thread *t)
{
    Tqueue *q;

    assert(t->state == Ready);
    _threaddebug(DBGSCHED, "readying %d.%d", t->proc->pid, t->id);

    q = &t->proc->ready;
    lock(&t->proc->readylock);
    t->next = nil;
    if(q->head==nil)
        q->head = t;
    else
        *q->tail = t;
    q->tail = &t->next;
    if(q->asleep){
        q->asleep = 0;
        /* lock passes to runthread */
        _threaddebug(DBGSCHED, "waking process %d", t->proc->pid);
        while(rendezvous(q, 0) == (void*)~0){
            if(_threadexitsallstatus)
```

```
                exits(_threadexitsallstatus);
        }
    }else
        unlock(&t->proc->readylock);
}
```

⟨*function* yield 110a⟩≡ (428b)

```
void
yield(void)
{
    _sched();
}
```

⟨*function* _schedexit 110b⟩≡ (430)

```
void
_schedexit(Proc *p)
{
    char ex[ERRMAX];
    Proc **l;

    lock(&_threadpq.lock);
    for(l=&_threadpq.head; *l; l=&(*l)->next){
        if(*l == p){
            *l = p->next;
            if(*l == nil)
                _threadpq.tail = l;
            break;
        }
    }
    unlock(&_threadpq.lock);

    utfecpy(ex, ex+sizeof ex, p->exitstr);
    free(p);
    _exits(ex);
}
```

17.6 alt()

⟨*function* alt 110c⟩≡ (429)

```
int
alt(Alt *alts)
{
    Alt *a, *xa, *ca;
    Channel volatile *c;
    int n, s, waiting, allreadycl;
    void* r;
    Thread *t;

    /*
     * The point of going splhi here is that note handlers
     * might reasonably want to use channel operations,
     * but that will hang if the note comes while we hold the
     * chanlock.  Instead, we delay the note until we've dropped
     * the lock.
     */
    t = _threadgetproc()->thread;
    if(t->moribund || _threadexitsallstatus)
        yield(); /* won't return */
```

```
s = _procsplhi();

lock(&chanlock);
t->alt = alts;
t->chan = Chanalt;

/* test whether any channels can proceed */
n = 0;
a = nil;

for(xa=alts; xa->op!=CHANEND && xa->op!=CHANNOBLK; xa++){
    xa->entryno = -1;
    if(xa->op == CHANNOP)
        continue;

    c = xa->c;
    if(c==nil){
        unlock(&chanlock);
        _procsplx(s);
        t->chan = Channone;
        return -1;
    }

    if(isopenfor(c, xa->op) && canexec(xa))
        if(nrand(++n) == 0)
            a = xa;
}


if(a==nil){
    /* nothing can proceed */
    if(xa->op == CHANNOBLK){
        unlock(&chanlock);
        _procsplx(s);
        t->chan = Channone;
        if(xa->op == CHANNOBLK)
            return xa - alts;
    }

    /* enqueue on all channels open for us. */
    c = nil;
    ca = nil;
    waiting = 0;
    allreadycl = 0;
    for(xa=alts; xa->op!=CHANEND; xa++)
        if(xa->op==CHANNOP)
            continue;
        else if(isopenfor(xa->c, xa->op)){
            waiting = 1;
            enqueue(xa, &c);
        } else if(xa->err != errcl)
            ca = xa;
        else
            allreadycl = 1;

    if(waiting == 0)
        if(ca != nil){
            /* everything was closed, select last channel */
            ca->err = errcl;
            unlock(&chanlock);
```

```
                _procsplx(s);
                t->chan = Channone;
                return ca - alts;
            } else if(allreadycl){
                /* everything was already closed */
                unlock(&chanlock);
                _procsplx(s);
                t->chan = Channone;
                return -1;
            }
        /*
         * wait for successful rendezvous.
         * we can't just give up if the rendezvous
         * is interrupted -- someone else might come
         * along and try to rendezvous with us, so
         * we need to be here.
         * if the channel was closed, the op is done
         * and we flag an error for the entry.
         */
        Again:
        unlock(&chanlock);
        _procsplx(s);
        r = _threadrendezvous(&c, 0);
        s = _procsplhi();
        lock(&chanlock);

        if(r==Intred){  /* interrupted */
            if(c!=nil) /* someone will meet us; go back */
                goto Again;
            c = (Channel*)~0; /* so no one tries to meet us */
        }

        /* dequeue from channels, find selected one */
        a = nil;
        for(xa=alts; xa->op!=CHANEND; xa++){
            if(xa->op==CHANNOP)
                continue;
            if(xa->c == c){
                a = xa;
                a->err = nil;
                if(r == Closed)
                    a->err = errcl;
            }
            dequeue(xa);
        }
        unlock(&chanlock);
        _procsplx(s);
        if(a == nil){ /* we were interrupted */
            assert(c==(Channel*)~0);
            return -1;
        }
    }else
        altexec(a, s); /* unlocks chanlock, does splx */
    _sched();
    t->chan = Channone;
    return a - alts;
}
```

⟨*function* isopenfor 112⟩≡ (429)

```
static bool
```

```
isopenfor(Channel *c, int op)
{
    return c->closed == 0 || (op == CHANRCV && c->n > 0);
}
```

⟨*function* canexec 113a⟩≡ (429)

```
static int
canexec(Alt *a)
{
    int i, otherop;
    Channel *c;

    c = a->c;
    /* are there senders or receivers blocked? */
    otherop = (CHANSND+CHANRCV) - a->op;
    for(i=0; i<c->nentry; i++)
        if(c->qentry[i] && c->qentry[i]->op==otherop && *c->qentry[i]->tag==nil){
            _threaddebug(DBGCHAN, "can rendez alt %p chan %p", a, c);
            return 1;
        }

    /* is there room in the channel? */
    if((a->op==CHANSND && c->n < c->s)
    || (a->op==CHANRCV && c->n > 0)){
        _threaddebug(DBGCHAN, "can buffer alt %p chan %p", a, c);
        return 1;
    }

    return 0;
}
```

⟨*global* errcl 113b⟩≡ (429)

```
static char errcl[] = "channel was closed";
```

⟨*function* enqueue 113c⟩≡ (429)

```
static void
enqueue(Alt *a, Channel **c)
{
    int i;

    _threaddebug(DBGCHAN, "Queuing alt %p on channel %p", a, a->c);
    a->tag = c;
    i = emptyentry(a->c);
    a->c->qentry[i] = a;
}
```

⟨*function* dequeue 113d⟩≡ (429)

```
static void
dequeue(Alt *a)
{
    int i;
    Channel *c;

    c = a->c;
    for(i=0; i<c->nentry; i++)
        if(c->qentry[i]==a){
            _threaddebug(DBGCHAN, "Dequeuing alt %p from channel %p", a, a->c);
            c->qentry[i] = nil;
            /* release if freed and not closing */
            if(c->freed && c->closed != 1)
```

```
                _chanfree(c);
            return;
        }
}
```

⟨*function* emptyentry 114a⟩≡ (429)

```
static int
emptyentry(Channel *c)
{
    int i, extra;

    assert((c->nentry==0 && c->qentry==nil) || (c->nentry && c->qentry));

    for(i=0; i<c->nentry; i++)
        if(c->qentry[i]==nil)
            return i;

    extra = 16;
    c->nentry += extra;
    c->qentry = realloc((void*)c->qentry, c->nentry*sizeof(c->qentry[0]));
    if(c->qentry == nil)
        sysfatal("realloc channel entries: %r");
    memset(&c->qentry[i], 0, extra*sizeof(c->qentry[0]));
    return i;
}
```

⟨*function* altexec 114b⟩≡ (429)

```
static int
altexec(Alt *a, int spl)
{
    volatile Alt *b;
    int i, n, otherop;
    Channel *c;
    void *me, *waiter, *buf;

    c = a->c;

    /* rendezvous with others */
    otherop = (CHANSND+CHANRCV) - a->op;
    n = 0;
    b = nil;
    me = a->v;
    for(i=0; i<c->nentry; i++)
        if(c->qentry[i] && c->qentry[i]->op==otherop && *c->qentry[i]->tag==nil)
            if(nrand(++n) == 0)
                b = c->qentry[i];
    if(b != nil){
        _threaddebug(DBGCHAN, "rendez %s alt %p chan %p alt %p", a->op==CHANRCV?"recv":"send", a, c, b);
        waiter = b->v;
        if(c->s && c->n){
            /*
             * if buffer is full and there are waiters
             * and we're meeting a waiter,
             * we must be receiving.
             *
             * we use the value in the channel buffer,
             * copy the waiter's value into the channel buffer
             * on behalf of the waiter, and then wake the waiter.
             */
            if(a->op!=CHANRCV)
```

```
                abort();
            buf = altexecbuffered(a, 1);
            altcopy(me, buf, c->e);
            altcopy(buf, waiter, c->e);
        }else{
            if(a->op==CHANRCV)
                altcopy(me, waiter, c->e);
            else
                altcopy(waiter, me, c->e);
        }
        *b->tag = c; /* commits us to rendezvous */
        _threaddebug(DBGCHAN, "unlocking the chanlock");
        unlock(&chanlock);
        _procsplx(spl);
        _threaddebug(DBGCHAN, "chanlock is %lud", *(ulong*)&chanlock);
        while(_threadrendezvous(b->tag, 0) == Intred)
            ;
        return 1;
    }

    buf = altexecbuffered(a, 0);
    if(a->op==CHANRCV)
        altcopy(me, buf, c->e);
    else
        altcopy(buf, me, c->e);

    unlock(&chanlock);
    _procsplx(spl);
    return 1;
}
```

⟨*function* altexecbuffered 115a⟩≡ (429)

```
static void*
altexecbuffered(Alt *a, int willreplace)
{
    uchar *v;
    Channel *c;

    c = a->c;
    /* use buffered channel queue */
    if(a->op==CHANRCV && c->n > 0){
        _threaddebug(DBGCHAN, "buffer recv alt %p chan %p", a, c);
        v = c->v + c->e*(c->f%c->s);
        if(!willreplace)
            c->n--;
        c->f++;
        return v;
    }
    if(a->op==CHANSND && c->n < c->s){
        _threaddebug(DBGCHAN, "buffer send alt %p chan %p", a, c);
        v = c->v + c->e*((c->f+c->n)%c->s);
        if(!willreplace)
            c->n++;
        return v;
    }
    abort();
    return nil;
}
```

⟨*function* altcopy 115b⟩≡ (429)

```
static void
altcopy(void *dst, void *src, int sz)
{
    if(dst){
        if(src)
            memmove(dst, src, sz);
        else
            memset(dst, 0, sz);
    }
}
```

17.7 Send/Receive

17.7.1 runop

⟨*function* runop 116⟩≡ (429)

```
static int
runop(int op, Channel *c, void *v, bool nb)
{
    int r;
    Alt a[2];

    /*
     * we could do this without calling alt,
     * but the only reason would be performance,
     * and i'm not convinced it matters.
     */
    a[0].op = op;
    a[0].c = c;
    a[0].v = v;
    a[0].err = nil;
    a[1].op = CHANEND;
    if(nb)
        a[1].op = CHANNOBLK;

    switch(r=alt(a)){
    case -1: /* interrupted */
        return -1;
    case 1: /* nonblocking, didn't accomplish anything */
        assert(nb);
        return 0;
    case 0:
        /*
         * Okay, but return -1 if the op is done because of a close.
         */
        if(a[0].err != nil)
            return -1;
        return 1;
    default:
        fprint(2, "ERROR: channel alt returned %d\n", r);
        abort();
        return -1;
    }
}
```

17.7.2 send()/recv()

⟨*function* send 117a⟩≡ (429)

```
int
send(Channel *c, void *v)
{
    return runop(CHANSND, c, v, 0);
}
```

⟨*function* recv 117b⟩≡ (429)

```
int
recv(Channel *c, void *v)
{
    return runop(CHANRCV, c, v, 0);
}
```

17.7.3 Non blocking operations: nbxxx()

⟨*function* nbrecv 117c⟩≡ (429)

```
int
nbrecv(Channel *c, void *v)
{
    return runop(CHANRCV, c, v, 1);
}
```

⟨*function* nbsend 117d⟩≡ (429)

```
int
nbsend(Channel *c, void *v)
{
    return runop(CHANSND, c, v, 1);
}
```

17.7.4 Typed send and receive

⟨*function* channelsize 117e⟩≡ (429)

```
static void
channelsize(Channel *c, int sz)
{
    if(c->e != sz){
        fprint(2, "expected channel with elements of size %d, got size %d\n",
            sz, c->e);
        abort();
    }
}
```

⟨*function* sendul 117f⟩≡ (429)

```
int
sendul(Channel *c, ulong v)
{
    channelsize(c, sizeof(ulong));
    return send(c, &v);
}
```

⟨*function* recvul 118a⟩≡ (429)

```
ulong
recvul(Channel *c)
{
    ulong v;

    channelsize(c, sizeof(ulong));
    if(recv(c, &v) < 0)
        return ~0;
    return v;
}
```

⟨*function* sendp 118b⟩≡ (429)

```
int
sendp(Channel *c, void *v)
{
    channelsize(c, sizeof(void*));
    return send(c, &v);
}
```

⟨*function* recvp 118c⟩≡ (429)

```
void*
recvp(Channel *c)
{
    void *v;

    channelsize(c, sizeof(void*));
    if(recv(c, &v) < 0)
        return nil;
    return v;
}
```

⟨*function* nbsendul 118d⟩≡ (429)

```
int
nbsendul(Channel *c, ulong v)
{
    channelsize(c, sizeof(ulong));
    return nbsend(c, &v);
}
```

⟨*function* nbrecvul 118e⟩≡ (429)

```
ulong
nbrecvul(Channel *c)
{
    ulong v;

    channelsize(c, sizeof(ulong));
    if(nbrecv(c, &v) == 0)
        return 0;
    return v;
}
```

⟨*function* nbsendp 118f⟩≡ (429)

```
int
nbsendp(Channel *c, void *v)
{
    channelsize(c, sizeof(void*));
    return nbsend(c, &v);
}
```

⟨*function* nbrecvp 119a⟩≡ (429)

```
void*
nbrecvp(Channel *c)
{
    void *v;

    channelsize(c, sizeof(void*));
    if(nbrecv(c, &v) == 0)
        return nil;
    return v;
}
```

17.8 IO processus

⟨*function* ioproc_arg 119b⟩≡ (423d)

```
#define ioproc_arg(io, type) (va_arg((io)->arg, type))
```

17.8.1 Ioproc

⟨*struct* Ioproc 119c⟩≡ (423d)

```
struct Ioproc
{
    int tid;

    Channel *c;
    Channel *creply;

    int inuse;

    long (*op)(va_list*);

    va_list arg;
    long ret;
    char err[ERRMAX];

    Ioproc *next;
};
```

⟨*constant* STACK 119d⟩≡ (431b)

```
STACK = 8192,
```

⟨*function* ioproc 119e⟩≡ (431c)

```
Ioproc*
ioproc(void)
{
    Ioproc *io;

    io = mallocz(sizeof(Ioproc), 1);
    if(io == nil)
        sysfatal("ioproc malloc: %r");
    io->c = chancreate(sizeof(void*), 0);
    io->creply = chancreate(sizeof(void*), 0);
    io->tid = proccreate(xioproc, io, STACK);
    return io;
}
```

⟨*function* closeioproc 120a⟩≡ (431c)

```
void
closeioproc(Ioproc *io)
{
    if(io == nil)
        return;
    iointerrupt(io);
    while(send(io->c, 0) == -1)
        ;
    chanfree(io->c);
    chanfree(io->creply);
    free(io);
}
```

⟨*function* iointerrupt 120b⟩≡ (431c)

```
void
iointerrupt(Ioproc *io)
{
    if(!io->inuse)
        return;
    threadint(io->tid);
}
```

17.8.2 xiproc()

⟨*function* xioproc 120c⟩≡ (431c)

```
static void
xioproc(void *a)
{
    Ioproc *io, *x;
    io = a;
    /*
     * first recvp acquires the ioproc.
     * second tells us that the data is ready.
     */
    for(;;){
        while(recv(io->c, &x) == -1)
            ;
        if(x == 0) /* our cue to leave */
            break;
        assert(x == io);

        /* caller is now committed -- even if interrupted he'll return */
        while(recv(io->creply, &x) == -1)
            ;
        if(x == 0) /* caller backed out */
            continue;
        assert(x == io);

        io->ret = io->op(&io->arg);
        if(io->ret < 0)
            rerrstr(io->err, sizeof io->err);
        while(send(io->creply, &io) == -1)
            ;
        while(recv(io->creply, &x) == -1)
            ;
    }
}
```

17.8.3 iocall()

⟨*function* iocall 121a⟩≡ (432a)

```
long
iocall(Ioproc *io, long (*op)(va_list*), ...)
{
    int ret, inted;
    Ioproc *msg;

    if(send(io->c, &io) == -1){
        werrstr("interrupted");
        return -1;
    }
    assert(!io->inuse);
    io->inuse = 1;
    io->op = op;
    va_start(io->arg, op);
    msg = io;
    inted = 0;
    while(send(io->creply, &msg) == -1){
        msg = nil;
        inted = 1;
    }
    if(inted){
        werrstr("interrupted");
        return -1;
    }

    /*
     * If we get interrupted, we have to stick around so that
     * the IO proc has someone to talk to.  Send it an interrupt
     * and try again.
     */
    inted = 0;
    while(recv(io->creply, nil) == -1){
        inted = 1;
        iointerrupt(io);
    }
    USED(inted);
    va_end(io->arg);
    ret = io->ret;
    if(ret < 0)
        errstr(io->err, sizeof io->err);
    io->inuse = 0;

    /* release resources */
    while(send(io->creply, &io) == -1)
        ;
    return ret;
}
```

17.8.4 IO wrappers

⟨*function* ioopen 121b⟩≡ (433a)

```
int
ioopen(Ioproc *io, char *path, int mode)
{
    return iocall(io, _ioopen, path, mode);
}
```

⟨*function* _ioopen 122a⟩≡ (433a)

```
static long
_ioopen(va_list *arg)
{
    char *path;
    int mode;

    path = va_arg(*arg, char*);
    mode = va_arg(*arg, int);
    return open(path, mode);
}
```

⟨*function* ioclose 122b⟩≡ (432c)

```
int
ioclose(Ioproc *io, int fd)
{
    return iocall(io, _ioclose, fd);
}
```

⟨*function* _ioclose 122c⟩≡ (432c)

```
static long
_ioclose(va_list *arg)
{
    int fd;

    fd = va_arg(*arg, int);
    return close(fd);
}
```

⟨*function* ioread 122d⟩≡ (433b)

```
long
ioread(Ioproc *io, int fd, void *a, long n)
{
    return iocall(io, _ioread, fd, a, n);
}
```

⟨*function* _ioread 122e⟩≡ (433b)

```
static long
_ioread(va_list *arg)
{
    int fd;
    void *a;
    long n;

    fd = va_arg(*arg, int);
    a = va_arg(*arg, void*);
    n = va_arg(*arg, long);
    return read(fd, a, n);
}
```

⟨*function* ioreadn 122f⟩≡ (433c)

```
long
ioreadn(Ioproc *io, int fd, void *a, long n)
{
    return iocall(io, _ioreadn, fd, a, n);
}
```

⟨*function* _ioreadn 123a⟩≡ (433c)

```
static long
_ioreadn(va_list *arg)
{
    int fd;
    void *a;
    long n;

    fd = va_arg(*arg, int);
    a = va_arg(*arg, void*);
    n = va_arg(*arg, long);
    return readn(fd, a, n);
}
```

⟨*function* iowrite 123b⟩≡ (433e)

```
long
iowrite(Ioproc *io, int fd, void *a, long n)
{
    return iocall(io, _iowrite, fd, a, n);
}
```

⟨*function* _iowrite 123c⟩≡ (433e)

```
static long
_iowrite(va_list *arg)
{
    int fd;
    void *a;
    long n;

    fd = va_arg(*arg, int);
    a = va_arg(*arg, void*);
    n = va_arg(*arg, long);
    return write(fd, a, n);
}
```

⟨*function* iosleep 123d⟩≡ (433d)

```
int
iosleep(Ioproc *io, long n)
{
    return iocall(io, _iosleep, n);
}
```

⟨*function* _iosleep 123e⟩≡ (433d)

```
static long
_iosleep(va_list *arg)
{
    long n;

    n = va_arg(*arg, long);
    return sleep(n);
}
```

17.9 Thread-aware libc

17.9.1 Memory

⟨*global* totalmalloc 123f⟩≡ (427a)

```
static long totalmalloc;
```

⟨*function* _threadmalloc 124a⟩≡ (427a)

```
void*
_threadmalloc(long size, int z)
{
    void *m;

    m = malloc(size);
    if (m == nil)
        sysfatal("Malloc of size %ld failed: %r", size);
    setmalloctag(m, getcallerpc(&size));
    totalmalloc += size;
    if (size > 100000000) {
        fprint(2, "Malloc of size %ld, total %ld\n", size, totalmalloc);
        abort();
    }
    if (z)
        memset(m, 0, size);
    return m;
}
```

17.9.2 Exit

⟨*global* _threadexitsallstatus 124b⟩≡ (426c)

```
char *_threadexitsallstatus;
```

⟨*function* threadexits 124c⟩≡ (426c)

```
void
threadexits(char *exitstr)
{
    Proc *p;
    Thread *t;

    p = _threadgetproc();
    t = p->thread;
    t->moribund = 1;
    if(exitstr==nil)
        exitstr="";
    utfecpy(p->exitstr, p->exitstr+ERRMAX, exitstr);
    _sched();
}
```

⟨*function* threadexitsall 124d⟩≡ (426c)

```
void
threadexitsall(char *exitstr)
{
    Proc *p;
    int pid[64];
    int i, npid, mypid;

    if(exitstr == nil)
        exitstr = "";
    _threadexitsallstatus = exitstr;
    _threaddebug(DBGSCHED, "_threadexitsallstatus set to %p", _threadexitsallstatus);
    mypid = _tos->pid; //getpid();

    /*
     * signal others.
     * copying all the pids first avoids other threads
     * teardown procedures getting in the way.
```

```
     *
     * avoid mallocs since malloc can post a note which can
     * call threadexitsall...
     */
    for(;;){
        lock(&_threadpq.lock);
        npid = 0;
        for(p = _threadpq.head; p && npid < nelem(pid); p=p->next){
            if(p->threadint == 0 && p->pid != mypid){
                pid[npid++] = p->pid;
                p->threadint = 1;
            }
        }
        unlock(&_threadpq.lock);
        if(npid == 0)
            break;
        for(i=0; i<npid; i++)
            postnote(PNPROC, pid[i], "threadint");
    }

    /* leave */
    exits(exitstr);
}
```

⟨*global* _threadwaitchan 125a⟩≡ (426c)

```
Channel *_threadwaitchan;
```

⟨*function* threadwaitchan 125b⟩≡ (426c)

```
Channel*
threadwaitchan(void)
{
    if(_threadwaitchan==nil)
        _threadwaitchan = chancreate(sizeof(Waitmsg*), 16);
    return _threadwaitchan;
}
```

17.9.3 Fork

⟨*function* _schedfork 125c⟩≡ (430)

```
int
_schedfork(Proc *p)
{
    int pid;

    switch(pid = rfork(RFPROC|RFMEM|RFNOWAIT|p->rforkflag)){
    case 0:
        *mainp = p; /* write to stack, so local to proc */
        longjmp(_mainjmp, 1);
    default:
        return pid;
    }
}
```

17.9.4 Exec

⟨*function* _schedexec 125d⟩≡ (430)

```
int
_schedexec(Execargs *e)
```

```
{
    int pid;

    switch(pid = rfork(RFREND|RFNOTEG|RFFDG|RFMEM|RFPROC)){
    case 0:
        efork(e);
    default:
        return pid;
    }
}
```

⟨*function* efork 126a⟩≡ (430)

```
static void
efork(Execargs *e)
{
    char buf[ERRMAX];

    _threaddebug(DBGEXEC, "_schedexec %s", e->prog);
    close(e->fd[0]);
    exec(e->prog, e->args);
    _threaddebug(DBGEXEC, "_schedexec failed: %r");
    rerrstr(buf, sizeof buf);
    if(buf[0]=='\0')
        strcpy(buf, "exec failed");
    write(e->fd[1], buf, strlen(buf));
    close(e->fd[1]);
    _exits(buf);
}
```

⟨*function* _schedexecwait 126b⟩≡ (430)

```
void
_schedexecwait(void)
{
    int pid;
    Channel *c;
    Proc *p;
    Thread *t;
    Waitmsg *w;

    p = _threadgetproc();
    t = p->thread;
    pid = t->ret;
    _threaddebug(DBGEXEC, "_schedexecwait %d", t->ret);

    rfork(RFCFDG);
    for(;;){
        w = wait();
        if(w == nil)
            break;
        if(w->pid == pid)
            break;
        free(w);
    }
    if(w != nil){
        if((c = _threadwaitchan) != nil)
            sendp(c, w);
        else
            free(w);
    }
    threadexits("procexec");
}
```

⟨*constant* PIPEMNT 127a⟩≡ (434b)

```
#define PIPEMNT "/mnt/temp"
```

⟨*function* procexec 127b⟩≡ (434b)

```
void
procexec(Channel *pidc, char *prog, char *args[])
{
    int n;
    Proc *p;
    Thread *t;

    _threaddebug(DBGEXEC, "procexec %s", prog);
    /* must be only thread in proc */
    p = _threadgetproc();
    t = p->thread;
    if(p->threads.head != t || p->threads.head->nextt != nil){
        werrstr("not only thread in proc");
    Bad:
        if(pidc)
            sendul(pidc, ~0);
        return;
    }

    /*
     * We want procexec to behave like exec; if exec succeeds,
     * never return, and if it fails, return with errstr set.
     * Unfortunately, the exec happens in another proc since
     * we have to wait for the exec'ed process to finish.
     * To provide the semantics, we open a pipe with the
     * write end close-on-exec and hand it to the proc that
     * is doing the exec.  If the exec succeeds, the pipe will
     * close so that our read below fails.  If the exec fails,
     * then the proc doing the exec sends the errstr down the
     * pipe to us.
     */
    if(bind("#|", PIPEMNT, MREPL) < 0)
        goto Bad;
    if((p->exec.fd[0] = open(PIPEMNT "/data", OREAD)) < 0){
        unmount(nil, PIPEMNT);
        goto Bad;
    }
    if((p->exec.fd[1] = open(PIPEMNT "/data1", OWRITE|OCEXEC)) < 0){
        close(p->exec.fd[0]);
        unmount(nil, PIPEMNT);
        goto Bad;
    }
    unmount(nil, PIPEMNT);

    /* exec in parallel via the scheduler */
    assert(p->needexec==0);
    p->exec.prog = prog;
    p->exec.args = args;
    p->needexec = 1;
    _sched();

    close(p->exec.fd[1]);
    if((n = read(p->exec.fd[0], p->exitstr, ERRMAX-1)) > 0){ /* exec failed */
        p->exitstr[n] = '\0';
        errstr(p->exitstr, ERRMAX);
        close(p->exec.fd[0]);
```

```
        goto Bad;
    }
    close(p->exec.fd[0]);

    if(pidc)
        sendul(pidc, t->ret);

    /* wait for exec'ed program, then exit */
    _schedexecwait();
}
```

⟨*function* procexecl 128a⟩≡ (434b)

```
void
procexecl(Channel *pidc, char *f, ...)
{
    procexec(pidc, f, &f+1);
}
```

17.9.5 Interruptions

⟨*constant* Intred 128b⟩≡ (429)

```
#define Intred ((void*)~0)  /* interrupted */
```

17.9.6 Notes

⟨*global* _threadnopasser 128c⟩≡ (427b)

```
int _threadnopasser;
```

⟨*constant NFN (libthread/note.c)* 128d⟩≡ (427b)

```
#define NFN  33
```

⟨*constant* ERRLEN 128e⟩≡ (427b)

```
#define ERRLEN 48
```

⟨*struct* Note 128f⟩≡ (427b)

```
struct Note
{
    Lock  inuse;
    Proc  *proc;  /* recipient */
    char  s[ERRMAX]; /* arg2 */
};
```

⟨*global* notes 128g⟩≡ (427b)

```
static Note notes[128];
```

⟨*global* enotes 128h⟩≡ (427b)

```
static Note *enotes = notes+nelem(notes);
```

⟨*global* onnote 128i⟩≡ (427b)

```
static int  (*onnote[NFN])(void*, char*);
```

⟨*global* onnotepid 128j⟩≡ (427b)

```
static int  onnotepid[NFN];
```

⟨*global* onnotelock 128k⟩≡ (427b)

```
static Lock onnotelock;
```

⟨*function* threadnotify 129a⟩≡ (427b)

```
int
threadnotify(int (*f)(void*, char*), int in)
{
    int i, topid;
    int (*from)(void*, char*), (*to)(void*, char*);

    if(in){
        from = nil;
        to = f;
        topid = _threadgetproc()->pid;
    }else{
        from = f;
        to = nil;
        topid = 0;
    }
    lock(&onnotelock);
    for(i=0; i<NFN; i++)
        if(onnote[i]==from){
            onnote[i] = to;
            onnotepid[i] = topid;
            break;
        }
    unlock(&onnotelock);
    return i<NFN;
}
```

⟨*function* delayednotes 129b⟩≡ (427b)

```
static void
delayednotes(Proc *p, void *v)
{
    int i;
    Note *n;
    int (*fn)(void*, char*);

    if(!p->pending)
        return;

    p->pending = 0;
    for(n=notes; n<enotes; n++){
        if(n->proc == p){
            for(i=0; i<NFN; i++){
                if(onnotepid[i]!=p->pid || (fn = onnote[i])==nil)
                    continue;
                if((*fn)(v, n->s))
                    break;
            }
            if(i==NFN){
                _threaddebug(DBGNOTE, "Unhandled note %s, proc %p\n", n->s, p);
                if(v != nil)
                    noted(NDFLT);
                else if(strncmp(n->s, "sys:", 4)==0)
                    abort();
                threadexitsall(n->s);
            }
            n->proc = nil;
            unlock(&n->inuse);
        }
    }
}
```

⟨*function* _threadnote 130a⟩≡ (427b)

```
void
_threadnote(void *v, char *s)
{
    Proc *p;
    Note *n;

    _threaddebug(DBGNOTE, "Got note %s", s);
    if(strncmp(s, "sys:", 4) == 0)
        noted(NDFLT);

    if(_threadexitsallstatus){
        _threaddebug(DBGNOTE, "Threadexitsallstatus = '%s'\n", _threadexitsallstatus);
        _exits(_threadexitsallstatus);
    }

    if(strcmp(s, "threadint")==0)
        noted(NCONT);

    p = _threadgetproc();
    if(p == nil)
        noted(NDFLT);

    for(n=notes; n<enotes; n++)
        if(canlock(&n->inuse))
            break;
    if(n==enotes)
        sysfatal("libthread: too many delayed notes");
    utfecpy(n->s, n->s+ERRMAX, s);
    n->proc = p;
    p->pending = 1;
    if(!p->splhi)
        delayednotes(p, v);
    noted(NCONT);
}
```

⟨*function* _procsplhi 130b⟩≡ (427b)

```
int
_procsplhi(void)
{
    int s;
    Proc *p;

    p = _threadgetproc();
    s = p->splhi;
    p->splhi = 1;
    return s;
}
```

⟨*function* _procsplx 130c⟩≡ (427b)

```
void
_procsplx(int s)
{
    Proc *p;

    p = _threadgetproc();
    p->splhi = s;
    if(s)
        return;
    if(p->pending)
```

```
        delayednotes(p, nil);
}
```

17.9.7 Dialing

⟨*function* _iodial 131a⟩≡ (432d)

```
static long
_iodial(va_list *arg)
{
    char *addr, *local, *dir;
    int *cdfp;

    addr = va_arg(*arg, char*);
    local = va_arg(*arg, char*);
    dir = va_arg(*arg, char*);
    cdfp = va_arg(*arg, int*);

    return dial(addr, local, dir, cdfp);
}
```

⟨*function* iodial 131b⟩≡ (432d)

```
int
iodial(Ioproc *io, char *addr, char *local, char *dir, int *cdfp)
{
    return iocall(io, _iodial, addr, local, dir, cdfp);
}
```

⟨*enum* _anon_ (libthread/dial.c) 131c⟩≡ (434a)

```
enum
{
    Maxstring = 128,
    Maxpath  = 256,
};
```

⟨*struct* DS*(libthread/dial.c)* 131d⟩≡ (434a)

```
struct DS {
    /* dist string */
    char buf[Maxstring];
    char *netdir;
    char *proto;
    char *rem;

    /* other args */
    char *local;
    char *dir;
    int *cfdp;
};
```

⟨*function* _threaddial 131e⟩≡ (434a)

```
/*
 *  the dialstring is of the form '[/net/]proto!dest'
 */
int
_threaddial(char *dest, char *local, char *dir, int *cfdp)
{
    DS ds;
    int rv;
    char err[ERRMAX], alterr[ERRMAX];
```

```
    ds.local = local;
    ds.dir = dir;
    ds.cfdp = cfdp;

    _dial_string_parse(dest, &ds);
    if(ds.netdir)
        return csdial(&ds);

    ds.netdir = "/net";
    rv = csdial(&ds);
    if(rv >= 0)
        return rv;
    err[0] = '\0';
    errstr(err, sizeof err);
    if(strstr(err, "refused") != 0){
        werrstr("%s", err);
        return rv;
    }
    ds.netdir = "/net.alt";
    rv = csdial(&ds);
    if(rv >= 0)
        return rv;

    alterr[0] = 0;
    errstr(alterr, sizeof alterr);
    if(strstr(alterr, "translate") || strstr(alterr, "does not exist"))
        werrstr("%s", err);
    else
        werrstr("%s", alterr);
    return rv;
}
```

⟨*function* csdial *(libthread/dial.c)* 132⟩≡ (434a)

```
static int
csdial(DS *ds)
{
    int n, fd, rv;
    char *p, buf[Maxstring], clone[Maxpath], err[ERRMAX], besterr[ERRMAX];

    /*
     *  open connection server
     */
    snprint(buf, sizeof(buf), "%s/cs", ds->netdir);
    fd = open(buf, ORDWR);
    if(fd < 0){
        /* no connection server, don't translate */
        snprint(clone, sizeof(clone), "%s/%s/clone", ds->netdir, ds->proto);
        return call(clone, ds->rem, ds);
    }

    /*
     *  ask connection server to translate
     */
    snprint(buf, sizeof(buf), "%s!%s", ds->proto, ds->rem);
    if(write(fd, buf, strlen(buf)) < 0){
        close(fd);
        return -1;
    }

    /*
```

```
     *  loop through each address from the connection server till
     *  we get one that works.
     */
    *besterr = 0;
    rv = -1;
    seek(fd, 0, 0);
    while((n = read(fd, buf, sizeof(buf) - 1)) > 0){
        buf[n] = 0;
        p = strchr(buf, ' ');
        if(p == 0)
            continue;
        *p++ = 0;
        rv = call(buf, p, ds);
        if(rv >= 0)
            break;
        err[0] = '\0';
        errstr(err, sizeof err);
        if(strstr(err, "does not exist") == 0)
            strcpy(besterr, err);
    }
    close(fd);

    if(rv < 0 && *besterr)
        werrstr("%s", besterr);
    else
        werrstr("%s", err);
    return rv;
}
```

⟨*function* `call` *(libthread/dial.c)* 133⟩≡ (434a)

```
static int
call(char *clone, char *dest, DS *ds)
{
    int fd, cfd, n;
    char cname[Maxpath], name[Maxpath], data[Maxpath], *p;

    /* because cs is in a different name space, replace the mount point */
    if(*clone == '/'){
        p = strchr(clone+1, '/');
        if(p == nil)
            p = clone;
        else
            p++;
    } else
        p = clone;
    snprint(cname, sizeof cname, "%s/%s", ds->netdir, p);

    cfd = open(cname, ORDWR);
    if(cfd < 0)
        return -1;

    /* get directory name */
    n = read(cfd, name, sizeof(name)-1);
    if(n < 0){
        close(cfd);
        return -1;
    }
    name[n] = 0;
    for(p = name; *p == ' '; p++)
        ;
```

```
    snprint(name, sizeof(name), "%ld", strtoul(p, 0, 0));
    p = strrchr(cname, '/');
    *p = 0;
    if(ds->dir)
        snprint(ds->dir, NETPATHLEN, "%s/%s", cname, name);
    snprint(data, sizeof(data), "%s/%s/data", cname, name);

    /* connect */
    if(ds->local)
        snprint(name, sizeof(name), "connect %s %s", dest, ds->local);
    else
        snprint(name, sizeof(name), "connect %s", dest);
    if(write(cfd, name, strlen(name)) < 0){
        close(cfd);
        return -1;
    }

    /* open data connection */
    fd = open(data, ORDWR);
    if(fd < 0){
        close(cfd);
        return -1;
    }
    if(ds->cfdp)
        *ds->cfdp = cfd;
    else
        close(cfd);
    return fd;
}
```

⟨*function* _dial_string_parse*(libthread/dial.c)* 134⟩≡ (434a)

```
/*
 *  parse a dial string
 */
static void
_dial_string_parse(char *str, DS *ds)
{
    char *p, *p2;

    strncpy(ds->buf, str, Maxstring);
    ds->buf[Maxstring-1] = 0;

    p = strchr(ds->buf, '!');
    if(p == 0) {
        ds->netdir = 0;
        ds->proto = "net";
        ds->rem = ds->buf;
    } else {
        if(*ds->buf != '/' && *ds->buf != '#'){
            ds->netdir = 0;
            ds->proto = ds->buf;
        } else {
            for(p2 = p; *p2 != '/'; p2--)
                ;
            *p2++ = 0;
            ds->netdir = ds->buf;
            ds->proto = p2;
        }
        *p = 0;
        ds->rem = p + 1;
```

```
    }
}
```

17.10 Error management

⟨*function* _threadsysfatal 135a⟩≡ (427a)

```
void
_threadsysfatal(char *fmt, va_list arg)
{
    char buf[1024]; /* size doesn't matter; we're about to exit */

    vseprint(buf, buf+sizeof(buf), fmt, arg);
    if(argv0)
        fprint(2, "%s: %s\n", argv0, buf);
    else
        fprint(2, "%s\n", buf);
    threadexitsall(buf);
}
```

⟨*function* _threadassert 135b⟩≡ (426a)

```
void
_threadassert(char *s)
{
    char buf[256];
    int n;
    Proc *p;

    p = _threadgetproc();
    if(p && p->thread)
        n = sprint(buf, "%d.%d ", p->pid, p->thread->id);
    else
        n = 0;
    snprint(buf+n, sizeof(buf)-n, "%s: assertion failed\n", s);
    write(2, buf, strlen(buf));
    abort();
}
```

17.11 Debugging

⟨*global* _threaddebuglevel 135c⟩≡ (426a)

```
// biset<enum<dbgxxx>>
int _threaddebuglevel;
```

⟨*constant* DBGAPPL 135d⟩≡ (423d)

```
#define DBGAPPL (1 << 0)
```

⟨*constant* DBGSCHED 135e⟩≡ (423d)

```
#define DBGSCHED (1 << 16)
```

⟨*constant* DBGCHAN 135f⟩≡ (423d)

```
#define DBGCHAN (1 << 17)
```

⟨*constant* DBGREND 135g⟩≡ (423d)

```
#define DBGREND (1 << 18)
```

⟨*constant* DBGNOTE 135h⟩≡ (423d)

```
#define DBGNOTE (1 << 20)
```

⟨*constant* DBGEXEC 136a⟩≡ (423d)

```
#define DBGEXEC (1 << 21)
```

⟨*function* _threaddebug 136b⟩≡ (426a)

```
void
_threaddebug(ulong flag, char *fmt, ...)
{
    char buf[128];
    va_list arg;
    Fmt f;
    Proc *p;

    if((_threaddebuglevel&flag) == 0)
        return;

    fmtfdinit(&f, 2, buf, sizeof buf);

    p = _threadgetproc();
    if(p==nil)
        fmtprint(&f, "noproc ");
    else if(p->thread)
        fmtprint(&f, "%d.%d ", p->pid, p->thread->id);
    else
        fmtprint(&f, "%d._ ", p->pid);

    va_start(arg, fmt);
    fmtvprint(&f, fmt, arg);
    va_end(arg);
    fmtprint(&f, "\n");
    fmtfdflush(&f);
}
```

17.12 Advanced topics

⟨*global* _threadnotefd 136c⟩≡ (430)

```
int _threadnotefd;
```

⟨*global* _threadpasserpid 136d⟩≡ (430)

```
int _threadpasserpid;
```

⟨*function* _times 136e⟩≡ (430)

```
static long
_times(long *t)
{
    char b[200], *p;
    int f;
    ulong r;

    memset(b, 0, sizeof(b));
    f = open("/dev/cputime", OREAD|OCEXEC);
    if(f < 0)
        return 0;
    if(read(f, b, sizeof(b)) <= 0){
        close(f);
        return 0;
    }
    p = b;
    if(t)
```

```
        t[0] = atol(p);
    p = skip(p);
    if(t)
        t[1] = atol(p);
    p = skip(p);
    r = atol(p);
    if(t){
        p = skip(p);
        t[2] = atol(p);
        p = skip(p);
        t[3] = atol(p);
    }
    return r;
}
```

⟨*function skip (libthread/main.c)* 137a⟩≡ (430)

```
static char*
skip(char *p)
{
    while(*p == ' ')
        p++;
    while(*p != ' ' && *p != 0)
        p++;
    return p;
}
```

17.12.1 Per thread private data

⟨*global privlock (libthread/id.c)* 137b⟩≡ (426b)

```
static Lock privlock;
```

⟨*global* `privmask` 137c⟩≡ (426b)

```
//array<bool> NPRIV at least
static int privmask = 1;
```

⟨*function* `tprivalloc` 137d⟩≡ (426b)

```
int
tprivalloc(void)
{
    int i;

    lock(&privlock);
    for(i=0; i<NPRIV; i++)
        if(!(privmask&(1<<i))){
            privmask |= 1<<i;
            unlock(&privlock);
            return i;
        }
    unlock(&privlock);
    return -1;
}
```

⟨*function* `tprivfree` 137e⟩≡ (426b)

```
void
tprivfree(int i)
{
    if(i < 0 || i >= NPRIV)
        abort();
    lock(&privlock);
    privmask &= ~(1<<i);
}
```

⟨*function* tprivaddr 138a⟩≡ (426b)

```
void**
tprivaddr(int i)
{
    return &_threadgetproc()->thread->udata[i];
}
```

⟨*function* threaddata 138b⟩≡ (426b)

```
void**
threaddata(void)
{
    return &_threadgetproc()->thread->udata[0];
}
```

⟨*function* _workerdata 138c⟩≡ (426b)

```
void**
_workerdata(void)
{
    return &_threadgetproc()->wdata;
}
```

⟨*function* procdata 138d⟩≡ (426b)

```
void**
procdata(void)
{
    return &_threadgetproc()->udata;
}
```

17.12.2 Thread groups

⟨*function* threadsetgrp 138e⟩≡ (426b)

```
int
threadsetgrp(int ng)
{
    int og;
    Thread *t;

    t = _threadgetproc()->thread;
    og = t->grp;
    t->grp = ng;
    return og;
}
```

⟨*function* threadgetgrp 138f⟩≡ (426b)

```
int
threadgetgrp(void)
{
    return _threadgetproc()->thread->grp;
}
```

17.12.3 Thread kills

⟨*function* tinterrupt 138g⟩≡ (431a)

```
static void
tinterrupt(Proc *p, Thread *t)
{
    switch(t->state){
    case Running:
```

```
        postnote(PNPROC, p->pid, "threadint");
        break;
    case Rendezvous:
        _threadflagrendez(t);
        break;
    }
}
```

⟨*function* threadkillgrp 139a⟩≡ (431a)

```
void
threadkillgrp(int grp)
{
    threadxxxgrp(grp, 1);
}
```

⟨*function* threadkill 139b⟩≡ (431a)

```
void
threadkill(int id)
{
    threadxxx(id, 1);
}
```

⟨*function* threadxxxgrp 139c⟩≡ (431a)

```
static void
threadxxxgrp(int grp, int dokill)
{
    Proc *p;
    Thread *t;

    lock(&_threadpq.lock);
    for(p=_threadpq.head; p; p=p->next){
        lock(&p->lock);
        for(t=p->threads.head; t; t=t->nextt)
            if(t->grp == grp){
                if(dokill)
                    t->moribund = 1;
                tinterrupt(p, t);
            }
        unlock(&p->lock);
    }
    unlock(&_threadpq.lock);
    _threadbreakrendez();
}
```

⟨*function* threadxxx 139d⟩≡ (431a)

```
static void
threadxxx(int id, int dokill)
{
    Proc *p;
    Thread *t;

    lock(&_threadpq.lock);
    for(p=_threadpq.head; p; p=p->next){
        lock(&p->lock);
        for(t=p->threads.head; t; t=t->nextt)
            if(t->id == id){
                if(dokill)
                    t->moribund = 1;
                tinterrupt(p, t);
                unlock(&p->lock);
```

```
                unlock(&_threadpq.lock);
                _threadbreakrendez();
                return;
            }
        unlock(&p->lock);
    }
    unlock(&_threadpq.lock);
    _threaddebug(DBGNOTE, "Can't find thread to kill");
    return;
}
```

⟨*function* threadintgrp 140a⟩≡ (431a)

```
void
threadintgrp(int grp)
{
    threadxxxgrp(grp, 0);
}
```

⟨*function* threadint 140b⟩≡ (431a)

```
void
threadint(int id)
{
    threadxxx(id, 0);
}
```

17.12.4 Rendez vous

⟨*constant* RENDHASH 140c⟩≡ (423c)

```
RENDHASH = 13,
```

⟨*struct* Rgrp 140d⟩≡ (423d)

```
struct Rgrp
{
    Lock lock;
    Thread *hash[RENDHASH];
};
```

⟨*global* _threadrgrp 140e⟩≡ (428a)

```
Rgrp _threadrgrp;
```

⟨*function* _threadrendezvous 140f⟩≡ (428a)

```
void*
_threadrendezvous(void *tag, void *val)
{
    void *ret;
    Thread *t, **l;

    lock(&_threadrgrp.lock);
    l = &_threadrgrp.hash[((uintptr)tag)%nelem(_threadrgrp.hash)];
    for(t=*l; t; l=&t->rendhash, t=*l){
        if(t->rendtag==tag){
            _threaddebug(DBGREND, "Rendezvous with thread %d.%d", t->proc->pid, t->id);
            *l = t->rendhash;
            ret = finish(t, val);
            unlock(&_threadrgrp.lock);
            return ret;
        }
    }
```

```
    /* Going to sleep here. */
    t = _threadgetproc()->thread;
    t->rendbreak = 0;
    t->inrendez = 1;
    t->rendtag = tag;
    t->rendval = val;
    t->rendhash = *l;
    *l = t;
    t->nextstate = Rendezvous;
    _threaddebug(DBGREND, "Rendezvous for tag %p", t->rendtag);
    unlock(&_threadrgrp.lock);
    _sched();
    t->inrendez = 0;
    _threaddebug(DBGREND, "Woke after rendezvous; val is %p", t->rendval);
    return t->rendval;
}
```

⟨*global* isdirty 141a⟩≡ (428a)

```
static int isdirty;
```

⟨*function* finish 141b⟩≡ (428a)

```
static void*
finish(Thread *t, void *val)
{
    void *ret;

    ret = t->rendval;
    t->rendval = val;
    while(t->state == Running)
        sleep(0);
    lock(&t->proc->lock);
    if(t->state == Rendezvous){ /* not always true: might be Dead */
        t->state = Ready;
        _threadready(t);
    }
    unlock(&t->proc->lock);
    return ret;
}
```

⟨*function* _threadflagrendez 141c⟩≡ (428a)

```
/*
 * This is called while holding _threadpq.lock and p->lock,
 * so we can't lock _threadrgrp.lock.  Instead our caller has
 * to call _threadbreakrendez after dropping those locks.
 */
void
_threadflagrendez(Thread *t)
{
    t->rendbreak = 1;
    isdirty = 1;
}
```

⟨*function* _threadbreakrendez 141d⟩≡ (428a)

```
void
_threadbreakrendez(void)
{
    int i;
    Thread *t, **l;

    if(isdirty == 0)
```

```
        return;
    lock(&_threadrgrp.lock);
    if(isdirty == 0){
        unlock(&_threadrgrp.lock);
        return;
    }
    isdirty = 0;
    for(i=0; i<nelem(_threadrgrp.hash); i++){
        l = &_threadrgrp.hash[i];
        for(t=*l; t; t=*l){
            if(t->rendbreak){
                *l = t->rendhash;
                finish(t, (void*)~0);
            }else
                 l=&t->rendhash;
        }
    }
    unlock(&_threadrgrp.lock);
}
```

17.12.5 chanprint()

⟨*function* chanprint 142⟩≡ (432b)

```
int
chanprint(Channel *c, char *fmt, ...)
{
    va_list arg;
    char *p;
    int n;

    va_start(arg, fmt);
    p = vsmprint(fmt, arg);
    va_end(arg);
    if(p == nil)
        sysfatal("vsmprint failed: %r");
    n = sendp(c, p);
    yield(); /* let recipient handle message immediately */
    return n;
}
```

Chapter 18

Network

⟨*type* NetConnInfo 143a⟩≡ (194b)

```
struct NetConnInfo {
    char    *dir;       /* connection directory */
    char    *root;      /* network root */
    char    *spec;      /* binding spec */
    char    *lsys;      /* local system */
    char    *lserv;     /* local service */
    char    *rsys;      /* remote system */
    char    *rserv;     /* remote service */
    char    *laddr;     /* local address */
    char    *raddr;     /* remote address */
};
```

18.1 Example: an echo server

18.2 netmkaddr()

⟨*function* netmkaddr 143b⟩≡ (236a)

```
/*
 *  make an address, add the defaults
 */
char *
netmkaddr(char *linear, char *defnet, char *defsrv)
{
    static char addr[256];
    char *cp;

    /*
     *  dump network name
     */
    cp = strchr(linear, '!');
    if(cp == 0){
        if(defnet==0){
            if(defsrv)
                snprint(addr, sizeof(addr), "net!%s!%s",
                    linear, defsrv);
            else
                snprint(addr, sizeof(addr), "net!%s", linear);
        }
        else {
            if(defsrv)
                snprint(addr, sizeof(addr), "%s!%s!%s", defnet,
```

```
                    linear, defsrv);
            else
                snprint(addr, sizeof(addr), "%s!%s", defnet,
                    linear);
        }
        return addr;
    }

    /*
     *  if there is already a service, use it
     */
    cp = strchr(cp+1, '!');
    if(cp)
        return linear;

    /*
     *  add default service
     */
    if(defsrv == 0)
        return linear;
    snprint(addr, sizeof(addr), "%s!%s", linear, defsrv);

    return addr;
}
```

18.3 announce(), accept(), listen(), reject()

⟨*function* announce 144⟩≡ (301a)

```
/*
 *  announce a network service.
 */
int
announce(char *addr, char *dir)
{
    int ctl, n, m;
    char buf[Maxpath];
    char buf2[Maxpath];
    char netdir[Maxpath];
    char naddr[Maxpath];
    char *cp;

    /*
     *  translate the address
     */
    if(nettrans(addr, naddr, sizeof(naddr), netdir, sizeof(netdir)) < 0)
        return -1;

    /*
     * get a control channel
     */
    ctl = open(netdir, ORDWR);
    if(ctl<0){
        werrstr("announce opening %s: %r", netdir);
        return -1;
    }
    cp = strrchr(netdir, '/');
    if(cp == nil){
        werrstr("announce arg format %s", netdir);
```

```
        close(ctl);
        return -1;
    }
    *cp = 0;

    /*
     *  find out which line we have
     */
    n = snprint(buf, sizeof(buf), "%s/", netdir);
    m = read(ctl, &buf[n], sizeof(buf)-n-1);
    if(m <= 0){
        werrstr("announce reading %s: %r", netdir);
        close(ctl);
        return -1;
    }
    buf[n+m] = 0;

    /*
     *  make the call
     */
    n = snprint(buf2, sizeof(buf2), "announce %s", naddr);
    if(write(ctl, buf2, n)!=n){
        werrstr("announce writing %s: %r", netdir);
        close(ctl);
        return -1;
    }

    /*
     *  return directory etc.
     */
    if(dir){
        strncpy(dir, buf, NETPATHLEN);
        dir[NETPATHLEN-1] = 0;
    }
    return ctl;
}
```

⟨*function* listen 145⟩≡ (301a)

```
/*
 *  listen for an incoming call
 */
int
listen(char *dir, char *newdir)
{
    int ctl, n, m;
    char buf[Maxpath];
    char *cp;

    /*
     *  open listen, wait for a call
     */
    snprint(buf, sizeof(buf), "%s/listen", dir);
    ctl = open(buf, ORDWR);
    if(ctl < 0){
        werrstr("listen opening %s: %r", buf);
        return -1;
    }

    /*
     *  find out which line we have
```

```
     */
    strncpy(buf, dir, sizeof(buf) - 1);
    buf[sizeof(buf) - 1] = 0;
    cp = strrchr(buf, '/');
    if(cp == nil){
        close(ctl);
        werrstr("listen arg format %s", dir);
        return -1;
    }
    *++cp = 0;
    n = cp-buf;
    m = read(ctl, cp, sizeof(buf) - n - 1);
    if(m <= 0){
        close(ctl);
        werrstr("listen reading %s/listen: %r", dir);
        return -1;
    }
    buf[n+m] = 0;

    /*
     *  return directory etc.
     */
    if(newdir){
        strncpy(newdir, buf, NETPATHLEN);
        newdir[NETPATHLEN-1] = 0;
    }
    return ctl;

}
```

⟨*function* accept 146a⟩≡ (301a)

```
/*
 *  accept a call, return an fd to the open data file
 */
int
accept(int ctl, char *dir)
{
    char buf[Maxpath];
    char *num;
    long n;

    num = strrchr(dir, '/');
    if(num == nil)
        num = dir;
    else
        num++;

    n = snprint(buf, sizeof(buf), "accept %s", num);
    write(ctl, buf, n); /* ignore return value, network might not need accepts */

    snprint(buf, sizeof(buf), "%s/data", dir);
    return open(buf, ORDWR);
}
```

⟨*function* reject 146b⟩≡ (301a)

```
/*
 *  reject a call, tell device the reason for the rejection
 */
int
reject(int ctl, char *dir, char *cause)
```

```
{
    char buf[Maxpath];
    char *num;
    long n;

    num = strrchr(dir, '/');
    if(num == 0)
        num = dir;
    else
        num++;
    snprint(buf, sizeof(buf), "reject %s %s", num, cause);
    n = strlen(buf);
    if(write(ctl, buf, n) != n)
        return -1;
    return 0;
}
```

18.4 dial()

⟨*function* dial 147a⟩≡ (316b)

```
int
dial(char *dest, char *local, char *dir, int *cfdp)
{
    return (*_dial)(dest, local, dir, cfdp);
}
```

⟨*global* _dial 147b⟩≡ (316b)

```
/*
 * the thread library can't cope with rfork(RFMEM|RFPROC),
 * so it must override this with a private version of dial.
 */
int (*_dial)(char *, char *, char *, int *) = dialimpl;
```

⟨*function* dialimpl 147c⟩≡ (316b)

```
/*
 *  the dialstring is of the form '[/net/]proto!dest'
 */
static int
dialimpl(char *dest, char *local, char *dir, int *cfdp)
{
    DS ds;
    int rv;
    char err[ERRMAX], alterr[ERRMAX];

    ds.local = local;
    ds.dir = dir;
    ds.cfdp = cfdp;

    _dial_string_parse(dest, &ds);
    if(ds.netdir)
        return csdial(&ds);

    ds.netdir = "/net";
    rv = csdial(&ds);
    if(rv >= 0)
        return rv;
    err[0] = '\0';
    errstr(err, sizeof err);
```

```
    if(strstr(err, "refused") != 0){
        werrstr("%s", err);
        return rv;
    }
    ds.netdir = "/net.alt";
    rv = csdial(&ds);
    if(rv >= 0)
        return rv;

    alterr[0] = 0;
    errstr(alterr, sizeof alterr);
    if(strstr(alterr, "translate") || strstr(alterr, "does not exist"))
        werrstr("%s", err);
    else
        werrstr("%s", alterr);
    return rv;
}
```

Chapter 19

Mathematics

19.1 Basics

⟨*function* abs 149a⟩≡ (212b)

```
int
abs(int a)
{
    if(a < 0)
        return -a;
    return a;
}
```

⟨*function* fabs 149b⟩≡ (222a)

```
double
fabs(double arg)
{

    if(arg < 0)
        return -arg;
    return arg;
}
```

⟨*function* floor 149c⟩≡ (222b)

```
/*
 * floor and ceil-- greatest integer <= arg
 * (resp least >=)
 */
double
floor(double d)
{
    double fract;

    if(d < 0) {
        fract = modf(-d, &d);
        if(fract != 0.0)
            d += 1;
        d = -d;
    } else
        modf(d, &d);
    return d;
}
```

⟨*function* ceil 149d⟩≡ (222b)

```
double
ceil(double d)
```

```
{
    return -floor(-d);
}
```

19.2 Integers

19.2.1 Parsing

⟨*function* atoi 150a⟩≡ (215e)

```
int
atoi(char *s)
{

    return atol(s);
}
```

⟨*function* atol 150b⟩≡ (215e)

```
long
atol(char *s)
{
    long n;
    int f, c;

    n = 0;
    f = 0;
    while(*s == ' ' || *s == '\t')
        s++;
    if(*s == '-' || *s == '+') {
        if(*s++ == '-')
            f = 1;
        while(*s == ' ' || *s == '\t')
            s++;
    }
    if(s[0]=='0' && s[1]) {
        if(s[1]=='x' || s[1]=='X'){
            s += 2;
            for(;;) {
                c = *s;
                if(c >= '0' && c <= '9')
                    n = n*16 + c - '0';
                else
                if(c >= 'a' && c <= 'f')
                    n = n*16 + c - 'a' + 10;
                else
                if(c >= 'A' && c <= 'F')
                    n = n*16 + c - 'A' + 10;
                else
                    break;
                s++;
            }
        } else
            while(*s >= '0' && *s <= '7')
                n = n*8 + *s++ - '0';
    } else
        while(*s >= '0' && *s <= '9')
            n = n*10 + *s++ - '0';
    if(f)
        n = -n;
```

```
    return n;
}
```

⟨*constant* LONG_MAX 151a⟩≡ (282d)

```
#define LONG_MAX    2147483647L
```

⟨*constant* LONG_MIN 151b⟩≡ (282d)

```
#define LONG_MIN    -2147483648L
```

⟨*function* strtol 151c⟩≡ (282d)

```
long
strtol(char *nptr, char **endptr, int base)
{
    char *p;
    long n, nn, m;
    int c, ovfl, v, neg, ndig;

    p = nptr;
    neg = 0;
    n = 0;
    ndig = 0;
    ovfl = 0;

    /*
     * White space
     */
    for(;; p++) {
        switch(*p) {
        case ' ':
        case '\t':
        case '\n':
        case '\f':
        case '\r':
        case '\v':
            continue;
        }
        break;
    }

    /*
     * Sign
     */
    if(*p=='-' || *p=='+')
        if(*p++ == '-')
            neg = 1;

    /*
     * Base
     */
    if(base==0) {
        base = 10;
        if(*p == '0') {
            base = 8;
            if(p[1]=='x' || p[1]=='X') {
                p += 2;
                base = 16;
            }
        }
    } else
    if(base==16 && *p=='0'){
```

```
        if(p[1]=='x' || p[1]=='X')
            p += 2;
    } else
    if(base<0 || 36<base)
        goto Return;

    /*
     * Non-empty sequence of digits
     */
    m = LONG_MAX/base;
    for(;; p++,ndig++){
        c = *p;
        v = base;
        if('0'<=c && c<='9')
            v = c - '0';
        else
        if('a'<=c && c<='z')
            v = c - 'a' + 10;
        else
        if('A'<=c && c<='Z')
            v = c - 'A' + 10;
        if(v >= base)
            break;
        if(n > m)
            ovfl = 1;
        nn = n*base + v;
        if(nn < n)
            ovfl = 1;
        n = nn;
    }

Return:
    if(ndig == 0)
        p = nptr;
    if(endptr)
        *endptr = p;
    if(ovfl){
        if(neg)
            return LONG_MIN;
        return LONG_MAX;
    }
    if(neg)
        return -n;
    return n;
}
```

19.3 Floats

19.3.1 Special numbers

⟨*function* NaN 152⟩≡ (235b)

```
double
NaN(void)
{
    FPdbleword a;

    a.hi = NANEXP;
    a.lo = 1;
```

```
    return a.x;
}
```

⟨*constant* NANEXP 153a⟩≡ (235b)

```
#define NANEXP  (2047<<20)
```

⟨*constant* NANMASK 153b⟩≡ (235b)

```
#define NANMASK (2047<<20)
```

⟨*function* isNaN 153c⟩≡ (235b)

```
bool
isNaN(double d)
{
    FPdbleword a;

    a.x = d;
    if((a.hi & NANMASK) != NANEXP)
        return false;
    return !isInf(d, 0);
}
```

⟨*constant* NANSIGN 153d⟩≡ (235b)

```
#define NANSIGN (1<<31)
```

⟨*function* Inf 153e⟩≡ (235b)

```
double
Inf(int sign)
{
    FPdbleword a;

    a.hi = NANEXP;
    a.lo = 0;
    if(sign < 0)
        a.hi |= NANSIGN;
    return a.x;
}
```

⟨*function* isInf 153f⟩≡ (235b)

```
bool
isInf(double d, int sign)
{
    FPdbleword a;

    a.x = d;
    if(a.lo != 0)
        return false;
    if(a.hi == NANEXP)
        return sign >= 0;
    if(a.hi == (NANEXP|NANSIGN))
        return sign <= 0;
    return false;
}
```

⟨*type* FPxxx 153g⟩≡ (194a)

```
/* VFP FCR */
#define FPINEX  (1<<12)         /* trap enables for exceptions */
#define FPUNFL  (1<<11)
#define FPOVFL  (1<<10)
#define FPZDIV  (1<<9)
#define FPINVAL (1<<8)
```

```
#define FPRNR   (0<<22)
#define FPRZ    (1<<22)
#define FPRPINF (2<<22)
#define FPRNINF (3<<22)
#define FPRMASK (3<<22)
#define FPPEXT  0
#define FPPSGL  0
#define FPPDBL  0
#define FPPMASK 0
/* FSR */
#define FPAINEX (1<<4)          /* accrued exceptions */
#define FPAUNFL (1<<3)
#define FPAOVFL (1<<2)
#define FPAZDIV (1<<1)
#define FPAINVAL    (1<<0)
```

19.3.2 Parsing

⟨*function* atof 154a⟩≡ (215d)

```
double
atof(char *cp)
{
    return strtod(cp, nil);
}
```

⟨*function* strtod 154b⟩≡ (281c)

```
double
strtod(char *as, char **aas)
{
    int na, ona, ex, dp, bp, c, i, flag, state;
    ulong low[Prec], hig[Prec], mid[Prec], num, den;
    double d;
    char *s, a[Ndig];

    flag = 0;   // Fsign, Fesign, Fdpoint
    na = 0;     // number of digits of a[]
    dp = 0;     // na of decimal point
    ex = 0;     // exonent

    state = S0;
    for(s=as;; s++) {
        c = *s;
        if(c >= '0' && c <= '9') {
            switch(state) {
            case S0:
            case S1:
            case S2:
                state = S2;
                break;
            case S3:
            case S4:
                state = S4;
                break;

            case S5:
            case S6:
            case S7:
                state = S7;
                ex = ex*10 + (c-'0');
```

```
            continue;
        }
        if(na == 0 && c == '0') {
            dp--;
            continue;
        }
        if(na < Ndig-50)
            a[na++] = c;
        continue;
    }
    switch(c) {
    case '\t':
    case '\n':
    case '\v':
    case '\f':
    case '\r':
    case ' ':
        if(state == S0)
            continue;
        break;
    case '-':
        if(state == S0)
            flag |= Fsign;
        else
            flag |= Fesign;
    case '+':
        if(state == S0)
            state = S1;
        else
        if(state == S5)
            state = S6;
        else
            break;  // syntax
        continue;
    case '.':
        flag |= Fdpoint;
        dp = na;
        if(state == S0 || state == S1) {
            state = S3;
            continue;
        }
        if(state == S2) {
            state = S4;
            continue;
        }
        break;
    case 'e':
    case 'E':
        if(state == S2 || state == S4) {
            state = S5;
            continue;
        }
        break;
    }
    break;
}

/*
 * clean up return char-pointer
 */
```

```
switch(state) {
case S0:
    if(xcmp(s, "nan") == 0) {
        if(aas != nil)
            *aas = s+3;
        goto retnan;
    }
case S1:
    if(xcmp(s, "infinity") == 0) {
        if(aas != nil)
            *aas = s+8;
        goto retinf;
    }
    if(xcmp(s, "inf") == 0) {
        if(aas != nil)
            *aas = s+3;
        goto retinf;
    }
case S3:
    if(aas != nil)
        *aas = as;
    goto ret0;  // no digits found
case S6:
    s--;        // back over +-
case S5:
    s--;        // back over e
    break;
}
if(aas != nil)
    *aas = s;

if(flag & Fdpoint)
while(na > 0 && a[na-1] == '0')
    na--;
if(na == 0)
    goto ret0;  // zero
a[na] = 0;
if(!(flag & Fdpoint))
    dp = na;
if(flag & Fesign)
    ex = -ex;
dp += ex;
if(dp < -Maxe-Nmant/3)  /* actually -Nmant*log(2)/log(10), but Nmant/3 close enough */
    goto ret0;  // underflow by exp
else
if(dp > +Maxe)
    goto retinf;    // overflow by exp

/*
 * normalize the decimal ascii number
 * to range .[5-9][0-9]* e0
 */
bp = 0;     // binary exponent
while(dp > 0)
    divascii(a, &na, &dp, &bp);
while(dp < 0 || a[0] < '5')
    mulascii(a, &na, &dp, &bp);
a[na] = 0;

/*
```

```
 * very small numbers are represented using
 * bp = -Bias+1.  adjust accordingly.
 */
if(bp < -Bias+1){
    ona = na;
    divby(a, &na, -bp-Bias+1);
    if(na < ona){
        memmove(a+ona-na, a, na);
        memset(a, '0', ona-na);
        na = ona;
    }
    a[na] = 0;
    bp = -Bias+1;
}

/* close approx by naive conversion */
num = 0;
den = 1;
for(i=0; i<9 && (c=a[i]); i++) {
    num = num*10 + (c-'0');
    den *= 10;
}
low[0] = umuldiv(num, One, den);
hig[0] = umuldiv(num+1, One, den);
for(i=1; i<Prec; i++) {
    low[i] = 0;
    hig[i] = One-1;
}

/* binary search for closest mantissa */
for(;;) {
    /* mid = (hig + low) / 2 */
    c = 0;
    for(i=0; i<Prec; i++) {
        mid[i] = hig[i] + low[i];
        if(c)
            mid[i] += One;
        c = mid[i] & 1;
        mid[i] >>= 1;
    }
    frnorm(mid);

    /* compare */
    c = fpcmp(a, mid);
    if(c > 0) {
        c = 1;
        for(i=0; i<Prec; i++)
            if(low[i] != mid[i]) {
                c = 0;
                low[i] = mid[i];
            }
        if(c)
            break;  // between mid and hig
        continue;
    }
    if(c < 0) {
        for(i=0; i<Prec; i++)
            hig[i] = mid[i];
        continue;
    }
```

```
        /* only hard part is if even/odd roundings wants to go up */
        c = mid[Prec-1] & (Sigbit-1);
        if(c == Sigbit/2 && (mid[Prec-1]&Sigbit) == 0)
            mid[Prec-1] -= c;
        break;  // exactly mid
    }

    /* normal rounding applies */
    c = mid[Prec-1] & (Sigbit-1);
    mid[Prec-1] -= c;
    if(c >= Sigbit/2) {
        mid[Prec-1] += Sigbit;
        frnorm(mid);
    }
    d = 0;
    for(i=0; i<Prec; i++)
        d = d*One + mid[i];
    if(flag & Fsign)
        d = -d;
    d = ldexp(d, bp - Prec*Nbits);
    return d;

ret0:
    return 0;

retnan:
    return NaN();

retinf:
    if(flag & Fsign)
        return Inf(-1);
    return Inf(+1);
}
```

⟨*enum* _anon_ (port/strtod.c) 158⟩≡ (281c)

```
/*
 * This routine will convert to arbitrary precision
 * floating point entirely in multi-precision fixed.
 * The answer is the closest floating point number to
 * the given decimal number. Exactly half way are
 * rounded ala ieee rules.
 * Method is to scale input decimal between .500 and .999...
 * with external power of 2, then binary search for the
 * closest mantissa to this decimal number.
 * Nmant is is the required precision. (53 for ieee dp)
 * Nbits is the max number of bits/word. (must be <= 28)
 * Prec is calculated - the number of words of fixed mantissa.
 */
enum
{
    Nbits   = 28,               // bits safely represented in a ulong
    Nmant   = 53,               // bits of precision required
    Bias        = 1022,
    Prec    = (Nmant+Nbits+1)/Nbits,    // words of Nbits each to represent mantissa
    Sigbit  = 1<<(Prec*Nbits-Nmant),    // first significant bit of Prec-th word
    Ndig    = 1500,
    One = (ulong)(1<<Nbits),
    Half    = (ulong)(One>>1),
    Maxe    = 310,
```

```
    Fsign   = 1<<0,     // found -
    Fesign  = 1<<1,     // found e-
    Fdpoint = 1<<2,     // found .

    S0  = 0,        // _         _S0 +S1 #S2 .S3
    S1,         // _+       #S2 .S3
    S2,         // _+#      #S2 .S4 eS5
    S3,         // _+.      #S4
    S4,         // _+#.#    #S4 eS5
    S5,         // _+#.#e   +S6 #S7
    S6,         // _+#.#e+  #S7
    S7,         // _+#.#e+# #S7
};
```

⟨*struct* Tab 159a⟩≡ (281c)

```
struct  Tab
{
    int bp;
    int siz;
    char*   cmp;
};
```

⟨*function* frnorm 159b⟩≡ (281c)

```
static void
frnorm(ulong *f)
{
    int i, c;

    c = 0;
    for(i=Prec-1; i>0; i--) {
        f[i] += c;
        c = f[i] >> Nbits;
        f[i] &= One-1;
    }
    f[0] += c;
}
```

⟨*function* fpcmp 159c⟩≡ (281c)

```
static int
fpcmp(char *a, ulong* f)
{
    ulong tf[Prec];
    int i, d, c;

    for(i=0; i<Prec; i++)
        tf[i] = f[i];

    for(;;) {
        /* tf *= 10 */
        for(i=0; i<Prec; i++)
            tf[i] = tf[i]*10;
        frnorm(tf);
        d = (tf[0] >> Nbits) + '0';
        tf[0] &= One-1;

        /* compare next digit */
        c = *a;
        if(c == 0) {
            if('0' < d)
                return -1;
```

```
            if(tf[0] != 0)
                goto cont;
            for(i=1; i<Prec; i++)
                if(tf[i] != 0)
                    goto cont;
            return 0;
        }
        if(c > d)
            return +1;
        if(c < d)
            return -1;
        a++;
    cont:;
    }
}
```

⟨*function* _divby 160⟩≡ (281c)

```
static void
_divby(char *a, int *na, int b)
{
    int n, c;
    char *p;

    p = a;
    n = 0;
    while(n>>b == 0) {
        c = *a++;
        if(c == 0) {
            while(n) {
                c = n*10;
                if(c>>b)
                    break;
                n = c;
            }
            goto xx;
        }
        n = n*10 + c-'0';
        (*na)--;
    }
    for(;;) {
        c = n>>b;
        n -= c<<b;
        *p++ = c + '0';
        c = *a++;
        if(c == 0)
            break;
        n = n*10 + c-'0';
    }
    (*na)++;
xx:
    while(n) {
        n = n*10;
        c = n>>b;
        n -= c<<b;
        *p++ = c + '0';
        (*na)++;
    }
    *p = 0;
}
```

⟨*function* divby 161a⟩≡ (281c)

```
static void
divby(char *a, int *na, int b)
{
    while(b > 9){
        _divby(a, na, 9);
        a[*na] = 0;
        b -= 9;
    }
    if(b > 0)
        _divby(a, na, b);
}
```

⟨*global* tab1 161b⟩≡ (281c)

```
static  Tab tab1[] =
{
     1,  0, "",
     3,  1, "7",
     6,  2, "63",
     9,  3, "511",
    13,  4, "8191",
    16,  5, "65535",
    19,  6, "524287",
    23,  7, "8388607",
    26,  8, "67108863",
    27,  9, "134217727",
};
```

⟨*function* divascii 161c⟩≡ (281c)

```
static void
divascii(char *a, int *na, int *dp, int *bp)
{
    int b, d;
    Tab *t;

    d = *dp;
    if(d >= nelem(tab1))
        d = nelem(tab1)-1;
    t = tab1 + d;
    b = t->bp;
    if(memcmp(a, t->cmp, t->siz) > 0)
        d--;
    *dp -= d;
    *bp += b;
    divby(a, na, b);
}
```

⟨*function* mulby 161d⟩≡ (281c)

```
static void
mulby(char *a, char *p, char *q, int b)
{
    int n, c;

    n = 0;
    *p = 0;
    for(;;) {
        q--;
        if(q < a)
            break;
        c = *q - '0';
```

```
        c = (c<<b) + n;
        n = c/10;
        c -= n*10;
        p--;
        *p = c + '0';
    }
    while(n) {
        c = n;
        n = c/10;
        c -= n*10;
        p--;
        *p = c + '0';
    }
}
```

⟨*global* tab2 162a⟩≡ (281c)

```
static  Tab tab2[] =
{
     1,  1, "",             // dp = 0-0
     3,  3, "125",
     6,  5, "15625",
     9,  7, "1953125",
    13, 10, "1220703125",
    16, 12, "152587890625",
    19, 14, "19073486328125",
    23, 17, "11920928955078125",
    26, 19, "1490116119384765625",
    27, 19, "7450580596923828125",      // dp 8-9
};
```

⟨*function* mulascii 162b⟩≡ (281c)

```
static void
mulascii(char *a, int *na, int *dp, int *bp)
{
    char *p;
    int d, b;
    Tab *t;

    d = -*dp;
    if(d >= nelem(tab2))
        d = nelem(tab2)-1;
    t = tab2 + d;
    b = t->bp;
    if(memcmp(a, t->cmp, t->siz) < 0)
        d--;
    p = a + *na;
    *bp -= b;
    *dp += d;
    *na += d;
    mulby(a, p+d, p, b);
}
```

⟨*function* xcmp 162c⟩≡ (281c)

```
static int
xcmp(char *a, char *b)
{
    int c1, c2;

    while(c1 = *b++) {
        c2 = *a++;
```

```
        if(isupper(c2))
            c2 = tolower(c2);
        if(c1 != c2)
            return 1;
    }
    return 0;
}
```

19.3.3 sqrt()

⟨*function* sqrt 163⟩≡ (276c)

```
double
sqrt(double arg)
{
    double x, temp;
    int exp, i;

    if(arg <= 0) {
        if(arg < 0)
            return NaN();
        return 0;
    }
    if(isInf(arg, 1))
        return arg;
    x = frexp(arg, &exp);
    while(x < 0.5) {
        x *= 2;
        exp--;
    }
    /*
     * NOTE
     * this wont work on 1's comp
     */
    if(exp & 1) {
        x *= 2;
        exp--;
    }
    temp = 0.5 * (1.0+x);

    while(exp > 60) {
        temp *= (1L<<30);
        exp -= 60;
    }
    while(exp < -60) {
        temp /= (1L<<30);
        exp += 60;
    }
    if(exp >= 0)
        temp *= 1L << (exp/2);
    else
        temp /= 1L << (-exp/2);
    for(i=0; i<=4; i++)
        temp = 0.5*(temp + arg/temp);
    return temp;
}
```

19.3.4 modf()

⟨*function* modf 164a⟩≡ (223b)

```
double
modf(double d, double *ip)
{
    FPdbleword x;
    int e;

    x.x = d;
    e = (x.hi >> SHIFT) & MASK;
    if(e == MASK){
        *ip = d;
        if(x.lo != 0 || (x.hi & 0xfffffL) != 0) /* NaN */
            return d;
        /* +/- Inf */
        x.hi &= 0x80000000L;
        return x.x;
    }
    if(d < 1) {
        if(d < 0) {
            x.x = modf(-d, ip);
            *ip = -*ip;
            return -x.x;
        }
        *ip = 0;
        return d;
    }
    e -= BIAS;
    if(e <= SHIFT+1) {
        x.hi &= ~(0x1fffffL >> e);
        x.lo = 0;
    } else
    if(e <= SHIFT+33)
        x.lo &= ~(0x7fffffffL >> (e-SHIFT-2));
    *ip = x.x;
    return d - x.x;
}
```

19.3.5 fmod()

⟨*function* fmod 164b⟩≡ (222c)

```
/*
 * floating-point mod function without infinity or NaN checking
 */
double
fmod (double x, double y)
{
    int sign, yexp, rexp;
    double r, yfr, rfr;

    if (y == 0)
        return x;
    if (y < 0)
        y = -y;
    yfr = frexp(y, &yexp);
    sign = 0;
    if(x < 0) {
        r = -x;
```

```
        sign++;
    } else
        r = x;
    while(r >= y) {
        rfr = frexp(r, &rexp);
        r -= ldexp(y, rexp - yexp - (rfr < yfr));
    }
    if(sign)
        r = -r;
    return r;
}
```

19.3.6 pow()

⟨*function* pow 165⟩≡ (263a)

```
double
pow(double x, double y) /* return x ^ y (exponentiation) */
{
    double xy, y1, ye;
    long i;
    int ex, ey, flip;

    if(y == 0.0)
        return 1.0;

    flip = 0;
    if(y < 0.){
        y = -y;
        flip = 1;
    }
    y1 = modf(y, &ye);
    if(y1 != 0.0){
        if(x <= 0.)
            goto zreturn;
        if(y1 > 0.5) {
            y1 -= 1.;
            ye += 1.;
        }
        xy = exp(y1 * log(x));
    }else
        xy = 1.0;
    if(ye > 0x7FFFFFFF){    /* should be ~0UL but compiler can't convert double to ulong */
        if(x <= 0.){
 zreturn:
            if(x==0. && !flip)
                return 0.;
            return NaN();
        }
        if(flip){
            if(y == .5)
                return 1./sqrt(x);
            y = -y;
        }else if(y == .5)
            return sqrt(x);
        return exp(y * log(x));
    }
    x = frexp(x, &ex);
    ey = 0;
    i = ye;
```

```
    if(i)
        for(;;){
            if(i & 1){
                xy *= x;
                ey += ex;
            }
            i >>= 1;
            if(i == 0)
                break;
            x *= x;
            ex <<= 1;
            if(x < .5){
                x += x;
                ex -= 1;
            }
        }
    if(flip){
        xy = 1. / xy;
        ey = -ey;
    }
    return ldexp(xy, ey);
}
```

⟨*function* pow10 166a⟩≡ (263b)

```
double
pow10(int n)
{
    int m;

    if(n < 0) {
        n = -n;
        if(n < sizeof(tab)/sizeof(tab[0]))
            return 1/tab[n];
        m = n/2;
        return 1/(pow10(m) * pow10(n-m));
    }
    if(n < sizeof(tab)/sizeof(tab[0]))
        return tab[n];
    m = n/2;
    return pow10(m) * pow10(n-m);
}
```

⟨*global* tab 166b⟩≡ (263b)

```
/*
 * this table might overflow 127-bit exponent representations.
 * in that case, truncate it after 1.0e38.
 * it is important to get all one can from this
 * routine since it is used in atof to scale numbers.
 * the presumption is that C converts fp numbers better
 * than multipication of lower powers of 10.
 */
static
double  tab[] =
{
    1.0e0,  1.0e1,  1.0e2,  1.0e3,  1.0e4,  1.0e5,  1.0e6,  1.0e7,  1.0e8,  1.0e9,
    1.0e10, 1.0e11, 1.0e12, 1.0e13, 1.0e14, 1.0e15, 1.0e16, 1.0e17, 1.0e18, 1.0e19,
    1.0e20, 1.0e21, 1.0e22, 1.0e23, 1.0e24, 1.0e25, 1.0e26, 1.0e27, 1.0e28, 1.0e29,
    1.0e30, 1.0e31, 1.0e32, 1.0e33, 1.0e34, 1.0e35, 1.0e36, 1.0e37, 1.0e38, 1.0e39,
    1.0e40, 1.0e41, 1.0e42, 1.0e43, 1.0e44, 1.0e45, 1.0e46, 1.0e47, 1.0e48, 1.0e49,
    1.0e50, 1.0e51, 1.0e52, 1.0e53, 1.0e54, 1.0e55, 1.0e56, 1.0e57, 1.0e58, 1.0e59,
```

```
    1.0e60, 1.0e61, 1.0e62, 1.0e63, 1.0e64, 1.0e65, 1.0e66, 1.0e67, 1.0e68, 1.0e69,
    1.0e70, 1.0e71, 1.0e72, 1.0e73, 1.0e74, 1.0e75, 1.0e76, 1.0e77, 1.0e78, 1.0e79,
    1.0e80, 1.0e81, 1.0e82, 1.0e83, 1.0e84, 1.0e85, 1.0e86, 1.0e87, 1.0e88, 1.0e89,
    1.0e90, 1.0e91, 1.0e92, 1.0e93, 1.0e94, 1.0e95, 1.0e96, 1.0e97, 1.0e98, 1.0e99,
    1.0e100,1.0e101,1.0e102,1.0e103,1.0e104,1.0e105,1.0e106,1.0e107,1.0e108,1.0e109,
    1.0e110,1.0e111,1.0e112,1.0e113,1.0e114,1.0e115,1.0e116,1.0e117,1.0e118,1.0e119,
    1.0e120,1.0e121,1.0e122,1.0e123,1.0e124,1.0e125,1.0e126,1.0e127,1.0e128,1.0e129,
    1.0e130,1.0e131,1.0e132,1.0e133,1.0e134,1.0e135,1.0e136,1.0e137,1.0e138,1.0e139,
    1.0e140,1.0e141,1.0e142,1.0e143,1.0e144,1.0e145,1.0e146,1.0e147,1.0e148,1.0e149,
    1.0e150,1.0e151,1.0e152,1.0e153,1.0e154,1.0e155,1.0e156,1.0e157,1.0e158,1.0e159,
};
```

19.4 Trigonometry

⟨*function* cos 167a⟩≡ (274c)

```
double
cos(double arg)
{
    if(arg < 0)
        arg = -arg;
    return sinus(arg, 1);
}
```

⟨*function* sin 167b⟩≡ (274c)

```
double
sin(double arg)
{
    return sinus(arg, 0);
}
```

⟨*constant p0 (port/sin.c)* 167c⟩≡ (274c)

```
#define p0      .135788409787737566909268 0e8
```

⟨*constant p1 (port/sin.c)* 167d⟩≡ (274c)

```
#define p1     -.494290810090284416115862 7e7
```

⟨*constant p2 (port/sin.c)* 167e⟩≡ (274c)

```
#define p2      .440103053537526650194491 8e6
```

⟨*constant p3 (port/sin.c)* 167f⟩≡ (274c)

```
#define p3     -.138472724998245287305445 7e5
```

⟨*constant p4 (port/sin.c)* 167g⟩≡ (274c)

```
#define p4      .145968840666576872222695 9e3
```

⟨*constant q0 (port/sin.c)* 167h⟩≡ (274c)

```
#define q0      .864455865292253442991514 9e7
```

⟨*constant q1 (port/sin.c)* 167i⟩≡ (274c)

```
#define q1      .408179225234329974939577 9e6
```

⟨*constant q2 (port/sin.c)* 167j⟩≡ (274c)

```
#define q2      .946309610153820818057125 7e4
```

⟨*constant q3 (port/sin.c)* 167k⟩≡ (274c)

```
#define q3      .132653490878613635891149 4e3
```

⟨*function* sinus 168a⟩≡ (274c)

```
static
double
sinus(double arg, int quad)
{
    double e, f, ysq, x, y, temp1, temp2;
    int k;

    x = arg;
    if(x < 0) {
        x = -x;
        quad += 2;
    }
    x *= 1/PIO2;    /* underflow? */
    if(x > 32764) {
        y = modf(x, &e);
        e += quad;
        modf(0.25*e, &f);
        quad = e - 4*f;
    } else {
        k = x;
        y = x - k;
        quad += k;
        quad &= 3;
    }
    if(quad & 1)
        y = 1-y;
    if(quad > 1)
        y = -y;

    ysq = y*y;
    temp1 = ((((p4*ysq+p3)*ysq+p2)*ysq+p1)*ysq+p0)*y;
    temp2 = ((((ysq+q3)*ysq+q2)*ysq+q1)*ysq+q0);
    return temp1/temp2;
}
```

⟨*global p0 (port/tan.c)* 168b⟩≡ (288b)

```
static double p0     = -0.1306820264754825668269611177e+5;
```

⟨*global p1 (port/tan.c)* 168c⟩≡ (288b)

```
static double p1      = 0.1055970901714953193602353981e+4;
```

⟨*global p2 (port/tan.c)* 168d⟩≡ (288b)

```
static double p2     = -0.1550685653483266376941705728e+2;
```

⟨*global p3 (port/tan.c)* 168e⟩≡ (288b)

```
static double p3      = 0.3422554387241003435328470489e-1;
```

⟨*global* p4 168f⟩≡ (288b)

```
static double p4      = 0.3386638642677172096076369e-4;
```

⟨*global q0 (port/tan.c)* 168g⟩≡ (288b)

```
static double q0     = -0.1663895238947119001851464661e+5;
```

⟨*global q1 (port/tan.c)* 168h⟩≡ (288b)

```
static double q1      = 0.4765751362916483698926655581e+4;
```

⟨*global q2 (port/tan.c)* 168i⟩≡ (288b)

```
static double q2     = -0.1555033164031709966900124574e+3;
```

⟨*function* tan 169a⟩≡ (288b)

```
double
tan(double arg)
{
    double temp, e, x, xsq;
    int flag, sign, i;

    flag = 0;
    sign = 0;
    if(arg < 0){
        arg = -arg;
        sign++;
    }
    arg = 2*arg/PIO2;   /* overflow? */
    x = modf(arg, &e);
    i = e;
    switch(i%4) {
    case 1:
        x = 1 - x;
        flag = 1;
        break;

    case 2:
        sign = !sign;
        flag = 1;
        break;

    case 3:
        x = 1 - x;
        sign = !sign;
        break;

    case 0:
        break;
    }

    xsq = x*x;
    temp = ((((p4*xsq+p3)*xsq+p2)*xsq+p1)*xsq+p0)*x;
    temp = temp/(((xsq+q2)*xsq+q1)*xsq+q0);

    if(flag) {
        if(temp == 0)
            return NaN();
        temp = 1/temp;
    }
    if(sign)
        temp = -temp;
    return temp;
}
```

⟨*function* asin 169b⟩≡ (212c)

```
double
asin(double arg)
{
    double temp;
    int sign;

    sign = 0;
    if(arg < 0) {
        arg = -arg;
```

```
        sign++;
    }
    if(arg > 1)
        return NaN();
    temp = sqrt(1 - arg*arg);
    if(arg > 0.7)
        temp = PIO2 - atan(temp/arg);
    else
        temp = atan(arg/temp);
    if(sign)
        temp = -temp;
    return temp;
}
```

⟨*function* acos 170a⟩≡ (212c)

```
double
acos(double arg)
{
    if(arg > 1 || arg < -1)
        return NaN();
    return PIO2 - asin(arg);
}
```

⟨*function* atan 170b⟩≡ (214a)

```
/*
    atan makes its argument positive and
    calls the inner routine satan.
 */

double
atan(double arg)
{

    if(arg > 0)
        return satan(arg);
    return -satan(-arg);
}
```

19.5 Logarithms and exponentials

19.5.1 exp()

⟨*function* exp 170c⟩≡ (221c)

```
double
exp(double arg)
{
    double fract, temp1, temp2, xsq;
    int ent;

    if(arg == 0)
        return 1;
    if(arg < -maxf)
        return 0;
    if(arg > maxf)
        return Inf(1);

    arg *= log2e;
    ent = floor(arg);
```

```
    fract = (arg-ent) - 0.5;
    xsq = fract*fract;
    temp1 = ((p2*xsq+p1)*xsq+p0)*fract;
    temp2 = ((xsq+q2)*xsq+q1)*xsq + q0;
    return ldexp(sqrt2*(temp2+temp1)/(temp2-temp1), ent);
}
```

⟨*constant* maxf 171a⟩≡ (221c)

```
#define maxf  10000
```

⟨*constant* log2e 171b⟩≡ (221c)

```
#define log2e  1.4426950408889634073599247
```

⟨*constant p0 (port/exp.c)* 171c⟩≡ (221c)

```
#define p0  .20803843466946630014438434 11e7
```

⟨*constant p1 (port/exp.c)* 171d⟩≡ (221c)

```
#define p1  .30286971697440362990760488 76e5
```

⟨*constant p2 (port/exp.c)* 171e⟩≡ (221c)

```
#define p2  .60614853300610808416155845 56e2
```

⟨*constant q0 (port/exp.c)* 171f⟩≡ (221c)

```
#define q0  .60027203602388325282309075 98e7
```

⟨*constant q1 (port/exp.c)* 171g⟩≡ (221c)

```
#define q1  .32772515180829144230579644 22e6
```

⟨*constant q2 (port/exp.c)* 171h⟩≡ (221c)

```
#define q2  .17492876890930764038449453 35e4
```

⟨*constant* sqrt2 171i⟩≡ (221c)

```
#define sqrt2  1.4142135623730950488016887
```

19.5.2 ldexp()

⟨*constant MASK (port/frexp.c)* 171j⟩≡ (223b)

```
/*
 * this is big/little endian non-portable
 * it gets the endian from the FPdbleword
 * union in u.h.
 */
#define MASK    0x7ffL
```

⟨*constant* SHIFT 171k⟩≡ (223b)

```
#define SHIFT   20
```

⟨*constant* BIAS 171l⟩≡ (223b)

```
#define BIAS    1022L
```

⟨*constant* SIG 171m⟩≡ (223b)

```
#define SIG 52
```

⟨*function* ldexp 172a⟩≡ (223b)

```
double
ldexp(double d, int deltae)
{
    int e, bits;
    FPdbleword x;
    ulong z;

    if(d == 0)
        return 0;
    x.x = d;
    e = (x.hi >> SHIFT) & MASK;
    if(deltae >= 0 || e+deltae >= 1){   /* no underflow */
        e += deltae;
        if(e >= MASK){      /* overflow */
            if(d < 0)
                return Inf(-1);
            return Inf(1);
        }
    }else{  /* underflow gracefully */
        deltae = -deltae;
        /* need to shift d right deltae */
        if(e > 1){      /* shift e-1 by exponent manipulation */
            deltae -= e-1;
            e = 1;
        }
        if(deltae > 0 && e==1){ /* shift 1 by switch from 1.fff to 0.1ff */
            deltae--;
            e = 0;
            x.lo >>= 1;
            x.lo |= (x.hi&1)<<31;
            z = x.hi & ((1<<SHIFT)-1);
            x.hi &= ~((1<<SHIFT)-1);
            x.hi |= (1<<(SHIFT-1)) | (z>>1);
        }
        while(deltae > 0){      /* shift bits down */
            bits = deltae;
            if(bits > SHIFT)
                bits = SHIFT;
            x.lo >>= bits;
            x.lo |= (x.hi&((1<<bits)-1)) << (32-bits);
            z = x.hi & ((1<<SHIFT)-1);
            x.hi &= ~((1<<SHIFT)-1);
            x.hi |= z>>bits;
            deltae -= bits;
        }
    }
    x.hi &= ~(MASK << SHIFT);
    x.hi |= (long)e << SHIFT;
    return x.x;
}
```

19.5.3 frexp()

⟨*function* frexp 172b⟩≡ (223b)

```
double
frexp(double d, int *ep)
{
    FPdbleword x, a;
```

```
    *ep = 0;
    /* order matters: only isNaN can operate on NaN */
    if(isNaN(d) || isInf(d, 0) || d == 0)
        return d;
    x.x = d;
    a.x = fabs(d);
    if((a.hi >> SHIFT) == 0){   /* normalize subnormal numbers */
        x.x = (double)(1ULL<<SIG) * d;
        *ep = -SIG;
    }
    *ep += ((x.hi >> SHIFT) & MASK) - BIAS;
    x.hi &= ~(MASK << SHIFT);
    x.hi |= BIAS << SHIFT;
    return x.x;
}
```

19.5.4 log()

⟨*function* log 173a⟩≡ (225e)

```
double
log(double arg)
{
    double x, z, zsq, temp;
    int exp;

    if(arg <= 0)
        return NaN();
    x = frexp(arg, &exp);
    while(x < 0.5) {
        x *= 2;
        exp--;
    }
    if(x < sqrto2) {
        x *= 2;
        exp--;
    }

    z = (x-1) / (x+1);
    zsq = z*z;

    temp = ((p3*zsq + p2)*zsq + p1)*zsq + p0;
    temp = temp/(((zsq + q2)*zsq + q1)*zsq + q0);
    temp = temp*z + exp*log2;
    return temp;
}
```

⟨*constant* log2 173b⟩≡ (225e)

```
#define log2    0.693147180559945309e0
```

⟨*constant* ln10o1 173c⟩≡ (225e)

```
#define ln10o1  .434294481903251827651 1
```

⟨*constant* sqrto2 173d⟩≡ (225e)

```
#define sqrto2  0.707106781186547524e0
```

⟨*constant p0 (port/log.c)* 173e⟩≡ (225e)

```
#define p0      -.240139179559210510e2
```

⟨*constant p1 (port/log.c)* 174a⟩≡ (225e)
```
#define p1      0.309572928215376501e2
```

⟨*constant p2 (port/log.c)* 174b⟩≡ (225e)
```
#define p2      -.963769093377840513e1
```

⟨*constant p3 (port/log.c)* 174c⟩≡ (225e)
```
#define p3      0.421087371217979714e0
```

⟨*constant q0 (port/log.c)* 174d⟩≡ (225e)
```
#define q0      -.120069589779605255e2
```

⟨*constant q1 (port/log.c)* 174e⟩≡ (225e)
```
#define q1      0.194809660700889731e2
```

⟨*constant q2 (port/log.c)* 174f⟩≡ (225e)
```
#define q2      -.891110902798312337e1
```

⟨*function* `log10` 174g⟩≡ (225e)
```
double
log10(double arg)
{

    if(arg <= 0)
        return NaN();
    return log(arg) * ln10o1;
}
```

19.6 Random numbers

⟨*function* `rand` 174h⟩≡ (269a)
```
int
rand(void)
{
    return lrand() & 0x7fff;
}
```

⟨*function* `lrand` 174i⟩≡ (227b)
```
long
lrand(void)
{
    ulong x;

    lock(&lk);

    rng_tap--;
    if(rng_tap < rng_vec) {
        if(rng_feed == 0) {
            isrand(1);
            rng_tap--;
        }
        rng_tap += LEN;
    }
    rng_feed--;
    if(rng_feed < rng_vec)
        rng_feed += LEN;
    x = (*rng_feed + *rng_tap) & MASK;
    *rng_feed = x;
```

```
    unlock(&lk);

    return x;
}
```

⟨*global* lk 175a⟩≡ (227b)

```
static  Lock    lk;
```

⟨*global* rng_vec 175b⟩≡ (227b)

```
static  ulong   rng_vec[LEN];
```

⟨*global* rng_tap 175c⟩≡ (227b)

```
static  ulong*  rng_tap = rng_vec;
```

⟨*global* rng_feed 175d⟩≡ (227b)

```
static  ulong*  rng_feed = 0;
```

⟨*function* truerand 175e⟩≡ (339b)

```
ulong
truerand(void)
{
    ulong x;
    static fdt randfd = -1;

    if(randfd < 0)
        randfd = open("/dev/random", OREAD|OCEXEC);
    if(randfd < 0)
        sysfatal("can't open /dev/random");
    if(read(randfd, &x, sizeof(x)) != sizeof(x))
        sysfatal("can't read /dev/random");

    return x;
}
```

19.7 muldiv

⟨*function* umuldiv 175f⟩≡ (235a)

```
ulong
umuldiv(ulong a, ulong b, ulong c)
{
    double d;

    d = ((double)a * (double)b) / (double)c;
    if(d >= 4294967296.)
        abort();
    return d;
}
```

⟨*function* muldiv 175g⟩≡ (235a)

```
long
muldiv(long a, long b, long c)
{
    int s;
    long v;

    s = 0;
    if(a < 0) {
```

```
        s = !s;
        a = -a;
    }
    if(b < 0) {
        s = !s;
        b = -b;
    }
    if(c < 0) {
        s = !s;
        c = -c;
    }
    v = umuldiv(a, b, c);
    if(s)
        v = -v;
    return v;
}
```

Chapter 20

Compression

Chapter 21

Error Management

⟨*constant* ERRMAX 178a⟩≡ (205)

```
#define ERRMAX 128 /* max length of error string */
```

21.1 Syscall: errstr()

21.2 rerrstr(), werrstr()

⟨*function* rerrstr 178b⟩≡ (333d)

```
void
rerrstr(char *buf, uint nbuf)
{
    char tmp[ERRMAX];

    tmp[0] = '\0';
    errstr(tmp, sizeof tmp);

    utfecpy(buf, buf+nbuf, tmp);
    errstr(tmp, sizeof tmp);
}
```

⟨*function* werrstr 178c⟩≡ (340b)

```
void
werrstr(char *fmt, ...)
{
    va_list arg;
    char buf[ERRMAX];

    va_start(arg, fmt);
    vseprint(buf, buf+ERRMAX, fmt, arg);
    va_end(arg);
    errstr(buf, sizeof buf);
}
```

21.3 perror()

⟨*function* perror 178d⟩≡ (237b)

```
void
perror(char *s)
{
    char buf[ERRMAX];
```

```
    buf[0] = '\0';
    errstr(buf, sizeof buf);
    if(s && *s)
        fprint(2, "%s: %s\n", s, buf);
    else
        fprint(2, "%s\n", buf);
}
```

21.4 Error return values

⟨*type* errorxxx 179a⟩≡ (194b)

```
// later: unify all of that to be more consistent!
typedef int error0; // 0 is the error value
typedef int error1; // 1 is the error value
typedef int errorneg1; // -1 is the error value
typedef int errorn; // 1 or more means error
```

⟨*constant* OKxxx 179b⟩≡ (194b)

```
#define OK_0 0
#define OK_1 1
```

⟨*constant* ERRORxxx 179c⟩≡ (194b)

```
#define ERROR_0 0
#define ERROR_1 1
#define ERROR_NEG1 (-1)
```

21.5 assert()

⟨*macro* assert 179d⟩≡ (194b)

```
#define assert(x)   do{ if(x) {} else _assert("x"); }while(0)
```

⟨*global* _assert 179e⟩≡ (212a)

```
void (*_assert)(char*) = default_assert;
```

⟨*function* default_assert 179f⟩≡ (212a)

```
void
default_assert(char *s)
{
    if(__assert)
        (*__assert)(s);
    fprint(2, "assert failed: %s\n", s);
    abort();
}
```

⟨*global* __assert 179g⟩≡ (212a)

```
void (*__assert)(char*);
```

⟨*function* abort 179h⟩≡ (298e)

```
void
abort(void)
{
    while(*(int*)0)
        ;
}
```

21.6 sysfatal()

⟨*function* sysfatal 180a⟩≡ (334c)

```
void
sysfatal(char *fmt, ...)
{
    va_list arg;

    va_start(arg, fmt);
    (*_sysfatal)(fmt, arg);
    va_end(arg);
}
```

⟨*global* _sysfatal 180b⟩≡ (334c)

```
void (*_sysfatal)(char *fmt, va_list arg) = _sysfatalimpl;
```

⟨*function* _sysfatalimpl 180c⟩≡ (334c)

```
static void
_sysfatalimpl(char *fmt, va_list arg)
{
    char buf[1024];

    vseprint(buf, buf+sizeof(buf), fmt, arg);
    if(argv0)
        fprint(2, "%s: %s\n", argv0, buf);
    else
        fprint(2, "%s\n", buf);
    exits(buf);
}
```

Chapter 22

Debugging Support

22.1 Logging

⟨*function* syslog 181⟩≡ (334d)

```
/*
 * Print
 *  sysname: time: mesg
 * on /sys/log/logname.
 * If cons or log file can't be opened, print on the system console, too.
 */
void
syslog(int cons, char *logname, char *fmt, ...)
{
    char buf[1024];
    char *ctim, *p;
    va_list arg;
    int n;
    Dir *d;
    char err[ERRMAX];

    err[0] = '\0';
    errstr(err, sizeof err);
    lock(&sl);

    /*
     *  paranoia makes us stat to make sure a fork+close
     *  hasn't broken our fd's
     */
    d = dirfstat(sl.fd);
    if(sl.fd < 0 || sl.name == nil || strcmp(sl.name, logname) != 0 ||
       !eqdirdev(d, sl.d)){
        free(sl.name);
        sl.name = strdup(logname);
        if(sl.name == nil)
            cons = 1;
        else{
            free(sl.d);
            sl.d = nil;
            _syslogopen();
            if(sl.fd < 0)
                cons = 1;
            else
                sl.d = dirfstat(sl.fd);
        }
    }
```

```
    free(d);
    if(cons){
        d = dirfstat(sl.consfd);
        if(sl.consfd < 0 || !eqdirdev(d, sl.consd)){
            free(sl.consd);
            sl.consd = nil;
            sl.consfd = open("#c/cons", OWRITE|OCEXEC);
            if(sl.consfd >= 0)
                sl.consd = dirfstat(sl.consfd);
        }
        free(d);
    }

    if(fmt == nil){
        unlock(&sl);
        return;
    }

    ctim = ctime(time(0));
    p = buf + snprint(buf, sizeof(buf)-1, "%s ", sysname());
    strncpy(p, ctim+4, 15);
    p += 15;
    *p++ = ' ';
    errstr(err, sizeof err);
    va_start(arg, fmt);
    p = vseprint(p, buf+sizeof(buf)-1, fmt, arg);
    va_end(arg);
    *p++ = '\n';
    n = p - buf;

    if(sl.fd >= 0){
        seek(sl.fd, 0, 2);
        write(sl.fd, buf, n);
    }

    if(cons && sl.consfd >=0)
        write(sl.consfd, buf, n);

    unlock(&sl);
}
```

⟨*global* sl 182a⟩≡ (334d)

```
static struct
{
    int fd;
    int consfd;
    char    *name;
    Dir *d;
    Dir *consd;
    Lock;
} sl =
{
    -1, -1,
};
```

⟨*function* _syslogopen 182b⟩≡ (334d)

```
static void
_syslogopen(void)
{
    char buf[1024];
```

```
    if(sl.fd >= 0)
        close(sl.fd);
    snprint(buf, sizeof(buf), "/sys/log/%s", sl.name);
    sl.fd = open(buf, OWRITE|OCEXEC);
}
```

⟨*function* eqdirdev 183⟩≡ (334d)

```
static int
eqdirdev(Dir *a, Dir *b)
{
    return a != nil && b != nil &&
        a->dev == b->dev && a->type == b->type &&
        a->qid.path == b->qid.path;
}
```

Chapter 23

Profiling Support

⟨*type* Prof 184a⟩≡ (194b)

```
enum Profiling {
    Profoff,        /* No profiling */

    Profuser,       /* Measure user time only (default) */
    Profkernel,     /* Measure user + kernel time */
    Proftime,       /* Measure total time */
    Profsample,     /* Use clock interrupt to sample (default when there is no cycle counter) */
}; /* what */
```

⟨*struct* Plink 184b⟩≡ (265)

```
struct  Plink
{
    Plink   *old;
    Plink   *down;
    Plink   *link;
    long    pc;
    long    count;
    vlong time;
};
```

⟨*function* prof 184c⟩≡ (265)

```
void
prof(void (*fn)(void*), void *arg, int entries, int what)
{
    _profinit(entries, what);
    _tos->prof.pp = _tos->prof.next;
    fn(arg);
    _profdump();
}
```

⟨*global* khz 184d⟩≡ (265)

```
static  ulong   khz;
```

⟨*global* perr 184e⟩≡ (265)

```
static  ulong   perr;
```

⟨*global* havecycles 184f⟩≡ (265)

```
static  int havecycles;
```

23.1 _profin()

⟨*function* _profin 185⟩≡ (265)

```
// Called at every procedure entry when use 5l -p, see Linker.nw
ulong
_profin(void)
{
    void *dummy;
    long pc;
    Plink *pp, *p;
    ulong arg;
    vlong t;

    arg = _savearg();
    pc = _callpc(&dummy);
    pp = _tos->prof.pp;
    if(pp == 0 || (_tos->prof.pid && _tos->pid != _tos->prof.pid))
        return arg;

    for(p=pp->down; p; p=p->link)
        if(p->pc == pc)
            goto out;
    p = _tos->prof.next + 1;
    if(p >= _tos->prof.last) {
        _tos->prof.pp = 0;
        perr++;
        return arg;
    }
    _tos->prof.next = p;
    p->link = pp->down;
    pp->down = p;
    p->pc = pc;
    p->old = pp;
    p->down = 0;
    p->count = 0;
    p->time = 0LL;

out:
    _tos->prof.pp = p;
    p->count++;
    switch(_tos->prof.what){
    case Profkernel:
        p->time = p->time - _tos->pcycles;
        goto proftime;
    case Profuser:
        /* Add kernel cycles on proc entry */
        p->time = p->time + _tos->kcycles;
        /* fall through */
    case Proftime:
    proftime:       /* Subtract cycle counter on proc entry */
        cycles((uvlong*)&t);
        p->time = p->time - t;
        break;
    case Profsample:
        p->time = p->time - _tos->clock;
        break;
    }
    return arg;     /* disgusting linkage */
}
```

23.2 _profout()

⟨*function* _profout 186a⟩≡ (265)

```
// Called at every procedure return when use 5l -p, see Linker.nw
ulong
_profout(void)
{
    Plink *p;
    ulong arg;
    vlong t;

    arg = _savearg();
    p = _tos->prof.pp;
    if (p == nil || (_tos->prof.pid != 0 && _tos->pid != _tos->prof.pid))
        return arg; /* Not our process */
    switch(_tos->prof.what){
    case Profkernel:        /* Add proc cycles on proc entry */
        p->time = p->time + _tos->pcycles;
        goto proftime;
    case Profuser:          /* Subtract kernel cycles on proc entry */
        p->time = p->time - _tos->kcycles;
        /* fall through */
    case Proftime:
    proftime:               /* Add cycle counter on proc entry */
        cycles((uvlong*)&t);
        p->time = p->time + t;
        break;
    case Profsample:
        p->time = p->time + _tos->clock;
        break;
    }
    _tos->prof.pp = p->old;
    return arg;
}
```

23.3 _profinit()

⟨*function* _profinit 186b⟩≡ (265)

```
void
_profinit(int entries, int what)
{
    if (_tos->prof.what == 0)
        return; /* Profiling not linked in */
    _tos->prof.pp = nil;
    _tos->prof.first = mallocz(entries*sizeof(Plink),1);
    _tos->prof.last = _tos->prof.first + entries;
    _tos->prof.next = _tos->prof.first;
    _tos->prof.pid = _tos->pid;
    _tos->prof.what = what;
    _tos->clock = 1;
}
```

23.4 _profmain()

⟨*function* _profmain 186c⟩≡ (265)

```
// called by _mainp in _main9p.s when 5l -p, see Linker
```

```
void
_profmain(void)
{
    char ename[50];
    int n, f;

    n = 2000;
    if (_tos->cyclefreq != 0LL){
        khz = _tos->cyclefreq / 1000;   /* Report times in milliseconds */
        havecycles = 1;
    }
    f = open("/env/profsize", OREAD);
    if(f >= 0) {
        memset(ename, 0, sizeof(ename));
        read(f, ename, sizeof(ename)-1);
        close(f);
        n = atol(ename);
    }
    _tos->prof.what = Profuser;
    f = open("/env/proftype", OREAD);
    if(f >= 0) {
        memset(ename, 0, sizeof(ename));
        read(f, ename, sizeof(ename)-1);
        close(f);
        if (strcmp(ename, "user") == 0)
            _tos->prof.what = Profuser;
        else if (strcmp(ename, "kernel") == 0)
            _tos->prof.what = Profkernel;
        else if (strcmp(ename, "elapsed") == 0 || strcmp(ename, "time") == 0)
            _tos->prof.what = Proftime;
        else if (strcmp(ename, "sample") == 0)
            _tos->prof.what = Profsample;
    }
    _tos->prof.first = sbrk(n*sizeof(Plink));
    _tos->prof.last = sbrk(0);
    _tos->prof.next = _tos->prof.first;
    _tos->prof.pp = nil;
    _tos->prof.pid = _tos->pid;
    atexit(_profdump);
    _tos->clock = 1;
}
```

23.5 cputime()

⟨*constant* HZ 187a⟩≡ (303b)

```
#define HZ  1000
```

⟨*function* cputime 187b⟩≡ (303b)

```
double
cputime(void)
{
    long t[4];
    int i;

    times(t);
    for(i=1; i<4; i++)
        t[0] += t[i];
    return t[0] / (double)HZ;
}
```

Chapter 24

Security

24.1 getuser()

⟨*function* getuser 188a⟩≡ (223f)

```
char*
getuser(void)
{
    static char user[64];
    fdt fd;
    int n;

    fd = open("/dev/user", OREAD);
    if(fd < 0)
        return "none";
    n = read(fd, user, (sizeof user)-1);
    close(fd);
    if(n <= 0)
        strcpy(user, "none");
    else
        user[n] = '\0';
    return user;
}
```

24.2 Cryptography

⟨*function* encrypt 188b⟩≡ (219d)

```
/*
 * destructively encrypt the buffer, which
 * must be at least 8 characters long.
 */
int
encrypt(void *key, void *vbuf, int n)
{
    ulong ekey[32];
    uchar *buf;
    int i, r;

    if(n < 8)
        return 0;
    key_setup(key, ekey);
    buf = vbuf;
    n--;
    r = n % 7;
```

```
    n /= 7;
    for(i = 0; i < n; i++){
        block_cipher(ekey, buf, 0);
        buf += 7;
    }
    if(r)
        block_cipher(ekey, buf - 7 + r, 0);
    return 1;
}
```

⟨*function* decrypt 189a⟩≡ (219d)

```
/*
 * destructively decrypt the buffer, which
 * must be at least 8 characters long.
 */
int
decrypt(void *key, void *vbuf, int n)
{
    ulong ekey[128];
    uchar *buf;
    int i, r;

    if(n < 8)
        return 0;
    key_setup(key, ekey);
    buf = vbuf;
    n--;
    r = n % 7;
    n /= 7;
    buf += n * 7;
    if(r)
        block_cipher(ekey, buf - 7 + r, 1);
    for(i = 0; i < n; i++){
        buf -= 7;
        block_cipher(ekey, buf, 1);
    }
    return 1;
}
```

⟨*function* netcrypt 189b⟩≡ (235e)

```
int
netcrypt(void *key, void *chal)
{
    uchar buf[8], *p;

    strncpy((char*)buf, chal, 7);
    buf[7] = '\0';
    for(p = buf; *p && *p != '\n'; p++)
        ;
    *p = '\0';
    encrypt(key, buf, 8);
    sprint(chal, "%.2ux%.2ux%.2ux%.2ux", buf[0], buf[1], buf[2], buf[3]);
    return 1;
}
```

24.3 SSL/TLS

⟨*function* pushssl 189c⟩≡ (329c)

```
/*
 * Since the SSL device uses decimal file descriptors to name channels,
 * it is impossible for a user-level file server to stand in for the kernel device.
 * Thus we hard-code #D rather than use /net/ssl.
 */

int
pushssl(int fd, char *alg, char *secin, char *secout, int *cfd)
{
    char buf[8];
    char dname[64];
    int n, data, ctl;

    ctl = open("#D/ssl/clone", ORDWR);
    if(ctl < 0)
        return -1;
    n = read(ctl, buf, sizeof(buf)-1);
    if(n < 0)
        goto error;
    buf[n] = 0;
    sprint(dname, "#D/ssl/%s/data", buf);
    data = open(dname, ORDWR);
    if(data < 0)
        goto error;
    if(fprint(ctl, "fd %d", fd) < 0 ||
       fprint(ctl, "secretin %s", secin) < 0 ||
       fprint(ctl, "secretout %s", secout) < 0 ||
       fprint(ctl, "alg %s", alg) < 0){
        close(data);
        goto error;
    }
    close(fd);
    if(cfd != 0)
        *cfd = ctl;
    else
        close(ctl);
    return data;
error:
    close(ctl);
    return -1;
}
```

⟨*function* pushtls 190⟩≡ (330b)

```
// given a plain fd and secrets established beforehand, return encrypted connection
int
pushtls(int fd, char *hashalg, char *encalg, int isclient, char *secret, char *dir)
{
    char buf[8];
    char dname[64];
    int n, data, ctl, hand;

    // open a new filter; get ctl fd
    data = hand = -1;
    // /net/tls uses decimal file descriptors to name channels, hence a
    // user-level file server can't stand in for #a; may as well hard-code it.
    ctl = open("#a/tls/clone", ORDWR);
    if(ctl < 0)
        goto error;
    n = read(ctl, buf, sizeof(buf)-1);
    if(n < 0)
```

```
        goto error;
    buf[n] = 0;
    if(dir)
        sprint(dir, "#a/tls/%s", buf);

    // get application fd
    sprint(dname, "#a/tls/%s/data", buf);
    data = open(dname, ORDWR);
    if(data < 0)
        goto error;

    // get handshake fd
    sprint(dname, "#a/tls/%s/hand", buf);
    hand = open(dname, ORDWR);
    if(hand < 0)
        goto error;

    // speak a minimal handshake
    if(fprint(ctl, "fd %d 0x301", fd) < 0 ||
       fprint(ctl, "version 0x301") < 0 ||
       fprint(ctl, "secret %s %s %d %s", hashalg, encalg, isclient, secret) < 0 ||
       fprint(ctl, "changecipher") < 0 ||
       finished(hand, isclient) < 0 ||
       fprint(ctl, "opened") < 0){
        close(hand);
        hand = -1;
        goto error;
    }
    close(ctl);
    close(hand);
    close(fd);
    return data;

error:
    if(data>=0)
        close(data);
    if(ctl>=0)
        close(ctl);
    if(hand>=0)
        close(hand);
    return -1;
}
```

Chapter 25

Advanced Topics

25.1 Arbitrary precision arithmetics

⟨*type* mpdigit 192a⟩≡ (194a)

```
typedef unsigned int    mpdigit;    /* for /sys/include/mp.h */
```

25.2 IO chunks

⟨*type* IOchunk 192b⟩≡ (205)

```
struct IOchunk {
 void *addr;
 ulong len;
};
```

25.3 Extra functions

25.3.1 Strings

Chapter 26

Conclusion

Appendix A

Extra Code

A.1 include/

A.1.1 include/arm/u.h

⟨include/arch/arm/u.h 194a⟩≡

```
⟨type uxxx 16b⟩

⟨type uxxxint 16c⟩

⟨type xxxvlong 16d⟩

typedef signed char schar;
⟨type usize 16e⟩

⟨constant nil 18l⟩
⟨type uintptr 18m⟩

⟨type Rune 18e⟩
⟨type mpdigit 192a⟩

⟨type jmpbufxxx 67c⟩
⟨type jmp_buf 67b⟩

⟨type FPxxx 153g⟩
⟨type FPdbleword 16f⟩
typedef     union FPdbleword FPdbleword;

⟨type va_list 21b⟩

⟨macro va_start 21c⟩
⟨macro va_end 21d⟩
⟨macro va_arg 21e⟩
```

A.1.2 include/core/libc.h

⟨include/core/libc.h 194b⟩≡

```
#pragma lib "libc.a"
#pragma src "/sys/src/libc"

// ---------------------------------------------
// pad's stuff
```

```
// -------------------------------------------
// More types! Types are good!
⟨type bool 16a⟩

⟨type byte 18n⟩

typedef uchar bool_byte;

//typedef char* string; // conflict
//typedef char* filename; // conflict in sam with function filename

⟨constant STDxxx 21g⟩
⟨type fdt 21f⟩

⟨constant OKxxx 179b⟩
⟨constant ERRORxxx 179c⟩
⟨type errorxxx 179a⟩

// -------------------------------------------
// end pad's stuff
// -------------------------------------------

⟨function nelem 19a⟩
⟨function offsetof 21a⟩

typedef struct Fmt Fmt;
typedef struct Tm Tm;
typedef struct Lock Lock;
typedef struct QLp QLp;
typedef struct QLock QLock;
typedef struct RWLock RWLock;
typedef struct Rendez Rendez;
typedef struct NetConnInfo NetConnInfo;
typedef struct Qid Qid;
typedef struct Dir Dir;
typedef struct Waitmsg Waitmsg;
typedef struct IOchunk IOchunk;

/*
 * mem routines
 */
extern  void*   memset(void*, int, ulong);
extern  void*   memcpy(void*, void*, ulong);
extern  int     memcmp(void*, void*, ulong);
extern  void*   memchr(void*, int, ulong);

// less useful?
extern  void*   memmove(void*, void*, ulong);
extern  void*   memccpy(void*, void*, int, ulong);



/*
 * string routines
 */
// memxxx equivalent, but with special handling for '\0' (no need pass ulong)
extern  char*   strcpy(char*, char*);
extern  int     strcmp(char*, char*);
extern  char*   strchr(char*, int);
extern  char*   strrchr(char*, int);
```

```
extern  long    strlen(char*);
extern  char*   strdup(char*);
extern  char*   strcat(char*, char*);
extern  char*   strstr(char*, char*);

extern  int tolower(int);
extern  int toupper(int);

// less useful?
extern  char*   strecpy(char*, char*, char*);
extern  char*   strncat(char*, char*, long);
extern  char*   strncpy(char*, char*, long);
extern  int     strncmp(char*, char*, long);

extern  char*   strpbrk(char*, char*);

extern  long    strspn(char*, char*);
extern  long    strcspn(char*, char*);

extern  int     cistrncmp(char*, char*, int);
extern  int     cistrcmp(char*, char*);
extern  char*   cistrstr(char*, char*);

extern  char*   strtok(char*, char*);
extern  int     tokenize(char*, char**, int);

enum
{
    ⟨constant UTFmax 18h⟩
    ⟨constant Runesync 18i⟩
    ⟨constant Runeself 18j⟩
    ⟨constant Runeerror 18k⟩
    ⟨constant Runemax 18f⟩
    ⟨constant Runemask 18g⟩
};

/*
 * rune routines
 */

// char <-> rune conversion
extern  int chartorune(Rune*, char*);
extern  int runetochar(char*, Rune*);

// strxxx equivalent for rune
extern  Rune*   runestrcpy(Rune*, Rune*);
extern  int     runestrcmp(Rune*, Rune*);
extern  Rune*   runestrchr(Rune*, Rune);
extern  Rune*   runestrrchr(Rune*, Rune);

extern  long    runestrlen(Rune*);
extern  Rune*   runestrdup(Rune*);
extern  Rune*   runestrcat(Rune*, Rune*);
extern  Rune*   runestrstr(Rune*, Rune*);

// less useful
extern  Rune*   runestrecpy(Rune*, Rune*, Rune*);
extern  Rune*   runestrncpy(Rune*, Rune*, long);
extern  Rune*   runestrncat(Rune*, Rune*, long);
```

```
extern  int     runestrncmp(Rune*, Rune*, long);

extern  Rune    tolowerrune(Rune);
extern  Rune    toupperrune(Rune);

extern  int isalpharune(Rune);
extern  int isdigitrune(Rune);
extern  int islowerrune(Rune);
extern  int isupperrune(Rune);
extern  int isspacerune(Rune);

// ??
extern  Rune    totitlerune(Rune);
extern  Rune    tobaserune(Rune);
extern  int istitlerune(Rune);
extern  int isbaserune(Rune);

// ?????
extern  int runelen(long);
extern  int runenlen(Rune*, int);
extern  int fullrune(char*, int);

// strxxx equivalent for utf
extern  int utflen(char*);
extern  char*   utfrune(char*, long);

// less useful?
extern  int utfnlen(char*, long);
extern  char*   utfrrune(char*, long);
extern  char*   utfutf(char*, char*);
extern  char*   utfecpy(char*, char*, char*);

/*
 * malloc
 */
extern  void*   malloc(ulong);
extern  void    free(void*);

// less useful
extern  void*   mallocz(ulong, bool);
extern  void*   calloc(ulong, ulong);
extern  void*   realloc(void*, ulong);
extern  ulong   msize(void*);
extern  void*   mallocalign(ulong, ulong, long, ulong);

// internals
extern  void    setmalloctag(void*, ulong);
extern  void    setrealloctag(void*, ulong);
extern  ulong   getmalloctag(void*);
extern  ulong   getrealloctag(void*);
extern  void*   malloctopoolblock(void*);

/*
 * print routines
 */
```
⟨*type* Fmt 42a⟩

⟨*type* Fmt_flag 42b⟩

```
// pad: used to be just print()? but for cg transformed in a pointer func
```

```
extern  int     (*print)(char*, ...);
extern  int     sprint(char*, char*, ...);
extern  int     fprint(fdt, char*, ...);

extern  int     vfprint(fdt, char*, va_list);
extern  char*   vseprint(char*, char*, char*, va_list);

extern  int     runesprint(Rune*, char*, ...);

// less useful
extern  char*   seprint(char*, char*, char*, ...);
extern  int     snprint(char*, int, char*, ...);
extern  char*   smprint(char*, ...);

extern  int     vsnprint(char*, int, char*, va_list);
extern  char*   vsmprint(char*, va_list);

extern  Rune*   runeseprint(Rune*, Rune*, char*, ...);
extern  int     runesnprint(Rune*, int, char*, ...);
extern  Rune*   runesmprint(char*, ...);

extern  Rune*   runevseprint(Rune*, Rune*, char*, va_list);
extern  int     runevsnprint(Rune*, int, char*, va_list);
extern  Rune*   runevsmprint(char*, va_list);

extern  int     fmtfdinit(Fmt*, int, char*, int);

extern  int     fmtfdflush(Fmt*);
extern  int     fmtstrinit(Fmt*);
extern  char*   fmtstrflush(Fmt*);
extern  int     runefmtstrinit(Fmt*);
extern  Rune*   runefmtstrflush(Fmt*);

#pragma varargck    argpos  fmtprint    2
#pragma varargck    argpos  fprint      2
#pragma varargck    argpos  print       1
#pragma varargck    argpos  runeseprint 3
#pragma varargck    argpos  runesmprint 1
#pragma varargck    argpos  runesnprint 3
#pragma varargck    argpos  runesprint  2
#pragma varargck    argpos  seprint     3
#pragma varargck    argpos  smprint     1
#pragma varargck    argpos  snprint     3
#pragma varargck    argpos  sprint      2

// %d = decimal, o = octal, x = hexa, b = binary?
#pragma varargck    type    "d" int
#pragma varargck    type    "o" int
#pragma varargck    type    "x" int
#pragma varargck    type    "b" int
// %f
#pragma varargck    type    "f" double
#pragma varargck    type    "e" double
#pragma varargck    type    "g" double
// %s
#pragma varargck    type    "s" char*
#pragma varargck    type    "q" char*
#pragma varargck    type    "S" Rune*
#pragma varargck    type    "Q" Rune*
#pragma varargck    type    "r" void
```

```
#pragma varargck    type    "%" void
#pragma varargck    type    "n" int*
#pragma varargck    type    "p" uintptr
#pragma varargck    type    "p" void*
#pragma varargck    type    "c" int
#pragma varargck    type    "C" int
#pragma varargck    type    "d" uint
#pragma varargck    type    "x" uint
#pragma varargck    type    "b" uint
#pragma varargck    type    "c" uint
#pragma varargck    type    "C" uint
#pragma varargck    type    "<" void*
#pragma varargck    type    "[" void*
#pragma varargck    type    "H" void*
#pragma varargck    type    "lH"    void*

#pragma varargck    type    "lld"   vlong
#pragma varargck    type    "llo"   vlong
#pragma varargck    type    "llx"   vlong
#pragma varargck    type    "llb"   vlong

#pragma varargck    type    "lld"   uvlong
#pragma varargck    type    "llo"   uvlong
#pragma varargck    type    "llx"   uvlong
#pragma varargck    type    "llb"   uvlong

#pragma varargck    type    "ld"    long
#pragma varargck    type    "lo"    long
#pragma varargck    type    "lx"    long
#pragma varargck    type    "lb"    long

#pragma varargck    type    "ld"    ulong
#pragma varargck    type    "lo"    ulong
#pragma varargck    type    "lx"    ulong
#pragma varargck    type    "lb"    ulong


#pragma varargck    flag    ','
#pragma varargck    flag    ' '
#pragma varargck    flag    'h'


extern  int fmtinstall(int, int (*)(Fmt*));

extern  int dofmt(Fmt*, char*);
extern  int dorfmt(Fmt*, Rune*);
extern  int fmtprint(Fmt*, char*, ...);
extern  int fmtvprint(Fmt*, char*, va_list);
extern  int fmtrune(Fmt*, int);
extern  int fmtstrcpy(Fmt*, char*);
extern  int fmtrunestrcpy(Fmt*, Rune*);
/*
 * error string for %r
 * supplied on per os basis, not part of fmt library
 */
extern  int errfmt(Fmt *f);

/*
 * quoted strings
 */
```

```
extern  char*   unquotestrdup(char*);
extern  Rune*   unquoterunestrdup(Rune*);
extern  char*   quotestrdup(char*);
extern  Rune*   quoterunestrdup(Rune*);
extern  int     quotestrfmt(Fmt*);
extern  int     quoterunestrfmt(Fmt*);
extern  void    quotefmtinstall(void);
extern  int     (*doquote)(int);
extern  int     needsrcquote(int);

/*
 * random number
 */
extern  void    srand(long);

extern  int     rand(void);
extern  double  frand(void);
extern  int     nrand(int);
extern  ulong   truerand(void);         /* uses /dev/random */

// less useful
extern  long    lrand(void);
extern  long    lnrand(long);
extern  ulong   ntruerand(ulong);       /* uses /dev/random */

/*
 * math
 */
extern  int     abs(int);
//extern  long    labs(long);
extern  double  frexp(double, int*);
extern  double  ldexp(double, int);
extern  double  modf(double, double*);
extern  double  pow10(int);

#define HUGE    3.4028234e38

extern  double  NaN(void);
extern  double  Inf(int);
extern  int     isNaN(double);
extern  int     isInf(double, int);

extern  double  fabs(double);
extern  double  floor(double);
extern  double  ceil(double);

extern  double  pow(double, double);
extern  double  log(double);
extern  double  log10(double);
extern  double  exp(double);
extern  double  sqrt(double);
extern  double  hypot(double, double);
extern  double  fmod(double, double);

#define PIO2    1.57079632679489661923 1e0
#define PI  (PIO2+PIO2)

extern  double  sin(double);
extern  double  cos(double);
extern  double  tan(double);
```

```
extern  double  asin(double);
extern  double  acos(double);
extern  double  atan(double);
extern  double  atan2(double, double);
extern  double  sinh(double);
extern  double  cosh(double);
extern  double  tanh(double);


// conversion
extern  double  atof(char*);
extern  int     atoi(char*);
extern  long    atol(char*);
extern  vlong   atoll(char*);

extern  double  strtod(char*, char**);
extern  long    strtol(char*, char**, int);
extern  ulong   strtoul(char*, char**, int);
extern  vlong   strtoll(char*, char**, int);
extern  uvlong  strtoull(char*, char**, int);

// internals?
extern  ulong   getfcr(void);
extern  void    setfsr(ulong);
extern  ulong   getfsr(void);
extern  void    setfcr(ulong);

extern  ulong   umuldiv(ulong, ulong, ulong);
extern  long    muldiv(long, long, long);



/*
 * Time-of-day
 */
```
⟨*type* Tm 22⟩

```
extern  long    time(long*);

extern  double  cputime(void);
extern  vlong   nsec(void);

extern  Tm*     gmtime(long);
extern  Tm*     localtime(long);

extern  long    tm2sec(Tm*);

// less useful?
extern  char*   asctime(Tm*);
extern  char*   ctime(long);
extern  long    times(long*);

extern  void    cycles(uvlong*);    /* 64-bit value of the cycle counter if there is one, 0 if there isn't */



/*
 * one-of-a-kind
 */
```

```
// misc
extern  double  charstod(int(*)(void*), void*);

// modified in place, so type should really be void cleanname(INOUT char*);
extern  char*   cleanname(char*);
extern  int     encodefmt(Fmt*);

extern  int     getfields(char*, char**, int, int, char*);
extern  int     gettokens(char *, char **, int, char *);

extern  int     iounit(fdt);

// ugly redefined by user code? see statusbar.c
extern  void    qsort(void*, long, long, int (*)(void*, void*));


/*
 | debugging tools
 */

⟨macro assert 179d⟩
extern  void    (*_assert)(char*);
extern  void    perror(char*);
extern  void    sysfatal(char*, ...);
extern  void    syslog(int, char*, char*, ...);

#pragma varargck    argpos  sysfatal    1
#pragma varargck    argpos  syslog  3

extern  uintptr getcallerpc(void*);

/*
 *  profiling
 */
⟨type Prof 184a⟩

extern  void    prof(void (*fn)(void*), void *arg, int entries, int what);


/*
 | concurrency
 */

extern  int     setjmp(jmp_buf);
extern  void    longjmp(jmp_buf, int);
extern  void    notejmp(void*, jmp_buf, int);

// IPC
⟨type PostnoteKind 71b⟩
extern  int     postnote(int, int, char *);
extern  int     atnotify(int(*)(void*, char*), int);

extern  int     atexit(void(*)(void));
extern  void    atexitdont(void(*)(void));

/*
 * atomic
 */
extern long    ainc(long*);
extern long    adec(long*);
```

```
extern int     cas32(u32int*, u32int, u32int);
extern int     casp(void**, void*, void*);
extern int     casl(ulong*, ulong, ulong);

/*
 *  synchronization
 */
```
⟨*type* Lock 60a⟩

```
extern int  _tas(int*);

extern  void    lock(Lock*);
extern  void    unlock(Lock*);
extern  int     canlock(Lock*);
```

⟨*type* QLp 61b⟩

⟨*type* QLock 61a⟩

```
extern  void    qlock(QLock*);
extern  void    qunlock(QLock*);
extern  int     canqlock(QLock*);

extern  void    _qlockinit(void* (*)(void*, void*));    /* called only by the thread library */
```

⟨*type* RWLock 62c⟩

```
extern  void    rlock(RWLock*);
extern  void    runlock(RWLock*);
extern  int     canrlock(RWLock*);
extern  void    wlock(RWLock*);
extern  void    wunlock(RWLock*);
extern  int     canwlock(RWLock*);
```

⟨*type* Rendez 65a⟩

```
extern  void    rsleep(Rendez*);    /* unlocks r->l, sleeps, locks r->l again */
extern  int     rwakeup(Rendez*);

extern  int     rwakeupall(Rendez*);
extern  void**  privalloc(void);
extern  void    privfree(void**);

/*
 *  network dialing
 */
#define NETPATHLEN 40
extern  int     accept(int, char*);
extern  int     announce(char*, char*);
extern  int     dial(char*, char*, char*, int*);
extern  void    setnetmtpt(char*, int, char*);
extern  int     hangup(int);
extern  int     listen(char*, char*);
extern  char*   netmkaddr(char*, char*, char*);
extern  int     reject(int, char*, char*);

/*
 *  network services
```

```
 */
⟨type NetConnInfo 143a⟩
extern  NetConnInfo*    getnetconninfo(char*, int);
extern  void            freenetconninfo(NetConnInfo*);


/*
 *  encryption
 */
extern  int pushssl(int, char*, char*, char*, int*);
extern  int pushtls(int, char*, char*, int, char*, char*);

// encryption
extern  int decrypt(void*, void*, int);
extern  int encrypt(void*, void*, int);
extern  int netcrypt(void*, void*);

extern  int dec64(uchar*, int, char*, int);
extern  int enc64(char*, int, uchar*, int);
extern  int dec32(uchar*, int, char*, int);
extern  int enc32(char*, int, uchar*, int);
extern  int dec16(uchar*, int, char*, int);
extern  int enc16(char*, int, uchar*, int);


/*
 * system calls
 *
 */
#include <syscall.h>



// getopt like macros
⟨signature global argv0 23a⟩
⟨macro ARGBEGIN 23b⟩
⟨macro ARGEND 23c⟩
⟨macro ARGF 23d⟩
⟨macro EARGF 23e⟩
⟨macro ARGC 23f⟩


/* this is used by sbrk and brk,  it's a really bad idea to redefine it */
extern  char    end[];
```

A.1.3 include/core/ctype.h

```
⟨include/core/ctype.h 204⟩≡
  #pragma src "/sys/src/libc/port"
  #pragma lib "libc.a"

  // could be merged in libc.h

  ⟨type Ctype_flag 17a⟩

  extern unsigned char _ctype[];

  ⟨macros isxxx 17c⟩
```

⟨*macro* _toupper 18a⟩
⟨*macro* _tolower 18c⟩
⟨*macro* toascii 18d⟩

A.1.4 include/core/syscall.h

⟨include/core/syscall.h 205⟩≡

```
// you must include libc.h instead of this file

//-----------------------------------------------------------------------------
// Data structures and constants
//-----------------------------------------------------------------------------

⟨type Namespace_flag 53⟩

⟨type Open_flag 44a⟩

⟨type Access_flag 44b⟩

⟨type Segattach_flag 73a⟩

⟨type Note_flag 71a⟩

⟨type Qid_type 44d⟩

⟨type Dir_mode 46b⟩

⟨type Rfork_flag 58a⟩

⟨constant STATMAX 46d⟩

⟨constant DIRMAX 46c⟩

⟨constant ERRMAX 178a⟩

⟨type Seek_cursor 45a⟩

⟨type Qid 44c⟩

⟨type DirEntry 46a⟩

⟨type Waitmsg 69a⟩

⟨type IOchunk 192b⟩

//-----------------------------------------------------------------------------
// Functions
//-----------------------------------------------------------------------------

// syscalls (and small wrappers around syscalls)

// process
extern int  rfork(int);
extern void _exits(char*);
extern int  exec(char*, char*[]);
extern int  await(char*, int);

extern int  fork(void);
extern  void    exits(char*);
```

```
extern int  execl(char*, ...);
extern void abort(void);
extern Waitmsg* wait(void);
extern int  waitpid(void);

extern int  getpid(void);
extern int  getppid(void);
extern  char*   getenv(char*);
extern  int     putenv(char*, char*);

// memory
extern void* sbrk(ulong);

// file
extern fdt  open(char*, int);
extern int  close(fdt);
extern long pread(fdt, void*, long, vlong);
extern long pwrite(fdt, void*, long, vlong);

extern long read(fdt, void*, long);
extern long write(fdt, void*, long);
extern long preadv(fdt, IOchunk*, int, vlong);
extern long pwritev(fdt, IOchunk*, int, vlong);
extern long readv(fdt, IOchunk*, int);
extern long writev(fdt, IOchunk*, int);
extern vlong seek(fdt, vlong, int);
// extern int fdflush(int);

extern long readn(fdt, void*, long);

// directory
extern int  create(char*, int, ulong);
extern int  remove(char*);
extern int  chdir(char*);
extern int  fd2path(fdt, char*, int);
extern int  fstat(int, uchar*, int);
extern int  stat(char*, uchar*, int);
extern int  fwstat(int, uchar*, int);
extern int  wstat(char*, uchar*, int);

extern  char*   getwd(char*, int);
extern  char*   mktemp(char*);

extern Dir* dirfstat(fdt);
extern Dir* dirstat(char*);
extern int  dirfwstat(int, Dir*);
extern int  dirwstat(char*, Dir*);
//
extern long dirread(int, Dir**);
extern void nulldir(Dir*);
extern long dirreadall(int, Dir**);
extern int  access(char*, int); // ???

// namespace
extern int  bind(char*, char*, int/*Mxxx*/);
extern int  mount(fdt, int, char*, int/*Mxxx*/, char*);
extern int  unmount(char*, char*);

// time
extern long alarm(ulong);
```

```
extern int  sleep(long); //less: could be void (ulong). 0 means yield.

// IPC
extern int  pipe(int*);
extern int  noted(int);
extern int  notify(void(*)(void*, char*));
extern void* segattach(int, char*, void*, ulong);
extern void* segbrk(void*, void*);
extern int  segdetach(void*);
extern int  segflush(void*, ulong);
extern int  segfree(void*, ulong);

// concurrency
extern void* rendezvous(void*, void*);
extern int  semacquire(long*, int);
extern long semrelease(long*, long);
extern int  tsemacquire(long*, ulong);

// special files
extern int  dup(int, int);

// security
extern int  fauth(int, char*);
extern int  fversion(int, int, char*, int);

extern  char*   getuser(void);

// error management
extern int  errstr(char*, uint);

extern void werrstr(char*, ...);
extern void rerrstr(char*, uint);

#pragma varargck argpos werrstr 1

//???
extern char* sysname(void);
```

A.1.5 include/regexp.h

⟨include/regexp.h 207⟩≡

```
#pragma src "/sys/src/libregexp"
#pragma lib "libregexp.a"

typedef struct Resub        Resub;
typedef struct Reclass      Reclass;
typedef struct Reinst       Reinst;
typedef struct Reprog       Reprog;

/*
 *  Sub expression matches
 */
struct Resub{
    // old: kencc-ext: this used to be anon unions but gcc don't accept it
    // and for goken we need mk to be compatible with a libregexp gcc can compile
    // hence the }s; and }e; below
    union
    {
```

```
        char *sp;
        Rune *rsp;
    }s;
    union
    {
        char *ep;
        Rune *rep;
    }e;
};

/*
 *  character class, each pair of rune's defines a range
 */
struct Reclass{
    Rune    *end;
    Rune    spans[64];
};

/*
 *  Machine instructions
 */
struct Reinst{
    int type;
    union   {
        Reclass *cp;        /* class pointer */
        Rune    r;      /* character */
        int subid;      /* sub-expression id for RBRA and LBRA */
        Reinst  *right;     /* right child of OR */
    };
    union { /* regexp relies on these two being in the same union */
        Reinst *left;       /* left child of OR */
        Reinst *next;       /* next instruction for CAT & LBRA */
    };
};

/*
 *  Reprogram definition
 */
struct Reprog{
    Reinst  *startinst; /* start pc */
    Reclass class[16];  /* .data */
    Reinst  firstinst[5];   /* .text */
};

extern Reprog   *regcomp(char*);
extern Reprog   *regcomplit(char*);
extern Reprog   *regcompnl(char*);
extern void regerror(char*);
extern int  regexec(Reprog*, char*, Resub*, int);
extern void regsub(char*, char*, int, Resub*, int);
extern int  rregexec(Reprog*, Rune*, Resub*, int);
extern void rregsub(Rune*, Rune*, int, Resub*, int);
```

A.1.6 include/ipc/thread.h

⟨include/ipc/thread.h 208⟩≡

```
#pragma src "/sys/src/libthread"
#pragma lib "libthread.a"
```

```
#pragma varargck    argpos  chanprint   2

typedef struct Alt  Alt;
typedef struct Channel  Channel;
typedef struct Ref  Ref;
/* slave I/O processes */
typedef struct Ioproc Ioproc;
#pragma incomplete Ioproc
```

⟨*struct* Channel 95b⟩

⟨*enum* chanop 105a⟩

```
typedef enum chanop ChanOp;
```

⟨*struct* Alt 105b⟩

⟨*struct* Ref 94a⟩

```
long    decref(Ref *r);         /* returns 0 iff value is now zero */
void    incref(Ref *r);

Channel* chancreate(int elemsize, int bufsize);
int chanclose(Channel*);
int chanclosing(Channel *c);
int chaninit(Channel *c, int elemsize, int elemcnt);
void chanfree(Channel *c);
int chanprint(Channel *, char *, ...);

// blocking API
int recv(Channel *c, void *v);
void* recvp(Channel *c);
ulong recvul(Channel *c);
int send(Channel *c, void *v);
int sendp(Channel *c, void *v);
int sendul(Channel *c, ulong v);

// non blocking API
int nbrecv(Channel *c, void *v);
void* nbrecvp(Channel *c);
ulong nbrecvul(Channel *c);
int nbsend(Channel *c, void *v);
int nbsendp(Channel *c, void *v);
int nbsendul(Channel *c, ulong v);

// select
int alt(Alt alts[]);

// process
int proccreate(void (*f)(void *arg), void *arg, uint stacksize);
int procrfork(void (*f)(void *arg), void *arg, uint stacksize, int flag);
void**  procdata(void);
void    procexec(Channel *, char *, char *[]);
void    procexecl(Channel *, char *, ...);

// threads
int threadcreate(void (*f)(void *arg), void *arg, uint stacksize);
void**  threaddata(void);
void    threadexits(char *);
void    threadexitsall(char *);
void    threadmain(int argc, char *argv[]);
```

```
int threadid(void);

int threadgetgrp(void); /* return thread group of current thread */
int threadsetgrp(int);      /* set thread group, return old */
char*   threadgetname(void);
void    threadsetname(char *fmt, ...);
void    threadint(int);     /* interrupt thread */
void    threadintgrp(int);  /* interrupt threads in grp */
void    threadkill(int);    /* kill thread */
void    threadkillgrp(int); /* kill threads in group */
void    threadnonotes(void);
int threadnotify(int (*f)(void*, char*), int in);
int threadpid(int);
Channel*threadwaitchan(void);

// Io proc
Ioproc* ioproc(void);
void    closeioproc(Ioproc*);
void    iointerrupt(Ioproc*);

int ioopen(Ioproc*, char*, int);
int ioclose(Ioproc*, int);
long    ioread(Ioproc*, int, void*, long);
long    ioreadn(Ioproc*, int, void*, long);
long    iowrite(Ioproc*, int, void*, long);
int iosleep(Ioproc*, long);
int iodial(Ioproc*, char*, char*, char*, int*);

long    iocall(Ioproc*, long (*)(va_list*), ...);
void    ioret(Ioproc*, int);

// misc
void    needstack(int);
int tprivalloc(void);
void    tprivfree(int);
void    **tprivaddr(int);
void    yield(void);

extern  int mainstacksize;
```

A.1.7 include/core/internals/pool.h

⟨include/core/internals/pool.h 210⟩≡

```
typedef struct Pool Pool;

⟨type Pool 28d⟩

extern void* poolalloc(Pool*, ulong);
extern void poolfree(Pool*, void*);

extern void* poolallocalign(Pool*, ulong, ulong, long, ulong);
extern ulong poolmsize(Pool*, void*);
extern void* poolrealloc(Pool*, void*, ulong);
extern void poolcheck(Pool*);
extern int poolcompact(Pool*);
extern void poolblockcheck(Pool*, void*);

// those globals are initialized in libc but also in the kernel itself
// so that the kernel can reuse the pool allocation code
```

```
extern Pool* mainmem;
extern Pool* imagmem;

⟨type Pool_flag 29a⟩
```

A.1.8 include/fcall.h

⟨include/fcall.h 211⟩≡

```
#pragma src "/sys/src/libc/9sys"
#pragma lib "libc.a"

#define VERSION9P   "9P2000"

⟨constant MAXWELEM 74b⟩

typedef struct  Fcall Fcall;

⟨type Fcall 73b⟩

⟨macros GBITxxx 77b⟩

⟨macros PBITxxx 77c⟩

⟨macros BITxxx 78a⟩

⟨constant QIDSZ 90b⟩

⟨constant STATFIXLEN 90c⟩


#define NOTAG       (ushort)~0U /* Dummy tag */
#define NOFID       (u32int)~0U /* Dummy fid */

⟨constant IOHDRSZ 78b⟩

⟨type FcallType 74a⟩


error0    convM2S(uchar*, uint, Fcall*);
error0    convS2M(Fcall*, uchar*, uint);
uint    sizeS2M(Fcall*);

int     statcheck(uchar *abuf, uint nbuf);
uint    convM2D(uchar*, uint, Dir*, char*);
uint    convD2M(Dir*, uchar*, uint);
uint    sizeD2M(Dir*);

// dumpers
int fcallfmt(Fmt*);
int dirfmt(Fmt*);
int dirmodefmt(Fmt*);

int read9pmsg(fdt, void*, uint);

#pragma varargck    type    "F" Fcall*
#pragma varargck    type    "M" ulong
#pragma varargck    type    "D" Dir*
```

A.2 lib_core/libc/port

A.2.1 libc/port/_assert.c

⟨libc/port/_assert.c 212a⟩≡
```
#include <u.h>
#include <libc.h>

⟨global __assert 179g⟩


⟨function default_assert 179f⟩

⟨global _assert 179e⟩
```

A.2.2 libc/port/abs.c

⟨libc/port/abs.c 212b⟩≡
```
#include <u.h>
#include <libc.h>

⟨function abs 149a⟩
```

A.2.3 libc/port/asin.c

⟨libc/port/asin.c 212c⟩≡
```
/*
 * asin(arg) and acos(arg) return the arcsin, arccos,
 * respectively of their arguments.
 *
 * Arctan is called after appropriate range reduction.
 */
#include <u.h>
#include <libc.h>

⟨function asin 169b⟩

⟨function acos 170a⟩
```

A.2.4 libc/port/atan.c

⟨constant sq2p1 212d⟩≡ (214a)
```
#define sq2p1 2.41421356237309504880 2e0
```

⟨constant sq2m1 212e⟩≡ (214a)
```
#define sq2m1  .41421356237309504880 2e0
```

⟨constant p4 212f⟩≡ (214a)
```
#define p4  .16153641298223022826 2e2
```

⟨constant p3 212g⟩≡ (214a)
```
#define p3  .26842548195503973794141e3
```

⟨constant p2 212h⟩≡ (214a)
```
#define p2  .11530293515404850115428136e4
```

⟨constant p1 212i⟩≡ (214a)
```
#define p1  .17804063164331969710546458 7e4
```

⟨*constant* p0 213a⟩≡ (214a)

```
#define p0  .8967859740366386195998748 8e3
```

⟨*constant* q4 213b⟩≡ (214a)

```
#define q4  .5895697050844462222791e2
```

⟨*constant* q3 213c⟩≡ (214a)

```
#define q3  .5362653740312153151042 35e3
```

⟨*constant* q2 213d⟩≡ (214a)

```
#define q2  .1666783814881633718452 1798e4
```

⟨*constant* q1 213e⟩≡ (214a)

```
#define q1  .2079334974445409812872 75926e4
```

⟨*constant* q0 213f⟩≡ (214a)

```
#define q0  .8967859740366386196248 1162e3
```

⟨*function* xatan 213g⟩≡ (214a)

```
/*
    xatan evaluates a series valid in the
    range [-0.414...,+0.414...]. (tan(pi/8))
 */

static
double
xatan(double arg)
{
    double argsq, value;

    argsq = arg*arg;
    value = ((((p4*argsq + p3)*argsq + p2)*argsq + p1)*argsq + p0);
    value = value/(((((argsq + q4)*argsq + q3)*argsq + q2)*argsq + q1)*argsq + q0);
    return value*arg;
}
```

⟨*function* satan 213h⟩≡ (214a)

```
/*
    satan reduces its argument (known to be positive)
    to the range [0,0.414...] and calls xatan.
 */

static
double
satan(double arg)
{

    if(arg < sq2m1)
        return xatan(arg);
    if(arg > sq2p1)
        return PIO2 - xatan(1/arg);
    return PIO2/2 + xatan((arg-1)/(arg+1));
}
```

⟨libc/port/atan.c 214a⟩≡

```
/*
    floating-point arctangent

    atan returns the value of the arctangent of its
    argument in the range [-pi/2,pi/2].

    atan2 returns the arctangent of arg1/arg2
    in the range [-pi,pi].

    there are no error returns.

    coefficients are #5077 from Hart & Cheney. (19.56D)
*/
#include <u.h>
#include <libc.h>

⟨constant sq2p1 212d⟩
⟨constant sq2m1 212e⟩
⟨constant p4 212f⟩
⟨constant p3 212g⟩
⟨constant p2 212h⟩
⟨constant p1 212i⟩
⟨constant p0 213a⟩
⟨constant q4 213b⟩
⟨constant q3 213c⟩
⟨constant q2 213d⟩
⟨constant q1 213e⟩
⟨constant q0 213f⟩

⟨function xatan 213g⟩
⟨function satan 213h⟩

⟨function atan 170b⟩
```

A.2.5 libc/port/atan2.c

⟨function atan2 214b⟩≡ (215a)

```
/*
    atan2 discovers what quadrant the angle
    is in and calls atan.
*/

double
atan2(double arg1, double arg2)
{

    if(arg1+arg2 == arg1) {
        if(arg1 >= 0)
            return PIO2;
        return -PIO2;
    }
    arg1 = atan(arg1/arg2);
    if(arg2 < 0) {
        if(arg1 <= 0)
            return arg1 + PI;
        return arg1 - PI;
    }
    return arg1;
```

```
}
```

⟨libc/port/atan2.c 215a⟩≡

```
#include <u.h>
#include <libc.h>

⟨function atan2 214b⟩
```

A.2.6 libc/port/atexit.c

⟨libc/port/atexit.c 215b⟩≡

```
#include <u.h>
#include <libc.h>

⟨constant NEXIT 24b⟩

typedef struct Onex Onex;
⟨struct Onex 24c⟩

⟨global onexlock 24d⟩
⟨global onex 24a⟩

⟨function atexit 24e⟩

#pragma profile off
⟨function exits 23g⟩
#pragma profile on
```

A.2.7 libc/port/atnotify.c

⟨libc/port/atnotify.c 215c⟩≡

```
#include <u.h>
#include <libc.h>

⟨constant NFN (port/atnotify.c) 72a⟩
⟨global onnot 72b⟩
⟨global onnotlock 72c⟩

⟨function notifier 72d⟩

⟨function atnotify 71d⟩
```

A.2.8 libc/port/atof.c

⟨libc/port/atof.c 215d⟩≡

```
#include <u.h>
#include <libc.h>

⟨function atof 154a⟩
```

A.2.9 libc/port/atol.c

⟨libc/port/atol.c 215e⟩≡

```
#include <u.h>
#include <libc.h>
```

⟨*function* atol 150b⟩

⟨*function* atoi 150a⟩

A.2.10 libc/port/atoll.c

⟨*function* atoll 216a⟩≡ (216b)

```
vlong
atoll(char *s)
{
    return strtoll(s, nil, 0);
}
```

⟨libc/port/atoll.c 216b⟩≡

```
#include <u.h>
#include <libc.h>
```

⟨*function* atoll 216a⟩

A.2.11 libc/port/charstod.c

⟨*constant* ADVANCE 216c⟩≡ (217a)

```
/*
 * Reads a floating-point number by interpreting successive characters
 * returned by (*f)(vp).  The last call it makes to f terminates the
 * scan, so is not a character in the number.  It may therefore be
 * necessary to back up the input stream up one byte after calling charstod.
 */

#define ADVANCE do{*s++ = c; if(s>=e) return NaN(); c = (*f)(vp);}while(0)
```

⟨*function* charstod 216d⟩≡ (217a)

```
double
charstod(int(*f)(void*), void *vp)
{
    char str[400], *s, *e, *start;
    int c;

    s = str;
    e = str + sizeof str - 1;
    c = (*f)(vp);
    while(c == ' ' || c == '\t')
        c = (*f)(vp);
    if(c == '-' || c == '+'){
        ADVANCE;
    }
    start = s;
    while(c >= '0' && c <= '9'){
        ADVANCE;
    }
    if(c == '.'){
        ADVANCE;
        while(c >= '0' && c <= '9'){
            ADVANCE;
        }
    }
    if(s > start && (c == 'e' || c == 'E')){
        ADVANCE;
```

```
        if(c == '-' || c == '+'){
            ADVANCE;
        }
        while(c >= '0' && c <= '9'){
            ADVANCE;
        }
    }else if(s == start && (c == 'i' || c == 'I')){
        ADVANCE;
        if(c != 'n' && c != 'N')
            return NaN();
        ADVANCE;
        if(c != 'f' && c != 'F')
            return NaN();
        ADVANCE;
        if(c != 'i' && c != 'I')
            return NaN();
        ADVANCE;
        if(c != 'n' && c != 'N')
            return NaN();
        ADVANCE;
        if(c != 'i' && c != 'I')
            return NaN();
        ADVANCE;
        if(c != 't' && c != 'T')
            return NaN();
        ADVANCE;
        if(c != 'y' && c != 'Y')
            return NaN();
        ADVANCE;  /* so caller can back up uniformly */
        USED(c);
    }else if(s == str && (c == 'n' || c == 'N')){
        ADVANCE;
        if(c != 'a' && c != 'A')
            return NaN();
        ADVANCE;
        if(c != 'n' && c != 'N')
            return NaN();
        ADVANCE;  /* so caller can back up uniformly */
        USED(c);
    }
    *s = 0;
    return strtod(str, &s);
}
```

⟨libc/port/charstod.c 217a⟩≡

```
#include <u.h>
#include <libc.h>

⟨constant ADVANCE 216c⟩

⟨function charstod 216d⟩
```

A.2.12 libc/port/cistrcmp.c

⟨*function* cistrcmp 217b⟩≡ (218a)

```
int
cistrcmp(char *s1, char *s2)
{
    int c1, c2;
```

```
    while(*s1){
        c1 = *(uchar*)s1++;
        c2 = *(uchar*)s2++;

        if(c1 == c2)
            continue;

        if(c1 >= 'A' && c1 <= 'Z')
            c1 -= 'A' - 'a';

        if(c2 >= 'A' && c2 <= 'Z')
            c2 -= 'A' - 'a';

        if(c1 != c2)
            return c1 - c2;
    }
    return -*s2;
}
```

⟨libc/port/cistrcmp.c 218a⟩≡

```
#include <u.h>
#include <libc.h>

⟨function cistrcmp 217b⟩
```

A.2.13 libc/port/cistrncmp.c

⟨*function* cistrncmp 218b⟩≡ (218c)

```
int
cistrncmp(char *s1, char *s2, int n)
{
    int c1, c2;

    while(*s1 && n-- > 0){
        c1 = *(uchar*)s1++;
        c2 = *(uchar*)s2++;

        if(c1 == c2)
            continue;

        if(c1 >= 'A' && c1 <= 'Z')
            c1 -= 'A' - 'a';

        if(c2 >= 'A' && c2 <= 'Z')
            c2 -= 'A' - 'a';

        if(c1 != c2)
            return c1 - c2;
    }
    if(n <= 0)
        return 0;
    return -*s2;
}
```

⟨libc/port/cistrncmp.c 218c⟩≡

```
#include <u.h>
#include <libc.h>

⟨function cistrncmp 218b⟩
```

A.2.14 libc/port/cistrstr.c

⟨*function* cistrstr 219a⟩≡ (219b)

```
char*
cistrstr(char *s, char *sub)
{
    int c, csub, n;

    csub = *sub;
    if(csub == '\0')
        return s;
    if(csub >= 'A' && csub <= 'Z')
        csub -= 'A' - 'a';
    sub++;
    n = strlen(sub);
    for(; c = *s; s++){
        if(c >= 'A' && c <= 'Z')
            c -= 'A' - 'a';
        if(c == csub && cistrncmp(s+1, sub, n) == 0)
            return s;
    }
    return nil;
}
```

⟨libc/port/cistrstr.c 219b⟩≡

```
#include <u.h>
#include <libc.h>

⟨function cistrstr 219a⟩
```

A.2.15 libc/port/cleanname.c

⟨libc/port/cleanname.c 219c⟩≡

```
#include <u.h>
#include <libc.h>

⟨macro SEP 50a⟩
⟨function cleanname 50b⟩
```

A.2.16 libc/port/crypt.c

⟨libc/port/crypt.c 219d⟩≡

```
/*
 *  Data Encryption Standard
 *  D.P.Mitchell  83/06/08.
 *
 *  block_cipher(key, block, decrypting)
 *
 *  these routines use the non-standard 7 byte format
 *  for DES keys.
 */
#include <u.h>
#include <libc.h>
#include <auth.h>
#include <libsec.h>

⟨function encrypt 188b⟩

⟨function decrypt 189a⟩
```

A.2.17 libc/port/ctype.c

⟨libc/port/ctype.c 220a⟩≡

```
#include <u.h>
#include <libc.h>
#include <ctype.h>

⟨global _ctype 17b⟩
```

A.2.18 libc/port/encodefmt.c

⟨*function* encodefmt 220b⟩≡ (221a)

```
int
encodefmt(Fmt *f)
{
    char *out;
    char *buf;
    int len;
    int ilen;
    int rv;
    uchar *b;
    char *p;
    char obuf[64];  // rsc optimization

    if(!(f->flags&FmtPrec) || f->prec < 1)
        goto error;

    b = va_arg(f->args, uchar*);
    if(b == 0)
        return fmtstrcpy(f, "<nil>");

    ilen = f->prec;
    f->prec = 0;
    f->flags &= ~FmtPrec;
    switch(f->r){
    case '<':
        len = (8*ilen+4)/5 + 3;
        break;
    case '[':
        len = (8*ilen+5)/6 + 4;
        break;
    case 'H':
        len = 2*ilen + 1;
        break;
    default:
        goto error;
    }

    if(len > sizeof(obuf)){
        buf = malloc(len);
        if(buf == nil)
            goto error;
    } else
        buf = obuf;

    // convert
    out = buf;
    switch(f->r){
    case '<':
```

```
        rv = enc32(out, len, b, ilen);
        break;
    case '[':
        rv = enc64(out, len, b, ilen);
        break;
    case 'H':
        rv = enc16(out, len, b, ilen);
        if(rv >= 0 && (f->flags & FmtLong))
            for(p = buf; *p; p++)
                *p = tolower(*p);
        break;
    default:
        rv = -1;
        break;
    }
    if(rv < 0)
        goto error;

    fmtstrcpy(f, buf);
    if(buf != obuf)
        free(buf);
    return 0;

error:
    return fmtstrcpy(f, "<encodefmt>");
}
```

⟨libc/port/encodefmt.c 221a⟩≡

```
#include <u.h>
#include <libc.h>
#include <ctype.h>

⟨function encodefmt 220b⟩
```

A.2.19 libc/port/execl.c

⟨libc/port/execl.c 221b⟩≡

```
#include <u.h>
#include <libc.h>

⟨function execl 58c⟩
```

A.2.20 libc/port/exp.c

⟨libc/port/exp.c 221c⟩≡

```
/*
    exp returns the exponential function of its
    floating-point argument.

    The coefficients are #1069 from Hart and Cheney. (22.35D)
*/

#include <u.h>
#include <libc.h>

⟨constant p0 (port/exp.c) 171c⟩
⟨constant p1 (port/exp.c) 171d⟩
⟨constant p2 (port/exp.c) 171e⟩
```

⟨*constant q0 (port/exp.c)* 171f⟩
⟨*constant q1 (port/exp.c)* 171g⟩
⟨*constant q2 (port/exp.c)* 171h⟩
⟨*constant* `log2e` 171b⟩
⟨*constant* `sqrt2` 171i⟩
⟨*constant* `maxf` 171a⟩

⟨*function* `exp` 170c⟩

A.2.21 libc/port/fabs.c

⟨`libc/port/fabs.c` 222a⟩≡
```
#include <u.h>
#include <libc.h>

⟨function fabs 149b⟩
```

A.2.22 libc/port/floor.c

⟨`libc/port/floor.c` 222b⟩≡
```
#include <u.h>
#include <libc.h>
⟨function floor 149c⟩

⟨function ceil 149d⟩
```

A.2.23 libc/port/fmod.c

⟨`libc/port/fmod.c` 222c⟩≡
```
#include <u.h>
#include <libc.h>

⟨function fmod 164b⟩
```

A.2.24 libc/port/frand.c

⟨*constant* `MASK` 222d⟩≡ (223a)
```
#define MASK    0x7fffffffL
```

⟨*constant* `NORM` 222e⟩≡ (223a)
```
#define NORM    (1.0/(1.0+MASK))
```

⟨*function* `frand` 222f⟩≡ (223a)
```
double
frand(void)
{
    double x;

    do {
        x = lrand() * NORM;
        x = (x + lrand()) * NORM;
    } while(x >= 1);
    return x;
}
```

⟨libc/port/frand.c 223a⟩≡
```
#include    <u.h>
#include    <libc.h>

⟨constant MASK 222d⟩
⟨constant NORM 222e⟩

⟨function frand 222f⟩
```

A.2.25 libc/port/frexp.c

⟨libc/port/frexp.c 223b⟩≡
```
#include <u.h>
#include <libc.h>

⟨constant MASK (port/frexp.c) 171j⟩
⟨constant SHIFT 171k⟩
⟨constant BIAS 171l⟩
⟨constant SIG 171m⟩

⟨function frexp 172b⟩

⟨function ldexp 172a⟩

⟨function modf 164a⟩
```

A.2.26 libc/port/getcallerpc.c

⟨function getcallerpc 223c⟩≡ (223d)
```
uintptr
getcallerpc(void*)
{
    return 0;
}
```

⟨libc/port/getcallerpc.c 223d⟩≡
```
#include <u.h>
#include <libc.h>

⟨function getcallerpc 223c⟩
```

A.2.27 libc/port/getfields.c

⟨libc/port/getfields.c 223e⟩≡
```
#include <u.h>
#include <libc.h>

⟨function getfields 68⟩
```

A.2.28 libc/port/getuser.c

⟨libc/port/getuser.c 223f⟩≡
```
#include <u.h>
#include <libc.h>

⟨function getuser 188a⟩
```

A.2.29 libc/port/hangup.c

⟨*function* hangup 224a⟩≡ (224b)

```
/*
 *  force a connection to hangup
 */
int
hangup(int ctl)
{
    return write(ctl, "hangup", sizeof("hangup")-1) != sizeof("hangup")-1;
}
```

⟨libc/port/hangup.c 224b⟩≡

```
#include <u.h>
#include <libc.h>
#include <ctype.h>

⟨function hangup 224a⟩
```

A.2.30 libc/port/hypot.c

⟨*function* hypot 224c⟩≡ (224d)

```
double
hypot(double p, double q)
{
    double r, s, pfac;

    if(p < 0)
        p = -p;
    if(q < 0)
        q = -q;
    if(p < q) {
        r = p;
        p = q;
        q = r;
    }
    if(p == 0)
        return 0;
    pfac = p;
    r = q = q/p;
    p = 1;
    for(;;) {
        r *= r;
        s = r+4;
        if(s == 4)
            return p*pfac;
        r /= s;
        p += 2*r*p;
        q *= r;
        r = q/p;
    }
}
```

⟨libc/port/hypot.c 224d⟩≡

```
/*
 * hypot -- sqrt(p*p+q*q), but overflows only if the result does.
 * See Cleve Moler and Donald Morrison,
 * ``Replacing Square Roots by Pythagorean Sums,''
 * IBM Journal of Research and Development,
```

```
 * Vol. 27, Number 6, pp. 577-581, Nov. 1983
 */

#include <u.h>
#include <libc.h>

⟨function hypot 224c⟩
```

A.2.31 libc/port/lnrand.c

⟨*constant MASK (port/lnrand.c)* 225a⟩≡ (225c)

```
#define MASK    0x7fffffffL
```

⟨*function* lnrand 225b⟩≡ (225c)

```
long
lnrand(long n)
{
    long slop, v;

    if(n < 0)
        return n;
    slop = MASK % n;
    do
        v = lrand();
    while(v <= slop);
    return v % n;
}
```

⟨libc/port/lnrand.c 225c⟩≡

```
#include    <u.h>
#include    <libc.h>

⟨constant MASK (port/lnrand.c) 225a⟩

⟨function lnrand 225b⟩
```

A.2.32 libc/port/lock.c

⟨libc/port/lock.c 225d⟩≡

```
#include <u.h>
#include <libc.h>

⟨function lock 60b⟩

⟨function unlock 60c⟩

⟨function canlock 60d⟩
```

A.2.33 libc/port/log.c

⟨libc/port/log.c 225e⟩≡

```
/*
    log returns the natural logarithm of its floating
    point argument.

    The coefficients are #2705 from Hart & Cheney. (19.38D)
```

```
    It calls frexp.
*/

#include <u.h>
#include <libc.h>

⟨constant log2 173b⟩
⟨constant ln10o1 173c⟩
⟨constant sqrto2 173d⟩
⟨constant p0 (port/log.c) 173e⟩
⟨constant p1 (port/log.c) 174a⟩
⟨constant p2 (port/log.c) 174b⟩
⟨constant p3 (port/log.c) 174c⟩
⟨constant q0 (port/log.c) 174d⟩
⟨constant q1 (port/log.c) 174e⟩
⟨constant q2 (port/log.c) 174f⟩

⟨function log 173a⟩

⟨function log10 174g⟩
```

A.2.34 libc/port/lrand.c

⟨*constant* LEN 226a⟩≡ (227b)

```
#define LEN 607
```

⟨*constant* TAP 226b⟩≡ (227b)

```
#define TAP 273
```

⟨*constant MASK (port/lrand.c)* 226c⟩≡ (227b)

```
#define MASK    0x7fffffffL
```

⟨*constant* A 226d⟩≡ (227b)

```
#define A   48271
```

⟨*constant* M 226e⟩≡ (227b)

```
#define M   2147483647
```

⟨*constant* Q 226f⟩≡ (227b)

```
#define Q   44488
```

⟨*constant* R 226g⟩≡ (227b)

```
#define R   3399
```

⟨*constant NORM (port/lrand.c)* 226h⟩≡ (227b)

```
#define NORM    (1.0/(1.0+MASK))
```

⟨*function* israND 226i⟩≡ (227b)

```
static void
israND(long seed)
{
    long lo, hi, x;
    int i;

    rng_tap = rng_vec;
    rng_feed = rng_vec+LEN-TAP;
    seed = seed%M;
    if(seed < 0)
        seed += M;
```

```
    if(seed == 0)
        seed = 89482311;
    x = seed;
    /*
     *  Initialize by x[n+1] = 48271 * x[n] mod (2**31 - 1)
     */
    for(i = -20; i < LEN; i++) {
        hi = x / Q;
        lo = x % Q;
        x = A*lo - R*hi;
        if(x < 0)
            x += M;
        if(i >= 0)
            rng_vec[i] = x;
    }
}
```

⟨*function* `srand` 227a⟩≡ (227b)

```
void
srand(long seed)
{
    lock(&lk);
    isrand(seed);
    unlock(&lk);
}
```

⟨`libc/port/lrand.c` 227b⟩≡

```
#include    <u.h>
#include    <libc.h>

/*
 *  algorithm by
 *  D. P. Mitchell & J. A. Reeds
 */

⟨constant LEN 226a⟩
⟨constant TAP 226b⟩
⟨constant MASK (port/lrand.c) 226c⟩
⟨constant A 226d⟩
⟨constant M 226e⟩
⟨constant Q 226f⟩
⟨constant R 226g⟩
⟨constant NORM (port/lrand.c) 226h⟩

⟨global rng_vec 175b⟩
⟨global rng_tap 175c⟩
⟨global rng_feed 175d⟩
⟨global lk 175a⟩

⟨function isrand 226i⟩

⟨function srand 227a⟩

⟨function lrand 174i⟩
```

A.2.35 `libc/port/malloc.c`

⟨*function* `sbrkalloc` 227c⟩≡ (232d)

```
/*
```

```
 * we do minimal bookkeeping so we can tell pool
 * whether two blocks are adjacent and thus mergeable.
 */
static void*
sbrkalloc(ulong n)
{
    ulong *x;

    n += 2*sizeof(ulong);   /* two longs for us */
    x = sbrk(n);
    if(x == (void*)-1)
        return nil;
    x[0] = (n+7)&~7;    /* sbrk rounds size up to mult. of 8 */
    x[1] = 0xDeadBeef;
    return x+2;
}
```

⟨*function* sbrkmerge 228a⟩≡ (232d)

```
static int
sbrkmerge(void *x, void *y)
{
    ulong *lx, *ly;

    lx = x;
    if(lx[-1] != 0xDeadBeef)
        abort();

    if((uchar*)lx+lx[-2] == (uchar*)y) {
        ly = y;
        lx[-2] += ly[-2];
        return 1;
    }
    return 0;
}
```

⟨*function* plock 228b⟩≡ (232d)

```
static void
plock(Pool *p)
{
    Private *pv;
    pv = p->private;
    lock(&pv->lk);
    if(pv->pid != 0)
        abort();
    pv->pid = _tos->pid;
}
```

⟨*function* punlock 228c⟩≡ (232d)

```
static void
punlock(Pool *p)
{
    Private *pv;
    pv = p->private;
    if(pv->pid != _tos->pid)
        abort();
    pv->pid = 0;
    unlock(&pv->lk);
}
```

⟨*function* checkenv 229a⟩≡ (232d)

```
static int
checkenv(void)
{
    int n, fd;
    char buf[20];
    fd = open("/env/MALLOCFD", OREAD);
    if(fd < 0)
        return -1;
    if((n = read(fd, buf, sizeof buf)) < 0) {
        close(fd);
        return -1;
    }
    if(n >= sizeof buf)
        n = sizeof(buf)-1;
    buf[n] = 0;
    n = atoi(buf);
    if(n == 0)
        n = -1;
    return n;
}
```

⟨*function* pprint 229b⟩≡ (232d)

```
static void
pprint(Pool *p, char *fmt, ...)
{
    va_list v;
    Private *pv;

    pv = p->private;
    if(pv->printfd == 0)
        pv->printfd = checkenv();

    if(pv->printfd <= 0)
        pv->printfd = 2;

    va_start(v, fmt);
    vfprint(pv->printfd, fmt, v);
    va_end(v);
}
```

⟨*global* panicbuf 229c⟩≡ (232d)

```
static char panicbuf[256];
```

⟨*function* ppanic 229d⟩≡ (232d)

```
static void
ppanic(Pool *p, char *fmt, ...)
{
    va_list v;
    int n;
    char *msg;
    Private *pv;

    pv = p->private;
    assert(canlock(&pv->lk)==0);

    if(pv->printfd == 0)
        pv->printfd = checkenv();
    if(pv->printfd <= 0)
        pv->printfd = 2;
```

```
    msg = panicbuf;
    va_start(v, fmt);
    n = vseprint(msg, msg+sizeof panicbuf, fmt, v) - msg;
    write(2, "panic: ", 7);
    write(2, msg, n);
    write(2, "\n", 1);
    if(pv->printfd != 2){
        write(pv->printfd, "panic: ", 7);
        write(pv->printfd, msg, n);
        write(pv->printfd, "\n", 1);
    }
    va_end(v);
//  unlock(&pv->lk);
    abort();
}
```

⟨*enum* _anon_ 230a⟩≡ (232d)

```
/* tracing */
enum {
    Npadlong    = 2,
    MallocOffset = 0,
    ReallocOffset = 1
};
```

⟨*function* mallocz 230b⟩≡ (232d)

```
void*
mallocz(ulong size, int clr)
{
    void *v;

    v = poolalloc(mainmem, size+Npadlong*sizeof(ulong));
    if(Npadlong && v != nil){
        v = (ulong*)v+Npadlong;
        setmalloctag(v, getcallerpc(&size));
        setrealloctag(v, 0);
    }
    if(clr && v != nil)
        memset(v, 0, size);
    return v;
}
```

⟨*function* mallocalign 230c⟩≡ (232d)

```
void*
mallocalign(ulong size, ulong align, long offset, ulong span)
{
    void *v;

    v = poolallocalign(mainmem, size+Npadlong*sizeof(ulong), align, offset-Npadlong*sizeof(ulong), span);
    if(Npadlong && v != nil){
        v = (ulong*)v+Npadlong;
        setmalloctag(v, getcallerpc(&size));
        setrealloctag(v, 0);
    }
    return v;
}
```

⟨*function* realloc 230d⟩≡ (232d)

```
void*
realloc(void *v, ulong size)
```

```
{
    void *nv;

    if(size == 0){
        free(v);
        return nil;
    }

    if(v)
        v = (ulong*)v-Npadlong;
    size += Npadlong*sizeof(ulong);

    if(nv = poolrealloc(mainmem, v, size)){
        nv = (ulong*)nv+Npadlong;
        setrealloctag(nv, getcallerpc(&v));
        if(v == nil)
            setmalloctag(nv, getcallerpc(&v));
    }
    return nv;
}
```

⟨*function* msize 231a⟩≡ (232d)

```
ulong
msize(void *v)
{
    return poolmsize(mainmem, (ulong*)v-Npadlong)-Npadlong*sizeof(ulong);
}
```

⟨*function* calloc 231b⟩≡ (232d)

```
void*
calloc(ulong n, ulong szelem)
{
    void *v;
    if(v = mallocz(n*szelem, 1))
        setmalloctag(v, getcallerpc(&n));
    return v;
}
```

⟨*function* setmalloctag 231c⟩≡ (232d)

```
void
setmalloctag(void *v, ulong pc)
{
    ulong *u;
    USED(v, pc);
    if(Npadlong <= MallocOffset || v == nil)
        return;
    u = v;
    u[-Npadlong+MallocOffset] = pc;
}
```

⟨*function* setrealloctag 231d⟩≡ (232d)

```
void
setrealloctag(void *v, ulong pc)
{
    ulong *u;
    USED(v, pc);
    if(Npadlong <= ReallocOffset || v == nil)
        return;
    u = v;
    u[-Npadlong+ReallocOffset] = pc;
}
```

⟨*function* getmalloctag 232a⟩≡ (232d)

```
ulong
getmalloctag(void *v)
{
    USED(v);
    if(Npadlong <= MallocOffset)
        return ~0;
    return ((ulong*)v)[-Npadlong+MallocOffset];
}
```

⟨*function* getrealloctag 232b⟩≡ (232d)

```
ulong
getrealloctag(void *v)
{
    USED(v);
    if(Npadlong <= ReallocOffset)
        return ((ulong*)v)[-Npadlong+ReallocOffset];
    return ~0;
}
```

⟨*function* malloctopoolblock 232c⟩≡ (232d)

```
void*
malloctopoolblock(void *v)
{
    if(v == nil)
        return nil;

    return &((ulong*)v)[-Npadlong];
}
```

⟨libc/port/malloc.c 232d⟩≡

```
#include <u.h>
#include <libc.h>
#include <pool.h>
#include <tos.h>

static void*    sbrkalloc(ulong);
static int      sbrkmerge(void*, void*);
static void     plock(Pool*);
static void     punlock(Pool*);
static void     pprint(Pool*, char*, ...);
static void     ppanic(Pool*, char*, ...);

typedef struct Private Private;
⟨struct Private 29b⟩

⟨global sbrkmempriv 29c⟩

⟨global sbrkmem 29d⟩
⟨global mainmem 30a⟩
⟨global imagmem 30b⟩

⟨function sbrkalloc 227c⟩

⟨function sbrkmerge 228a⟩

⟨function plock 228b⟩

⟨function punlock 228c⟩
```

⟨*function* checkenv 229a⟩

⟨*function* pprint 229b⟩

⟨*global* panicbuf 229c⟩
⟨*function* ppanic 229d⟩

```
/* - everything from here down should be the same in libc, libdebugmalloc, and the kernel - */
/* - except the code for malloc(), which alternately doesn't clear or does. - */

/*
 * Npadlong is the number of 32-bit longs to leave at the beginning of
 * each allocated buffer for our own bookkeeping.  We return to the callers
 * a pointer that points immediately after our bookkeeping area.  Incoming pointers
 * must be decremented by that much, and outgoing pointers incremented.
 * The malloc tag is stored at MallocOffset from the beginning of the block,
 * and the realloc tag at ReallocOffset.  The offsets are from the true beginning
 * of the block, not the beginning the caller sees.
 *
 * The extra if(Npadlong != 0) in various places is a hint for the compiler to
 * compile out function calls that would otherwise be no-ops.
 */

/*  non tracing
 *
enum {
    Npadlong    = 0,
    MallocOffset = 0,
    ReallocOffset = 0,
};
 *
 */
```

⟨*enum* _anon_ 230a⟩

⟨*function* malloc 32e⟩

⟨*function* mallocz 230b⟩

⟨*function* mallocalign 230c⟩

⟨*function* free 32f⟩

⟨*function* realloc 230d⟩

⟨*function* msize 231a⟩

⟨*function* calloc 231b⟩

⟨*function* setmalloctag 231c⟩

⟨*function* setrealloctag 231d⟩

⟨*function* getmalloctag 232a⟩

⟨*function* getrealloctag 232b⟩

⟨*function* malloctopoolblock 232c⟩

A.2.36 libc/port/memccpy.c

⟨libc/port/memccpy.c 234a⟩≡

```
#include    <u.h>
#include    <libc.h>

⟨function memccpy 34d⟩
```

A.2.37 libc/port/memchr.c

⟨libc/port/memchr.c 234b⟩≡

```
#include    <u.h>
#include    <libc.h>

⟨function memchr 27b⟩
```

A.2.38 libc/port/memcmp.c

⟨libc/port/memcmp.c 234c⟩≡

```
#include    <u.h>
#include    <libc.h>

⟨function memcmp 27a⟩
```

A.2.39 libc/port/memmove.c

⟨libc/port/memmove.c 234d⟩≡

```
#include    <u.h>
#include    <libc.h>

⟨function memmove 26c⟩

⟨function memcpy 26b⟩
```

A.2.40 libc/port/memset.c

⟨libc/port/memset.c 234e⟩≡

```
#include    <u.h>
#include    <libc.h>

⟨function memset 26a⟩
```

A.2.41 libc/port/mktemp.c

⟨libc/port/mktemp.c 234f⟩≡

```
#include <u.h>
#include <libc.h>

⟨function mktemp 49b⟩
```

A.2.42 libc/port/muldiv.c

⟨libc/port/muldiv.c 235a⟩≡
```
#include <u.h>
#include <libc.h>

⟨function umuldiv 175f⟩

⟨function muldiv 175g⟩
```

A.2.43 libc/port/nan.c

⟨libc/port/nan.c 235b⟩≡
```
#include <u.h>
#include <libc.h>

⟨constant NANEXP 153a⟩
⟨constant NANMASK 153b⟩
⟨constant NANSIGN 153d⟩

⟨function NaN 152⟩

⟨function isNaN 153c⟩

⟨function Inf 153e⟩

⟨function isInf 153f⟩
```

A.2.44 libc/port/needsrcquote.c

⟨function needsrcquote 235c⟩≡ (235d)
```
int
needsrcquote(int c)
{
    if(c <= ' ')
        return 1;
    if(utfrune("`^#*[]=|\\?${}()'<>&;", c))
        return 1;
    return 0;
}
```

⟨libc/port/needsrcquote.c 235d⟩≡
```
#include <u.h>
#include <libc.h>

⟨function needsrcquote 235c⟩
```

A.2.45 libc/port/netcrypt.c

⟨libc/port/netcrypt.c 235e⟩≡
```
#include <u.h>
#include <libc.h>
#include <auth.h>

⟨function netcrypt 189b⟩
```

A.2.46 libc/port/netmkaddr.c

⟨libc/port/netmkaddr.c 236a⟩≡
```
#include <u.h>
#include <libc.h>
#include <ctype.h>

⟨function netmkaddr 143b⟩
```

A.2.47 libc/port/nrand.c

⟨constant MASK (port/nrand.c) 236b⟩≡ (236d)
```
#define MASK    0x7fffffffL
```

⟨function nrand 236c⟩≡ (236d)
```
int
nrand(int n)
{
    long slop, v;

    if(n < 0)
        return n;
    if(n == 1)
        return 0;
    /* and if n == 0, you deserve what you get */
    slop = MASK % n;
    do
        v = lrand();
    while(v <= slop);
    return v % n;
}
```

⟨libc/port/nrand.c 236d⟩≡
```
#include    <u.h>
#include    <libc.h>

⟨constant MASK (port/nrand.c) 236b⟩

⟨function nrand 236c⟩
```

A.2.48 libc/port/ntruerand.c

⟨function ntruerand 236e⟩≡ (237a)
```
ulong
ntruerand(ulong n)
{
    ulong m, r;

    /*
     * set m to the one less than the maximum multiple of n <= 2^32,
     * so we want a random number <= m.
     */
    if(n > (1UL<<31))
        m = n-1;
    else
        /* 2^32 - 2^32%n - 1 = (2^32 - 1) - (2*(2^31%n))%n */
        m = 0xFFFFFFFFUL - (2*((1UL<<31)%n))%n;
```

```
    while((r = truerand()) > m)
        ;

    return r%n;
}
```

⟨libc/port/ntruerand.c 237a⟩≡

```
#include <u.h>
#include <libc.h>

⟨function ntruerand 236e⟩
```

A.2.49 libc/port/perror.c

⟨libc/port/perror.c 237b⟩≡

```
#include <u.h>
#include <libc.h>

⟨function perror 178d⟩
```

A.2.50 libc/port/pool.c

⟨*function* checklist 237c⟩≡ (258d)

```
/*
 * Tree walking
 */

static void
checklist(Free *t)
{
    Free *q;

    for(q=t->next; q!=t; q=q->next){
        assert(q->size == t->size);
        assert(q->next==nil || q->next->prev==q);
        assert(q->prev==nil || q->prev->next==q);
    //  assert(q->left==nil);
    //  assert(q->right==nil);
        assert(q->magic==FREE_MAGIC);
    }
}
```

⟨*function* checktree 237d⟩≡ (258d)

```
static void
checktree(Free *t, int a, int b)
{
    assert(t->magic==FREE_MAGIC);
    assert(a < t->size && t->size < b);
    assert(t->next==nil || t->next->prev==t);
    assert(t->prev==nil || t->prev->next==t);
    checklist(t);
    if(t->left)
        checktree(t->left, a, t->size);
    if(t->right)
        checktree(t->right, t->size, b);

}
```

⟨*function* ltreewalk 238a⟩≡ (258d)

```
/* ltreewalk: return address of pointer to node of size == size */
static Free**
ltreewalk(Free **t, ulong size)
{
    assert(t != nil /* ltreewalk */);

    for(;;) {
        if(*t == nil)
            return t;

        assert((*t)->magic == FREE_MAGIC);

        if(size == (*t)->size)
            return t;
        if(size < (*t)->size)
            t = &(*t)->left;
        else
            t = &(*t)->right;
    }
}
```

⟨*function* treelookup 238b⟩≡ (258d)

```
/* treelookup: find node in tree with size == size */
static Free*
treelookup(Free *t, ulong size)
{
    return *ltreewalk(&t, size);
}
```

⟨*function* treeinsert 238c⟩≡ (258d)

```
/* treeinsert: insert node into tree */
static Free*
treeinsert(Free *tree, Free *node)
{
    Free **loc, *repl;

    assert(node != nil /* treeinsert */);

    loc = ltreewalk(&tree, node->size);
    if(*loc == nil) {
        node->left = nil;
        node->right = nil;
    } else {    /* replace existing node */
        repl = *loc;
        node->left = repl->left;
        node->right = repl->right;
    }
    *loc = node;
    return tree;
}
```

⟨*function* treedelete 238d⟩≡ (258d)

```
/* treedelete: remove node from tree */
static Free*
treedelete(Free *tree, Free *node)
{
    Free **loc, **lsucc, *succ;

    assert(node != nil /* treedelete */);
```

```
    loc = ltreewalk(&tree, node->size);
    assert(*loc == node);

    if(node->left == nil)
        *loc = node->right;
    else if(node->right == nil)
        *loc = node->left;
    else {
        /* have two children, use inorder successor as replacement */
        for(lsucc = &node->right; (*lsucc)->left; lsucc = &(*lsucc)->left)
            ;
        succ = *lsucc;
        *lsucc = succ->right;
        succ->left = node->left;
        succ->right = node->right;
        *loc = succ;
    }

    node->left = node->right = Poison;
    return tree;
}
```

⟨*function* treelookupgt 239a⟩≡ (258d)

```
/* treelookupgt: find smallest node in tree with size >= size */
static Free*
treelookupgt(Free *t, ulong size)
{
    Free *lastgood; /* last node we saw that was big enough */

    lastgood = nil;
    for(;;) {
        if(t == nil)
            return lastgood;
        if(size == t->size)
            return t;
        if(size < t->size) {
            lastgood = t;
            t = t->left;
        } else
            t = t->right;
    }
}
```

⟨*function* listadd 239b⟩≡ (258d)

```
/* listadd: add a node to a doubly linked list */
static Free*
listadd(Free *list, Free *node)
{
    if(list == nil) {
        node->next = node;
        node->prev = node;
        return node;
    }

    node->prev = list->prev;
    node->next = list;

    node->prev->next = node;
    node->next->prev = node;
```

```
    return list;
}
```

⟨*function* listdelete 240a⟩≡ (258d)

```
/* listdelete: remove node from a doubly linked list */
static Free*
listdelete(Pool *p, Free *list, Free *node)
{
    if(node->next == node) {    /* singular list */
        node->prev = node->next = Poison;
        return nil;
    }
    if(node->next == nil)
        p->panic(p, "pool->next");
    if(node->prev == nil)
        p->panic(p, "pool->prev");
    node->next->prev = node->prev;
    node->prev->next = node->next;

    if(list == node)
        list = node->next;

    node->prev = node->next = Poison;
    return list;
}
```

⟨*function* pooladd 240b⟩≡ (258d)

```
/* pooladd: add anode to the free pool */
static Free*
pooladd(Pool *p, Alloc *anode)
{
    Free *lst, *olst;
    Free *node;
    Free **parent;

    antagonism {
        memmark(_B2D(anode), 0xF7, anode->size-sizeof(Bhdr)-sizeof(Btail));
    }

    node = (Free*)anode;
    node->magic = FREE_MAGIC;
    parent = ltreewalk(&p->freeroot, node->size);
    olst = *parent;
    lst = listadd(olst, node);
    if(olst != lst) /* need to update tree */
        *parent = treeinsert(*parent, lst);
    p->curfree += node->size;
    return node;
}
```

⟨*function* pooldel 240c⟩≡ (258d)

```
/* pooldel: remove node from the free pool */
static Alloc*
pooldel(Pool *p, Free *node)
{
    Free *lst, *olst;
    Free **parent;

    parent = ltreewalk(&p->freeroot, node->size);
```

```
    olst = *parent;
    assert(olst != nil /* pooldel */);

    lst = listdelete(p, olst, node);
    if(lst == nil)
        *parent = treedelete(*parent, olst);
    else if(lst != olst)
        *parent = treeinsert(*parent, lst);

    node->left = node->right = Poison;
    p->curfree -= node->size;

    antagonism {
        memmark(_B2D(node), 0xF9, node->size-sizeof(Bhdr)-sizeof(Btail));
    }

    node->magic = UNALLOC_MAGIC;
    return (Alloc*)node;
}
```

⟨*function* dsize2bsize 241a⟩≡ (258d)

```
/* block allocation */
static ulong
dsize2bsize(Pool *p, ulong sz)
{
    sz += sizeof(Bhdr)+sizeof(Btail);
    if(sz < p->minblock)
        sz = p->minblock;
    if(sz < MINBLOCKSIZE)
        sz = MINBLOCKSIZE;
    sz = (sz+p->quantum-1)&~(p->quantum-1);
    return sz;
}
```

⟨*function* bsize2asize 241b⟩≡ (258d)

```
static ulong
bsize2asize(Pool *p, ulong sz)
{
    sz += sizeof(Arena)+sizeof(Btail);
    if(sz < p->minarena)
        sz = p->minarena;
    sz = (sz+p->quantum)&~(p->quantum-1);
    return sz;
}
```

⟨*function* blockmerge 241c⟩≡ (258d)

```
/* both are removed from pool if necessary. */
static Alloc*
blockmerge(Pool *pool, Bhdr *a, Bhdr *b)
{
    Btail *t;

    assert(B2NB(a) == b);

    if(a->magic == FREE_MAGIC)
        pooldel(pool, (Free*)a);
    if(b->magic == FREE_MAGIC)
        pooldel(pool, (Free*)b);

    t = B2T(a);
```

```
    t->size = (ulong)Poison;
    t->magic0 = NOT_MAGIC;
    t->magic1 = NOT_MAGIC;
    PSHORT(t->datasize, NOT_MAGIC);

    a->size += b->size;
    t = B2T(a);
    t->size = a->size;
    PSHORT(t->datasize, 0xFFFF);

    b->size = NOT_MAGIC;
    b->magic = NOT_MAGIC;

    a->magic = UNALLOC_MAGIC;
    return (Alloc*)a;
}
```

⟨*function* blocksetsize 242a⟩≡ (258d)

```
/* blocksetsize: set the total size of a block, fixing tail pointers */
static Bhdr*
blocksetsize(Bhdr *b, ulong bsize)
{
    Btail *t;

    assert(b->magic != FREE_MAGIC /* blocksetsize */);

    b->size = bsize;
    t = B2T(b);
    t->size = b->size;
    t->magic0 = TAIL_MAGIC0;
    t->magic1 = TAIL_MAGIC1;
    return b;
}
```

⟨*function* getdsize 242b⟩≡ (258d)

```
/* getdsize: return the requested data size for an allocated block */
static ulong
getdsize(Alloc *b)
{
    Btail *t;
    t = B2T(b);
    return b->size - SHORT(t->datasize);
}
```

⟨*function* trim 242c⟩≡ (258d)

```
/* trim: trim a block down to what is needed to hold dsize bytes of user data */
Alloc*
trim(Pool *p, Alloc *b, ulong dsize)
{
    ulong extra, bsize;
    Alloc *frag;

    bsize = dsize2bsize(p, dsize);
    extra = b->size - bsize;
    if(b->size - dsize >= 0x10000 ||
      (extra >= bsize>>2 && extra >= MINBLOCKSIZE && extra >= p->minblock)) {
        blocksetsize(b, bsize);
        frag = (Alloc*) B2NB(b);

        antagonism {
```

```
            memmark(frag, 0xF1, extra);
        }

        frag->magic = UNALLOC_MAGIC;
        blocksetsize(frag, extra);
        pooladd(p, frag);
    }

    b->magic = ALLOC_MAGIC;
    blocksetdsize(p, b, dsize);
    return b;
}
```

⟨*function* freefromfront 243a⟩≡ (258d)

```
static Alloc*
freefromfront(Pool *p, Alloc *b, ulong skip)
{
    Alloc *bb;

    skip = skip&~(p->quantum-1);
    if(skip >= 0x1000 || (skip >= b->size>>2 && skip >= MINBLOCKSIZE && skip >= p->minblock)){
        bb = (Alloc*)((uchar*)b+skip);
        blocksetsize(bb, b->size-skip);
        bb->magic = UNALLOC_MAGIC;
        blocksetsize(b, skip);
        b->magic = UNALLOC_MAGIC;
        pooladd(p, b);
        return bb;
    }
    return b;
}
```

⟨*function* arenasetsize 243b⟩≡ (258d)

```
/* arenasetsize: set arena size, updating tail */
static void
arenasetsize(Arena *a, ulong asize)
{
    Bhdr *atail;

    a->asize = asize;
    atail = A2TB(a);
    atail->magic = ARENATAIL_MAGIC;
    atail->size = 0;
}
```

⟨*function* poolnewarena 243c⟩≡ (258d)

```
/* poolnewarena: allocate new arena */
static void
poolnewarena(Pool *p, ulong asize)
{
    Arena *a;
    Arena *ap, *lastap;
    Alloc *b;

    LOG(p, "newarena %lud\n", asize);
    if(p->cursize+asize > p->maxsize) {
        if(poolcompactl(p) == 0){
            LOG(p, "pool too big: %lud+%lud > %lud\n",
                p->cursize, asize, p->maxsize);
            werrstr("memory pool too large");
```

```
        }
        return;
    }

    if((a = p->alloc(asize)) == nil) {
        /* assume errstr set by p->alloc */
        return;
    }

    p->cursize += asize;

    /* arena hdr */
    a->magic = ARENA_MAGIC;
    blocksetsize(a, sizeof(Arena));
    arenasetsize(a, asize);
    blockcheck(p, a);

    /* create one large block in arena */
    b = (Alloc*)A2B(a);
    b->magic = UNALLOC_MAGIC;
    blocksetsize(b, (uchar*)A2TB(a)-(uchar*)b);
    blockcheck(p, b);
    pooladd(p, b);
    blockcheck(p, b);

    /* sort arena into descending sorted arena list */
    for(lastap=nil, ap=p->arenalist; ap > a; lastap=ap, ap=ap->down)
        ;

    if(a->down = ap)    /* assign = */
        a->down->aup = a;

    if(a->aup = lastap) /* assign = */
        a->aup->down = a;
    else
        p->arenalist = a;

    /* merge with surrounding arenas if possible */
    /* must do a with up before down with a (think about it) */
    if(a->aup)
        arenamerge(p, a, a->aup);
    if(a->down)
        arenamerge(p, a->down, a);
}
```

⟨*function* blockgrow 244⟩≡ (258d)

```
/* trimming it into two different blocks. */
static void
blockgrow(Pool *p, Bhdr *b, ulong nsize)
{
    if(b->magic == FREE_MAGIC) {
        Alloc *a;
        Bhdr *bnxt;
        a = pooldel(p, (Free*)b);
        blockcheck(p, a);
        blocksetsize(a, nsize);
        blockcheck(p, a);
        bnxt = B2NB(a);
        if(bnxt->magic == FREE_MAGIC)
            a = blockmerge(p, a, bnxt);
```

```
        blockcheck(p, a);
        pooladd(p, a);
    } else {
        Alloc *a;
        ulong dsize;

        a = (Alloc*)b;
        dsize = getdsize(a);
        blocksetsize(a, nsize);
        trim(p, a, dsize);
    }
}
```

⟨*function* arenamerge 245a⟩≡ (258d)

```
/* arenamerge: attempt to coalesce to arenas that might be adjacent */
static Arena*
arenamerge(Pool *p, Arena *bot, Arena *top)
{
    Bhdr *bbot, *btop;
    Btail *t;

    blockcheck(p, bot);
    blockcheck(p, top);
    assert(bot->aup == top && top > bot);

    if(p->merge == nil || p->merge(bot, top) == 0)
        return nil;

    /* remove top from list */
    if(bot->aup = top->aup) /* assign = */
        bot->aup->down = bot;
    else
        p->arenalist = bot;

    /* save ptrs to last block in bot, first block in top */
    t = B2PT(A2TB(bot));
    bbot = T2HDR(t);
    btop = A2B(top);
    blockcheck(p, bbot);
    blockcheck(p, btop);

    /* grow bottom arena to encompass top */
    arenasetsize(bot, top->asize + ((uchar*)top - (uchar*)bot));

    /* grow bottom block to encompass space between arenas */
    blockgrow(p, bbot, (uchar*)btop-(uchar*)bbot);
    blockcheck(p, bbot);
    return bot;
}
```

⟨*function* dumpblock 245b⟩≡ (258d)

```
/* dumpblock: print block's vital stats */
static void
dumpblock(Pool *p, Bhdr *b)
{
    ulong *dp;
    ulong dsize;
    uchar *cp;

    dp = (ulong*)b;
```

```
    p->print(p, "pool %s block %p\nhdr %.8lux %.8lux %.8lux %.8lux %.8lux %.8lux\n",
        p->name, b, dp[0], dp[1], dp[2], dp[3], dp[4], dp[5], dp[6]);

    dp = (ulong*)B2T(b);
    p->print(p, "tail %.8lux %.8lux %.8lux %.8lux %.8lux %.8lux | %.8lux %.8lux\n",
        dp[-6], dp[-5], dp[-4], dp[-3], dp[-2], dp[-1], dp[0], dp[1]);

    if(b->magic == ALLOC_MAGIC){
        dsize = getdsize((Alloc*)b);
        if(dsize >= b->size)    /* user data size corrupt */
            return;

        cp = (uchar*)_B2D(b)+dsize;
        p->print(p, "user data ");
        p->print(p, "%.2ux %.2ux %.2ux %.2ux  %.2ux %.2ux %.2ux %.2ux",
            cp[-8], cp[-7], cp[-6], cp[-5], cp[-4], cp[-3], cp[-2], cp[-1]);
        p->print(p, " | %.2ux %.2ux %.2ux %.2ux  %.2ux %.2ux %.2ux %.2ux\n",
            cp[0], cp[1], cp[2], cp[3], cp[4], cp[5], cp[6], cp[7]);
    }
}
```

⟨*function* printblock 246a⟩≡ (258d)

```
static void
printblock(Pool *p, Bhdr *b, char *msg)
{
    p->print(p, "%s\n", msg);
    dumpblock(p, b);
}
```

⟨*function* panicblock 246b⟩≡ (258d)

```
static void
panicblock(Pool *p, Bhdr *b, char *msg)
{
    p->print(p, "%s\n", msg);
    dumpblock(p, b);
    p->panic(p, "pool panic");
}
```

⟨*function* blockcheck 246c⟩≡ (258d)

```
/* should only be called when holding pool lock */
static void
blockcheck(Pool *p, Bhdr *b)
{
    Alloc *a;
    Btail *t;
    int i, n;
    uchar *q, *bq, *eq;
    ulong dsize;

    switch(b->magic) {
    default:
        panicblock(p, b, "bad magic");
    case FREE_MAGIC:
    case UNALLOC_MAGIC:
        t = B2T(b);
        if(t->magic0 != TAIL_MAGIC0 || t->magic1 != TAIL_MAGIC1)
            panicblock(p, b, "corrupt tail magic");
        if(T2HDR(t) != b)
            panicblock(p, b, "corrupt tail ptr");
        break;
```

```
    case DEAD_MAGIC:
        t = B2T(b);
        if(t->magic0 != TAIL_MAGIC0 || t->magic1 != TAIL_MAGIC1)
            panicblock(p, b, "corrupt tail magic");
        if(T2HDR(t) != b)
            panicblock(p, b, "corrupt tail ptr");
        n = getdsize((Alloc*)b);
        q = _B2D(b);
        q += 8;
        for(i=8; i<n; i++)
            if(*q++ != 0xDA)
                panicblock(p, b, "dangling pointer write");
        break;
    case ARENA_MAGIC:
        b = A2TB((Arena*)b);
        if(b->magic != ARENATAIL_MAGIC)
            panicblock(p, b, "bad arena size");
        /* fall through */
    case ARENATAIL_MAGIC:
        if(b->size != 0)
            panicblock(p, b, "bad arena tail size");
        break;
    case ALLOC_MAGIC:
        a = (Alloc*)b;
        t = B2T(b);
        dsize = getdsize(a);
        bq = (uchar*)_B2D(a)+dsize;
        eq = (uchar*)t;

        if(t->magic0 != TAIL_MAGIC0){
            /* if someone wrote exactly one byte over and it was a NUL, we sometimes only complain. */
            if((p->flags & POOL_TOLERANCE) && bq == eq && t->magic0 == 0)
                printblock(p, b, "mem user overflow (magic0)");
            else
                panicblock(p, b, "corrupt tail magic0");
        }

        if(t->magic1 != TAIL_MAGIC1)
            panicblock(p, b, "corrupt tail magic1");
        if(T2HDR(t) != b)
            panicblock(p, b, "corrupt tail ptr");

        if(dsize2bsize(p, dsize)  > a->size)
            panicblock(p, b, "too much block data");

        if(eq > bq+4)
            eq = bq+4;
        for(q=bq; q<eq; q++){
            if(*q != datamagic[((uintptr)q)%nelem(datamagic)]){
                if(q == bq && *q == 0 && (p->flags & POOL_TOLERANCE)){
                    printblock(p, b, "mem user overflow");
                    continue;
                }
                panicblock(p, b, "mem user overflow");
            }
        }
        break;
    }
}
```

⟨*enum* _anon_ (port/pool.c)*8* 248a⟩≡ (258d)

```
/*
 * compact an arena by shifting all the free blocks to the end.
 * assumes pool lock is held.
 */
enum {
    FLOATING_MAGIC = 0xCBCBCBCB,    /* temporarily neither allocated nor in the free tree */
};
```

⟨*function* arenacompact 248b⟩≡ (258d)

```
static int
arenacompact(Pool *p, Arena *a)
{
    Bhdr *b, *wb, *eb, *nxt;
    int compacted;

    if(p->move == nil)
        p->panic(p, "don't call me when pool->move is nil\n");

    poolcheckarena(p, a);
    eb = A2TB(a);
    compacted = 0;
    for(b=wb=A2B(a); b && b < eb; b=nxt) {
        nxt = B2NB(b);
        switch(b->magic) {
        case FREE_MAGIC:
            pooldel(p, (Free*)b);
            b->magic = FLOATING_MAGIC;
            break;
        case ALLOC_MAGIC:
            if(wb != b) {
                memmove(wb, b, b->size);
                p->move(_B2D(b), _B2D(wb));
                compacted = 1;
            }
            wb = B2NB(wb);
            break;
        }
    }

    /*
     * the only free data is now at the end of the arena, pointed
     * at by wb.  all we need to do is set its size and get out.
     */
    if(wb < eb) {
        wb->magic = UNALLOC_MAGIC;
        blocksetsize(wb, (uchar*)eb-(uchar*)wb);
        pooladd(p, (Alloc*)wb);
    }

    return compacted;
}
```

⟨*function* poolcompactl 248c⟩≡ (258d)

```
/*
 * compact a pool by compacting each individual arena.
 * 'twould be nice to shift blocks from one arena to the
 * next but it's a pain to code.
 */
static int
```

```
poolcompactl(Pool *pool)
{
    Arena *a;
    int compacted;

    if(pool->move == nil || pool->lastcompact == pool->nfree)
        return 0;

    pool->lastcompact = pool->nfree;
    compacted = 0;
    for(a=pool->arenalist; a; a=a->down)
        compacted |= arenacompact(pool, a);
    return compacted;
}
```

⟨*function* B2D 249a⟩≡ (258d)

```
/*
static void*
_B2D(void *a)
{
    return (uchar*)a+sizeof(Bhdr);
}
*/

static void*
B2D(Pool *p, Alloc *a)
{
    if(a->magic != ALLOC_MAGIC)
        p->panic(p, "B2D called on unworthy block");
    return _B2D(a);
}
```

⟨*function* D2B 249b⟩≡ (258d)

```
/*
static void*
_D2B(void *v)
{
    Alloc *a;
    a = (Alloc*)((uchar*)v-sizeof(Bhdr));
    return a;
}
*/

static Alloc*
D2B(Pool *p, void *v)
{
    Alloc *a;
    ulong *u;

    if((uintptr)v&(sizeof(ulong)-1))
        v = (char*)v - ((uintptr)v&(sizeof(ulong)-1));
    u = v;
    while(u[-1] == ALIGN_MAGIC)
        u--;
    a = _D2B(u);
    if(a->magic != ALLOC_MAGIC)
        p->panic(p, "D2B called on non-block %p (double-free?)", v);
    return a;
}
```

⟨*function* poolallocl 250a⟩≡ (258d)

```
/* poolallocl: attempt to allocate block to hold dsize user bytes; assumes lock held */
static void*
poolallocl(Pool *p, ulong dsize)
{
    ulong bsize;
    Free *fb;
    Alloc *ab;

    if(dsize >= 0x80000000UL){  /* for sanity, overflow */
        werrstr("invalid allocation size");
        return nil;
    }

    bsize = dsize2bsize(p, dsize);

    fb = treelookupgt(p->freeroot, bsize);
    if(fb == nil) {
        poolnewarena(p, bsize2asize(p, bsize));
        if((fb = treelookupgt(p->freeroot, bsize)) == nil) {
            /* assume poolnewarena failed and set %r */
            return nil;
        }
    }

    ab = trim(p, pooldel(p, fb), dsize);
    p->curalloc += ab->size;
    antagonism {
        memset(B2D(p, ab), 0xDF, dsize);
    }
    return B2D(p, ab);
}
```

⟨*function* poolreallocl 250b⟩≡ (258d)

```
/* poolreallocl: attempt to grow v to ndsize bytes; assumes lock held */
static void*
poolreallocl(Pool *p, void *v, ulong ndsize)
{
    Alloc *a;
    Bhdr *left, *right, *newb;
    Btail *t;
    ulong nbsize;
    ulong odsize;
    ulong obsize;
    void *nv;

    if(v == nil)    /* for ANSI */
        return poolallocl(p, ndsize);
    if(ndsize == 0) {
        poolfreel(p, v);
        return nil;
    }
    a = D2B(p, v);
    blockcheck(p, a);
    odsize = getdsize(a);
    obsize = a->size;

    /* can reuse the same block? */
    nbsize = dsize2bsize(p, ndsize);
    if(nbsize <= a->size) {
```

```
    Returnblock:
        if(v != _B2D(a))
            memmove(_B2D(a), v, odsize);
        a = trim(p, a, ndsize);
        p->curalloc -= obsize;
        p->curalloc += a->size;
        v = B2D(p, a);
        return v;
    }

    /* can merge with surrounding blocks? */
    right = B2NB(a);
    if(right->magic == FREE_MAGIC && a->size+right->size >= nbsize) {
        a = blockmerge(p, a, right);
        goto Returnblock;
    }

    t = B2PT(a);
    left = T2HDR(t);
    if(left->magic == FREE_MAGIC && left->size+a->size >= nbsize) {
        a = blockmerge(p, left, a);
        goto Returnblock;
    }

    if(left->magic == FREE_MAGIC && right->magic == FREE_MAGIC
    && left->size+a->size+right->size >= nbsize) {
        a = blockmerge(p, blockmerge(p, left, a), right);
        goto Returnblock;
    }

    if((nv = poolallocl(p, ndsize)) == nil)
        return nil;

    /* maybe the new block is next to us; if so, merge */
    left = T2HDR(B2PT(a));
    right = B2NB(a);
    newb = D2B(p, nv);
    if(left == newb || right == newb) {
        if(left == newb || left->magic == FREE_MAGIC)
            a = blockmerge(p, left, a);
        if(right == newb || right->magic == FREE_MAGIC)
            a = blockmerge(p, a, right);
        assert(a->size >= nbsize);
        goto Returnblock;
    }

    /* enough cleverness */
    memmove(nv, v, odsize);
    antagonism {
        memset((char*)nv+odsize, 0xDE, ndsize-odsize);
    }
    poolfreel(p, v);
    return nv;
}
```

⟨*function* alignptr 251⟩≡ (258d)

```
static void*
alignptr(void *v, ulong align, long offset)
{
    char *c;
```

```
    ulong off;

    c = v;
    if(align){
        off = (uintptr)c%align;
        if(off != offset){
            c += offset - off;
            if(off > offset)
                c += align;
        }
    }
    return c;
}
```

⟨*function* poolallocalignl 252⟩≡ (258d)

```
/* poolallocalignl: allocate as described below; assumes pool locked */
static void*
poolallocalignl(Pool *p, ulong dsize, ulong align, long offset, ulong span)
{
    ulong asize;
    void *v;
    char *c;
    ulong *u;
    int skip;
    Alloc *b;

    /*
     * allocate block
     *  dsize bytes
     *  addr == offset (modulo align)
     *  does not cross span-byte block boundary
     *
     * to satisfy alignment, just allocate an extra
     * align bytes and then shift appropriately.
     *
     * to satisfy span, try once and see if we're
     * lucky.  the second time, allocate 2x asize
     * so that we definitely get one not crossing
     * the boundary.
     */
    if(align){
        if(offset < 0)
            offset = align - ((-offset)%align);
        else
            offset %= align;
    }
    asize = dsize+align;
    v = poolallocl(p, asize);
    if(v == nil)
        return nil;
    if(span && (uintptr)v/span != ((uintptr)v+asize)/span){
        /* try again */
        poolfreel(p, v);
        v = poolallocl(p, 2*asize);
        if(v == nil)
            return nil;
    }

    /*
     * figure out what pointer we want to return
```

```
     */
    c = alignptr(v, align, offset);
    if(span && (uintptr)c/span != (uintptr)(c+dsize-1)/span){
        c += span - (uintptr)c%span;
        c = alignptr(c, align, offset);
        if((uintptr)c/span != (uintptr)(c+dsize-1)/span){
            poolfreel(p, v);
            werrstr("cannot satisfy dsize %lud span %lud with align %lud+%ld", dsize, span, align, offset);
            return nil;
        }
    }
    skip = c - (char*)v;

    /*
     * free up the skip bytes before that pointer
     * or mark it as unavailable.
     */
    b = _D2B(v);
    b = freefromfront(p, b, skip);
    v = _B2D(b);
    skip = c - (char*)v;
    if(c > (char*)v){
        u = v;
        while(c >= (char*)u+sizeof(ulong))
            *u++ = ALIGN_MAGIC;
    }
    trim(p, b, skip+dsize);
    assert(D2B(p, c) == b);
    antagonism {
        memset(c, 0xDD, dsize);
    }
    return c;
}
```

⟨*function* poolfreel 253⟩≡ (258d)

```
/* poolfree: free block obtained from poolalloc; assumes lock held */
static void
poolfreel(Pool *p, void *v)
{
    Alloc *ab;
    Bhdr *back, *fwd;

    if(v == nil)    /* for ANSI */
        return;

    ab = D2B(p, v);
    blockcheck(p, ab);

    if(p->flags&POOL_NOREUSE){
        int n;

        ab->magic = DEAD_MAGIC;
        n = getdsize(ab)-8;
        if(n > 0)
            memset((uchar*)v+8, 0xDA, n);
        return;
    }

    p->nfree++;
    p->curalloc -= ab->size;
```

```
    back = T2HDR(B2PT(ab));
    if(back->magic == FREE_MAGIC)
        ab = blockmerge(p, back, ab);

    fwd = B2NB(ab);
    if(fwd->magic == FREE_MAGIC)
        ab = blockmerge(p, ab, fwd);

    pooladd(p, ab);
}
```

⟨*function* poolalloc 254a⟩≡ (258d)

```
void*
poolalloc(Pool *p, ulong n)
{
    void *v;

    p->lock(p);
    paranoia {
        poolcheckl(p);
    }
    verbosity {
        pooldumpl(p);
    }
    v = poolallocl(p, n);
    paranoia {
        poolcheckl(p);
    }
    verbosity {
        pooldumpl(p);
    }
    if(p->logstack && (p->flags & POOL_LOGGING)) p->logstack(p);
    LOG(p, "poolalloc %p %lud = %p\n", p, n, v);
    p->unlock(p);
    return v;
}
```

⟨*function* poolallocalign 254b⟩≡ (258d)

```
void*
poolallocalign(Pool *p, ulong n, ulong align, long offset, ulong span)
{
    void *v;

    p->lock(p);
    paranoia {
        poolcheckl(p);
    }
    verbosity {
        pooldumpl(p);
    }
    v = poolallocalignl(p, n, align, offset, span);
    paranoia {
        poolcheckl(p);
    }
    verbosity {
        pooldumpl(p);
    }
    if(p->logstack && (p->flags & POOL_LOGGING)) p->logstack(p);
    LOG(p, "poolalignspanalloc %p %lud %lud %lud %ld = %p\n", p, n, align, span, offset, v);
    p->unlock(p);
```

```
    return v;
}
```

⟨*function* poolcompact 255a⟩≡ (258d)

```
int
poolcompact(Pool *p)
{
    int rv;

    p->lock(p);
    paranoia {
        poolcheckl(p);
    }
    verbosity {
        pooldumpl(p);
    }
    rv = poolcompactl(p);
    paranoia {
        poolcheckl(p);
    }
    verbosity {
        pooldumpl(p);
    }
    LOG(p, "poolcompact %p\n", p);
    p->unlock(p);
    return rv;
}
```

⟨*function* poolrealloc 255b⟩≡ (258d)

```
void*
poolrealloc(Pool *p, void *v, ulong n)
{
    void *nv;

    p->lock(p);
    paranoia {
        poolcheckl(p);
    }
    verbosity {
        pooldumpl(p);
    }
    nv = poolreallocl(p, v, n);
    paranoia {
        poolcheckl(p);
    }
    verbosity {
        pooldumpl(p);
    }
    if(p->logstack && (p->flags & POOL_LOGGING)) p->logstack(p);
    LOG(p, "poolrealloc %p %p %ld = %p\n", p, v, n, nv);
    p->unlock(p);
    return nv;
}
```

⟨*function* poolfree 255c⟩≡ (258d)

```
void
poolfree(Pool *p, void *v)
{
    p->lock(p);
    paranoia {
```

```
        poolcheckl(p);
    }
    verbosity {
        pooldumpl(p);
    }
    poolfreel(p, v);
    paranoia {
        poolcheckl(p);
    }
    verbosity {
        pooldumpl(p);
    }
    if(p->logstack && (p->flags & POOL_LOGGING)) p->logstack(p);
    LOG(p, "poolfree %p %p\n", p, v);
    p->unlock(p);
}
```

⟨*function* poolmsize 256a⟩≡ (258d)

```
/*
 * Return the real size of a block, and let the user use it.
 */
ulong
poolmsize(Pool *p, void *v)
{
    Alloc *b;
    ulong dsize;

    p->lock(p);
    paranoia {
        poolcheckl(p);
    }
    verbosity {
        pooldumpl(p);
    }
    if(v == nil)    /* consistency with other braindead ANSI-ness */
        dsize = 0;
    else {
        b = D2B(p, v);
        dsize = (b->size&~(p->quantum-1)) - sizeof(Bhdr) - sizeof(Btail);
        assert(dsize >= getdsize(b));
        blocksetdsize(p, b, dsize);
    }
    paranoia {
        poolcheckl(p);
    }
    verbosity {
        pooldumpl(p);
    }
    if(p->logstack && (p->flags & POOL_LOGGING)) p->logstack(p);
    LOG(p, "poolmsize %p %p = %ld\n", p, v, dsize);
    p->unlock(p);
    return dsize;
}
```

⟨*function* poolcheckarena 256b⟩≡ (258d)

```
/*
 * Debugging
 */

static void
```

```
poolcheckarena(Pool *p, Arena *a)
{
    Bhdr *b;
    Bhdr *atail;

    atail = A2TB(a);
    for(b=a; b->magic != ARENATAIL_MAGIC && b<atail; b=B2NB(b))
        blockcheck(p, b);
    blockcheck(p, b);
    if(b != atail)
        p->panic(p, "found wrong tail");
}
```

⟨*function* poolcheckl 257a⟩≡ (258d)

```
static void
poolcheckl(Pool *p)
{
    Arena *a;

    for(a=p->arenalist; a; a=a->down)
        poolcheckarena(p, a);
    if(p->freeroot)
        checktree(p->freeroot, 0, 1<<30);
}
```

⟨*function* poolcheck 257b⟩≡ (258d)

```
void
poolcheck(Pool *p)
{
    p->lock(p);
    poolcheckl(p);
    p->unlock(p);
}
```

⟨*function* poolblockcheck 257c⟩≡ (258d)

```
void
poolblockcheck(Pool *p, void *v)
{
    if(v == nil)
        return;

    p->lock(p);
    blockcheck(p, D2B(p, v));
    p->unlock(p);
}
```

⟨*function* pooldumpl 257d⟩≡ (258d)

```
static void
pooldumpl(Pool *p)
{
    Arena *a;

    p->print(p, "pool %p %s\n", p, p->name);
    for(a=p->arenalist; a; a=a->down)
        pooldumparena(p, a);
}
```

⟨*function* pooldump 258a⟩≡ (258d)

```
void
pooldump(Pool *p)
{
    p->lock(p);
    pooldumpl(p);
    p->unlock(p);
}
```

⟨*function* pooldumparena 258b⟩≡ (258d)

```
static void
pooldumparena(Pool *p, Arena *a)
{
    Bhdr *b;

    for(b=a; b->magic != ARENATAIL_MAGIC; b=B2NB(b))
        p->print(p, "(%p %.8lux %lud)", b, b->magic, b->size);
    p->print(p, "\n");
}
```

⟨*function* memmark 258c⟩≡ (258d)

```
/*
 * mark the memory in such a way that we know who marked it
 * (via the signature) and we know where the marking started.
 */
static void
memmark(void *v, int sig, ulong size)
{
    uchar *p, *ep;
    ulong *lp, *elp;
    lp = v;
    elp = lp+size/4;
    while(lp < elp)
        *lp++ = (sig<<24) ^ ((uintptr)lp-(uintptr)v);
    p = (uchar*)lp;
    ep = (uchar*)v+size;
    while(p<ep)
        *p++ = sig;
}
```

⟨libc/port/pool.c 258d⟩≡

```
/*
 * This allocator takes blocks from a coarser allocator (p->alloc) and
 * uses them as arenas.
 *
 * An arena is split into a sequence of blocks of variable size.  The
 * blocks begin with a Bhdr that denotes the length (including the Bhdr)
 * of the block.  An arena begins with an Arena header block (Arena,
 * ARENA_MAGIC) and ends with a Bhdr block with magic ARENATAIL_MAGIC and
 * size 0.  Intermediate blocks are either allocated or free.  At the end
 * of each intermediate block is a Btail, which contains information
 * about where the block starts.  This is useful for walking backwards.
 *
 * Free blocks (Free*) have a magic value of FREE_MAGIC in their Bhdr
 * headers.  They are kept in a binary tree (p->freeroot) traversible by
 * walking ->left and ->right.  Each node of the binary tree is a pointer
 * to a circular doubly-linked list (next, prev) of blocks of identical
 * size.  Blocks are added to this ``tree of lists'' by pooladd(), and
 * removed by pooldel().
 *
```

```
 * When freed, adjacent blocks are coalesced to create larger blocks when
 * possible.
 *
 * Allocated blocks (Alloc*) have one of two magic values: ALLOC_MAGIC or
 * UNALLOC_MAGIC.  When blocks are released from the pool, they have
 * magic value UNALLOC_MAGIC.  Once the block has been trimmed by trim()
 * and the amount of user-requested data has been recorded in the
 * datasize field of the tail, the magic value is changed to ALLOC_MAGIC.
 * All blocks returned to callers should be of type ALLOC_MAGIC, as
 * should all blocks passed to us by callers.  The amount of data the user
 * asked us for can be found by subtracting the short in tail->datasize
 * from header->size.  Further, the up to at most four bytes between the
 * end of the user-requested data block and the actual Btail structure are
 * marked with a magic value, which is checked to detect user overflow.
 *
 * The arenas returned by p->alloc are kept in a doubly-linked list
 * (p->arenalist) running through the arena headers, sorted by descending
 * base address (prev, next).  When a new arena is allocated, we attempt
 * to merge it with its two neighbors via p->merge.
 */

#include <u.h>
#include <libc.h>
#include <pool.h>

typedef struct Alloc    Alloc;
typedef struct Arena    Arena;
typedef struct Bhdr Bhdr;
typedef struct Btail    Btail;
typedef struct Free Free;

⟨struct Bhdr 30c⟩
⟨enum _anon_ (port/pool.c) 30d⟩
⟨macro B2NB 30e⟩

⟨macro SHORT 30f⟩
⟨macro PSHORT 30g⟩

⟨enum _anon_ (port/pool.c)2 30h⟩
⟨struct Btail 30i⟩
⟨macro B2T 30j⟩
⟨macro B2PT 30k⟩
⟨macro T2HDR 30l⟩
⟨struct Free 31a⟩
⟨enum _anon_ (port/pool.c)3 31b⟩

⟨struct Alloc 31c⟩
⟨enum _anon_ (port/pool.c)4 31d⟩

⟨struct Arena 31e⟩
⟨enum _anon_ (port/pool.c)5 31f⟩
⟨macro A2TB 31g⟩
⟨macro A2B 31h⟩

⟨enum _anon_ (port/pool.c)6 31i⟩

⟨enum _anon_ (port/pool.c)7 31j⟩

⟨global datamagic 32a⟩
```

⟨*constant* Poison 32b⟩

⟨*macro* _B2D 32c⟩
⟨*macro* _D2B 32d⟩

```
// static void* _B2D(void*);
// static void* _D2B(void*);
static void*    B2D(Pool*, Alloc*);
static Alloc*   D2B(Pool*, void*);
static Arena*   arenamerge(Pool*, Arena*, Arena*);
static void     blockcheck(Pool*, Bhdr*);
static Alloc*   blockmerge(Pool*, Bhdr*, Bhdr*);
static Alloc*   blocksetdsize(Pool*, Alloc*, ulong);
static Bhdr*    blocksetsize(Bhdr*, ulong);
static ulong    bsize2asize(Pool*, ulong);
static ulong    dsize2bsize(Pool*, ulong);
static ulong    getdsize(Alloc*);
static Alloc*   trim(Pool*, Alloc*, ulong);
static Free*    listadd(Free*, Free*);
static void     logstack(Pool*);
static Free**   ltreewalk(Free**, ulong);
static void     memmark(void*, int, ulong);
static Free*    pooladd(Pool*, Alloc*);
static void*    poolallocl(Pool*, ulong);
static void     poolcheckl(Pool*);
static void     poolcheckarena(Pool*, Arena*);
static int      poolcompactl(Pool*);
static Alloc*   pooldel(Pool*, Free*);
static void     pooldumpl(Pool*);
static void     pooldumparena(Pool*, Arena*);
static void     poolfreel(Pool*, void*);
static void     poolnewarena(Pool*, ulong);
static void*    poolreallocl(Pool*, void*, ulong);
static Free*    treedelete(Free*, Free*);
static Free*    treeinsert(Free*, Free*);
static Free*    treelookup(Free*, ulong);
static Free*    treelookupgt(Free*, ulong);

/*
 * Debugging
 *
 * Antagonism causes blocks to always be filled with garbage if their
 * contents are undefined.  This tickles both programs and the library.
 * It's a linear time hit but not so noticeable during nondegenerate use.
 * It would be worth leaving in except that it negates the benefits of the
 * kernel's demand-paging.  The tail magic and end-of-data magic
 * provide most of the user-visible benefit that antagonism does anyway.
 *
 * Paranoia causes the library to recheck the entire pool on each lock
 * or unlock.  A failed check on unlock means we tripped over ourselves,
 * while a failed check on lock tends to implicate the user.  Paranoia has
 * the potential to slow things down a fair amount for pools with large
 * numbers of allocated blocks.  It completely negates all benefits won
 * by the binary tree.  Turning on paranoia in the kernel makes it painfully
 * slow.
 *
 * Verbosity induces the dumping of the pool via p->print at each lock operation.
 * By default, only one line is logged for each alloc, free, and realloc.
 */
```

```
/* the if(!x);else avoids ``dangling else'' problems */
#define antagonism  if(!(p->flags & POOL_ANTAGONISM)){}else
#define paranoia    if(!(p->flags & POOL_PARANOIA)){}else
#define verbosity   if(!(p->flags & POOL_VERBOSITY)){}else

#define DPRINT  if(!(p->flags & POOL_DEBUGGING)){}else p->print
#define LOG     if(!(p->flags & POOL_LOGGING)){}else p->print
```

⟨*function* checklist 237c⟩

⟨*function* checktree 237d⟩

⟨*function* ltreewalk 238a⟩

⟨*function* treelookup 238b⟩

⟨*function* treeinsert 238c⟩

⟨*function* treedelete 238d⟩

⟨*function* treelookupgt 239a⟩

```
/*
 * List maintenance
 */
```

⟨*function* listadd 239b⟩

⟨*function* listdelete 240a⟩

```
/*
 * Pool maintenance
 */
```

⟨*function* pooladd 240b⟩

⟨*function* pooldel 240c⟩

```
/*
 * Block maintenance
 */
```

⟨*function* dsize2bsize 241a⟩

⟨*function* bsize2asize 241b⟩

```
/* blockmerge: merge a and b, known to be adjacent */
```

⟨*function* blockmerge 241c⟩

⟨*function* blocksetsize 242a⟩

⟨*function* getdsize 242b⟩

```
/* blocksetdsize: set the user data size of a block */
static Alloc*
blocksetdsize(Pool *p, Alloc *b, ulong dsize)
{
    Btail *t;
    uchar *q, *eq;

    assert(b->size >= dsize2bsize(p, dsize));
```

```
    assert(b->size - dsize < 0x10000);

    t = B2T(b);
    PSHORT(t->datasize, b->size - dsize);

    q=(uchar*)_B2D(b)+dsize;
    eq = (uchar*)t;
    if(eq > q+4)
        eq = q+4;
    for(; q<eq; q++)
        *q = datamagic[((ulong)(uintptr)q)%nelem(datamagic)];

    return b;
}
```

⟨*function* trim 242c⟩

⟨*function* freefromfront 243a⟩

```
/*
 * Arena maintenance
 */
```

⟨*function* arenasetsize 243b⟩

⟨*function* poolnewarena 243c⟩

```
/* blockresize: grow a block to encompass space past its end, possibly by */
```
⟨*function* blockgrow 244⟩

⟨*function* arenamerge 245a⟩

⟨*function* dumpblock 245b⟩

⟨*function* printblock 246a⟩

⟨*function* panicblock 246b⟩

```
/* blockcheck: ensure a block consistent with our expectations */
```
⟨*function* blockcheck 246c⟩

⟨*enum* _anon_ (port/pool.c)*8* 248a⟩

⟨*function* arenacompact 248b⟩

⟨*function* poolcompactl 248c⟩

```
/*
static int
poolcompactl(Pool*)
{
    return 0;
}
*/

/*
 * Actual allocators
 */
```

⟨*function* B2D 249a⟩

⟨*function* D2B 249b⟩

⟨*function* poolallocl 250a⟩

⟨*function* poolreallocl 250b⟩

⟨*function* alignptr 251⟩

⟨*function* poolallocalignl 252⟩

⟨*function* poolfreel 253⟩

⟨*function* poolalloc 254a⟩

⟨*function* poolallocalign 254b⟩

⟨*function* poolcompact 255a⟩

⟨*function* poolrealloc 255b⟩

⟨*function* poolfree 255c⟩

⟨*function* poolmsize 256a⟩

⟨*function* poolcheckarena 256b⟩

⟨*function* poolcheckl 257a⟩

⟨*function* poolcheck 257b⟩

⟨*function* poolblockcheck 257c⟩

⟨*function* pooldumpl 257d⟩

⟨*function* pooldump 258a⟩

⟨*function* pooldumparena 258b⟩

⟨*function* memmark 258c⟩

A.2.51 libc/port/pow.c

⟨libc/port/pow.c 263a⟩≡

```
#include <u.h>
#include <libc.h>
```

⟨*function* pow 165⟩

A.2.52 libc/port/pow10.c

⟨libc/port/pow10.c 263b⟩≡

```
#include    <u.h>
#include    <libc.h>
```

⟨*global* tab 166b⟩

⟨*function* pow10 166a⟩

A.2.53 libc/port/profile.c

⟨*function* _profdump 264⟩≡ (265)

```
// called by??
void
_profdump(void)
{
    int f;
    long n;
    Plink *p;
    char *vp;
    char filename[64];

    if (_tos->prof.what == 0)
        return; /* No profiling */
    if (_tos->prof.pid != 0 && _tos->pid != _tos->prof.pid)
        return; /* Not our process */
    if(perr)
        fprint(2, "%lud Prof errors\n", perr);
    _tos->prof.pp = nil;
    if (_tos->prof.pid)
        snprint(filename, sizeof filename - 1, "prof.%ld", _tos->prof.pid);
    else
        snprint(filename, sizeof filename - 1, "prof.out");
    f = create(filename, 1, 0666);
    if(f < 0) {
        perror("create prof.out");
        return;
    }
    _tos->prof.pid = ~0;    /* make sure data gets dumped once */
    switch(_tos->prof.what){
    case Profkernel:
        cycles((uvlong*)&_tos->prof.first->time);
        _tos->prof.first->time = _tos->prof.first->time + _tos->pcycles;
        break;
    case Profuser:
        cycles((uvlong*)&_tos->prof.first->time);
        _tos->prof.first->time = _tos->prof.first->time - _tos->kcycles;
        break;
    case Proftime:
        cycles((uvlong*)&_tos->prof.first->time);
        break;
    case Profsample:
        _tos->prof.first->time = _tos->clock;
        break;
    }
    vp = (char*)_tos->prof.first;

    for(p = _tos->prof.first; p <= _tos->prof.next; p++) {

        /*
         * short down
         */
        n = 0xffff;
        if(p->down)
            n = p->down - _tos->prof.first;
        vp[0] = n>>8;
        vp[1] = n;

        /*
```

```
         * short right
         */
        n = 0xffff;
        if(p->link)
            n = p->link - _tos->prof.first;
        vp[2] = n>>8;
        vp[3] = n;
        vp += 4;

        /*
         * long pc
         */
        n = p->pc;
        vp[0] = n>>24;
        vp[1] = n>>16;
        vp[2] = n>>8;
        vp[3] = n;
        vp += 4;

        /*
         * long count
         */
        n = p->count;
        vp[0] = n>>24;
        vp[1] = n>>16;
        vp[2] = n>>8;
        vp[3] = n;
        vp += 4;

        /*
         * vlong time
         */
        if (havecycles){
            n = (vlong)(p->time / (vlong)khz);
        }else
            n = p->time;

        vp[0] = n>>24;
        vp[1] = n>>16;
        vp[2] = n>>8;
        vp[3] = n;
        vp += 4;
    }
    write(f, (char*)_tos->prof.first, vp - (char*)_tos->prof.first);
    close(f);
}
```

⟨libc/port/profile.c 265⟩≡

```
#include    <u.h>
#include    <libc.h>
#include    <tos.h>

extern  long    _callpc(void**);
extern  long    _savearg(void);

⟨global khz 184d⟩
⟨global perr 184e⟩
⟨global havecycles 184f⟩

typedef struct  Plink   Plink;
```

⟨*struct* Plink 184b⟩

```
#pragma profile off
```

⟨*function* _profin 185⟩

⟨*function* _profout 186a⟩

⟨*function* _profdump 264⟩

⟨*function* _profinit 186b⟩

⟨*function* _profmain 186c⟩

⟨*function* prof 184c⟩

```
#pragma profile on
```

A.2.54 libc/port/qsort.c

⟨libc/port/qsort.c 266a⟩≡

```
/*
 * qsort -- simple quicksort
 */
#include <u.h>
```

⟨*type* Sort 19b⟩

⟨*function* swapb 19d⟩

⟨*function* swapi 19e⟩

⟨*function* pivot 20a⟩

⟨*function* qsorts 20b⟩

⟨*function* qsort 19c⟩

A.2.55 libc/port/quote.c

⟨*global* doquote 266b⟩≡ (268c)

```
int (*doquote)(int);
```

⟨*function* unquotestrdup 266c⟩≡ (268c)

```
char*
unquotestrdup(char *s)
{
    char *t, *ret;
    int quoting;

    ret = s = strdup(s);    /* return unquoted copy */
    if(ret == nil)
        return ret;
    quoting = 0;
    t = s;  /* s is output string, t is input string */
    while(*t!='\0' && (quoting || (*t!=' ' && *t!='\t'))){
        if(*t != '\''){
            *s++ = *t++;
```

```
            continue;
        }
        /* *t is a quote */
        if(!quoting){
            quoting = 1;
            t++;
            continue;
        }
        /* quoting and we're on a quote */
        if(t[1] != '\''){
            /* end of quoted section; absorb closing quote */
            t++;
            quoting = 0;
            continue;
        }
        /* doubled quote; fold one quote into two */
        t++;
        *s++ = *t++;
    }
    if(t != s)
        memmove(s, t, strlen(t)+1);
    return ret;
}
```

⟨*function* unquoterunestrdup 267⟩≡ (268c)

```
Rune*
unquoterunestrdup(Rune *s)
{
    Rune *t, *ret;
    int quoting;

    ret = s = runestrdup(s);    /* return unquoted copy */
    if(ret == nil)
        return ret;
    quoting = 0;
    t = s;  /* s is output string, t is input string */
    while(*t!='\0' && (quoting || (*t!=' ' && *t!='\t'))){
        if(*t != '\''){
            *s++ = *t++;
            continue;
        }
        /* *t is a quote */
        if(!quoting){
            quoting = 1;
            t++;
            continue;
        }
        /* quoting and we're on a quote */
        if(t[1] != '\''){
            /* end of quoted section; absorb closing quote */
            t++;
            quoting = 0;
            continue;
        }
        /* doubled quote; fold one quote into two */
        t++;
        *s++ = *t++;
    }
    if(t != s)
        memmove(s, t, (runestrlen(t)+1)*sizeof(Rune));
```

```
    return ret;
}
```

⟨*function* quotestrdup 268a⟩≡ (268c)

```
char*
quotestrdup(char *s)
{
    char *t, *u, *ret;
    int quotelen;
    Rune r;

    if(_needsquotes(s, &quotelen) == 0)
        return strdup(s);

    ret = malloc(quotelen+1);
    if(ret == nil)
        return nil;
    u = ret;
    *u++ = '\'';
    for(t=s; *t; t++){
        r = *t;
        if(r == L'\'')
            *u++ = r;   /* double the quote */
        *u++ = r;
    }
    *u++ = '\'';
    *u = '\0';
    return ret;
}
```

⟨*function* quoterunestrdup 268b⟩≡ (268c)

```
Rune*
quoterunestrdup(Rune *s)
{
    Rune *t, *u, *ret;
    int quotelen;
    Rune r;

    if(_runeneedsquotes(s, &quotelen) == 0)
        return runestrdup(s);

    ret = malloc((quotelen+1)*sizeof(Rune));
    if(ret == nil)
        return nil;
    u = ret;
    *u++ = '\'';
    for(t=s; *t; t++){
        r = *t;
        if(r == L'\'')
            *u++ = r;   /* double the quote */
        *u++ = r;
    }
    *u++ = '\'';
    *u = '\0';
    return ret;
}
```

⟨libc/port/quote.c 268c⟩≡

```
#include <u.h>
#include <libc.h>
```

⟨*global* doquote 266b⟩

```
extern int _needsquotes(char*, int*);
extern int _runeneedsquotes(Rune*, int*);
```

⟨*function* unquotestrdup 266c⟩

⟨*function* unquoterunestrdup 267⟩

⟨*function* quotestrdup 268a⟩

⟨*function* quoterunestrdup 268b⟩

A.2.56 libc/port/rand.c

⟨libc/port/rand.c 269a⟩≡

```
#include    <u.h>
#include    <libc.h>
```

⟨*function* rand 174h⟩

A.2.57 libc/port/readn.c

⟨libc/port/readn.c 269b⟩≡

```
#include <u.h>
#include <libc.h>
```

⟨*function* readn 45d⟩

A.2.58 libc/port/rune.c

⟨*enum* _anon_ (port/rune.c) 269c⟩≡ (270c)

```
enum
{
    Bitx    = Bit(1),

    Tx  = T(1),         /* 1000 0000 */
    Rune1 = (1<<(Bit(0)+0*Bitx))-1, /* 0000 0000 0000 0000 0111 1111 */

    Maskx   = (1<<Bitx)-1,      /* 0011 1111 */
    Testx   = Maskx ^ 0xFF,     /* 1100 0000 */

    SurrogateMin    = 0xD800,
    SurrogateMax    = 0xDFFF,

    Bad = Runeerror,
};
```

⟨*function* runelen 269d⟩≡ (270c)

```
int
runelen(long c)
{
    Rune rune;
    char str[10];

    rune = c;
```

```
    return runetochar(str, &rune);
}
```

⟨*function* runenlen 270a⟩≡ (270c)

```
int
runenlen(Rune *r, int nrune)
{
    int nb, i;
    Rune c;

    nb = 0;
    while(nrune--) {
        c = *r++;
        if(c <= Rune1){
            nb++;
        } else {
            for(i = 2; i < UTFmax + 1; i++)
                if(c <= RuneX(i) || i == UTFmax){
                    nb += i;
                    break;
                }
        }
    }
    return nb;
}
```

⟨*function* fullrune 270b⟩≡ (270c)

```
int
fullrune(char *str, int n)
{
    int  i;
    Rune c;

    if(n <= 0)
        return 0;
    c = *(uchar*)str;
    if(c < Tx)
        return 1;
    for(i = 3; i < UTFmax + 1; i++)
        if(c < T(i))
            return n >= i - 1;
    return n >= UTFmax;
}
```

⟨libc/port/rune.c 270c⟩≡

```
#include    <u.h>
#include    <libc.h>

⟨macro Bit 36b⟩
⟨macro T 36c⟩
⟨macro RuneX 36d⟩

⟨enum _anon_ (port/rune.c) 269c⟩

⟨function chartorune 36e⟩

⟨function runetochar 37⟩

⟨function runelen 269d⟩
```

⟨*function* runenlen 270a⟩

⟨*function* fullrune 270b⟩

A.2.59 libc/port/runebase.c

A.2.60 libc/port/runebsearch.c

⟨libc/port/runebsearch.c 271a⟩≡

```
#include <u.h>
#include <libc.h>
```

⟨*function* _runebsearch 36a⟩

A.2.61 libc/port/runestrcat.c

⟨libc/port/runestrcat.c 271b⟩≡

```
#include <u.h>
#include <libc.h>
```

⟨*function* runestrcat 39d⟩

A.2.62 libc/port/runestrchr.c

⟨libc/port/runestrchr.c 271c⟩≡

```
#include <u.h>
#include <libc.h>
```

⟨*function* runestrchr 39a⟩

A.2.63 libc/port/runestrcmp.c

⟨libc/port/runestrcmp.c 271d⟩≡

```
#include <u.h>
#include <libc.h>
```

⟨*function* runestrcmp 38c⟩

A.2.64 libc/port/runestrcpy.c

⟨libc/port/runestrcpy.c 271e⟩≡

```
#include <u.h>
#include <libc.h>
```

⟨*function* runestrcpy 38b⟩

A.2.65 libc/port/runestrdup.c

⟨libc/port/runestrdup.c 271f⟩≡

```
#include <u.h>
#include <libc.h>
```

⟨*function* runestrdup 39c⟩

A.2.66 libc/port/runestrecpy.c

⟨*function* runestrecpy 272a⟩≡ (272b)

```
Rune*
runestrecpy(Rune *s1, Rune *es1, Rune *s2)
{
    if(s1 >= es1)
        return s1;

    while(*s1++ = *s2++){
        if(s1 == es1){
            *--s1 = '\0';
            break;
        }
    }
    return s1;
}
```

⟨libc/port/runestrecpy.c 272b⟩≡

```
#include <u.h>
#include <libc.h>

⟨function runestrecpy 272a⟩
```

A.2.67 libc/port/runestrlen.c

⟨libc/port/runestrlen.c 272c⟩≡

```
#include <u.h>
#include <libc.h>

⟨function runestrlen 38a⟩
```

A.2.68 libc/port/runestrncat.c

⟨*function* runestrncat 272d⟩≡ (272e)

```
Rune*
runestrncat(Rune *s1, Rune *s2, long n)
{
    Rune *os1;

    os1 = s1;
    s1 = runestrchr(s1, 0);
    while(*s1++ = *s2++)
        if(--n < 0) {
            s1[-1] = 0;
            break;
        }
    return os1;
}
```

⟨libc/port/runestrncat.c 272e⟩≡

```
#include <u.h>
#include <libc.h>

⟨function runestrncat 272d⟩
```

A.2.69 libc/port/runestrncmp.c

⟨*function* runestrncmp 273a⟩≡ (273b)

```
int
runestrncmp(Rune *s1, Rune *s2, long n)
{
    Rune c1, c2;

    while(n > 0) {
        c1 = *s1++;
        c2 = *s2++;
        n--;
        if(c1 != c2) {
            if(c1 > c2)
                return 1;
            return -1;
        }
        if(c1 == 0)
            break;
    }
    return 0;
}
```

⟨libc/port/runestrncmp.c 273b⟩≡

```
#include <u.h>
#include <libc.h>

⟨function runestrncmp 273a⟩
```

A.2.70 libc/port/runestrncpy.c

⟨*function* runestrncpy 273c⟩≡ (273d)

```
Rune*
runestrncpy(Rune *s1, Rune *s2, long n)
{
    int i;
    Rune *os1;

    os1 = s1;
    for(i = 0; i < n; i++)
        if((*s1++ = *s2++) == 0) {
            while(++i < n)
                *s1++ = 0;
            return os1;
        }
    return os1;
}
```

⟨libc/port/runestrncpy.c 273d⟩≡

```
#include <u.h>
#include <libc.h>

⟨function runestrncpy 273c⟩
```

A.2.71 libc/port/runestrrchr.c

⟨libc/port/runestrrchr.c 273e⟩≡

```
#include <u.h>
```

```
#include <libc.h>

⟨function runestrrchr 39b⟩
```

A.2.72 libc/port/runestrstr.c

```
⟨libc/port/runestrstr.c 274a⟩≡
#include <u.h>
#include <libc.h>

⟨function runestrstr 39e⟩
```

A.2.73 libc/port/runetype.c and runetypebody-6.2.0.h

```
⟨libc/port/runetype.c 274b⟩≡
#include    <u.h>
#include    <libc.h>

Rune*_runebsearch(Rune c, Rune *t, int n, int ne);

#include "runetypebody-6.2.0.h"
```

A.2.74 libc/port/sin.c

```
⟨libc/port/sin.c 274c⟩≡
/*
    C program for floating point sin/cos.
    Calls modf.
    There are no error exits.
    Coefficients are #3370 from Hart & Cheney (18.80D).
*/

#include <u.h>
#include <libc.h>

⟨constant p0 (port/sin.c) 167c⟩
⟨constant p1 (port/sin.c) 167d⟩
⟨constant p2 (port/sin.c) 167e⟩
⟨constant p3 (port/sin.c) 167f⟩
⟨constant p4 (port/sin.c) 167g⟩
⟨constant q0 (port/sin.c) 167h⟩
⟨constant q1 (port/sin.c) 167i⟩
⟨constant q2 (port/sin.c) 167j⟩
⟨constant q3 (port/sin.c) 167k⟩

⟨function sinus 168a⟩

⟨function cos 167a⟩

⟨function sin 167b⟩
```

A.2.75 libc/port/sinh.c

```
⟨global p0 274d⟩≡                                                    (276b)
/*
 * sinh(arg) returns the hyperbolic sine of its floating-
```

```
 * point argument.
 *
 * The exponential function is called for arguments
 * greater in magnitude than 0.5.
 *
 * A series is used for arguments smaller in magnitude than 0.5.
 * The coefficients are #2029 from Hart & Cheney. (20.36D)
 *
 * cosh(arg) is computed from the exponential function for
 * all arguments.
 */

static  double  p0  = -0.630767364049771699118478725 1e+6;
```

⟨*global* p1 275a⟩≡ (276b)

```
static  double  p1  = -0.899127202203950935539801351 1e+5;
```

⟨*global* p2 275b⟩≡ (276b)

```
static  double  p2  = -0.289421135598956380728466036 6e+4;
```

⟨*global* p3 275c⟩≡ (276b)

```
static  double  p3  = -0.263056321339749706281948 9e+2;
```

⟨*global* q0 275d⟩≡ (276b)

```
static  double  q0  = -0.630767364049771699121207727 7e+6;
```

⟨*global* q1 275e⟩≡ (276b)

```
static  double  q1   = 0.152151737879001907069648517 6e+5;
```

⟨*global* q2 275f⟩≡ (276b)

```
static  double  q2  = -0.173678953558233699533450911e+3;
```

⟨*function* sinh 275g⟩≡ (276b)

```
double
sinh(double arg)
{
    double temp, argsq;
    int sign;

    sign = 0;
    if(arg < 0) {
        arg = -arg;
        sign++;
    }
    if(arg > 21) {
        temp = exp(arg)/2;
        goto out;
    }
    if(arg > 0.5) {
        temp = (exp(arg) - exp(-arg))/2;
        goto out;
    }
    argsq = arg*arg;
    temp = (((p3*argsq+p2)*argsq+p1)*argsq+p0)*arg;
    temp /= (((argsq+q2)*argsq+q1)*argsq+q0);
out:
    if(sign)
        temp = -temp;
    return temp;
}
```

⟨*function* cosh 276a⟩≡ (276b)

```
double
cosh(double arg)
{
    if(arg < 0)
        arg = - arg;
    if(arg > 21)
        return exp(arg)/2;
    return (exp(arg) + exp(-arg))/2;
}
```

⟨libc/port/sinh.c 276b⟩≡

```
#include <u.h>
#include <libc.h>

⟨global p0 274d⟩
⟨global p1 275a⟩
⟨global p2 275b⟩
⟨global p3 275c⟩
⟨global q0 275d⟩
⟨global q1 275e⟩
⟨global q2 275f⟩

⟨function sinh 275g⟩

⟨function cosh 276a⟩
```

A.2.76 libc/port/sqrt.c

⟨libc/port/sqrt.c 276c⟩≡

```
/*
    sqrt returns the square root of its floating
    point argument. Newton's method.

    calls frexp
*/

#include <u.h>
#include <libc.h>

⟨function sqrt 163⟩
```

A.2.77 libc/port/strcat.c

⟨libc/port/strcat.c 276d⟩≡

```
#include <u.h>
#include <libc.h>

⟨function strcat 33b⟩
```

A.2.78 libc/port/strchr.c

⟨libc/port/strchr.c 276e⟩≡

```
#include <u.h>
#include <libc.h>

⟨function strchr 33c⟩
```

A.2.79 libc/port/strcmp.c

⟨libc/port/strcmp.c 277a⟩≡
```
#include <u.h>
#include <libc.h>

⟨function strcmp 34a⟩
```

A.2.80 libc/port/strcpy.c

⟨libc/port/strcpy.c 277b⟩≡
```
#include <u.h>
#include <libc.h>
⟨constant N 34b⟩

⟨function strcpy 34c⟩
```

A.2.81 libc/port/strcspn.c

⟨*constant N (port/strcspn.c)* 277c⟩≡ (277e)
```
#define N   256
```

⟨*function* strcspn 277d⟩≡ (277e)
```
long
strcspn(char *s, char *b)
{
    char map[N], *os;

    memset(map, 0, N);
    for(;;) {
        map[*(uchar*)b] = 1;
        if(*b++ == 0)
            break;
    }
    os = s;
    while(map[*(uchar*)s++] == 0)
        ;
    return s - os - 1;
}
```

⟨libc/port/strcspn.c 277e⟩≡
```
#include <u.h>
#include <libc.h>

⟨constant N (port/strcspn.c) 277c⟩

⟨function strcspn 277d⟩
```

A.2.82 libc/port/strdup.c

⟨libc/port/strdup.c 277f⟩≡
```
#include <u.h>
#include <libc.h>

⟨function strdup 35a⟩
```

A.2.83 libc/port/strecpy.c

⟨*function* strecpy 278a⟩≡ (278b)

```
char*
strecpy(char *to, char *e, char *from)
{
    if(to >= e)
        return to;
    to = memccpy(to, from, '\0', e - to);
    if(to == nil){
        to = e - 1;
        *to = '\0';
    }else{
        to--;
    }
    return to;
}
```

⟨libc/port/strecpy.c 278b⟩≡

```
#include <u.h>
#include <libc.h>

⟨function strecpy 278a⟩
```

A.2.84 libc/port/strlen.c

⟨libc/port/strlen.c 278c⟩≡

```
#include <u.h>
#include <libc.h>

⟨function strlen 33a⟩
```

A.2.85 libc/port/strncat.c

⟨*function* strncat 278d⟩≡ (278e)

```
char*
strncat(char *s1, char *s2, long n)
{
    char *os1;

    os1 = s1;
    while(*s1++)
        ;
    s1--;
    while(*s1++ = *s2++)
        if(--n < 0) {
            s1[-1] = 0;
            break;
        }
    return os1;
}
```

⟨libc/port/strncat.c 278e⟩≡

```
#include <u.h>
#include <libc.h>

⟨function strncat 278d⟩
```

A.2.86 libc/port/strncmp.c

⟨*function* strncmp 279a⟩≡ (279b)

```
int
strncmp(char *s1, char *s2, long n)
{
    unsigned c1, c2;

    while(n > 0) {
        c1 = *s1++;
        c2 = *s2++;
        n--;
        if(c1 != c2) {
            if(c1 > c2)
                return 1;
            return -1;
        }
        if(c1 == 0)
            break;
    }
    return 0;
}
```

⟨libc/port/strncmp.c 279b⟩≡

```
#include <u.h>
#include <libc.h>

⟨function strncmp 279a⟩
```

A.2.87 libc/port/strncpy.c

⟨*function* strncpy 279c⟩≡ (279d)

```
char*
strncpy(char *s1, char *s2, long n)
{
    int i;
    char *os1;

    os1 = s1;
    for(i = 0; i < n; i++)
        if((*s1++ = *s2++) == 0) {
            while(++i < n)
                *s1++ = 0;
            return os1;
        }
    return os1;
}
```

⟨libc/port/strncpy.c 279d⟩≡

```
#include <u.h>
#include <libc.h>

⟨function strncpy 279c⟩
```

A.2.88 libc/port/strpbrk.c

⟨*constant N (port/strpbrk.c)* 279e⟩≡ (280b)

```
#define N   256
```

⟨*function* strpbrk 280a⟩≡ (280b)

```
char*
strpbrk(char *cs, char *cb)
{
    char map[N];
    uchar *s=(uchar*)cs, *b=(uchar*)cb;

    memset(map, 0, N);
    for(;;) {
        map[*b] = 1;
        if(*b++ == 0)
            break;
    }
    while(map[*s++] == 0)
        ;
    if(*--s)
        return (char*)s;
    return 0;
}
```

⟨libc/port/strpbrk.c 280b⟩≡

```
#include <u.h>
#include <libc.h>
```

⟨*constant N (port/strpbrk.c)* 279e⟩

⟨*function* strpbrk 280a⟩

A.2.89 libc/port/strrchr.c

⟨libc/port/strrchr.c 280c⟩≡

```
#include <u.h>
#include <libc.h>
```

⟨*function* strrchr 33d⟩

A.2.90 libc/port/strspn.c

⟨*constant N (port/strspn.c)* 280d⟩≡ (281a)

```
#define N   256
```

⟨*function* strspn 280e⟩≡ (281a)

```
long
strspn(char *s, char *b)
{
    char map[N], *os;

    memset(map, 0, N);
    while(*b)
        map[*(uchar *)b++] = 1;
    os = s;
    while(map[*(uchar *)s++])
        ;
    return s - os - 1;
}
```

⟨libc/port/strspn.c 281a⟩≡

```
#include <u.h>
#include <libc.h>

⟨constant N (port/strspn.c) 280d⟩

⟨function strspn 280e⟩
```

A.2.91 libc/port/strstr.c

⟨libc/port/strstr.c 281b⟩≡

```
#include <u.h>
#include <libc.h>

⟨function strstr 35b⟩
```

A.2.92 libc/port/strtod.c

⟨libc/port/strtod.c 281c⟩≡

```
#include <u.h>
#include <libc.h>
#include <ctype.h>

⟨enum _anon_ (port/strtod.c) 158⟩

static  int xcmp(char*, char*);
static  int fpcmp(char*, ulong*);
static  void    frnorm(ulong*);
static  void    divascii(char*, int*, int*, int*);
static  void    mulascii(char*, int*, int*, int*);
static  void    divby(char*, int*, int);

typedef struct  Tab Tab;
⟨struct Tab 159a⟩

⟨function strtod 154b⟩

⟨function frnorm 159b⟩

⟨function fpcmp 159c⟩

⟨function _divby 160⟩

⟨function divby 161a⟩

⟨global tab1 161b⟩

⟨function divascii 161c⟩

⟨function mulby 161d⟩

⟨global tab2 162a⟩

⟨function mulascii 162b⟩

⟨function xcmp 162c⟩
```

A.2.93 libc/port/strtok.c

⟨*constant N (port/strtok.c)* 282a⟩≡ (282c)

```
#define N   256
```

⟨*function* strtok 282b⟩≡ (282c)

```
char*
strtok(char *s, char *b)
{
    static char *under_rock;
    char map[N], *os;

    memset(map, 0, N);
    while(*b)
        map[*(uchar*)b++] = 1;
    if(s == 0)
        s = under_rock;
    while(map[*(uchar*)s++])
        ;
    if(*--s == 0)
        return 0;
    os = s;
    while(map[*(uchar*)s] == 0)
        if(*s++ == 0) {
            under_rock = s-1;
            return os;
        }
    *s++ = 0;
    under_rock = s;
    return os;
}
```

⟨libc/port/strtok.c 282c⟩≡

```
#include <u.h>
#include <libc.h>
```

⟨*constant N (port/strtok.c)* 282a⟩

⟨*function* strtok 282b⟩

A.2.94 libc/port/strtol.c

⟨libc/port/strtol.c 282d⟩≡

```
#include <u.h>
#include <libc.h>
```

⟨*constant* LONG_MAX 151a⟩
⟨*constant* LONG_MIN 151b⟩

⟨*function* strtol 151c⟩

A.2.95 libc/port/strtoll.c

⟨*constant* VLONG_MAX 282e⟩≡ (284a)

```
#define VLONG_MAX   ~(1LL<<63)
```

⟨*constant* VLONG_MIN 282f⟩≡ (284a)

```
#define VLONG_MIN   (1LL<<63)
```

⟨*function* strtoll 283⟩≡ (284a)

```
vlong
strtoll(char *nptr, char **endptr, int base)
{
    char *p;
    vlong n, nn, m;
    int c, ovfl, v, neg, ndig;

    p = nptr;
    neg = 0;
    n = 0;
    ndig = 0;
    ovfl = 0;

    /*
     * White space
     */
    for(;; p++) {
        switch(*p) {
        case ' ':
        case '\t':
        case '\n':
        case '\f':
        case '\r':
        case '\v':
            continue;
        }
        break;
    }

    /*
     * Sign
     */
    if(*p=='-' || *p=='+')
        if(*p++ == '-')
            neg = 1;

    /*
     * Base
     */
    if(base==0){
        base = 10;
        if(*p == '0') {
            base = 8;
            if(p[1]=='x' || p[1]=='X') {
                p += 2;
                base = 16;
            }
        }
    } else
    if(base==16 && *p=='0') {
        if(p[1]=='x' || p[1]=='X')
            p += 2;
    } else
    if(base<0 || 36<base)
        goto Return;

    /*
     * Non-empty sequence of digits
     */
```

```
    m = VLONG_MAX/base;
    for(;; p++,ndig++) {
        c = *p;
        v = base;
        if('0'<=c && c<='9')
            v = c - '0';
        else
        if('a'<=c && c<='z')
            v = c - 'a' + 10;
        else
        if('A'<=c && c<='Z')
            v = c - 'A' + 10;
        if(v >= base)
            break;
        if(n > m)
            ovfl = 1;
        nn = n*base + v;
        if(nn < n)
            ovfl = 1;
        n = nn;
    }

Return:
    if(ndig == 0)
        p = nptr;
    if(endptr)
        *endptr = p;
    if(ovfl){
        if(neg)
            return VLONG_MIN;
        return VLONG_MAX;
    }
    if(neg)
        return -n;
    return n;
}
```

⟨libc/port/strtoll.c 284a⟩≡

```
#include <u.h>
#include <libc.h>

⟨constant VLONG_MAX 282e⟩
⟨constant VLONG_MIN 282f⟩

⟨function strtoll 283⟩
```

A.2.96 libc/port/strtoul.c

⟨*constant* ULONG_MAX 284b⟩≡ (286a)

```
#define ULONG_MAX   4294967295UL
```

⟨*function* strtoul 284c⟩≡ (286a)

```
ulong
strtoul(char *nptr, char **endptr, int base)
{
    char *p;
    ulong n, nn, m;
    int c, ovfl, neg, v, ndig;
```

```
p = nptr;
neg = 0;
n = 0;
ndig = 0;
ovfl = 0;

/*
 * White space
 */
for(;;p++){
    switch(*p){
    case ' ':
    case '\t':
    case '\n':
    case '\f':
    case '\r':
    case '\v':
        continue;
    }
    break;
}

/*
 * Sign
 */
if(*p=='-' || *p=='+')
    if(*p++ == '-')
        neg = 1;

/*
 * Base
 */
if(base==0){
    if(*p != '0')
        base = 10;
    else{
        base = 8;
        if(p[1]=='x' || p[1]=='X')
            base = 16;
    }
}
if(base<2 || 36<base)
    goto Return;
if(base==16 && *p=='0'){
    if(p[1]=='x' || p[1]=='X')
        if(('0' <= p[2] && p[2] <= '9')
         ||('a' <= p[2] && p[2] <= 'f')
         ||('A' <= p[2] && p[2] <= 'F'))
            p += 2;
}
/*
 * Non-empty sequence of digits
 */
n = 0;
m = ULONG_MAX/base;
for(;; p++,ndig++){
    c = *p;
    v = base;
    if('0'<=c && c<='9')
        v = c - '0';
```

```
        else if('a'<=c && c<='z')
            v = c - 'a' + 10;
        else if('A'<=c && c<='Z')
            v = c - 'A' + 10;
        if(v >= base)
            break;
        if(n > m)
            ovfl = 1;
        nn = n*base + v;
        if(nn < n)
            ovfl = 1;
        n = nn;
    }

    Return:
    if(ndig == 0)
        p = nptr;
    if(endptr)
        *endptr = p;
    if(ovfl)
        return ULONG_MAX;
    if(neg)
        return -n;
    return n;
}
```

⟨libc/port/strtoul.c 286a⟩≡

```
#include <u.h>
#include <libc.h>

⟨constant ULONG_MAX 284b⟩

⟨function strtoul 284c⟩
```

A.2.97 libc/port/strtoull.c

⟨*constant* UVLONG_MAX 286b⟩≡ (288a)

```
#define UVLONG_MAX  (1LL<<63)
```

⟨*function* strtoull 286c⟩≡ (288a)

```
uvlong
strtoull(char *nptr, char **endptr, int base)
{
    char *p;
    uvlong n, nn, m;
    int c, ovfl, v, neg, ndig;

    p = nptr;
    neg = 0;
    n = 0;
    ndig = 0;
    ovfl = 0;

    /*
     * White space
     */
    for(;; p++) {
        switch(*p) {
        case ' ':
```

```
    case '\t':
    case '\n':
    case '\f':
    case '\r':
    case '\v':
        continue;
    }
    break;
}

/*
 * Sign
 */
if(*p == '-' || *p == '+')
    if(*p++ == '-')
        neg = 1;

/*
 * Base
 */
if(base == 0) {
    base = 10;
    if(*p == '0') {
        base = 8;
        if(p[1] == 'x' || p[1] == 'X'){
            p += 2;
            base = 16;
        }
    }
} else
if(base == 16 && *p == '0') {
    if(p[1] == 'x' || p[1] == 'X')
        p += 2;
} else
if(base < 0 || 36 < base)
    goto Return;

/*
 * Non-empty sequence of digits
 */
m = UVLONG_MAX/base;
for(;; p++,ndig++) {
    c = *p;
    v = base;
    if('0' <= c && c <= '9')
        v = c - '0';
    else
    if('a' <= c && c <= 'z')
        v = c - 'a' + 10;
    else
    if('A' <= c && c <= 'Z')
        v = c - 'A' + 10;
    if(v >= base)
        break;
    if(n > m)
        ovfl = 1;
    nn = n*base + v;
    if(nn < n)
        ovfl = 1;
    n = nn;
```

```
    }

Return:
    if(ndig == 0)
        p = nptr;
    if(endptr)
        *endptr = p;
    if(ovfl)
        return UVLONG_MAX;
    if(neg)
        return -n;
    return n;
}
```

⟨libc/port/strtoull.c 288a⟩≡

```
#include <u.h>
#include <libc.h>

⟨constant UVLONG_MAX 286b⟩

⟨function strtoull 286c⟩
```

A.2.98 libc/port/tan.c

⟨libc/port/tan.c 288b⟩≡

```
/*
    floating point tangent

    A series is used after range reduction.
    Coefficients are #4285 from Hart & Cheney. (19.74D)
 */

#include <u.h>
#include <libc.h>

⟨global p0 (port/tan.c) 168b⟩
⟨global p1 (port/tan.c) 168c⟩
⟨global p2 (port/tan.c) 168d⟩
⟨global p3 (port/tan.c) 168e⟩
⟨global p4 168f⟩
⟨global q0 (port/tan.c) 168g⟩
⟨global q1 (port/tan.c) 168h⟩
⟨global q2 (port/tan.c) 168i⟩

⟨function tan 169a⟩
```

A.2.99 libc/port/tanh.c

⟨function tanh 288c⟩≡ (289a)

```
/*
    tanh(arg) computes the hyperbolic tangent of its floating
    point argument.

    sinh and cosh are called except for large arguments, which
    would cause overflow improperly.
 */

double
```

```
tanh(double arg)
{

    if(arg < 0) {
        arg = -arg;
        if(arg > 21)
            return -1;
        return -sinh(arg)/cosh(arg);
    }
    if(arg > 21)
        return 1;
    return sinh(arg)/cosh(arg);
}
```

⟨libc/port/tanh.c 289a⟩≡

```
#include <u.h>
#include <libc.h>

⟨function tanh 288c⟩
```

A.2.100 libc/port/tokenize.c

⟨*global* qsep 289b⟩≡ (291a)

```
static char qsep[] = " \t\r\n";
```

⟨*function* qtoken 289c⟩≡ (291a)

```
static char*
qtoken(char *s, char *sep)
{
    int quoting;
    char *t;

    quoting = 0;
    t = s;  /* s is output string, t is input string */
    while(*t!='\0' && (quoting || utfrune(sep, *t)==nil)){
        if(*t != '\''){
            *s++ = *t++;
            continue;
        }
        /* *t is a quote */
        if(!quoting){
            quoting = 1;
            t++;
            continue;
        }
        /* quoting and we're on a quote */
        if(t[1] != '\''){
            /* end of quoted section; absorb closing quote */
            t++;
            quoting = 0;
            continue;
        }
        /* doubled quote; fold one quote into two */
        t++;
        *s++ = *t++;
    }
    if(*s != '\0'){
        *s = '\0';
        if(t == s)
```

```
            t++;
        }
        return t;
    }
```

⟨*function* etoken 290a⟩≡ (291a)

```
    static char*
    etoken(char *t, char *sep)
    {
        int quoting;

        /* move to end of next token */
        quoting = 0;
        while(*t!='\0' && (quoting || utfrune(sep, *t)==nil)){
            if(*t != '\''){
                t++;
                continue;
            }
            /* *t is a quote */
            if(!quoting){
                quoting = 1;
                t++;
                continue;
            }
            /* quoting and we're on a quote */
            if(t[1] != '\''){
                /* end of quoted section; absorb closing quote */
                t++;
                quoting = 0;
                continue;
            }
            /* doubled quote; fold one quote into two */
            t += 2;
        }
        return t;
    }
```

⟨*function* gettokens 290b⟩≡ (291a)

```
    int
    gettokens(char *s, char **args, int maxargs, char *sep)
    {
        int nargs;

        for(nargs=0; nargs<maxargs; nargs++){
            while(*s!='\0' && utfrune(sep, *s)!=nil)
                *s++ = '\0';
            if(*s == '\0')
                break;
            args[nargs] = s;
            s = etoken(s, sep);
        }

        return nargs;
    }
```

⟨*function* tokenize 290c⟩≡ (291a)

```
    int
    tokenize(char *s, char **args, int maxargs)
    {
        int nargs;
```

```
    for(nargs=0; nargs<maxargs; nargs++){
        while(*s!='\0' && utfrune(qsep, *s)!=nil)
            s++;
        if(*s == '\0')
            break;
        args[nargs] = s;
        s = qtoken(s, qsep);
    }

    return nargs;
}
```

⟨libc/port/tokenize.c 291a⟩≡

```
#include <u.h>
#include <libc.h>

⟨global qsep 289b⟩

⟨function qtoken 289c⟩

⟨function etoken 290a⟩

⟨function gettokens 290b⟩

⟨function tokenize 290c⟩
```

A.2.101 libc/port/toupper.c

⟨libc/port/toupper.c 291b⟩≡

```
#include    <ctype.h>

⟨function toupper 17d⟩

⟨function tolower 18b⟩
```

A.2.102 libc/port/u16.c

⟨global t16e 291c⟩≡ (292b)

```
static char t16e[] = "0123456789ABCDEF";
```

⟨function dec16 291d⟩≡ (292b)

```
int
dec16(uchar *out, int lim, char *in, int n)
{
    int c, w = 0, i = 0;
    uchar *start = out;
    uchar *eout = out + lim;

    while(n-- > 0){
        c = *in++;
        if('0' <= c && c <= '9')
            c = c - '0';
        else if('a' <= c && c <= 'z')
            c = c - 'a' + 10;
        else if('A' <= c && c <= 'Z')
            c = c - 'A' + 10;
        else
```

```
            continue;
        w = (w<<4) + c;
        i++;
        if(i == 2){
            if(out + 1 > eout)
                goto exhausted;
            *out++ = w;
            w = 0;
            i = 0;
        }
    }
exhausted:
    return out - start;
}
```

⟨*function* enc16 292a⟩≡ (292b)

```
int
enc16(char *out, int lim, uchar *in, int n)
{
    uint c;
    char *eout = out + lim;
    char *start = out;

    while(n-- > 0){
        c = *in++;
        if(out + 2 >= eout)
            goto exhausted;
        *out++ = t16e[c>>4];
        *out++ = t16e[c&0xf];
    }
exhausted:
    *out = 0;
    return out - start;
}
```

⟨libc/port/u16.c 292b⟩≡

```
#include <u.h>
#include <libc.h>
⟨global t16e 291c⟩

⟨function dec16 291d⟩

⟨function enc16 292a⟩
```

A.2.103 libc/port/u32.c

⟨*function* dec32 292c⟩≡ (294a)

```
int
dec32(uchar *dest, int ndest, char *src, int nsrc)
{
    char *s, *tab;
    uchar *start;
    int i, u[8];

    if(ndest+1 < (5*nsrc+7)/8)
        return -1;
    start = dest;
    tab = "23456789abcdefghijkmnpqrstuvwxyz";
    while(nsrc>=8){
```

```
        for(i=0; i<8; i++){
            s = strchr(tab,(int)src[i]);
            u[i] = s ? s-tab : 0;
        }
        *dest++ = (u[0]<<3) | (0x7 & (u[1]>>2));
        *dest++ = ((0x3 & u[1])<<6) | (u[2]<<1) | (0x1 & (u[3]>>4));
        *dest++ = ((0xf & u[3])<<4) | (0xf & (u[4]>>1));
        *dest++ = ((0x1 & u[4])<<7) | (u[5]<<2) | (0x3 & (u[6]>>3));
        *dest++ = ((0x7 & u[6])<<5) | u[7];
        src  += 8;
        nsrc -= 8;
    }
    if(nsrc > 0){
        if(nsrc == 1 || nsrc == 3 || nsrc == 6)
            return -1;
        for(i=0; i<nsrc; i++){
            s = strchr(tab,(int)src[i]);
            u[i] = s ? s-tab : 0;
        }
        *dest++ = (u[0]<<3) | (0x7 & (u[1]>>2));
        if(nsrc == 2)
            goto out;
        *dest++ = ((0x3 & u[1])<<6) | (u[2]<<1) | (0x1 & (u[3]>>4));
        if(nsrc == 4)
            goto out;
        *dest++ = ((0xf & u[3])<<4) | (0xf & (u[4]>>1));
        if(nsrc == 5)
            goto out;
        *dest++ = ((0x1 & u[4])<<7) | (u[5]<<2) | (0x3 & (u[6]>>3));
    }
out:
    return dest-start;
}
```

⟨*function* enc32 293⟩≡ (294a)

```
int
enc32(char *dest, int ndest, uchar *src, int nsrc)
{
    char *tab, *start;
    int j;

    if(ndest <= (8*nsrc+4)/5 )
        return -1;
    start = dest;
    tab = "23456789abcdefghijkmnpqrstuvwxyz";
    while(nsrc>=5){
        j = (0x1f & (src[0]>>3));
        *dest++ = tab[j];
        j = (0x1c & (src[0]<<2)) | (0x03 & (src[1]>>6));
        *dest++ = tab[j];
        j = (0x1f & (src[1]>>1));
        *dest++ = tab[j];
        j = (0x10 & (src[1]<<4)) | (0x0f & (src[2]>>4));
        *dest++ = tab[j];
        j = (0x1e & (src[2]<<1)) | (0x01 & (src[3]>>7));
        *dest++ = tab[j];
        j = (0x1f & (src[3]>>2));
        *dest++ = tab[j];
        j = (0x18 & (src[3]<<3)) | (0x07 & (src[4]>>5));
        *dest++ = tab[j];
```

```
        j = (0x1f & (src[4]));
        *dest++ = tab[j];
        src  += 5;
        nsrc -= 5;
    }
    if(nsrc){
        j = (0x1f & (src[0]>>3));
        *dest++ = tab[j];
        j = (0x1c & (src[0]<<2));
        if(nsrc == 1)
            goto out;
        j |= (0x03 & (src[1]>>6));
        *dest++ = tab[j];
        j = (0x1f & (src[1]>>1));
        if(nsrc == 2)
            goto out;
        *dest++ = tab[j];
        j = (0x10 & (src[1]<<4));
        if(nsrc == 3)
            goto out;
        j |= (0x0f & (src[2]>>4));
        *dest++ = tab[j];
        j = (0x1e & (src[2]<<1));
        if(nsrc == 4)
            goto out;
        j |= (0x01 & (src[3]>>7));
        *dest++ = tab[j];
        j = (0x1f & (src[3]>>2));
        *dest++ = tab[j];
        j = (0x18 & (src[3]<<3));
 out:
        *dest++ = tab[j];
    }
    *dest = 0;
    return dest-start;
 }
```

⟨libc/port/u32.c 294a⟩≡

```
 #include <u.h>
 #include <libc.h>

 ⟨function dec32 292c⟩

 ⟨function enc32 293⟩
```

A.2.104 libc/port/u64.c

⟨*enum* _anon_ (port/u64.c) 294b⟩≡ (296b)

```
 enum {
     INVAL=  255
 };
```

⟨*global* t64d 294c⟩≡ (296b)

```
 static uchar t64d[256] = {
    INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,
    INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,
    INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,   62,INVAL,INVAL,INVAL,   63,
       52,   53,   54,   55,   56,   57,   58,   59,   60,   61,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,
    INVAL,    0,    1,    2,    3,    4,    5,    6,    7,    8,    9,   10,   11,   12,   13,   14,
```

```
      15,   16,   17,   18,   19,   20,   21,   22,   23,   24,   25,INVAL,INVAL,INVAL,INVAL,INVAL,
   INVAL,   26,   27,   28,   29,   30,   31,   32,   33,   34,   35,   36,   37,   38,   39,   40,
      41,   42,   43,   44,   45,   46,   47,   48,   49,   50,   51,INVAL,INVAL,INVAL,INVAL,INVAL,
   INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,
   INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,
   INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,
   INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,
   INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,
   INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,
   INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL,INVAL
};
```

⟨*global* t64e 295a⟩≡ (296b)

```
static char t64e[] = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/";
```

⟨*function* dec64 295b⟩≡ (296b)

```
int
dec64(uchar *out, int lim, char *in, int n)
{
    ulong b24;
    uchar *start = out;
    uchar *e = out + lim;
    int i, c;

    b24 = 0;
    i = 0;
    while(n-- > 0){

        c = t64d[*(uchar*)in++];
        if(c == INVAL)
            continue;
        switch(i){
        case 0:
            b24 = c<<18;
            break;
        case 1:
            b24 |= c<<12;
            break;
        case 2:
            b24 |= c<<6;
            break;
        case 3:
            if(out + 3 > e)
                goto exhausted;

            b24 |= c;
            *out++ = b24>>16;
            *out++ = b24>>8;
            *out++ = b24;
            i = -1;
            break;
        }
        i++;
    }
    switch(i){
    case 2:
        if(out + 1 > e)
            goto exhausted;
        *out++ = b24>>16;
```

```
        break;
    case 3:
        if(out + 2 > e)
            goto exhausted;
        *out++ = b24>>16;
        *out++ = b24>>8;
        break;
    }
exhausted:
    return out - start;
}
```

⟨*function* enc64 296a⟩≡ (296b)

```
int
enc64(char *out, int lim, uchar *in, int n)
{
    int i;
    ulong b24;
    char *start = out;
    char *e = out + lim;

    for(i = n/3; i > 0; i--){
        b24 = (*in++)<<16;
        b24 |= (*in++)<<8;
        b24 |= *in++;
        if(out + 4 >= e)
            goto exhausted;
        *out++ = t64e[(b24>>18)];
        *out++ = t64e[(b24>>12)&0x3f];
        *out++ = t64e[(b24>>6)&0x3f];
        *out++ = t64e[(b24)&0x3f];
    }

    switch(n%3){
    case 2:
        b24 = (*in++)<<16;
        b24 |= (*in)<<8;
        if(out + 4 >= e)
            goto exhausted;
        *out++ = t64e[(b24>>18)];
        *out++ = t64e[(b24>>12)&0x3f];
        *out++ = t64e[(b24>>6)&0x3f];
        *out++ = '=';
        break;
    case 1:
        b24 = (*in)<<16;
        if(out + 4 >= e)
            goto exhausted;
        *out++ = t64e[(b24>>18)];
        *out++ = t64e[(b24>>12)&0x3f];
        *out++ = '=';
        *out++ = '=';
        break;
    }
exhausted:
    *out = 0;
    return out - start;
}
```

⟨libc/port/u64.c 296b⟩≡

```
#include <u.h>
#include <libc.h>

⟨enum _anon_ (port/u64.c) 294b⟩

⟨global t64d 294c⟩
⟨global t64e 295a⟩

⟨function dec64 295b⟩

⟨function enc64 296a⟩
```

A.2.105 libc/port/utfecpy.c

⟨*function* utfecpy 297a⟩≡ (297b)

```
char*
utfecpy(char *to, char *e, char *from)
{
    char *end;

    if(to >= e)
        return to;
    end = memccpy(to, from, '\0', e - to);
    if(end == nil){
        end = e;
        while(end>to && (*--end&0xC0)==0x80)
            ;
        *end = '\0';
    }else{
        end--;
    }
    return end;
}
```

⟨libc/port/utfecpy.c 297b⟩≡

```
#include <u.h>
#include <libc.h>

⟨function utfecpy 297a⟩
```

A.2.106 libc/port/utflen.c

⟨libc/port/utflen.c 297c⟩≡

```
#include <u.h>
#include <libc.h>

⟨function utflen 40a⟩
```

A.2.107 libc/port/utfnlen.c

⟨*function* utfnlen 297d⟩≡ (298a)

```
int
utfnlen(char *s, long m)
{
    int c;
    long n;
    Rune rune;
```

```
    char *es;

    es = s + m;
    for(n = 0; s < es; n++) {
        c = *(uchar*)s;
        if(c < Runeself){
            if(c == '\0')
                break;
            s++;
            continue;
        }
        if(!fullrune(s, es-s))
            break;
        s += chartorune(&rune, s);
    }
    return n;
}
```

⟨libc/port/utfnlen.c 298a⟩≡

```
#include <u.h>
#include <libc.h>

⟨function utfnlen 297d⟩
```

A.2.108 libc/port/utfrrune.c

⟨libc/port/utfrrune.c 298b⟩≡

```
#include <u.h>
#include <libc.h>

⟨function utfrrune 41a⟩
```

A.2.109 libc/port/utfrune.c

⟨libc/port/utfrune.c 298c⟩≡

```
#include <u.h>
#include <libc.h>

⟨function utfrune 40b⟩
```

A.2.110 libc/port/utfutf.c

⟨libc/port/utfutf.c 298d⟩≡

```
#include <u.h>
#include <libc.h>

⟨function utfutf 41b⟩
```

A.3 lib_core/libc/9sys/

A.3.1 libc/9sys/abort.c

⟨libc/9sys/abort.c 298e⟩≡

```
#include <u.h>
#include <libc.h>
```

⟨*function* abort 179h⟩

A.3.2 libc/9sys/access.c

⟨libc/9sys/access.c 299a⟩≡
```
#include <u.h>
#include <libc.h>

⟨function access 51⟩
```

A.3.3 libc/9sys/announce.c

⟨*enum* _anon_ (9sys/announce.c) 299b⟩≡ (301a)
```
enum
{
    Maxpath=    256,
};
```

⟨*function* identtrans 299c⟩≡ (301a)
```
/*
 *  perform the identity translation (in case we can't reach cs)
 */
static int
identtrans(char *netdir, char *addr, char *naddr, int na, char *file, int nf)
{
    char proto[Maxpath];
    char *p;

    USED(nf);

    /* parse the protocol */
    strncpy(proto, addr, sizeof(proto));
    proto[sizeof(proto)-1] = 0;
    p = strchr(proto, '!');
    if(p)
        *p++ = 0;

    snprint(file, nf, "%s/%s/clone", netdir, proto);
    strncpy(naddr, p, na);
    naddr[na-1] = 0;

    return 1;
}
```

⟨*function* nettrans 299d⟩≡ (301a)
```
/*
 *  call up the connection server and get a translation
 */
static int
nettrans(char *addr, char *naddr, int na, char *file, int nf)
{
    int i, fd;
    char buf[Maxpath];
    char netdir[Maxpath];
    char *p, *p2;
    long n;
```

```
/*
 *  parse, get network directory
 */
p = strchr(addr, '!');
if(p == 0){
    werrstr("bad dial string: %s", addr);
    return -1;
}
if(*addr != '/'){
    strncpy(netdir, "/net", sizeof(netdir));
    netdir[sizeof(netdir) - 1] = 0;
} else {
    for(p2 = p; *p2 != '/'; p2--)
        ;
    i = p2 - addr;
    if(i == 0 || i >= sizeof(netdir)){
        werrstr("bad dial string: %s", addr);
        return -1;
    }
    strncpy(netdir, addr, i);
    netdir[i] = 0;
    addr = p2 + 1;
}

/*
 *  ask the connection server
 */
snprint(buf, sizeof(buf), "%s/cs", netdir);
fd = open(buf, ORDWR);
if(fd < 0)
    return identtrans(netdir, addr, naddr, na, file, nf);
if(write(fd, addr, strlen(addr)) < 0){
    close(fd);
    return -1;
}
seek(fd, 0, 0);
n = read(fd, buf, sizeof(buf)-1);
close(fd);
if(n <= 0)
    return -1;
buf[n] = 0;

/*
 *  parse the reply
 */
p = strchr(buf, ' ');
if(p == 0)
    return -1;
*p++ = 0;
strncpy(naddr, p, na);
naddr[na-1] = 0;

if(buf[0] == '/'){
    p = strchr(buf+1, '/');
    if(p == nil)
        p = buf;
    else
        p++;
}
snprint(file, nf, "%s/%s", netdir, p);
```

```
    return 0;
}
```

⟨libc/9sys/announce.c 301a⟩≡

```
#include <u.h>
#include <libc.h>
#include <ctype.h>

static int  nettrans(char*, char*, int na, char*, int);

⟨enum _anon_ (9sys/announce.c) 299b⟩

⟨function announce 144⟩

⟨function listen 145⟩

⟨function accept 146a⟩

⟨function reject 146b⟩

⟨function identtrans 299c⟩

⟨function nettrans 299d⟩
```

A.3.4 libc/9sys/convD2M.c

⟨function sizeD2M 301b⟩≡ (301c)

```
uint
sizeD2M(Dir *d)
{
    char *sv[4];
    int i, ns;

    sv[0] = d->name;
    sv[1] = d->uid;
    sv[2] = d->gid;
    sv[3] = d->muid;

    ns = 0;
    for(i = 0; i < 4; i++)
        if(sv[i])
            ns += strlen(sv[i]);

    return STATFIXLEN + ns;
}
```

⟨libc/9sys/convD2M.c 301c⟩≡

```
#include    <u.h>
#include    <libc.h>
#include    <fcall.h>

⟨function sizeD2M 301b⟩

⟨function convD2M 91⟩
```

A.3.5 libc/9sys/convM2D.c

⟨*function* statcheck 302a⟩≡ (302c)

```
int
statcheck(uchar *buf, uint nbuf)
{
    uchar *ebuf;
    int i;

    ebuf = buf + nbuf;

    if(nbuf < STATFIXLEN || nbuf != BIT16SZ + GBIT16(buf))
        return -1;

    buf += STATFIXLEN - 4 * BIT16SZ;

    for(i = 0; i < 4; i++){
        if(buf + BIT16SZ > ebuf)
            return -1;
        buf += BIT16SZ + GBIT16(buf);
    }

    if(buf != ebuf)
        return -1;

    return 0;
}
```

⟨*global* nullstring 302b⟩≡ (302c)

```
static char nullstring[] = "";
```

⟨libc/9sys/convM2D.c 302c⟩≡

```
#include    <u.h>
#include    <libc.h>
#include    <fcall.h>

⟨function statcheck 302a⟩

⟨global nullstring 302b⟩

⟨function convM2D 90d⟩
```

A.3.6 libc/9sys/convM2S.c

⟨libc/9sys/convM2S.c 302d⟩≡

```
#include    <u.h>
#include    <libc.h>
#include    <fcall.h>

⟨function gstring 83a⟩

⟨function gqid 83b⟩

⟨function convM2S 78d⟩
```

A.3.7 libc/9sys/convS2M.c

⟨libc/9sys/convS2M.c 303a⟩≡
```
#include    <u.h>
#include    <libc.h>
#include    <fcall.h>

⟨function pstring 87a⟩

⟨function pqid 87b⟩

⟨function stringsz 90a⟩

⟨function sizeS2M 87c⟩

⟨function convS2M 84⟩
```

A.3.8 libc/9sys/cputime.c

⟨libc/9sys/cputime.c 303b⟩≡
```
#include <u.h>
#include <libc.h>

⟨constant HZ 187a⟩

⟨function cputime 187b⟩
```

A.3.9 libc/9sys/ctime.c

⟨global dmsize 303c⟩≡ (307)
```
static  char    dmsize[12] =
{
    31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31
};
```

⟨constant TZSIZE 303d⟩≡ (307)
```
#define TZSIZE  150
```

⟨global timezone 303e⟩≡ (307)
```
static
struct
{
    char    stname[4];
    char    dlname[4];
    long    stdiff;
    long    dldiff;
    long    dlpairs[TZSIZE];
} timezone;
```

⟨function ctime 303f⟩≡ (307)
```
char*
ctime(long t)
{
    return asctime(localtime(t));
}
```

⟨*function* gmtime 304⟩≡ (307)

```
Tm*
gmtime(long tim)
{
    int d0, d1;
    long hms, day;
    static Tm xtime;

    /*
     * break initial number into days
     */
    hms = (ulong)tim % 86400L;
    day = (ulong)tim / 86400L;
    if(hms < 0) {
        hms += 86400L;
        day -= 1;
    }

    /*
     * generate hours:minutes:seconds
     */
    xtime.sec = hms % 60;
    d1 = hms / 60;
    xtime.min = d1 % 60;
    d1 /= 60;
    xtime.hour = d1;

    /*
     * day is the day number.
     * generate day of the week.
     * The addend is 4 mod 7 (1/1/1970 was Thursday)
     */

    xtime.wday = (day + 7340036L) % 7;

    /*
     * year number
     */
    if(day >= 0)
        for(d1 = 1970; day >= dysize(d1); d1++)
            day -= dysize(d1);
    else
        for (d1 = 1970; day < 0; d1--)
            day += dysize(d1-1);
    xtime.year = d1-1900;
    xtime.yday = d0 = day;

    /*
     * generate month
     */

    if(dysize(d1) == 366)
        dmsize[1] = 29;
    for(d1 = 0; d0 >= dmsize[d1]; d1++)
        d0 -= dmsize[d1];
    dmsize[1] = 28;
    xtime.mday = d0 + 1;
    xtime.mon = d1;
    strcpy(xtime.zone, "GMT");
    return &xtime;
```

```
}
```

⟨*function* dysize 305a⟩≡ (307)

```
static int
dysize(int y)
{

    if(y%4 == 0 && (y%100 != 0 || y%400 == 0))
        return 366;
    return 365;
}
```

⟨*function* ct_numb 305b⟩≡ (307)

```
static
void
ct_numb(char *cp, int n)
{

    cp[0] = ' ';
    if(n >= 10)
        cp[0] = (n/10)%10 + '0';
    cp[1] = n%10 + '0';
}
```

⟨*function* readtimezone 305c⟩≡ (307)

```
static
void
readtimezone(void)
{
    char buf[TZSIZE*11+30], *p;
    int i;

    memset(buf, 0, sizeof(buf));
    i = open("/env/timezone", 0);
    if(i < 0)
        goto error;
    if(read(i, buf, sizeof(buf)) >= sizeof(buf)){
        close(i);
        goto error;
    }
    close(i);
    p = buf;
    if(rd_name(&p, timezone.stname))
        goto error;
    if(rd_long(&p, &timezone.stdiff))
        goto error;
    if(rd_name(&p, timezone.dlname))
        goto error;
    if(rd_long(&p, &timezone.dldiff))
        goto error;
    for(i=0; i<TZSIZE; i++) {
        if(rd_long(&p, &timezone.dlpairs[i]))
            goto error;
        if(timezone.dlpairs[i] == 0)
            return;
    }

error:
    timezone.stdiff = 0;
    strcpy(timezone.stname, "GMT");
```

```
    timezone.dlpairs[0] = 0;
}
```

⟨*function* rd_name 306a⟩≡ (307)

```
static int
rd_name(char **f, char *p)
{
    int c, i;

    for(;;) {
        c = *(*f)++;
        if(c != ' ' && c != '\n')
            break;
    }
    for(i=0; i<3; i++) {
        if(c == ' ' || c == '\n')
            return 1;
        *p++ = c;
        c = *(*f)++;
    }
    if(c != ' ' && c != '\n')
        return 1;
    *p = 0;
    return 0;
}
```

⟨*function* rd_long 306b⟩≡ (307)

```
static int
rd_long(char **f, long *p)
{
    int c, s;
    long l;

    s = 0;
    for(;;) {
        c = *(*f)++;
        if(c == '-') {
            s++;
            continue;
        }
        if(c != ' ' && c != '\n')
            break;
    }
    if(c == 0) {
        *p = 0;
        return 0;
    }
    l = 0;
    for(;;) {
        if(c == ' ' || c == '\n')
            break;
        if(c < '0' || c > '9')
            return 1;
        l = l*10 + c-'0';
        c = *(*f)++;
    }
    if(s)
        l = -l;
    *p = l;
    return 0;
}
```

⟨libc/9sys/ctime.c 307⟩≡

```
/*
 * This routine converts time as follows.
 * The epoch is 0000 Jan 1 1970 GMT.
 * The argument time is in seconds since then.
 * The localtime(t) entry returns a pointer to an array
 * containing
 *
 *  seconds (0-59)
 *  minutes (0-59)
 *  hours (0-23)
 *  day of month (1-31)
 *  month (0-11)
 *  year-1970
 *  weekday (0-6, Sun is 0)
 *  day of the year
 *  daylight savings flag
 *
 * The routine gets the daylight savings time from the environment.
 *
 * asctime(tvec))
 * where tvec is produced by localtime
 * returns a ptr to a character string
 * that has the ascii time in the form
 *
 *                              \\
 *  Thu Jan 01 00:00:00 GMT 1970n0
 *  012345678901234567890123456789
 *  0     1     2
 *
 * ctime(t) just calls localtime, then asctime.
 */

#include <u.h>
#include <libc.h>

⟨global dmsize 303c⟩

/*
 * The following table is used for 1974 and 1975 and
 * gives the day number of the first day after the Sunday of the
 * change.
 */

static  int dysize(int);
static  void    ct_numb(char*, int);

⟨constant TZSIZE 303d⟩
static  void    readtimezone(void);
static  int rd_name(char**, char*);
static  int rd_long(char**, long*);
⟨global timezone 303e⟩

⟨function ctime 303f⟩

⟨function localtime 54a⟩

⟨function gmtime 304⟩

⟨function asctime 54b⟩
```

⟨*function* dysize 305a⟩

⟨*function* ct_numb 305b⟩

⟨*function* readtimezone 305c⟩

⟨*function* rd_name 306a⟩

⟨*function* rd_long 306b⟩

A.3.10 libc/9sys/dial.c

⟨*enum* _anon_ (9sys/dial.c) 308a⟩≡ (316b)

```
enum
{
    Maxstring   = 128,
    Maxpath     = 256,

    Maxcsreply  = 64*80,    /* this is probably overly generous */
    /*
     * this should be a plausible slight overestimate for non-interactive
     * use even if it's ridiculously long for interactive use.
     */
    Maxconnms   = 2*60*1000,    /* 2 minutes */
};
```

⟨*struct* DS *(9sys/dial.c)* 308b⟩≡ (316b)

```
struct DS {
    /* dist string */
    char    buf[Maxstring];
    char    *netdir;
    char    *proto;
    char    *rem;

    /* other args */
    char    *local;
    char    *dir;
    int *cfdp;
};
```

⟨*struct* Conn 308c⟩≡ (316b)

```
/*
 * malloc these; they need to be writable by this proc & all children.
 * the stack is private to each proc, and static allocation in the data
 * segment would not permit concurrent dials within a multi-process program.
 */
struct Conn {
    int pid;
    int dead;

    int dfd;
    int cfd;
    char    dir[NETPATHLEN+1];
    char    err[ERRMAX];
};
```

⟨*struct* Dest 309a⟩≡ (316b)

```
struct Dest {
    Conn    *conn;          /* allocated array */
    Conn    *connend;
    int nkid;

    long    oalarm;
    int naddrs;

    QLock   winlck;
    int winner;         /* index into conn[] */

    char    *nextaddr;
    char    addrlist[Maxcsreply];
};
```

⟨*function* connsalloc 309b⟩≡ (316b)

```
static int
connsalloc(Dest *dp, int addrs)
{
    Conn *conn;

    free(dp->conn);
    dp->connend = nil;
    assert(addrs > 0);

    dp->conn = mallocz(addrs * sizeof *dp->conn, 1);
    if(dp->conn == nil)
        return -1;
    dp->connend = dp->conn + addrs;
    for(conn = dp->conn; conn < dp->connend; conn++)
        conn->cfd = conn->dfd = -1;
    return 0;
}
```

⟨*function* freedest 309c⟩≡ (316b)

```
static void
freedest(Dest *dp)
{
    long oalarm;

    if (dp == nil)
        return;
    oalarm = dp->oalarm;
    free(dp->conn);
    free(dp);
    if (oalarm >= 0)
        alarm(oalarm);
}
```

⟨*function* closeopenfd 309d⟩≡ (316b)

```
static void
closeopenfd(int *fdp)
{
    if (*fdp >= 0) {
        close(*fdp);
        *fdp = -1;
    }
}
```

⟨*function* notedeath 310a⟩≡ (316b)

```
static void
notedeath(Dest *dp, char *exitsts)
{
    int i, n, pid;
    char *fields[5];            /* pid + 3 times + error */
    Conn *conn;

    for (i = 0; i < nelem(fields); i++)
        fields[i] = "";
    n = tokenize(exitsts, fields, nelem(fields));
    if (n < 4)
        return;
    pid = atoi(fields[0]);
    if (pid <= 0)
        return;
    for (conn = dp->conn; conn < dp->connend; conn++)
        if (conn->pid == pid && !conn->dead) {  /* it's one we know? */
            if (conn - dp->conn != dp->winner) {
                closeopenfd(&conn->dfd);
                closeopenfd(&conn->cfd);
            }
            strncpy(conn->err, fields[4], sizeof conn->err - 1);
            conn->err[sizeof conn->err - 1] = '\0';
            conn->dead = 1;
            return;
        }
    /* not a proc that we forked */
}
```

⟨*function* outstandingprocs 310b⟩≡ (316b)

```
static int
outstandingprocs(Dest *dp)
{
    Conn *conn;

    for (conn = dp->conn; conn < dp->connend; conn++)
        if (!conn->dead)
            return 1;
    return 0;
}
```

⟨*function* reap 310c⟩≡ (316b)

```
static int
reap(Dest *dp)
{
    char exitsts[2*ERRMAX];

    if (outstandingprocs(dp) && await(exitsts, sizeof exitsts) >= 0) {
        notedeath(dp, exitsts);
        return 0;
    }
    return -1;
}
```

⟨*function* fillinds 310d⟩≡ (316b)

```
static int
fillinds(DS *ds, Dest *dp)
{
    Conn *conn;
```

```
    if (dp->winner < 0)
        return -1;
    conn = &dp->conn[dp->winner];
    if (ds->cfdp)
        *ds->cfdp = conn->cfd;
    if (ds->dir) {
        strncpy(ds->dir, conn->dir, NETPATHLEN);
        ds->dir[NETPATHLEN-1] = '\0';
    }
    return conn->dfd;
}
```

⟨*function* connectwait 311a⟩≡ (316b)

```
static int
connectwait(Dest *dp, char *besterr)
{
    Conn *conn;

    /* wait for a winner or all attempts to time out */
    while (dp->winner < 0 && reap(dp) >= 0)
        ;

    /* kill all of our still-live kids & reap them */
    for (conn = dp->conn; conn < dp->connend; conn++)
        if (!conn->dead)
            postnote(PNPROC, conn->pid, "alarm");
    while (reap(dp) >= 0)
        ;

    /* rummage about and report some error string */
    for (conn = dp->conn; conn < dp->connend; conn++)
        if (conn - dp->conn != dp->winner && conn->dead &&
            conn->err[0]) {
            strncpy(besterr, conn->err, ERRMAX-1);
            besterr[ERRMAX-1] = '\0';
            break;
        }
    return dp->winner;
}
```

⟨*function* parsecs 311b⟩≡ (316b)

```
static int
parsecs(Dest *dp, char **clonep, char **destp)
{
    char *dest, *p;

    dest = strchr(dp->nextaddr, ' ');
    if(dest == nil) {
        p = strchr(dp->nextaddr, '\n');
        if(p)
            *p = '\0';
        werrstr("malformed clone cmd from cs `%s'", dp->nextaddr);
        if(p)
            *p = '\n';
        return -1;
    }
    *dest++ = '\0';
    p = strchr(dest, '\n');
    if(p == nil)
```

```
        return -1;
    *p++ = '\0';
    *clonep = dp->nextaddr;
    *destp = dest;
    dp->nextaddr = p;       /* advance to next line */
    return 0;
}
```

⟨*function* pickuperr 312a⟩≡ (316b)

```
static void
pickuperr(char *besterr, char *err)
{
    err[0] = '\0';
    errstr(err, ERRMAX);
    if(strstr(err, "does not exist") == 0)
        strcpy(besterr, err);
}
```

⟨*function* catcher 312b⟩≡ (316b)

```
static int
catcher(void *, char *s)
{
    return strstr(s, "alarm") != nil;
}
```

⟨*function* dialmulti 312c⟩≡ (316b)

```
/*
 * try all addresses in parallel and take the first one that answers;
 * this helps when systems have ip v4 and v6 addresses but are
 * only reachable from here on one (or some) of them.
 */
static int
dialmulti(DS *ds, Dest *dp)
{
    int rv, kid, kidme;
    char *clone, *dest;
    char besterr[ERRMAX];

    dp->winner = -1;
    dp->nkid = 0;
    while(dp->winner < 0 && *dp->nextaddr != '\0' &&
        parsecs(dp, &clone, &dest) >= 0) {
        kidme = dp->nkid++;     /* make private copy on stack */
        kid = rfork(RFPROC|RFMEM);  /* spin off a call attempt */
        if (kid < 0)
            --dp->nkid;
        else if (kid == 0) {
            char err[ERRMAX];

            /* only in kid, to avoid atnotify callbacks in parent */
            atnotify(catcher, 1);

            *besterr = '\0';
            rv = call(clone, dest, ds, dp, &dp->conn[kidme]);
            if(rv < 0)
                pickuperr(besterr, err);
            _exits(besterr);    /* avoid atexit callbacks */
        }
    }
    *besterr = '\0';
```

```
    rv = connectwait(dp, besterr);
    if(rv < 0)
        werrstr("%s", (*besterr? besterr: "unknown error"));
    return rv;
}
```

⟨*function* csdial*(9sys/dial.c)* 313⟩≡ (316b)

```
static int
csdial(DS *ds)
{
    int n, fd, rv, addrs, bleft;
    char c;
    char *addrp, *clone2, *dest;
    char buf[Maxstring], clone[Maxpath], err[ERRMAX], besterr[ERRMAX];
    Dest *dp;

    werrstr("");
    dp = mallocz(sizeof *dp, 1);
    if(dp == nil)
        return -1;
    dp->winner = -1;
    dp->oalarm = alarm(0);
    if (connsalloc(dp, 1) < 0) {        /* room for a single conn. */
        freedest(dp);
        return -1;
    }

    /*
     *  open connection server
     */
    snprint(buf, sizeof(buf), "%s/cs", ds->netdir);
    fd = open(buf, ORDWR);
    if(fd < 0){
        /* no connection server, don't translate */
        snprint(clone, sizeof(clone), "%s/%s/clone", ds->netdir, ds->proto);
        rv = call(clone, ds->rem, ds, dp, &dp->conn[0]);
        fillinds(ds, dp);
        freedest(dp);
        return rv;
    }

    /*
     *  ask connection server to translate
     *  e.g., net!cs.bell-labs.com!smtp
     */
    snprint(buf, sizeof(buf), "%s!%s", ds->proto, ds->rem);
    if(write(fd, buf, strlen(buf)) < 0){
        close(fd);
        freedest(dp);
        return -1;
    }

    /*
     *  read all addresses from the connection server:
     *  /net/tcp/clone 135.104.9.78!25
     *  /net/tcp/clone 2620:0:dc0:1805::29!25
     *
     *  assumes that we'll get one record per read.
     */
    seek(fd, 0, 0);
```

```
    addrs = 0;
    addrp = dp->nextaddr = dp->addrlist;
    bleft = sizeof dp->addrlist - 2;    /* 2 is room for \n\0 */
    while(bleft > 0 && (n = read(fd, addrp, bleft)) > 0) {
        if (addrp[n-1] != '\n')
            addrp[n++] = '\n';
        addrs++;
        addrp += n;
        bleft -= n;
    }
    *addrp = '\0';

    /*
     * if we haven't read all of cs's output, assume the last line might
     * have been truncated and ignore it.  we really don't expect this
     * to happen.
     */
    if (addrs > 0 && bleft <= 0 && read(fd, &c, 1) == 1)
        addrs--;
    close(fd);

    *besterr = 0;
    rv = -1;                /* pessimistic default */
    dp->naddrs = addrs;
    if (addrs == 0)
        werrstr("no address to dial");
    else if (addrs == 1) {
        /* common case: dial one address without forking */
        if (parsecs(dp, &clone2, &dest) >= 0 &&
            (rv = call(clone2, dest, ds, dp, &dp->conn[0])) < 0) {
            pickuperr(besterr, err);
            werrstr("%s", besterr);
        }
    } else if (connsalloc(dp, addrs) >= 0)
        rv = dialmulti(ds, dp);

    /* fill in results */
    if (rv >= 0 && dp->winner >= 0)
        rv = fillinds(ds, dp);

    freedest(dp);
    return rv;
}
```

⟨*function* call *(9sys/dial.c)* 314⟩≡ (316b)

```
static int
call(char *clone, char *dest, DS *ds, Dest *dp, Conn *conn)
{
    int fd, cfd, n, calleralarm, oalarm;
    char cname[Maxpath], name[Maxpath], data[Maxpath], *p;

    /* because cs is in a different name space, replace the mount point */
    if(*clone == '/'){
        p = strchr(clone+1, '/');
        if(p == nil)
            p = clone;
        else
            p++;
    } else
        p = clone;
```

```
snprint(cname, sizeof cname, "%s/%s", ds->netdir, p);

conn->pid = getpid();
conn->cfd = cfd = open(cname, ORDWR);
if(cfd < 0)
    return -1;

/* get directory name */
n = read(cfd, name, sizeof(name)-1);
if(n < 0){
    closeopenfd(&conn->cfd);
    return -1;
}
name[n] = 0;
for(p = name; *p == ' '; p++)
    ;
snprint(name, sizeof(name), "%ld", strtoul(p, 0, 0));
p = strrchr(cname, '/');
*p = 0;
if(ds->dir)
    snprint(conn->dir, NETPATHLEN, "%s/%s", cname, name);
snprint(data, sizeof(data), "%s/%s/data", cname, name);

/* should be no alarm pending now; re-instate caller's alarm, if any */
calleralarm = dp->oalarm > 0;
if (calleralarm)
    alarm(dp->oalarm);
else if (dp->naddrs > 1)    /* in a sub-process? */
    alarm(Maxconnms);

/* connect */
if(ds->local)
    snprint(name, sizeof(name), "connect %s %s", dest, ds->local);
else
    snprint(name, sizeof(name), "connect %s", dest);
if(write(cfd, name, strlen(name)) < 0){
    closeopenfd(&conn->cfd);
    return -1;
}

oalarm = alarm(0);  /* don't let alarm interrupt critical section */
if (calleralarm)
    dp->oalarm = oalarm;    /* time has passed, so update user's */

/* open data connection */
conn->dfd = fd = open(data, ORDWR);
if(fd < 0){
    closeopenfd(&conn->cfd);
    alarm(dp->oalarm);
    return -1;
}
if(ds->cfdp == nil)
    closeopenfd(&conn->cfd);

n = conn - dp->conn;
if (dp->winner < 0) {
    qlock(&dp->winlck);
    if (dp->winner < 0 && conn < dp->connend)
        dp->winner = n;
    qunlock(&dp->winlck);
```

```
    }
    alarm(calleralarm? dp->oalarm: 0);
    return fd;
}
```

⟨*function* _dial_string_parse*(9sys/dial.c)* 316a⟩≡ (316b)

```
/*
 *  parse a dial string
 */
static void
_dial_string_parse(char *str, DS *ds)
{
    char *p, *p2;

    strncpy(ds->buf, str, Maxstring);
    ds->buf[Maxstring-1] = 0;

    p = strchr(ds->buf, '!');
    if(p == 0) {
        ds->netdir = 0;
        ds->proto = "net";
        ds->rem = ds->buf;
    } else {
        if(*ds->buf != '/' && *ds->buf != '#'){
            ds->netdir = 0;
            ds->proto = ds->buf;
        } else {
            /* expecting /net.alt/tcp!foo or #I1/tcp!foo */
            for(p2 = p; p2 > ds->buf && *p2 != '/'; p2--)
                ;
            *p2++ = 0;
            ds->netdir = ds->buf;
            ds->proto = p2;
        }
        *p = 0;
        ds->rem = p + 1;
    }
}
```

⟨libc/9sys/dial.c 316b⟩≡

```
/*
 * dial - connect to a service (parallel version)
 */
#include <u.h>
#include <libc.h>

typedef struct Conn Conn;
typedef struct Dest Dest;
typedef struct DS DS;

⟨enum _anon_ (9sys/dial.c) 308a⟩

⟨struct DS(9sys/dial.c) 308b⟩

⟨struct Conn 308c⟩
⟨struct Dest 309a⟩

static int  call(char*, char*, DS*, Dest*, Conn*);
static int  csdial(DS*);
static void _dial_string_parse(char*, DS*);
```

⟨*function* dialimpl 147c⟩

⟨*global* _dial 147b⟩

⟨*function* dial 147a⟩

⟨*function* connsalloc 309b⟩

⟨*function* freedest 309c⟩

⟨*function* closeopenfd 309d⟩

⟨*function* notedeath 310a⟩

⟨*function* outstandingprocs 310b⟩

⟨*function* reap 310c⟩

⟨*function* fillinds 310d⟩

⟨*function* connectwait 311a⟩

⟨*function* parsecs 311b⟩

⟨*function* pickuperr 312a⟩

⟨*function* catcher 312b⟩

⟨*function* dialmulti 312c⟩

⟨*function* csdial*(9sys/dial.c)* 313⟩

⟨*function* call*(9sys/dial.c)* 314⟩

⟨*function* _dial_string_parse*(9sys/dial.c)* 316a⟩

A.3.11 libc/9sys/dirfstat.c

⟨libc/9sys/dirfstat.c 317a⟩≡

```
#include <u.h>
#include <libc.h>
#include <fcall.h>
```

⟨*enum* _anon_ (9sys/dirfstat.c) 47b⟩

⟨*function* dirfstat 47c⟩

A.3.12 libc/9sys/dirfwstat.c

⟨libc/9sys/dirfwstat.c 317b⟩≡

```
#include <u.h>
#include <libc.h>
#include <fcall.h>
```

⟨*function* dirfwstat 47d⟩

A.3.13 libc/9sys/dirmodefmt.c

⟨*global* modes 318a⟩≡ (318d)

```
static char *modes[] =
{
    "---",
    "--x",
    "-w-",
    "-wx",
    "r--",
    "r-x",
    "rw-",
    "rwx",
};
```

⟨*function* rwx 318b⟩≡ (318d)

```
static void
rwx(long m, char *s)
{
    strncpy(s, modes[m], 3);
}
```

⟨*function* dirmodefmt 318c⟩≡ (318d)

```
int
dirmodefmt(Fmt *f)
{
    static char buf[16];
    ulong m;

    m = va_arg(f->args, ulong);

    if(m & DMDIR)
        buf[0]='d';
    else if(m & DMAPPEND)
        buf[0]='a';
    else if(m & DMAUTH)
        buf[0]='A';
    else
        buf[0]='-';
    if(m & DMEXCL)
        buf[1]='l';
    else
        buf[1]='-';
    rwx((m>>6)&7, buf+2);
    rwx((m>>3)&7, buf+5);
    rwx((m>>0)&7, buf+8);
    buf[11] = 0;
    return fmtstrcpy(f, buf);
}
```

⟨libc/9sys/dirmodefmt.c 318d⟩≡

```
#include <u.h>
#include <libc.h>
#include <fcall.h>

⟨global modes 318a⟩

⟨function rwx 318b⟩

⟨function dirmodefmt 318c⟩
```

A.3.14 libc/9sys/dirread.c

⟨*function* dirpackage 319a⟩≡ (320b)

```
static
long
dirpackage(uchar *buf, long ts, Dir **d)
{
    char *s;
    long ss, i, n, nn, m;

    *d = nil;
    if(ts <= 0)
        return 0;

    /*
     * first find number of all stats, check they look like stats, & size all associated strings
     */
    ss = 0;
    n = 0;
    for(i = 0; i < ts; i += m){
        m = BIT16SZ + GBIT16(&buf[i]);
        if(statcheck(&buf[i], m) < 0)
            break;
        ss += m;
        n++;
    }

    if(i != ts)
        return -1;

    *d = malloc(n * sizeof(Dir) + ss);
    if(*d == nil)
        return -1;

    /*
     * then convert all buffers
     */
    s = (char*)*d + n * sizeof(Dir);
    nn = 0;
    for(i = 0; i < ts; i += m){
        m = BIT16SZ + GBIT16((uchar*)&buf[i]);
        if(nn >= n || convM2D(&buf[i], m, *d + nn, s) != m){
            free(*d);
            *d = nil;
            return -1;
        }
        nn++;
        s += m;
    }

    return nn;
}
```

⟨*function* dirread 319b⟩≡ (320b)

```
long
dirread(int fd, Dir **d)
{
    uchar *buf;
    long ts;
```

```
    buf = malloc(DIRMAX);
    if(buf == nil)
        return -1;
    ts = read(fd, buf, DIRMAX);
    if(ts >= 0)
        ts = dirpackage(buf, ts, d);
    free(buf);
    return ts;
}
```

⟨*function* dirreadall 320a⟩≡ (320b)

```
long
dirreadall(int fd, Dir **d)
{
    uchar *buf, *nbuf;
    long n, ts;

    buf = nil;
    ts = 0;
    for(;;){
        nbuf = realloc(buf, ts+DIRMAX);
        if(nbuf == nil){
            free(buf);
            return -1;
        }
        buf = nbuf;
        n = read(fd, buf+ts, DIRMAX);
        if(n <= 0)
            break;
        ts += n;
    }
    if(ts >= 0)
        ts = dirpackage(buf, ts, d);
    free(buf);
    if(ts == 0 && n < 0)
        return -1;
    return ts;
}
```

⟨libc/9sys/dirread.c 320b⟩≡

```
#include <u.h>
#include <libc.h>
#include <fcall.h>

⟨function dirpackage 319a⟩

⟨function dirread 319b⟩

⟨function dirreadall 320a⟩
```

A.3.15 libc/9sys/dirstat.c

⟨libc/9sys/dirstat.c 320c⟩≡

```
#include <u.h>
#include <libc.h>
#include <fcall.h>

⟨enum _anon_ (9sys/dirstat.c) 48a⟩

⟨function dirstat 48b⟩
```

A.3.16 libc/9sys/dirwstat.c

⟨libc/9sys/dirwstat.c 321a⟩≡
```
#include <u.h>
#include <libc.h>
#include <fcall.h>

⟨function dirwstat 48c⟩
```

A.3.17 libc/9sys/fcallfmt.c

⟨*function* dirfmt 321b⟩≡ (322a)
```
int
dirfmt(Fmt *fmt)
{
    char buf[160];

    fdirconv(buf, buf+sizeof buf, va_arg(fmt->args, Dir*));
    return fmtstrcpy(fmt, buf);
}
```

⟨*function* fdirconv 321c⟩≡ (322a)
```
static void
fdirconv(char *buf, char *e, Dir *d)
{
    char tmp[16];

    seprint(buf, e, "'%s' '%s' '%s' '%s' "
        "q " QIDFMT " m %#luo "
        "at %ld mt %ld l %lld "
        "t %d d %d",
            d->name, d->uid, d->gid, d->muid,
            d->qid.path, d->qid.vers, qidtype(tmp, d->qid.type), d->mode,
            d->atime, d->mtime, d->length,
            d->type, d->dev);
}
```

⟨*constant* DUMPL 321d⟩≡ (322a)
```
/*
 * dump out count (or DUMPL, if count is bigger) bytes from
 * buf to ans, as a string if they are all printable,
 * else as a series of hex bytes
 */
#define DUMPL 64
```

⟨*function* dumpsome 321e⟩≡ (322a)
```
static uint
dumpsome(char *ans, char *e, char *buf, long count)
{
    int i, printable;
    char *p;

    if(buf == nil){
        seprint(ans, e, "<no data>");
        return strlen(ans);
    }
    printable = 1;
    if(count > DUMPL)
        count = DUMPL;
```

```
    for(i=0; i<count && printable; i++)
        if((buf[i]<32 && buf[i] !='\n' && buf[i] !='\t') || (uchar)buf[i]>127)
            printable = 0;
    p = ans;
    *p++ = '\'';
    if(printable){
        if(count > e-p-2)
            count = e-p-2;
        memmove(p, buf, count);
        p += count;
    }else{
        if(2*count > e-p-2)
            count = (e-p-2)/2;
        for(i=0; i<count; i++){
            if(i>0 && i%4==0)
                *p++ = ' ';
            sprint(p, "%2.2ux", (uchar)buf[i]);
            p += 2;
        }
    }
    *p++ = '\'';
    *p = 0;
    return p - ans;
}
```

⟨libc/9sys/fcallfmt.c 322a⟩≡

```
#include <u.h>
#include <libc.h>
#include <fcall.h>

static uint dumpsome(char*, char*, char*, long);
static void fdirconv(char*, char*, Dir*);
static char *qidtype(char*, uchar);

⟨constant QIDFMT 74c⟩

⟨function fcallfmt 75⟩

⟨function qidtype 77a⟩

⟨function dirfmt 321b⟩

⟨function fdirconv 321c⟩

⟨constant DUMPL 321d⟩

⟨function dumpsome 321e⟩
```

A.3.18 libc/9sys/fork.c

⟨libc/9sys/fork.c 322b⟩≡

```
#include <u.h>
#include <libc.h>

⟨function fork 58b⟩
```

A.3.19 libc/9sys/getenv.c

⟨libc/9sys/getenv.c 323a⟩≡

```
#include <u.h>
#include <libc.h>

⟨function getenv 70a⟩
```

A.3.20 libc/9sys/getnetconninfo.c

⟨global unknown 323b⟩≡ (325c)

```
static char *unknown = "???";
```

⟨function getendpoint 323c⟩≡ (325c)

```
static void
getendpoint(char *dir, char *file, char **sysp, char **servp)
{
    int fd, n;
    char buf[128];
    char *sys, *serv;

    sys = serv = 0;

    snprint(buf, sizeof buf, "%s/%s", dir, file);
    fd = open(buf, OREAD);
    if(fd >= 0){
        n = read(fd, buf, sizeof(buf)-1);
        if(n>0){
            buf[n-1] = 0;
            serv = strchr(buf, '!');
            if(serv){
                *serv++ = 0;
                serv = strdup(serv);
            }
            sys = strdup(buf);
        }
        close(fd);
    }
    if(serv == 0)
        serv = unknown;
    if(sys == 0)
        sys = unknown;
    *servp = serv;
    *sysp = sys;
}
```

⟨function getnetconninfo 323d⟩≡ (325c)

```
NetConnInfo*
getnetconninfo(char *dir, int fd)
{
    NetConnInfo *nci;
    char *cp;
    Dir *d;
    char spec[10];
    char path[128];
    char netname[128], *p;

    /* get a directory address via fd */
    if(dir == nil || *dir == 0){
```

```
        if(fd2path(fd, path, sizeof(path)) < 0)
            return nil;
        cp = strrchr(path, '/');
        if(cp == nil)
            return nil;
        *cp = 0;
        dir = path;
    }

    nci = mallocz(sizeof *nci, 1);
    if(nci == nil)
        return nil;

    /* copy connection directory */
    nci->dir = strdup(dir);
    if(nci->dir == nil)
        goto err;

    /* get netroot */
    nci->root = strdup(dir);
    if(nci->root == nil)
        goto err;
    cp = strchr(nci->root+1, '/');
    if(cp == nil)
        goto err;
    *cp = 0;

    /* figure out bind spec */
    d = dirstat(nci->dir);
    if(d != nil){
        sprint(spec, "#%C%d", d->type, d->dev);
        nci->spec = strdup(spec);
    }
    if(nci->spec == nil)
        nci->spec = unknown;
    free(d);

    /* get the two end points */
    getendpoint(nci->dir, "local", &nci->lsys, &nci->lserv);
    if(nci->lsys == nil || nci->lserv == nil)
        goto err;
    getendpoint(nci->dir, "remote", &nci->rsys, &nci->rserv);
    if(nci->rsys == nil || nci->rserv == nil)
        goto err;

    strecpy(netname, netname+sizeof netname, nci->dir);
    if((p = strrchr(netname, '/')) != nil)
        *p = 0;
    if(strncmp(netname, "/net/", 5) == 0)
        memmove(netname, netname+5, strlen(netname+5)+1);
    nci->laddr = smprint("%s!%s!%s", netname, nci->lsys, nci->lserv);
    nci->raddr = smprint("%s!%s!%s", netname, nci->rsys, nci->rserv);
    if(nci->laddr == nil || nci->raddr == nil)
        goto err;
    return nci;
err:
    freenetconninfo(nci);
    return nil;
}
```

⟨*function* xfree 325a⟩≡ (325c)

```
static void
xfree(char *x)
{
    if(x == nil || x == unknown)
        return;
    free(x);
}
```

⟨*function* freenetconninfo 325b⟩≡ (325c)

```
void
freenetconninfo(NetConnInfo *nci)
{
    if(nci == nil)
        return;
    xfree(nci->root);
    xfree(nci->dir);
    xfree(nci->spec);
    xfree(nci->lsys);
    xfree(nci->lserv);
    xfree(nci->rsys);
    xfree(nci->rserv);
    xfree(nci->laddr);
    xfree(nci->raddr);
    free(nci);
}
```

⟨libc/9sys/getnetconninfo.c 325c⟩≡

```
#include <u.h>
#include <libc.h>

⟨global unknown 323b⟩

⟨function getendpoint 323c⟩

⟨function getnetconninfo 323d⟩

⟨function xfree 325a⟩

⟨function freenetconninfo 325b⟩
```

A.3.21 libc/9sys/getpid.c

⟨libc/9sys/getpid.c 325d⟩≡

```
#include    <u.h>
#include    <libc.h>

⟨function getpid 59a⟩
```

A.3.22 libc/9sys/getppid.c

⟨libc/9sys/getppid.c 325e⟩≡

```
#include    <u.h>
#include    <libc.h>

⟨function getppid 59b⟩
```

A.3.23 libc/9sys/getwd.c

⟨libc/9sys/getwd.c 326a⟩≡
```
#include <u.h>
#include <libc.h>

static char *nsgetwd(char*, int);

⟨function getwd 49a⟩
```

A.3.24 libc/9sys/iounit.c

⟨*function* iounit 326b⟩≡ (326c)
```
/*
 * Format:
  3 r  M    4 (0000000000457def 11 00)   8192      512 /rc/lib/rcmain
 */

int
iounit(int fd)
{
    int i, cfd;
    char buf[128], *args[10];

    snprint(buf, sizeof buf, "#d/%dctl", fd);
    cfd = open(buf, OREAD);
    if(cfd < 0)
        return 0;
    i = read(cfd, buf, sizeof buf-1);
    close(cfd);
    if(i <= 0)
        return 0;
    buf[i] = '\0';
    if(tokenize(buf, args, nelem(args)) != nelem(args))
        return 0;
    return atoi(args[7]);
}
```

⟨libc/9sys/iounit.c 326c⟩≡
```
#include <u.h>
#include <libc.h>

⟨function iounit 326b⟩
```

A.3.25 libc/9sys/nsec.c

⟨*global* order 326d⟩≡ (328a)
```
static uvlong order = 0x0001020304050607ULL;
```

⟨*function* be2vlong 326e⟩≡ (328a)
```
static void
be2vlong(vlong *to, uchar *f)
{
    uchar *t, *o;
    int i;

    t = (uchar*)to;
    o = (uchar*)&order;
```

```
    for(i = 0; i < sizeof order; i++)
        t[o[i]] = f[i];
}
```

⟨*global* fd 327a⟩≡ (328a)

```
static int fd = -1;
```

⟨*global* fds 327b⟩≡ (328a)

```
static struct {
    int pid;
    int fd;
} fds[64];
```

⟨*function* nsec 327c⟩≡ (328a)

```
vlong
nsec(void)
{
    uchar b[8];
    vlong t;
    int pid, i, f, tries;

    /*
     * Threaded programs may have multiple procs
     * with different fd tables, so we may need to open
     * /dev/bintime on a per-pid basis
     */

    /* First, look if we've opened it for this particular pid */
    if((pid = _tos->pid) == 0)      /* 9vx bug, perhaps? */
        _tos->pid = pid = getpid();
    do{
        f = -1;
        for(i = 0; i < nelem(fds); i++)
            if(fds[i].pid == pid){
                f = fds[i].fd;
                break;
            }
        tries = 0;
        if(f < 0){
            /* If it's not open for this pid, try the global pid */
            if(fd >= 0)
                f = fd;
            else{
                /* must open */
                if((f = open("/dev/bintime", OREAD|OCEXEC)) < 0)
                    return 0;
                fd = f;
                for(i = 0; i < nelem(fds); i++)
                    if(fds[i].pid == pid || fds[i].pid == 0){
                        fds[i].pid = pid;
                        fds[i].fd = f;
                        break;
                    }
            }
        }
        if(pread(f, b, sizeof b, 0) == sizeof b){
            be2vlong(&t, b);
            return t;
        }
        close(f);
```

```
        if(i < nelem(fds))
            fds[i].fd = -1;
    }while(tries++ == 0);   /* retry once */
    USED(tries);
    return 0;
}
```

⟨libc/9sys/nsec.c 328a⟩≡

```
#include <u.h>
#include <libc.h>
#include <tos.h>

⟨global order 326d⟩

⟨function be2vlong 326e⟩

⟨global fd 327a⟩
⟨global fds 327b⟩

⟨function nsec 327c⟩
```

A.3.26 libc/9sys/nulldir.c

⟨libc/9sys/nulldir.c 328b⟩≡

```
#include <u.h>
#include <libc.h>

⟨function nulldir 47a⟩
```

A.3.27 libc/9sys/postnote.c

⟨libc/9sys/postnote.c 328c⟩≡

```
#include <u.h>
#include <libc.h>

⟨function postnote 71c⟩
```

A.3.28 libc/9sys/privalloc.c

⟨*global privlock (9sys/privalloc.c)* 328d⟩≡ (329b)

```
static Lock privlock;
```

⟨*global* privinit 328e⟩≡ (329b)

```
static int  privinit;
```

⟨*global* privs 328f⟩≡ (329b)

```
static void **privs;
```

⟨*function* privalloc 328g⟩≡ (329b)

```
void **
privalloc(void)
{
    void **p;
    int i;

    lock(&privlock);
    if(!privinit){
```

```
        privinit = 1;
        if(_nprivates){
            _privates[0] = 0;
            for(i = 1; i < _nprivates; i++)
                _privates[i] = &_privates[i - 1];
            privs = &_privates[i - 1];
        }
    }
    p = privs;
    if(p != nil){
        privs = *p;
        *p = nil;
    }
    unlock(&privlock);
    return p;
}
```

⟨*function* privfree 329a⟩≡ (329b)

```
void
privfree(void **p)
{
    lock(&privlock);
    if(p != nil && privinit){
        *p = privs;
        privs = p;
    }
    unlock(&privlock);
}
```

⟨libc/9sys/privalloc.c 329b⟩≡

```
#include <u.h>
#include <libc.h>

⟨global privlock (9sys/privalloc.c) 328d⟩
⟨global privinit 328e⟩
⟨global privs 328f⟩

extern void **_privates;
extern int  _nprivates;

⟨function privalloc 328g⟩

⟨function privfree 329a⟩
```

A.3.29 libc/9sys/pushssl.c

⟨libc/9sys/pushssl.c 329c⟩≡

```
#include <u.h>
#include <libc.h>

⟨function pushssl 189c⟩
```

A.3.30 libc/9sys/pushtls.c

⟨*enum* _anon_ (9sys/pushtls.c) 329d⟩≡ (330b)

```
enum {
    TLSFinishedLen = 12,
    HFinished = 20,
};
```

⟨*function* finished 330a⟩≡ (330b)

```
static int
finished(int hand, int isclient)
{
    int i, n;
    uchar buf[500], buf2[500];

    buf[0] = HFinished;
    buf[1] = TLSFinishedLen>>16;
    buf[2] = TLSFinishedLen>>8;
    buf[3] = TLSFinishedLen;
    n = TLSFinishedLen+4;

    for(i=0; i<2; i++){
        if(i==0)
            memmove(buf+4, "client finished", TLSFinishedLen);
        else
            memmove(buf+4, "server finished", TLSFinishedLen);
        if(isclient == 1-i){
            if(write(hand, buf, n) != n)
                return -1;
        }else{
            if(readn(hand, buf2, n) != n || memcmp(buf,buf2,n) != 0)
                return -1;
        }
    }
    return 1;
}
```

⟨libc/9sys/pushtls.c 330b⟩≡

```
#include <u.h>
#include <libc.h>
#include <mp.h>
#include <libsec.h>

⟨enum _anon_ (9sys/pushtls.c) 329d⟩

⟨function finished 330a⟩

⟨function pushtls 190⟩
```

A.3.31 libc/9sys/putenv.c

⟨libc/9sys/putenv.c 330c⟩≡

```
#include <u.h>
#include <libc.h>

⟨function putenv 70b⟩
```

A.3.32 libc/9sys/qlock.c

⟨*global* ql 330d⟩≡ (331e)

```
static struct {
    QLp *p;
    QLp x[1024];
} ql = {
    ql.x
};
```

⟨*enum* _anon_ (9sys/qlock.c) 331a⟩≡ (331e)

```
enum
{
    Queuing,
    QueuingR,
    QueuingW,
    Sleeping,
};
```

⟨*global* _rendezvousp 331b⟩≡ (331e)

```
static void*    (*_rendezvousp)(void*, void*) = rendezvous;
```

⟨*function* _qlockinit 331c⟩≡ (331e)

```
/* this gets called by the thread library ONLY to get us to use its rendezvous */
void
_qlockinit(void* (*r)(void*, void*))
{
    _rendezvousp = r;
}
```

⟨*function* getqlp 331d⟩≡ (331e)

```
/* find a free shared memory location to queue ourselves in */
static QLp*
getqlp(void)
{
    QLp *p, *op;

    op = ql.p;
    for(p = op+1; ; p++){
        if(p == &ql.x[nelem(ql.x)])
            p = ql.x;
        if(p == op)
            abort();
        if(_tas(&(p->inuse)) == 0){
            ql.p = p;
            p->next = nil;
            break;
        }
    }
    return p;
}
```

⟨libc/9sys/qlock.c 331e⟩≡

```
#include <u.h>
#include <libc.h>

⟨global ql 330d⟩

⟨enum _anon_ (9sys/qlock.c) 331a⟩

⟨global _rendezvousp 331b⟩

⟨function _qlockinit 331c⟩

⟨function getqlp 331d⟩

⟨function qlock 61c⟩

⟨function qunlock 62a⟩
```

⟨*function* canqlock 62b⟩

⟨*function* rlock 62d⟩

⟨*function* canrlock 63a⟩

⟨*function* runlock 63b⟩

⟨*function* wlock 64a⟩

⟨*function* canwlock 64b⟩

⟨*function* wunlock 64c⟩

⟨*function* rsleep 65b⟩

⟨*function* rwakeup 66⟩

⟨*function* rwakeupall 67a⟩

A.3.33 libc/9sys/read.c

⟨libc/9sys/read.c 332a⟩≡

```
#include    <u.h>
#include    <libc.h>

⟨function read 45b⟩
```

A.3.34 libc/9sys/read9pmsg.c

⟨libc/9sys/read9pmsg.c 332b⟩≡

```
#include <u.h>
#include <libc.h>
#include <fcall.h>

⟨function read9pmsg 78c⟩
```

A.3.35 libc/9sys/readv.c

⟨*function* ioreadv 332c⟩≡ (333c)

```
static
long
ioreadv(int fd, IOchunk *io, int nio, vlong offset)
{
    int i;
    long m, n, tot;
    char *buf, *p;

    tot = 0;
    for(i=0; i<nio; i++)
        tot += io[i].len;
    buf = malloc(tot);
    if(buf == nil)
        return -1;

    tot = pread(fd, buf, tot, offset);
```

```
    p = buf;
    n = tot;
    for(i=0; i<nio; i++){
        if(n <= 0)
            break;
        m = io->len;
        if(m > n)
            m = n;
        memmove(io->addr, p, m);
        n -= m;
        p += m;
        io++;
    }

    free(buf);
    return tot;
}
```

⟨*function* readv 333a⟩≡ (333c)

```
long
readv(int fd, IOchunk *io, int nio)
{
    return ioreadv(fd, io, nio, -1LL);
}
```

⟨*function* preadv 333b⟩≡ (333c)

```
long
preadv(int fd, IOchunk *io, int nio, vlong off)
{
    return ioreadv(fd, io, nio, off);
}
```

⟨libc/9sys/readv.c 333c⟩≡

```
#include <u.h>
#include <libc.h>

⟨function ioreadv 332c⟩

⟨function readv 333a⟩

⟨function preadv 333b⟩
```

A.3.36 libc/9sys/rerrstr.c

⟨libc/9sys/rerrstr.c 333d⟩≡

```
#include <u.h>
#include <libc.h>

⟨function rerrstr 178b⟩
```

A.3.37 libc/9sys/sbrk.c

⟨libc/9sys/sbrk.c 333e⟩≡

```
#include <u.h>
#include <libc.h>

extern  char    end[];
```

⟨*global* bloc 28a⟩

```
// the syscall
extern  int brk(void*);
```

⟨*enum* _anon_ (9sys/sbrk.c) 28c⟩

⟨*function* sbrk 28b⟩

A.3.38 libc/9sys/setnetmtpt.c

⟨*function* setnetmtpt 334a⟩≡ (334b)

```
void
setnetmtpt(char *net, int n, char *x)
{
    if(x == nil)
        x = "/net";

    if(*x == '/'){
        strncpy(net, x, n);
        net[n-1] = 0;
    } else {
        snprint(net, n, "/net%s", x);
    }
}
```

⟨libc/9sys/setnetmtpt.c 334b⟩≡

```
#include <u.h>
#include <libc.h>
```

⟨*function* setnetmtpt 334a⟩

A.3.39 libc/9sys/sysfatal.c

⟨libc/9sys/sysfatal.c 334c⟩≡

```
#include <u.h>
#include <libc.h>
```

⟨*function* _sysfatalimpl 180c⟩

⟨*global* _sysfatal 180b⟩

⟨*function* sysfatal 180a⟩

A.3.40 libc/9sys/syslog.c

⟨libc/9sys/syslog.c 334d⟩≡

```
#include <u.h>
#include <libc.h>
```

⟨*global* sl 182a⟩

⟨*function* _syslogopen 182b⟩

⟨*function* eqdirdev 183⟩

⟨*function* syslog 181⟩

A.3.41 libc/9sys/sysname.c

⟨*function* sysname 335a⟩≡ (335b)

```
char*
sysname(void)
{
    int f, n;
    static char b[128];

    if(b[0])
        return b;

    f = open("#c/sysname", 0);
    if(f >= 0) {
        n = read(f, b, sizeof(b)-1);
        if(n > 0)
            b[n] = 0;
        close(f);
    }
    return b;
}
```

⟨libc/9sys/sysname.c 335b⟩≡

```
#include    <u.h>
#include    <libc.h>

⟨function sysname 335a⟩
```

A.3.42 libc/9sys/time.c

⟨libc/9sys/time.c 335c⟩≡

```
#include <u.h>
#include <libc.h>

⟨function oldtime 55b⟩

⟨function time 55a⟩
```

A.3.43 libc/9sys/times.c

⟨*function skip (9sys/times.c)* 335d⟩≡ (336a)

```
static
char*
skip(char *p)
{

    while(*p == ' ')
        p++;
    while(*p != ' ' && *p != 0)
        p++;
    return p;
}
```

⟨*function* times 335e⟩≡ (336a)

```
/*
 *  after a fork with fd's copied, both fd's are pointing to
 *  the same Chan structure.  Since the offset is kept in the Chan
 *  structure, the seek's and read's in the two processes can be
```

```
 *  are competing moving the offset around.  Hence the unusual loop
 *  in the middle of this routine.
 */
long
times(long *t)
{
    char b[200], *p;
    static int f = -1;
    int i, retries;
    ulong r;

    memset(b, 0, sizeof(b));
    for(retries = 0; retries < 100; retries++){
        if(f < 0)
            f = open("/dev/cputime", OREAD|OCEXEC);
        if(f < 0)
            break;
        if(seek(f, 0, 0) < 0 || (i = read(f, b, sizeof(b))) < 0){
            close(f);
            f = -1;
        } else {
            if(i != 0)
                break;
        }
    }
    p = b;
    if(t)
        t[0] = atol(p);
    p = skip(p);
    if(t)
        t[1] = atol(p);
    p = skip(p);
    r = atol(p);
    if(t){
        p = skip(p);
        t[2] = atol(p);
        p = skip(p);
        t[3] = atol(p);
    }
    return r;
}
```

⟨libc/9sys/times.c 336a⟩≡

```
#include <u.h>
#include <libc.h>

⟨function skip (9sys/times.c) 335d⟩

⟨function times 335e⟩
```

A.3.44 libc/9sys/tm2sec.c

⟨*constant* TZSIZE*(9sys/tm2sec.c)* 336b⟩≡ (339a)

```
#define TZSIZE  150
```

⟨*global* timezone*(9sys/tm2sec.c)* 336c⟩≡ (339a)

```
static
struct
{
```

```
    char    stname[4];
    char    dlname[4];
    long    stdiff;
    long    dldiff;
    long    dlpairs[TZSIZE];
} timezone;
```

⟨*constant* SEC2MIN 337a⟩≡ (339a)

```
#define SEC2MIN 60L
```

⟨*constant* SEC2HOUR 337b⟩≡ (339a)

```
#define SEC2HOUR (60L*SEC2MIN)
```

⟨*constant* SEC2DAY 337c⟩≡ (339a)

```
#define SEC2DAY (24L*SEC2HOUR)
```

⟨*global* dmsize*(9sys/tm2sec.c)* 337d⟩≡ (339a)

```
/*
 *  days per month plus days/year
 */
static  int dmsize[] =
{
    365, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31
};
```

⟨*global* ldmsize 337e⟩≡ (339a)

```
static  int ldmsize[] =
{
    366, 31, 29, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31
};
```

⟨*function* yrsize 337f⟩≡ (339a)

```
/*
 *  return the days/month for the given year
 */
static int *
yrsize(int y)
{
    if((y%4) == 0 && ((y%100) != 0 || (y%400) == 0))
        return ldmsize;
    else
        return dmsize;
}
```

⟨*function* readtimezone*(9sys/tm2sec.c)* 337g⟩≡ (339a)

```
static
void
readtimezone(void)
{
    char buf[TZSIZE*11+30], *p;
    int i;

    memset(buf, 0, sizeof(buf));
    i = open("/env/timezone", 0);
    if(i < 0)
        goto error;
    if(read(i, buf, sizeof(buf)) >= sizeof(buf))
        goto error;
    close(i);
    p = buf;
```

```
    if(rd_name(&p, timezone.stname))
        goto error;
    if(rd_long(&p, &timezone.stdiff))
        goto error;
    if(rd_name(&p, timezone.dlname))
        goto error;
    if(rd_long(&p, &timezone.dldiff))
        goto error;
    for(i=0; i<TZSIZE; i++) {
        if(rd_long(&p, &timezone.dlpairs[i]))
            goto error;
        if(timezone.dlpairs[i] == 0)
            return;
    }

error:
    timezone.stdiff = 0;
    strcpy(timezone.stname, "GMT");
    timezone.dlpairs[0] = 0;
}
```

⟨*function* rd_name*(9sys/tm2sec.c)* 338a⟩≡ (339a)

```
static int
rd_name(char **f, char *p)
{
    int c, i;

    for(;;) {
        c = *(*f)++;
        if(c != ' ' && c != '\n')
            break;
    }
    for(i=0; i<3; i++) {
        if(c == ' ' || c == '\n')
            return 1;
        *p++ = c;
        c = *(*f)++;
    }
    if(c != ' ' && c != '\n')
        return 1;
    *p = 0;
    return 0;
}
```

⟨*function* rd_long*(9sys/tm2sec.c)* 338b⟩≡ (339a)

```
static int
rd_long(char **f, long *p)
{
    int c, s;
    long l;

    s = 0;
    for(;;) {
        c = *(*f)++;
        if(c == '-') {
            s++;
            continue;
        }
        if(c != ' ' && c != '\n')
            break;
```

```
    }
    if(c == 0) {
        *p = 0;
        return 0;
    }
    l = 0;
    for(;;) {
        if(c == ' ' || c == '\n')
            break;
        if(c < '0' || c > '9')
            return 1;
        l = l*10 + c-'0';
        c = *(*f)++;
    }
    if(s)
        l = -l;
    *p = l;
    return 0;
}
```

⟨libc/9sys/tm2sec.c 339a⟩≡

```
#include <u.h>
#include <libc.h>

⟨constant TZSIZE(9sys/tm2sec.c) 336b⟩
static  void    readtimezone(void);
static  int rd_name(char**, char*);
static  int rd_long(char**, long*);
⟨global timezone(9sys/tm2sec.c) 336c⟩

⟨constant SEC2MIN 337a⟩
⟨constant SEC2HOUR 337b⟩
⟨constant SEC2DAY 337c⟩

⟨global dmsize(9sys/tm2sec.c) 337d⟩
⟨global ldmsize 337e⟩

⟨function yrsize 337f⟩

⟨function tm2sec 56⟩

⟨function readtimezone(9sys/tm2sec.c) 337g⟩

⟨function rd_name(9sys/tm2sec.c) 338a⟩

⟨function rd_long(9sys/tm2sec.c) 338b⟩
```

A.3.45 libc/9sys/truerand.c

⟨libc/9sys/truerand.c 339b⟩≡

```
#include <u.h>
#include <libc.h>

⟨function truerand 175e⟩
```

A.3.46 libc/9sys/wait.c

⟨libc/9sys/wait.c 339c⟩≡

```
#include <u.h>
#include <libc.h>
#include <fcall.h>

⟨function wait 69b⟩
```

A.3.47 libc/9sys/waitpid.c

```
⟨libc/9sys/waitpid.c 340a⟩≡
#include <u.h>
#include <libc.h>
#include <fcall.h>

⟨function waitpid 69c⟩
```

A.3.48 libc/9sys/werrstr.c

```
⟨libc/9sys/werrstr.c 340b⟩≡
#include <u.h>
#include <libc.h>

⟨function werrstr 178c⟩
```

A.3.49 libc/9sys/write.c

```
⟨libc/9sys/write.c 340c⟩≡
#include    <u.h>
#include    <libc.h>

⟨function write 45c⟩
```

A.3.50 libc/9sys/writev.c

```
⟨function iowritev 340d⟩≡                                    (341c)
static
long
iowritev(int fd, IOchunk *io, int nio, vlong offset)
{
    int i;
    long tot;
    char *buf, *p;

    tot = 0;
    for(i=0; i<nio; i++)
        tot += io[i].len;
    buf = malloc(tot);
    if(buf == nil)
        return -1;

    p = buf;
    for(i=0; i<nio; i++){
        memmove(p, io->addr, io->len);
        p += io->len;
        io++;
    }
```

```
    tot = pwrite(fd, buf, tot, offset);

    free(buf);
    return tot;
}
```

⟨*function* writev 341a⟩≡ (341c)

```
long
writev(int fd, IOchunk *io, int nio)
{
    return iowritev(fd, io, nio, -1LL);
}
```

⟨*function* pwritev 341b⟩≡ (341c)

```
long
pwritev(int fd, IOchunk *io, int nio, vlong off)
{
    return iowritev(fd, io, nio, off);
}
```

⟨libc/9sys/writev.c 341c⟩≡

```
#include <u.h>
#include <libc.h>

⟨function iowritev 340d⟩

⟨function writev 341a⟩

⟨function pwritev 341b⟩
```

A.4 lib_core/libc/fmt/

A.4.1 libc/fmt/fmtdef.h

⟨libc/fmt/fmtdef.h 341d⟩≡

```
/*
 * dofmt -- format to a buffer
 * the number of characters formatted is returned,
 * or -1 if there was an error.
 * if the buffer is ever filled, flush is called.
 * it should reset the buffer and return whether formatting should continue.
 */

typedef int (*Fmts)(Fmt*);

typedef struct Quoteinfo Quoteinfo;
struct Quoteinfo
{
    int quoted;     /* if set, string must be quoted */
    int nrunesin;   /* number of input runes that can be accepted */
    int nbytesin;   /* number of input bytes that can be accepted */
    int nrunesout;  /* number of runes that will be generated */
    int nbytesout;  /* number of bytes that will be generated */
};

void    *_fmtflush(Fmt*, void*, int);
void    *_fmtdispatch(Fmt*, void*, int);
int _floatfmt(Fmt*, double);
```

```
int _fmtpad(Fmt*, int);
int _rfmtpad(Fmt*, int);
int _fmtFdFlush(Fmt*);

int _efgfmt(Fmt*);
int _charfmt(Fmt*);
int _countfmt(Fmt*);
int _flagfmt(Fmt*);
int _percentfmt(Fmt*);
int _ifmt(Fmt*);
int _runefmt(Fmt*);
int _runesfmt(Fmt*);
int _strfmt(Fmt*);
int _badfmt(Fmt*);
int _fmtcpy(Fmt*, void*, int, int);
int _fmtrcpy(Fmt*, void*, int n);

void    _fmtlock(void);
void    _fmtunlock(void);

#define FMTCHAR(f, t, s, c)\
    do{\
    if(t + 1 > (char*)s){\
        t = _fmtflush(f, t, 1);\
        if(t != nil)\
            s = f->stop;\
        else\
            return -1;\
    }\
    *t++ = c;\
    }while(0)

#define FMTRCHAR(f, t, s, c)\
    do{\
    if(t + 1 > (Rune*)s){\
        t = _fmtflush(f, t, sizeof(Rune));\
        if(t != nil)\
            s = f->stop;\
        else\
            return -1;\
    }\
    *t++ = c;\
    }while(0)

#define FMTRUNE(f, t, s, r)\
    do{\
    Rune _rune;\
    int _runelen;\
    if(t + UTFmax > (char*)s && t + (_runelen = runelen(r)) > (char*)s){\
        t = _fmtflush(f, t, _runelen);\
        if(t != nil)\
            s = f->stop;\
        else\
            return -1;\
    }\
    if(r < Runeself)\
        *t++ = r;\
    else{\
        _rune = r;\
        t += runetochar(t, &_rune);\
```

```
    }\
    }while(0)
```

A.4.2 libc/fmt/dofmt.c

⟨*function* dofmt 343⟩≡ (352a)

```
/* format the output into f->to and return the number of characters fmted  */
int
dofmt(Fmt *f, char *fmt)
{
    Rune rune, *rt, *rs;
    int r;
    char *t, *s;
    int n, nfmt;

    nfmt = f->nfmt;
    for(;;){
        if(f->runes){
            rt = f->to;
            rs = f->stop;
            while((r = *(uchar*)fmt) && r != '%'){
                if(r < Runeself)
                    fmt++;
                else{
                    fmt += chartorune(&rune, fmt);
                    r = rune;
                }
                FMTRCHAR(f, rt, rs, r);
            }
            fmt++;
            f->nfmt += rt - (Rune *)f->to;
            f->to = rt;
            if(!r)
                return f->nfmt - nfmt;
            f->stop = rs;
        }else{
            t = f->to;
            s = f->stop;
            while((r = *(uchar*)fmt) && r != '%'){
                if(r < Runeself){
                    FMTCHAR(f, t, s, r);
                    fmt++;
                }else{
                    n = chartorune(&rune, fmt);
                    if(t + n > s){
                        t = _fmtflush(f, t, n);
                        if(t != nil)
                            s = f->stop;
                        else
                            return -1;
                    }
                    while(n--)
                        *t++ = *fmt++;
                }
            }
            fmt++;
            f->nfmt += t - (char *)f->to;
            f->to = t;
            if(!r)
```

```
                return f->nfmt - nfmt;
            f->stop = s;
        }

        fmt = _fmtdispatch(f, fmt, 0);
        if(fmt == nil)
            return -1;
    }
}
```

⟨*function* _fmtflush 344a⟩≡ (352a)

```
void *
_fmtflush(Fmt *f, void *t, int len)
{
    if(f->runes)
        f->nfmt += (Rune*)t - (Rune*)f->to;
    else
        f->nfmt += (char*)t - (char *)f->to;
    f->to = t;
    if(f->flush == 0 || (*f->flush)(f) == 0 || (char*)f->to + len > (char*)f->stop){
        f->stop = f->to;
        return nil;
    }
    return f->to;
}
```

⟨*function* _fmtpad 344b⟩≡ (352a)

```
/*
 * put a formatted block of memory sz bytes long of n runes into the output buffer,
 * left/right justified in a field of at least f->width charactes
 */
int
_fmtpad(Fmt *f, int n)
{
    char *t, *s;
    int i;

    t = f->to;
    s = f->stop;
    for(i = 0; i < n; i++)
        FMTCHAR(f, t, s, ' ');
    f->nfmt += t - (char *)f->to;
    f->to = t;
    return 0;
}
```

⟨*function* _rfmtpad 344c⟩≡ (352a)

```
int
_rfmtpad(Fmt *f, int n)
{
    Rune *t, *s;
    int i;

    t = f->to;
    s = f->stop;
    for(i = 0; i < n; i++)
        FMTRCHAR(f, t, s, ' ');
    f->nfmt += t - (Rune *)f->to;
    f->to = t;
    return 0;
}
```

⟨*function* _fmtcpy 345a⟩≡ (352a)

```
int
_fmtcpy(Fmt *f, void *vm, int n, int sz)
{
    Rune *rt, *rs, r;
    char *t, *s, *m, *me;
    ulong fl;
    int nc, w;

    m = vm;
    me = m + sz;
    w = f->width;
    fl = f->flags;
    if((fl & FmtPrec) && n > f->prec)
        n = f->prec;
    if(f->runes){
        if(!(fl & FmtLeft) && _rfmtpad(f, w - n) < 0)
            return -1;
        rt = f->to;
        rs = f->stop;
        for(nc = n; nc > 0; nc--){
            r = *(uchar*)m;
            if(r < Runeself)
                m++;
            else if((me - m) >= UTFmax || fullrune(m, me-m))
                m += chartorune(&r, m);
            else
                break;
            FMTRCHAR(f, rt, rs, r);
        }
        f->nfmt += rt - (Rune *)f->to;
        f->to = rt;
        if(fl & FmtLeft && _rfmtpad(f, w - n) < 0)
            return -1;
    }else{
        if(!(fl & FmtLeft) && _fmtpad(f, w - n) < 0)
            return -1;
        t = f->to;
        s = f->stop;
        for(nc = n; nc > 0; nc--){
            r = *(uchar*)m;
            if(r < Runeself)
                m++;
            else if((me - m) >= UTFmax || fullrune(m, me-m))
                m += chartorune(&r, m);
            else
                break;
            FMTRUNE(f, t, s, r);
        }
        f->nfmt += t - (char *)f->to;
        f->to = t;
        if(fl & FmtLeft && _fmtpad(f, w - n) < 0)
            return -1;
    }
    return 0;
}
```

⟨*function* _fmtrcpy 345b⟩≡ (352a)

```
int
_fmtrcpy(Fmt *f, void *vm, int n)
```

```
{
    Rune r, *m, *me, *rt, *rs;
    char *t, *s;
    ulong fl;
    int w;

    m = vm;
    w = f->width;
    fl = f->flags;
    if((fl & FmtPrec) && n > f->prec)
        n = f->prec;
    if(f->runes){
        if(!(fl & FmtLeft) && _rfmtpad(f, w - n) < 0)
            return -1;
        rt = f->to;
        rs = f->stop;
        for(me = m + n; m < me; m++)
            FMTRCHAR(f, rt, rs, *m);
        f->nfmt += rt - (Rune *)f->to;
        f->to = rt;
        if(fl & FmtLeft && _rfmtpad(f, w - n) < 0)
            return -1;
    }else{
        if(!(fl & FmtLeft) && _fmtpad(f, w - n) < 0)
            return -1;
        t = f->to;
        s = f->stop;
        for(me = m + n; m < me; m++){
            r = *m;
            FMTRUNE(f, t, s, r);
        }
        f->nfmt += t - (char *)f->to;
        f->to = t;
        if(fl & FmtLeft && _fmtpad(f, w - n) < 0)
            return -1;
    }
    return 0;
}
```

⟨*function* _charfmt 346a⟩≡ (352a)

```
/* fmt out one character */
int
_charfmt(Fmt *f)
{
    char x[1];

    x[0] = va_arg(f->args, int);
    f->prec = 1;
    return _fmtcpy(f, x, 1, 1);
}
```

⟨*function* _runefmt 346b⟩≡ (352a)

```
/* fmt out one rune */
int
_runefmt(Fmt *f)
{
    Rune x[1];

    x[0] = va_arg(f->args, int);
    return _fmtrcpy(f, x, 1);
}
```

⟨*function* fmtstrcpy 347a⟩≡ (352a)

```
/* public helper routine: fmt out a null terminated string already in hand */
int
fmtstrcpy(Fmt *f, char *s)
{
    int i, j;
    Rune r;

    if(!s)
        return _fmtcpy(f, "<nil>", 5, 5);
    /* if precision is specified, make sure we don't wander off the end */
    if(f->flags & FmtPrec){
        i = 0;
        for(j=0; j<f->prec && s[i]; j++)
            i += chartorune(&r, s+i);
        return _fmtcpy(f, s, j, i);
    }
    return _fmtcpy(f, s, utflen(s), strlen(s));
}
```

⟨*function* _strfmt 347b⟩≡ (352a)

```
/* fmt out a null terminated utf string */
int
_strfmt(Fmt *f)
{
    char *s;

    s = va_arg(f->args, char *);
    return fmtstrcpy(f, s);
}
```

⟨*function* fmtrunestrcpy 347c⟩≡ (352a)

```
/* public helper routine: fmt out a null terminated rune string already in hand */
int
fmtrunestrcpy(Fmt *f, Rune *s)
{
    Rune *e;
    int n, p;

    if(!s)
        return _fmtcpy(f, "<nil>", 5, 5);
    /* if precision is specified, make sure we don't wander off the end */
    if(f->flags & FmtPrec){
        p = f->prec;
        for(n = 0; n < p; n++)
            if(s[n] == 0)
                break;
    }else{
        for(e = s; *e; e++)
            ;
        n = e - s;
    }
    return _fmtrcpy(f, s, n);
}
```

⟨*function* _runesfmt 347d⟩≡ (352a)

```
/* fmt out a null terminated rune string */
int
_runesfmt(Fmt *f)
{
```

```
    Rune *s;

    s = va_arg(f->args, Rune *);
    return fmtrunestrcpy(f, s);
}
```

⟨*function* _percentfmt 348a⟩≡ (352a)

```
/* fmt a % */
int
_percentfmt(Fmt *f)
{
    Rune x[1];

    x[0] = f->r;
    f->prec = 1;
    return _fmtrcpy(f, x, 1);
}
```

⟨*enum* _anon_ (fmt/dofmt.c) 348b⟩≡ (352a)

```
enum {
    /* %,#llb could emit a sign, "0b" and 64 digits with 21 commas */
    Maxintwidth = 1 + 2 + 64 + 64/3,
};
```

⟨*function* _ifmt 348c⟩≡ (352a)

```
/* fmt an integer */
int
_ifmt(Fmt *f)
{
    char buf[Maxintwidth + 1], *p, *conv;
    uvlong vu;
    ulong u;
    uintptr pu;
    int neg, base, i, n, fl, w, isv;

    neg = 0;
    fl = f->flags;
    isv = 0;
    vu = 0;
    u = 0;
    if(f->r == 'p'){
        pu = va_arg(f->args, uintptr);
        if(sizeof(uintptr) == sizeof(uvlong)){
            vu = pu;
            isv = 1;
        }else
            u = pu;
        f->r = 'x';
        fl |= FmtUnsigned;
    }else if(fl & FmtVLong){
        isv = 1;
        if(fl & FmtUnsigned)
            vu = va_arg(f->args, uvlong);
        else
            vu = va_arg(f->args, vlong);
    }else if(fl & FmtLong){
        if(fl & FmtUnsigned)
            u = va_arg(f->args, ulong);
        else
            u = va_arg(f->args, long);
```

```
}else if(fl & FmtByte){
    if(fl & FmtUnsigned)
        u = (uchar)va_arg(f->args, int);
    else
        u = (char)va_arg(f->args, int);
}else if(fl & FmtShort){
    if(fl & FmtUnsigned)
        u = (ushort)va_arg(f->args, int);
    else
        u = (short)va_arg(f->args, int);
}else{
    if(fl & FmtUnsigned)
        u = va_arg(f->args, uint);
    else
        u = va_arg(f->args, int);
}
conv = "0123456789abcdef";
switch(f->r){
case 'd':
    base = 10;
    break;
case 'x':
    base = 16;
    break;
case 'X':
    base = 16;
    conv = "0123456789ABCDEF";
    break;
case 'b':
    base = 2;
    break;
case 'o':
    base = 8;
    break;
default:
    return -1;
}
if(!(fl & FmtUnsigned)){
    if(isv && (vlong)vu < 0){
        vu = -(vlong)vu;
        neg = 1;
    }else if(!isv && (long)u < 0){
        u = -(long)u;
        neg = 1;
    }
}
p = buf + sizeof buf - 1;
n = 0;
if(isv){
    while(vu){
        i = vu % base;
        vu /= base;
        if((fl & FmtComma) && n % 4 == 3){
            *p-- = ',';
            n++;
        }
        *p-- = conv[i];
        n++;
    }
}else{
```

```
        while(u){
            i = u % base;
            u /= base;
            if((fl & FmtComma) && n % 4 == 3){
                *p-- = ',';
                n++;
            }
            *p-- = conv[i];
            n++;
        }
    }
    if(n == 0){
        *p-- = '0';
        n = 1;
    }
    for(w = f->prec; n < w && p > buf+3; n++)
        *p-- = '0';
    if(neg || (fl & (FmtSign|FmtSpace)))
        n++;
    if(fl & FmtSharp){
        if(base == 16)
            n += 2;
        else if(base == 8){
            if(p[1] == '0')
                fl &= ~FmtSharp;
            else
                n++;
        }
    }
    if((fl & FmtZero) && !(fl & (FmtLeft|FmtPrec))){
        for(w = f->width; n < w && p > buf+3; n++)
            *p-- = '0';
        f->width = 0;
    }
    if(fl & FmtSharp){
        if(base == 16)
            *p-- = f->r;
        if(base == 16 || base == 8)
            *p-- = '0';
    }
    if(neg)
        *p-- = '-';
    else if(fl & FmtSign)
        *p-- = '+';
    else if(fl & FmtSpace)
        *p-- = ' ';
    f->flags &= ~FmtPrec;
    return _fmtcpy(f, p + 1, n, n);
}
```

⟨*function* _countfmt 350⟩≡ (352a)

```
int
_countfmt(Fmt *f)
{
    void *p;
    ulong fl;

    fl = f->flags;
    p = va_arg(f->args, void*);
    if(fl & FmtVLong){
```

```
        *(vlong*)p = f->nfmt;
    }else if(fl & FmtLong){
        *(long*)p = f->nfmt;
    }else if(fl & FmtByte){
        *(char*)p = f->nfmt;
    }else if(fl & FmtShort){
        *(short*)p = f->nfmt;
    }else{
        *(int*)p = f->nfmt;
    }
    return 0;
}
```

⟨*function* _flagfmt 351a⟩≡ (352a)

```
int
_flagfmt(Fmt *f)
{
    switch(f->r){
    case ',':
        f->flags |= FmtComma;
        break;
    case '-':
        f->flags |= FmtLeft;
        break;
    case '+':
        f->flags |= FmtSign;
        break;
    case '#':
        f->flags |= FmtSharp;
        break;
    case ' ':
        f->flags |= FmtSpace;
        break;
    case 'u':
        f->flags |= FmtUnsigned;
        break;
    case 'h':
        if(f->flags & FmtShort)
            f->flags |= FmtByte;
        f->flags |= FmtShort;
        break;
    case 'l':
        if(f->flags & FmtLong)
            f->flags |= FmtVLong;
        f->flags |= FmtLong;
        break;
    }
    return 1;
}
```

⟨*function* _badfmt 351b⟩≡ (352a)

```
/* default error format */
int
_badfmt(Fmt *f)
{
    Rune x[3];

    x[0] = '%';
    x[1] = f->r;
    x[2] = '%';
```

```
    f->prec = 3;
    _fmtrcpy(f, x, 3);
    return 0;
}
```

⟨libc/fmt/dofmt.c 352a⟩≡

```
#include <u.h>
#include <libc.h>
#include "fmtdef.h"

⟨function dofmt 343⟩

⟨function _fmtflush 344a⟩

⟨function _fmtpad 344b⟩

⟨function _rfmtpad 344c⟩

⟨function _fmtcpy 345a⟩

⟨function _fmtrcpy 345b⟩

⟨function _charfmt 346a⟩

⟨function _runefmt 346b⟩

⟨function fmtstrcpy 347a⟩

⟨function _strfmt 347b⟩

⟨function fmtrunestrcpy 347c⟩

⟨function _runesfmt 347d⟩

⟨function _percentfmt 348a⟩

⟨enum _anon_ (fmt/dofmt.c) 348b⟩

⟨function _ifmt 348c⟩

⟨function _countfmt 350⟩

⟨function _flagfmt 351a⟩

⟨function _badfmt 351b⟩
```

A.4.3 libc/fmt/dorfmt.c

⟨function dorfmt 352b⟩≡ (353a)

```
/* format the output into f->to and return the number of characters fmted  */

int
dorfmt(Fmt *f, Rune *fmt)
{
    Rune *rt, *rs;
    int r;
    char *t, *s;
    int nfmt;
```

```
    nfmt = f->nfmt;
    for(;;){
        if(f->runes){
            rt = f->to;
            rs = f->stop;
            while((r = *fmt++) && r != '%'){
                FMTRCHAR(f, rt, rs, r);
            }
            f->nfmt += rt - (Rune *)f->to;
            f->to = rt;
            if(!r)
                return f->nfmt - nfmt;
            f->stop = rs;
        }else{
            t = f->to;
            s = f->stop;
            while((r = *fmt++) && r != '%'){
                FMTRUNE(f, t, f->stop, r);
            }
            f->nfmt += t - (char *)f->to;
            f->to = t;
            if(!r)
                return f->nfmt - nfmt;
            f->stop = s;
        }

        fmt = _fmtdispatch(f, fmt, 1);
        if(fmt == nil)
            return -1;
    }
}
```

⟨libc/fmt/dorfmt.c 353a⟩≡

```
#include <u.h>
#include <libc.h>
#include "fmtdef.h"

⟨function dorfmt 352b⟩
```

A.4.4 libc/fmt/errfmt.c

⟨*function* errfmt 353b⟩≡ (353c)

```
int
errfmt(Fmt *f)
{
    char buf[ERRMAX];

    rerrstr(buf, sizeof buf);
    return _fmtcpy(f, buf, utflen(buf), strlen(buf));
}
```

⟨libc/fmt/errfmt.c 353c⟩≡

```
#include <u.h>
#include <libc.h>
#include "fmtdef.h"

⟨function errfmt 353b⟩
```

A.4.5 libc/fmt/fltfmt.c

⟨*enum* _anon_ (fmt/fltfmt.c) 354a⟩≡ (359b)

```
enum
{
    FDIGIT  = 30,
    FDEFLT  = 6,
    NSIGNIF = 17,
    NEXP10  = 308,
};
```

⟨*function* xadd 354b⟩≡ (359b)

```
static int
xadd(char *a, int n, int v)
{
    char *b;
    int c;

    if(n < 0 || n >= NSIGNIF)
        return 0;
    for(b = a+n; b >= a; b--) {
        c = *b + v;
        if(c <= '9') {
            *b = c;
            return 0;
        }
        *b = '0';
        v = 1;
    }
    *a = '1';   // overflow adding
    return 1;
}
```

⟨*function* xsub 354c⟩≡ (359b)

```
static int
xsub(char *a, int n, int v)
{
    char *b;
    int c;

    for(b = a+n; b >= a; b--) {
        c = *b - v;
        if(c >= '0') {
            *b = c;
            return 0;
        }
        *b = '9';
        v = 1;
    }
    *a = '9';   // underflow subtracting
    return 1;
}
```

⟨*function* xdtoa 354d⟩≡ (359b)

```
static void
xdtoa(Fmt *fmt, char *s2, double f)
{
    char s1[NSIGNIF+10];
    double g, h;
    int e, d, i, n;
```

```
int c1, c2, c3, c4, ucase, sign, chr, prec;

prec = FDEFLT;
if(fmt->flags & FmtPrec)
    prec = fmt->prec;
if(prec > FDIGIT)
    prec = FDIGIT;
if(isNaN(f)) {
    strcpy(s2, "NaN");
    return;
}
if(isInf(f, 1)) {
    strcpy(s2, "+Inf");
    return;
}
if(isInf(f, -1)) {
    strcpy(s2, "-Inf");
    return;
}
sign = 0;
if(f < 0) {
    f = -f;
    sign++;
}
ucase = 0;
chr = fmt->r;
if(isupper(chr)) {
    ucase = 1;
    chr = tolower(chr);
}

e = 0;
g = f;
if(g != 0) {
    frexp(f, &e);
    e = e * .301029995664;
    if(e >= -150 && e <= +150) {
        d = 0;
        h = f;
    } else {
        d = e/2;
        h = f * pow10(-d);
    }
    g = h * pow10(d-e);
    while(g < 1) {
        e--;
        g = h * pow10(d-e);
    }
    while(g >= 10) {
        e++;
        g = h * pow10(d-e);
    }
}

/*
 * convert NSIGNIF digits and convert
 * back to get accuracy.
 */
for(i=0; i<NSIGNIF; i++) {
    d = g;
```

```
        s1[i] = d + '0';
        g = (g - d) * 10;
    }
    s1[i] = 0;

    /*
     * try decimal rounding to eliminate 9s
     */
    c2 = prec + 1;
    if(chr == 'f')
        c2 += e;
    if(c2 >= NSIGNIF-2) {
        strcpy(s2, s1);
        d = e;
        s1[NSIGNIF-2] = '0';
        s1[NSIGNIF-1] = '0';
        sprint(s1+NSIGNIF, "e%d", e-NSIGNIF+1);
        g = strtod(s1, nil);
        if(g == f)
            goto found;
        if(xadd(s1, NSIGNIF-3, 1)) {
            e++;
            sprint(s1+NSIGNIF, "e%d", e-NSIGNIF+1);
        }
        g = strtod(s1, nil);
        if(g == f)
            goto found;
        strcpy(s1, s2);
        e = d;
    }

    /*
     * convert back so s1 gets exact answer
     */
    for(;;) {
        sprint(s1+NSIGNIF, "e%d", e-NSIGNIF+1);
        g = strtod(s1, nil);
        if(f > g) {
            if(xadd(s1, NSIGNIF-1, 1))
                e--;
            continue;
        }
        if(f < g) {
            if(xsub(s1, NSIGNIF-1, 1))
                e++;
            continue;
        }
        break;
    }

found:
    /*
     * sign
     */
    d = 0;
    i = 0;
    if(sign)
        s2[d++] = '-';
    else if(fmt->flags & FmtSign)
        s2[d++] = '+';
```

```
else if(fmt->flags & FmtSpace)
    s2[d++] = ' ';

/*
 * copy into final place
 * c1 digits of leading '0'
 * c2 digits from conversion
 * c3 digits of trailing '0'
 * c4 digits after '.'
 */
c1 = 0;
c2 = prec + 1;
c3 = 0;
c4 = prec;
switch(chr) {
default:
    if(xadd(s1, c2, 5))
        e++;
    break;
case 'g':
    /*
     * decide on 'e' of 'f' style convers
     */
    if(xadd(s1, c2, 5))
        e++;
    if(e >= -5 && e <= prec) {
        c1 = -e - 1;
        c4 = prec - e;
        chr = 'h';  // flag for 'f' style
    }
    break;
case 'f':
    if(xadd(s1, c2+e, 5))
        e++;
    c1 = -e;
    if(c1 > prec)
        c1 = c2;
    c2 += e;
    break;
}

/*
 * clean up c1 c2 and c3
 */
if(c1 < 0)
    c1 = 0;
if(c2 < 0)
    c2 = 0;
if(c2 > NSIGNIF) {
    c3 = c2-NSIGNIF;
    c2 = NSIGNIF;
}

/*
 * copy digits
 */
while(c1 > 0) {
    if(c1+c2+c3 == c4)
        s2[d++] = '.';
    s2[d++] = '0';
```

```
        c1--;
    }
    while(c2 > 0) {
        if(c2+c3 == c4)
            s2[d++] = '.';
        s2[d++] = s1[i++];
        c2--;
    }
    while(c3 > 0) {
        if(c3 == c4)
            s2[d++] = '.';
        s2[d++] = '0';
        c3--;
    }

    /*
     * strip trailing '0' on g conv
     */
    if(fmt->flags & FmtSharp) {
        if(0 == c4)
            s2[d++] = '.';
    } else
    if(chr == 'g' || chr == 'h') {
        for(n=d-1; n>=0; n--)
            if(s2[n] != '0')
                break;
        for(i=n; i>=0; i--)
            if(s2[i] == '.') {
                d = n;
                if(i != n)
                    d++;
                break;
            }
    }
    if(chr == 'e' || chr == 'g') {
        if(ucase)
            s2[d++] = 'E';
        else
            s2[d++] = 'e';
        c1 = e;
        if(c1 < 0) {
            s2[d++] = '-';
            c1 = -c1;
        } else
            s2[d++] = '+';
        if(c1 >= 100) {
            s2[d++] = c1/100 + '0';
            c1 = c1%100;
        }
        s2[d++] = c1/10 + '0';
        s2[d++] = c1%10 + '0';
    }
    s2[d] = 0;
}
```

⟨*function* _floatfmt 358⟩≡ (359b)

```
int
_floatfmt(Fmt *fmt, double f)
{
    char s[1+NEXP10+1+FDIGIT+1];
```

```
    /*
     * The max length of a %f string is
     *  '[+-]'+"max exponent"+'.'+"max precision"+'\0'
     * which is 341 currently.
     */
    xdtoa(fmt, s, f);
    fmt->flags &= FmtWidth|FmtLeft;
    _fmtcpy(fmt, s, strlen(s), strlen(s));
    return 0;
}
```

⟨*function* _efgfmt 359a⟩≡ (359b)

```
int
_efgfmt(Fmt *f)
{
    double d;

    d = va_arg(f->args, double);
    return _floatfmt(f, d);
}
```

⟨libc/fmt/fltfmt.c 359b⟩≡

```
#include <u.h>
#include <libc.h>
#include <ctype.h>
#include "fmtdef.h"

⟨enum _anon_ (fmt/fltfmt.c) 354a⟩

⟨function xadd 354b⟩

⟨function xsub 354c⟩

⟨function xdtoa 354d⟩

⟨function _floatfmt 358⟩

⟨function _efgfmt 359a⟩
```

A.4.6 libc/fmt/fmt.c

⟨*enum* _anon_ (fmt/fmt.c) 359c⟩≡ (363a)

```
enum
{
    Maxfmt = 64
};
```

⟨*struct* Convfmt 359d⟩≡ (363a)

```
struct Convfmt
{
    int c;
    volatile    Fmts    fmt;    /* for spin lock in fmtfmt; avoids race due to write order */
};
```

⟨*global* fmtalloc 360a⟩≡ (363a)

```
struct
{
    /* lock by calling _fmtlock, _fmtunlock */
    int nfmt;
    Convfmt fmt[Maxfmt];
} fmtalloc;
```

⟨*global* knownfmt 360b⟩≡ (363a)

```
static Convfmt knownfmt[] = {
    ' ',    _flagfmt,
    '#',    _flagfmt,
    '%',    _percentfmt,
    '+',    _flagfmt,
    ',',    _flagfmt,
    '-',    _flagfmt,
    'C',    _runefmt,
    'E',    _efgfmt,
    'G',    _efgfmt,
    'S',    _runesfmt,
    'X',    _ifmt,
    'b',    _ifmt,
    'c',    _charfmt,
    'd',    _ifmt,
    'e',    _efgfmt,
    'f',    _efgfmt,
    'g',    _efgfmt,
    'h',    _flagfmt,
    'l',    _flagfmt,
    'n',    _countfmt,
    'o',    _ifmt,
    'p',    _ifmt,
    'r',    errfmt,
    's',    _strfmt,
    'u',    _flagfmt,
    'x',    _ifmt,
    0,  nil,
};
```

⟨*global doquote (fmt/fmt.c)* 360c⟩≡ (363a)

```
int (*doquote)(int);
```

⟨*function* _fmtinstall 360d⟩≡ (363a)

```
/*
 * _fmtlock() must be set
 */
static int
_fmtinstall(int c, Fmts f)
{
    Convfmt *p, *ep;

    if(c<=0 || c>=65536)
        return -1;
    if(!f)
        f = _badfmt;

    ep = &fmtalloc.fmt[fmtalloc.nfmt];
    for(p=fmtalloc.fmt; p<ep; p++)
        if(p->c == c)
            break;
```

```
    if(p == &fmtalloc.fmt[Maxfmt])
        return -1;

    p->fmt = f;
    if(p == ep){    /* installing a new format character */
        fmtalloc.nfmt++;
        p->c = c;
    }

    return 0;
}
```

⟨*function* fmtinstall 361a⟩≡ (363a)

```
int
fmtinstall(int c, Fmts f)
{
    int ret;

    _fmtlock();
    ret = _fmtinstall(c, f);
    _fmtunlock();
    return ret;
}
```

⟨*function* fmtfmt 361b⟩≡ (363a)

```
static Fmts
fmtfmt(int c)
{
    Convfmt *p, *ep;

    ep = &fmtalloc.fmt[fmtalloc.nfmt];
    for(p=fmtalloc.fmt; p<ep; p++)
        if(p->c == c){
            while(p->fmt == nil)    /* loop until value is updated */
                ;
            return p->fmt;
        }

    /* is this a predefined format char? */
    _fmtlock();
    for(p=knownfmt; p->c; p++)
        if(p->c == c){
            _fmtinstall(p->c, p->fmt);
            _fmtunlock();
            return p->fmt;
        }
    _fmtunlock();

    return _badfmt;
}
```

⟨*function* _fmtdispatch 361c⟩≡ (363a)

```
void*
_fmtdispatch(Fmt *f, void *fmt, int isrunes)
{
    Rune rune, r;
    int i, n, w, p;
    ulong fl;
    void *ret;
```

```
w = f->width;
p = f->prec;
fl = f->flags;

f->flags = 0;
f->width = f->prec = 0;

for(;;){
    if(isrunes){
        r = *(Rune*)fmt;
        fmt = (Rune*)fmt + 1;
    }else{
        fmt = (char*)fmt + chartorune(&rune, fmt);
        r = rune;
    }
    f->r = r;
    switch(r){
    case '\0':
        ret = nil;
        goto end;
    case '.':
        f->flags |= FmtWidth|FmtPrec;
        continue;
    case '0':
        if(!(f->flags & FmtWidth)){
            f->flags |= FmtZero;
            continue;
        }
        /* fall through */
    case '1': case '2': case '3': case '4':
    case '5': case '6': case '7': case '8': case '9':
        i = 0;
        while(r >= '0' && r <= '9'){
            i = i * 10 + r - '0';
            if(isrunes){
                r = *(Rune*)fmt;
                fmt = (Rune*)fmt + 1;
            }else{
                r = *(char*)fmt;
                fmt = (char*)fmt + 1;
            }
        }
        if(isrunes)
            fmt = (Rune*)fmt - 1;
        else
            fmt = (char*)fmt - 1;
    numflag:
        if(f->flags & FmtWidth){
            f->flags |= FmtPrec;
            f->prec = i;
        }else{
            f->flags |= FmtWidth;
            f->width = i;
        }
        continue;
    case '*':
        i = va_arg(f->args, int);
        if(i < 0){
            /*
```

```
                 * negative precision =>
                 * ignore the precision.
                 */
                if(f->flags & FmtPrec){
                    f->flags &= ~FmtPrec;
                    f->prec = 0;
                    continue;
                }
                i = -i;
                f->flags |= FmtLeft;
            }
            goto numflag;
        }
        n = (*fmtfmt(r))(f);
        if(n < 0){
            ret = nil;
            break;
        }
        if(n == 0){
            ret = fmt;
            break;
        }
    }
end:
    f->width = w;
    f->prec = p;
    f->flags = fl;
    return ret;
}
```

⟨libc/fmt/fmt.c 363a⟩≡

```
#include <u.h>
#include <libc.h>
#include "fmtdef.h"

⟨enum _anon_ (fmt/fmt.c) 359c⟩

typedef struct Convfmt Convfmt;
⟨struct Convfmt 359d⟩

⟨global fmtalloc 360a⟩

⟨global knownfmt 360b⟩

⟨global doquote (fmt/fmt.c) 360c⟩

⟨function _fmtinstall 360d⟩

⟨function fmtinstall 361a⟩

⟨function fmtfmt 361b⟩

⟨function _fmtdispatch 361c⟩
```

A.4.7 libc/fmt/fmtfd.c

⟨function fmtfdflush 363b⟩≡ (364b)

```
/*
 * public routine for final flush of a formatting buffer
```

```
 * to a file descriptor; returns total char count.
 */
int
fmtfdflush(Fmt *f)
{
    if(_fmtFdFlush(f) <= 0)
        return -1;
    return f->nfmt;
}
```

⟨*function* fmtfdinit 364a⟩≡ (364b)

```
/*
 * initialize an output buffer for buffered printing
 */
int
fmtfdinit(Fmt *f, int fd, char *buf, int size)
{
    f->runes = 0;
    f->start = buf;
    f->to = buf;
    f->stop = buf + size;
    f->flush = _fmtFdFlush;
    f->farg = (void*)fd;
    f->nfmt = 0;
    return 0;
}
```

⟨libc/fmt/fmtfd.c 364b⟩≡

```
#include <u.h>
#include <libc.h>
#include "fmtdef.h"

⟨function fmtfdflush 363b⟩

⟨function fmtfdinit 364a⟩
```

A.4.8 libc/fmt/fmtlock.c

⟨*global* fmtl 364c⟩≡ (365a)

```
static Lock fmtl;
```

⟨*function* _fmtlock 364d⟩≡ (365a)

```
void
_fmtlock(void)
{
    lock(&fmtl);
}
```

⟨*function* _fmtunlock 364e⟩≡ (365a)

```
void
_fmtunlock(void)
{
    unlock(&fmtl);
}
```

⟨libc/fmt/fmtlock.c 365a⟩≡

```
#include <u.h>
#include <libc.h>

⟨global fmtl 364c⟩

⟨function _fmtlock 364d⟩

⟨function _fmtunlock 364e⟩
```

A.4.9 libc/fmt/fmtprint.c

⟨function fmtprint 365b⟩≡ (365c)

```
/*
 * format a string into the output buffer
 * designed for formats which themselves call fmt
 */
int
fmtprint(Fmt *f, char *fmt, ...)
{
    va_list va;
    int n;

    va_start(va, fmt);
    n = fmtvprint(f, fmt, va);
    va_end(va);
    return n;
}
```

⟨libc/fmt/fmtprint.c 365c⟩≡

```
#include <u.h>
#include <libc.h>
#include "fmtdef.h"

⟨function fmtprint 365b⟩
```

A.4.10 libc/fmt/fmtquote.c

⟨function _quotesetup 365d⟩≡ (369f)

```
/*
 * How many bytes of output UTF will be produced by quoting (if necessary) this string?
 * How many runes? How much of the input will be consumed?
 * The parameter q is filled in by _quotesetup.
 * The string may be UTF or Runes (s or r).
 * Return count does not include NUL.
 * Terminate the scan at the first of:
 *  NUL in input
 *  count exceeded in input
 *  count exceeded on output
 * *ninp is set to number of input bytes accepted.
 * nin may be <0 initially, to avoid checking input by count.
 */
void
_quotesetup(char *s, Rune *r, int nin, int nout, Quoteinfo *q, int sharp, int runesout)
{
    int w;
    Rune c;
```

```
q->quoted = 0;
q->nbytesout = 0;
q->nrunesout = 0;
q->nbytesin = 0;
q->nrunesin = 0;
if(sharp || nin==0 || (s && *s=='\0') || (r && *r=='\0')){
    if(nout < 2)
        return;
    q->quoted = 1;
    q->nbytesout = 2;
    q->nrunesout = 2;
}
for(; nin!=0; nin--){
    if(s)
        w = chartorune(&c, s);
    else{
        c = *r;
        w = runelen(c);
    }

    if(c == '\0')
        break;
    if(runesout){
        if(q->nrunesout+1 > nout)
            break;
    }else{
        if(q->nbytesout+w > nout)
            break;
    }

    if((c <= L' ') || (c == L'\'') || (doquote!=nil && doquote(c))){
        if(!q->quoted){
            if(runesout){
                if(1+q->nrunesout+1+1 > nout)   /* no room for quotes */
                    break;
            }else{
                if(1+q->nbytesout+w+1 > nout)   /* no room for quotes */
                    break;
            }
            q->nrunesout += 2;  /* include quotes */
            q->nbytesout += 2;  /* include quotes */
            q->quoted = 1;
        }
        if(c == '\'')   {
            if(runesout){
                if(1+q->nrunesout+1 > nout) /* no room for quotes */
                    break;
            }else{
                if(1+q->nbytesout+w > nout) /* no room for quotes */
                    break;
            }
            q->nbytesout++;
            q->nrunesout++; /* quotes reproduce as two characters */
        }
    }

    /* advance input */
    if(s)
        s += w;
    else
```

```
            r++;
        q->nbytesin += w;
        q->nrunesin++;

        /* advance output */
        q->nbytesout += w;
        q->nrunesout++;
    }
}
```

⟨*function* qstrfmt 367⟩≡ (369f)

```
static int
qstrfmt(char *sin, Rune *rin, Quoteinfo *q, Fmt *f)
{
    Rune r, *rm, *rme;
    char *t, *s, *m, *me;
    Rune *rt, *rs;
    ulong fl;
    int nc, w;

    m = sin;
    me = m + q->nbytesin;
    rm = rin;
    rme = rm + q->nrunesin;

    w = f->width;
    fl = f->flags;
    if(f->runes){
        if(!(fl & FmtLeft) && _rfmtpad(f, w - q->nrunesout) < 0)
            return -1;
    }else{
        if(!(fl & FmtLeft) && _fmtpad(f, w - q->nbytesout) < 0)
            return -1;
    }
    t = f->to;
    s = f->stop;
    rt = f->to;
    rs = f->stop;
    if(f->runes)
        FMTRCHAR(f, rt, rs, '\'');
    else
        FMTRUNE(f, t, s, '\'');
    for(nc = q->nrunesin; nc > 0; nc--){
        if(sin){
            r = *(uchar*)m;
            if(r < Runeself)
                m++;
            else if((me - m) >= UTFmax || fullrune(m, me-m))
                m += chartorune(&r, m);
            else
                break;
        }else{
            if(rm >= rme)
                break;
            r = *(uchar*)rm++;
        }
        if(f->runes){
            FMTRCHAR(f, rt, rs, r);
            if(r == '\'')
                FMTRCHAR(f, rt, rs, r);
```

```
        }else{
            FMTRUNE(f, t, s, r);
            if(r == '\'')
                FMTRUNE(f, t, s, r);
        }
    }

    if(f->runes){
        FMTRCHAR(f, rt, rs, '\'');
        USED(rs);
        f->nfmt += rt - (Rune *)f->to;
        f->to = rt;
        if(fl & FmtLeft && _rfmtpad(f, w - q->nrunesout) < 0)
            return -1;
    }else{
        FMTRUNE(f, t, s, '\'');
        USED(s);
        f->nfmt += t - (char *)f->to;
        f->to = t;
        if(fl & FmtLeft && _fmtpad(f, w - q->nbytesout) < 0)
            return -1;
    }
    return 0;
}
```

⟨*function* _quotestrfmt 368⟩≡ (369f)

```
int
_quotestrfmt(int runesin, Fmt *f)
{
    int nin, outlen;
    Rune *r;
    char *s;
    Quoteinfo q;

    nin = -1;
    if(f->flags&FmtPrec)
        nin = f->prec;
    if(runesin){
        r = va_arg(f->args, Rune *);
        s = nil;
    }else{
        s = va_arg(f->args, char *);
        r = nil;
    }
    if(!s && !r)
        return _fmtcpy(f, "<nil>", 5, 5);

    if(f->flush)
        outlen = 0x7FFFFFFF;    /* if we can flush, no output limit */
    else if(f->runes)
        outlen = (Rune*)f->stop - (Rune*)f->to;
    else
        outlen = (char*)f->stop - (char*)f->to;

    _quotesetup(s, r, nin, outlen, &q, f->flags&FmtSharp, f->runes);
//print("bytes in %d bytes out %d runes in %d runesout %d\n", q.nbytesin, q.nbytesout, q.nrunesin, q.nrunesout)

    if(runesin){
        if(!q.quoted)
            return _fmtrcpy(f, r, q.nrunesin);
```

```
        return qstrfmt(nil, r, &q, f);
    }

    if(!q.quoted)
        return _fmtcpy(f, s, q.nrunesin, q.nbytesin);
    return qstrfmt(s, nil, &q, f);
}
```

⟨*function* quotestrfmt 369a⟩≡ (369f)

```
int
quotestrfmt(Fmt *f)
{
    return _quotestrfmt(0, f);
}
```

⟨*function* quoterunestrfmt 369b⟩≡ (369f)

```
int
quoterunestrfmt(Fmt *f)
{
    return _quotestrfmt(1, f);
}
```

⟨*function* quotefmtinstall 369c⟩≡ (369f)

```
void
quotefmtinstall(void)
{
    fmtinstall('q', quotestrfmt);
    fmtinstall('Q', quoterunestrfmt);
}
```

⟨*function* _needsquotes 369d⟩≡ (369f)

```
int
_needsquotes(char *s, int *quotelenp)
{
    Quoteinfo q;

    _quotesetup(s, nil, -1, 0x7FFFFFFF, &q, 0, 0);
    *quotelenp = q.nbytesout;

    return q.quoted;
}
```

⟨*function* _runeneedsquotes 369e⟩≡ (369f)

```
int
_runeneedsquotes(Rune *r, int *quotelenp)
{
    Quoteinfo q;

    _quotesetup(nil, r, -1, 0x7FFFFFFF, &q, 0, 0);
    *quotelenp = q.nrunesout;

    return q.quoted;
}
```

⟨libc/fmt/fmtquote.c 369f⟩≡

```
#include <u.h>
#include <libc.h>
#include "fmtdef.h"

⟨function _quotesetup 365d⟩
```

⟨*function* qstrfmt 367⟩

⟨*function* _quotestrfmt 368⟩

⟨*function* quotestrfmt 369a⟩

⟨*function* quoterunestrfmt 369b⟩

⟨*function* quotefmtinstall 369c⟩

⟨*function* _needsquotes 369d⟩

⟨*function* _runeneedsquotes 369e⟩

A.4.11 libc/fmt/fmtrune.c

⟨*function* fmtrune 370a⟩≡ (370b)

```
int
fmtrune(Fmt *f, int r)
{
    Rune *rt;
    char *t;
    int n;

    if(f->runes){
        rt = f->to;
        FMTRCHAR(f, rt, f->stop, r);
        f->to = rt;
        n = 1;
    }else{
        t = f->to;
        FMTRUNE(f, t, f->stop, r);
        n = t - (char*)f->to;
        f->to = t;
    }
    f->nfmt += n;
    return 0;
}
```

⟨libc/fmt/fmtrune.c 370b⟩≡

```
#include <u.h>
#include <libc.h>
#include "fmtdef.h"

⟨function fmtrune 370a⟩
```

A.4.12 libc/fmt/fmtstr.c

⟨*function* fmtstrflush 370c⟩≡ (371a)

```
char*
fmtstrflush(Fmt *f)
{
    if(f->start == nil)
        return nil;
    *(char*)f->to = '\0';
    return f->start;
}
```

⟨libc/fmt/fmtstr.c 371a⟩≡
```
#include <u.h>
#include <libc.h>

⟨function fmtstrflush 370c⟩
```

A.4.13 libc/fmt/fmtvprint.c

⟨*function* fmtvprint 371b⟩≡ (371c)
```
/*
 * format a string into the output buffer
 * designed for formats which themselves call fmt
 */
int
fmtvprint(Fmt *f, char *fmt, va_list args)
{
    va_list va;
    int n;

    va = f->args;
    f->args = args;
    n = dofmt(f, fmt);
    f->args = va;
    if(n >= 0)
        return 0;
    return n;
}
```

⟨libc/fmt/fmtvprint.c 371c⟩≡
```
#include <u.h>
#include <libc.h>
#include "fmtdef.h"

⟨function fmtvprint 371b⟩
```

A.4.14 libc/fmt/fprint.c

⟨*function* fprint 371d⟩≡ (371e)
```
int
fprint(int fd, char *fmt, ...)
{
    int n;
    va_list args;

    va_start(args, fmt);
    n = vfprint(fd, fmt, args);
    va_end(args);
    return n;
}
```

⟨libc/fmt/fprint.c 371e⟩≡
```
#include    <u.h>
#include    <libc.h>

⟨function fprint 371d⟩
```

A.4.15 libc/fmt/print.c

⟨*function* libc_print 372a⟩≡ (372c)

```
int
libc_print(char *fmt, ...)
{
    int n;
    va_list args;

    va_start(args, fmt);
    n = vfprint(1, fmt, args);
    va_end(args);
    return n;
}
```

⟨*global* print 372b⟩≡ (372c)

```
int (*print)(char *fmt, ...) = &libc_print;
```

⟨libc/fmt/print.c 372c⟩≡

```
#include    <u.h>
#include    <libc.h>

⟨function libc_print 372a⟩

⟨global print 372b⟩
```

A.4.16 libc/fmt/runefmtstr.c

⟨*function* runefmtstrflush 372d⟩≡ (372e)

```
Rune*
runefmtstrflush(Fmt *f)
{
    if(f->start == nil)
        return nil;
    *(Rune*)f->to = '\0';
    return f->start;
}
```

⟨libc/fmt/runefmtstr.c 372e⟩≡

```
#include <u.h>
#include <libc.h>

⟨function runefmtstrflush 372d⟩
```

A.4.17 libc/fmt/runeseprint.c

⟨*function* runeseprint 372f⟩≡ (373a)

```
Rune*
runeseprint(Rune *buf, Rune *e, char *fmt, ...)
{
    Rune *p;
    va_list args;

    va_start(args, fmt);
    p = runevseprint(buf, e, fmt, args);
    va_end(args);
    return p;
}
```

⟨libc/fmt/runeseprint.c 373a⟩≡
```
#include <u.h>
#include <libc.h>

⟨function runeseprint 372f⟩
```

A.4.18 libc/fmt/runesmprint.c

⟨*function* runesmprint 373b⟩≡ (373c)
```
Rune*
runesmprint(char *fmt, ...)
{
    va_list args;
    Rune *p;

    va_start(args, fmt);
    p = runevsmprint(fmt, args);
    va_end(args);
    return p;
}
```

⟨libc/fmt/runesmprint.c 373c⟩≡
```
#include <u.h>
#include <libc.h>

⟨function runesmprint 373b⟩
```

A.4.19 libc/fmt/runesnprint.c

⟨*function* runesnprint 373d⟩≡ (373e)
```
int
runesnprint(Rune *buf, int len, char *fmt, ...)
{
    int n;
    va_list args;

    va_start(args, fmt);
    n = runevsnprint(buf, len, fmt, args);
    va_end(args);
    return n;
}
```

⟨libc/fmt/runesnprint.c 373e⟩≡
```
#include <u.h>
#include <libc.h>

⟨function runesnprint 373d⟩
```

A.4.20 libc/fmt/runesprint.c

⟨*function* runesprint 373f⟩≡ (374a)
```
int
runesprint(Rune *buf, char *fmt, ...)
{
    int n;
    va_list args;
```

```
    va_start(args, fmt);
    n = runevsnprint(buf, 256, fmt, args);
    va_end(args);
    return n;
}
```

⟨libc/fmt/runesprint.c 374a⟩≡

```
#include <u.h>
#include <libc.h>

⟨function runesprint 373f⟩
```

A.4.21 libc/fmt/runevseprint.c

⟨*function* runevseprint 374b⟩≡ (374c)

```
Rune*
runevseprint(Rune *buf, Rune *e, char *fmt, va_list args)
{
    Fmt f;

    if(e <= buf)
        return nil;
    f.runes = 1;
    f.start = buf;
    f.to = buf;
    f.stop = e - 1;
    f.flush = nil;
    f.farg = nil;
    f.nfmt = 0;
    f.args = args;
    dofmt(&f, fmt);
    *(Rune*)f.to = '\0';
    return f.to;
}
```

⟨libc/fmt/runevseprint.c 374c⟩≡

```
#include <u.h>
#include <libc.h>

⟨function runevseprint 374b⟩
```

A.4.22 libc/fmt/runevsmprint.c

⟨*function* runeFmtStrFlush 374d⟩≡ (375c)

```
static int
runeFmtStrFlush(Fmt *f)
{
    Rune *s;
    int n;

    if(f->start == nil)
        return 0;
    n = (int)(uintptr)f->farg;
    n *= 2;
    s = f->start;
    f->start = realloc(s, sizeof(Rune)*n);
    if(f->start == nil){
        f->farg = nil;
```

```
        f->to = nil;
        f->stop = nil;
        free(s);
        return 0;
    }
    f->farg = (void*)n;
    f->to = (Rune*)f->start + ((Rune*)f->to - s);
    f->stop = (Rune*)f->start + n - 1;
    return 1;
}
```

⟨*function* runefmtstrinit 375a⟩≡ (375c)

```
int
runefmtstrinit(Fmt *f)
{
    int n;

    memset(f, 0, sizeof *f);
    f->runes = 1;
    n = 32;
    f->start = malloc(sizeof(Rune)*n);
    if(f->start == nil)
        return -1;
    setmalloctag(f->start, getcallerpc(&f));
    f->to = f->start;
    f->stop = (Rune*)f->start + n - 1;
    f->flush = runeFmtStrFlush;
    f->farg = (void*)n;
    f->nfmt = 0;
    return 0;
}
```

⟨*function* runevsmprint 375b⟩≡ (375c)

```
/*
 * print into an allocated string buffer
 */
Rune*
runevsmprint(char *fmt, va_list args)
{
    Fmt f;
    int n;

    if(runefmtstrinit(&f) < 0)
        return nil;
    f.args = args;
    n = dofmt(&f, fmt);
    if(f.start == nil)      /* realloc failed? */
        return nil;
    if(n < 0){
        free(f.start);
        return nil;
    }
    setmalloctag(f.start, getcallerpc(&fmt));
    *(Rune*)f.to = '\0';
    return f.start;
}
```

⟨libc/fmt/runevsmprint.c 375c⟩≡

```
#include <u.h>
#include <libc.h>
```

```
#include "fmtdef.h"

⟨function runeFmtStrFlush 374d⟩

⟨function runefmtstrinit 375a⟩

⟨function runevsmprint 375b⟩
```

A.4.23 libc/fmt/runevsnprint.c

⟨*function* runevsnprint 376a⟩≡ (376b)

```
int
runevsnprint(Rune *buf, int len, char *fmt, va_list args)
{
    Fmt f;

    if(len <= 0)
        return -1;
    f.runes = 1;
    f.start = buf;
    f.to = buf;
    f.stop = buf + len - 1;
    f.flush = nil;
    f.farg = nil;
    f.nfmt = 0;
    f.args = args;
    dofmt(&f, fmt);
    *(Rune*)f.to = '\0';
    return (Rune*)f.to - buf;
}
```

⟨libc/fmt/runevsnprint.c 376b⟩≡

```
#include <u.h>
#include <libc.h>

⟨function runevsnprint 376a⟩
```

A.4.24 libc/fmt/seprint.c

⟨*function* seprint 376c⟩≡ (376d)

```
char*
seprint(char *buf, char *e, char *fmt, ...)
{
    char *p;
    va_list args;

    va_start(args, fmt);
    p = vseprint(buf, e, fmt, args);
    va_end(args);
    return p;
}
```

⟨libc/fmt/seprint.c 376d⟩≡

```
#include <u.h>
#include <libc.h>

⟨function seprint 376c⟩
```

A.4.25 libc/fmt/smprint.c

⟨*function* smprint 377a⟩≡ (377b)

```
char*
smprint(char *fmt, ...)
{
    va_list args;
    char *p;

    va_start(args, fmt);
    p = vsmprint(fmt, args);
    va_end(args);
    setmalloctag(p, getcallerpc(&fmt));
    return p;
}
```

⟨libc/fmt/smprint.c 377b⟩≡

```
#include <u.h>
#include <libc.h>

⟨function smprint 377a⟩
```

A.4.26 libc/fmt/snprint.c

⟨*function* snprint 377c⟩≡ (377d)

```
int
snprint(char *buf, int len, char *fmt, ...)
{
    int n;
    va_list args;

    va_start(args, fmt);
    n = vsnprint(buf, len, fmt, args);
    va_end(args);
    return n;
}
```

⟨libc/fmt/snprint.c 377d⟩≡

```
#include <u.h>
#include <libc.h>

⟨function snprint 377c⟩
```

A.4.27 libc/fmt/sprint.c

⟨*function* sprint 377e⟩≡ (378a)

```
int
sprint(char *buf, char *fmt, ...)
{
    int n;
    va_list args;

    va_start(args, fmt);
    n = vsnprint(buf, 65536, fmt, args);    /* big number, but sprint is deprecated anyway */
    va_end(args);
    return n;
}
```

⟨libc/fmt/sprint.c 378a⟩≡
```
#include <u.h>
#include <libc.h>

⟨function sprint 377e⟩
```

A.4.28 libc/fmt/vfprint.c

⟨*function* _fmtFdFlush 378b⟩≡ (378d)
```
/*
 * generic routine for flushing a formatting buffer
 * to a file descriptor
 */
int
_fmtFdFlush(Fmt *f)
{
    int n;

    n = (char*)f->to - (char*)f->start;
    if(n && write((int)(uintptr)f->farg, f->start, n) != n)
        return 0;
    f->to = f->start;
    return 1;
}
```

⟨*function* vfprint 378c⟩≡ (378d)
```
int
vfprint(int fd, char *fmt, va_list args)
{
    Fmt f;
    char buf[256];
    int n;

    fmtfdinit(&f, fd, buf, sizeof(buf));
    f.args = args;
    n = dofmt(&f, fmt);
    if(n > 0 && _fmtFdFlush(&f) == 0)
        return -1;
    return n;
}
```

⟨libc/fmt/vfprint.c 378d⟩≡
```
#include <u.h>
#include <libc.h>
#include "fmtdef.h"

⟨function _fmtFdFlush 378b⟩

⟨function vfprint 378c⟩
```

A.4.29 libc/fmt/vseprint.c

⟨*function* vseprint 378e⟩≡ (379a)
```
char*
vseprint(char *buf, char *e, char *fmt, va_list args)
{
    Fmt f;
```

```
    if(e <= buf)
        return nil;
    f.runes = 0;
    f.start = buf;
    f.to = buf;
    f.stop = e - 1;
    f.flush = nil;
    f.farg = nil;
    f.nfmt = 0;
    f.args = args;
    dofmt(&f, fmt);
    *(char*)f.to = '\0';
    return f.to;
}
```

⟨libc/fmt/vseprint.c 379a⟩≡

```
#include <u.h>
#include <libc.h>

⟨function vseprint 378e⟩
```

A.4.30 libc/fmt/vsmprint.c

⟨*function* fmtStrFlush 379b⟩≡ (380b)

```
static int
fmtStrFlush(Fmt *f)
{
    char *s;
    int n;

    if(f->start == nil)
        return 0;
    n = (int)(uintptr)f->farg;
    n *= 2;
    s = f->start;
    f->start = realloc(s, n);
    if(f->start == nil){
        f->farg = nil;
        f->to = nil;
        f->stop = nil;
        free(s);
        return 0;
    }
    f->farg = (void*)n;
    f->to = (char*)f->start + ((char*)f->to - s);
    f->stop = (char*)f->start + n - 1;
    return 1;
}
```

⟨*function* fmtstrinit 379c⟩≡ (380b)

```
int
fmtstrinit(Fmt *f)
{
    int n;

    memset(f, 0, sizeof *f);
    f->runes = 0;
    n = 32;
    f->start = malloc(n);
```

```
    if(f->start == nil)
        return -1;
    setmalloctag(f->start, getcallerpc(&f));
    f->to = f->start;
    f->stop = (char*)f->start + n - 1;
    f->flush = fmtStrFlush;
    f->farg = (void*)n;
    f->nfmt = 0;
    return 0;
}
```

⟨*function* vsmprint 380a⟩≡ (380b)

```
/*
 * print into an allocated string buffer
 */
char*
vsmprint(char *fmt, va_list args)
{
    Fmt f;
    int n;

    if(fmtstrinit(&f) < 0)
        return nil;
    f.args = args;
    n = dofmt(&f, fmt);
    if(f.start == nil)      /* realloc failed? */
        return nil;
    if(n < 0){
        free(f.start);
        return nil;
    }
    setmalloctag(f.start, getcallerpc(&fmt));
    *(char*)f.to = '\0';
    return f.start;
}
```

⟨libc/fmt/vsmprint.c 380b⟩≡

```
#include <u.h>
#include <libc.h>
#include "fmtdef.h"

⟨function fmtStrFlush 379b⟩

⟨function fmtstrinit 379c⟩

⟨function vsmprint 380a⟩
```

A.4.31 libc/fmt/vsnprint.c

⟨*function* vsnprint 380c⟩≡ (381a)

```
int
vsnprint(char *buf, int len, char *fmt, va_list args)
{
    Fmt f;

    if(len <= 0)
        return -1;
    f.runes = 0;
    f.start = buf;
```

```
    f.to = buf;
    f.stop = buf + len - 1;
    f.flush = nil;
    f.farg = nil;
    f.nfmt = 0;
    f.args = args;
    dofmt(&f, fmt);
    *(char*)f.to = '\0';
    return (char*)f.to - buf;
}
```

⟨libc/fmt/vsnprint.c 381a⟩≡

```
#include <u.h>
#include <libc.h>

⟨function vsnprint 380c⟩
```

A.5 lib_core/libc/arm/

A.5.1 libc/arm/cycles.c

⟨*function* cycles*(arm)* 381b⟩≡ (381c)

```
void
cycles(uvlong* u)
{
    *u = 0LL;
}
```

⟨libc/arm/cycles.c 381c⟩≡

```
#include <u.h>
#include <libc.h>

#pragma profile off

⟨function cycles(arm) 381b⟩
```

A.5.2 libc/arm/notejmp.c

⟨*function* notejmp*(arm)* 381d⟩≡ (381e)

```
void
notejmp(void *vr, jmp_buf j, int ret)
{
    struct Ureg *r = vr;

    r->r0 = ret;
    if(ret == 0)
        r->r0 = 1;
    r->pc = j[JMPBUFPC];
    r->r13 = j[JMPBUFSP];
    noted(NCONT);
}
```

⟨libc/arm/notejmp.c 381e⟩≡

```
#include <u.h>
#include <libc.h>
#include <ureg.h>

⟨function notejmp(arm) 381d⟩
```

A.5.3 libc/arm/vlrt.c

⟨*macro* SIGN*(arm)* 382a⟩≡ (394d)

```
#define SIGN(n) (1UL<<(n-1))
```

⟨*struct* Vlong*(arm)* 382b⟩≡ (394d)

```
struct  Vlong
{
    ulong   lo;
    ulong   hi;
};
```

⟨*function* _addv*(arm)* 382c⟩≡ (394d)

```
void
_addv(Vlong *r, Vlong a, Vlong b)
{
    ulong lo, hi;

    lo = a.lo + b.lo;
    hi = a.hi + b.hi;
    if(lo < a.lo)
        hi++;
    r->lo = lo;
    r->hi = hi;
}
```

⟨*function* _subv*(arm)* 382d⟩≡ (394d)

```
void
_subv(Vlong *r, Vlong a, Vlong b)
{
    ulong lo, hi;

    lo = a.lo - b.lo;
    hi = a.hi - b.hi;
    if(lo > a.lo)
        hi--;
    r->lo = lo;
    r->hi = hi;
}
```

⟨*function* _d2v*(arm)* 382e⟩≡ (394d)

```
void
_d2v(Vlong *y, double d)
{
    union { double d; struct Vlong; } x;
    ulong xhi, xlo, ylo, yhi;
    int sh;

    x.d = d;

    xhi = (x.hi & 0xfffff) | 0x100000;
    xlo = x.lo;
    sh = 1075 - ((x.hi >> 20) & 0x7ff);

    ylo = 0;
    yhi = 0;
    if(sh >= 0) {
        /* v = (hi||lo) >> sh */
        if(sh < 32) {
            if(sh == 0) {
                ylo = xlo;
```

```
                yhi = xhi;
            } else {
                ylo = (xlo >> sh) | (xhi << (32-sh));
                yhi = xhi >> sh;
            }
        } else {
            if(sh == 32) {
                ylo = xhi;
            } else
            if(sh < 64) {
                ylo = xhi >> (sh-32);
            }
        }
    } else {
        /* v = (hi||lo) << -sh */
        sh = -sh;
        if(sh <= 10) {
            ylo = xlo << sh;
            yhi = (xhi << sh) | (xlo >> (32-sh));
        } else {
            /* overflow */
            yhi = d;    /* causes something awful */
        }
    }
    if(x.hi & SIGN(32)) {
        if(ylo != 0) {
            ylo = -ylo;
            yhi = ~yhi;
        } else
            yhi = -yhi;
    }

    y->hi = yhi;
    y->lo = ylo;
}
```

⟨*function* _f2v*(arm)* 383a⟩≡ (394d)

```
void
_f2v(Vlong *y, float f)
{
    _d2v(y, f);
}
```

⟨*function* _v2d*(arm)* 383b⟩≡ (394d)

```
double
_v2d(Vlong x)
{
    if(x.hi & SIGN(32)) {
        if(x.lo) {
            x.lo = -x.lo;
            x.hi = ~x.hi;
        } else
            x.hi = -x.hi;
        return -((long)x.hi*4294967296. + x.lo);
    }
    return (long)x.hi*4294967296. + x.lo;
}
```

⟨*function* _v2f*(arm)* 383c⟩≡ (394d)

```
float
```

```
_v2f(Vlong x)
{
    return _v2d(x);
}
```

⟨*function* dodiv*(arm)* 384⟩≡ (394d)

```
static void
dodiv(Vlong num, Vlong den, Vlong *q, Vlong *r)
{
    ulong numlo, numhi, denhi, denlo, quohi, quolo, t;
    int i;

    numhi = num.hi;
    numlo = num.lo;
    denhi = den.hi;
    denlo = den.lo;
    /*
     * get a divide by zero
     */
    if(denlo==0 && denhi==0) {
        numlo = numlo / denlo;
    }

    /*
     * set up the divisor and find the number of iterations needed
     */
    if(numhi >= SIGN(32)) {
        quohi = SIGN(32);
        quolo = 0;
    } else {
        quohi = numhi;
        quolo = numlo;
    }
    i = 0;
    while(denhi < quohi || (denhi == quohi && denlo < quolo)) {
        denhi = (denhi<<1) | (denlo>>31);
        denlo <<= 1;
        i++;
    }

    quohi = 0;
    quolo = 0;
    for(; i >= 0; i--) {
        quohi = (quohi<<1) | (quolo>>31);
        quolo <<= 1;
        if(numhi > denhi || (numhi == denhi && numlo >= denlo)) {
            t = numlo;
            numlo -= denlo;
            if(numlo > t)
                numhi--;
            numhi -= denhi;
            quolo |= 1;
        }
        denlo = (denlo>>1) | (denhi<<31);
        denhi >>= 1;
    }

    if(q) {
        q->lo = quolo;
        q->hi = quohi;
```

```
    }
    if(r) {
        r->lo = numlo;
        r->hi = numhi;
    }
}
```

⟨*function* _divvu*(arm)* 385a⟩≡ (394d)

```
void
_divvu(Vlong *q, Vlong n, Vlong d)
{
    if(n.hi == 0 && d.hi == 0) {
        q->hi = 0;
        q->lo = n.lo / d.lo;
        return;
    }
    dodiv(n, d, q, 0);
}
```

⟨*function* _modvu*(arm)* 385b⟩≡ (394d)

```
void
_modvu(Vlong *r, Vlong n, Vlong d)
{

    if(n.hi == 0 && d.hi == 0) {
        r->hi = 0;
        r->lo = n.lo % d.lo;
        return;
    }
    dodiv(n, d, 0, r);
}
```

⟨*function* vneg*(arm)* 385c⟩≡ (394d)

```
static void
vneg(Vlong *v)
{

    if(v->lo == 0) {
        v->hi = -v->hi;
        return;
    }
    v->lo = -v->lo;
    v->hi = ~v->hi;
}
```

⟨*function* _divv*(arm)* 385d⟩≡ (394d)

```
void
_divv(Vlong *q, Vlong n, Vlong d)
{
    long nneg, dneg;

    if(n.hi == (((long)n.lo)>>31) && d.hi == (((long)d.lo)>>31)) {
        q->lo = (long)n.lo / (long)d.lo;
        q->hi = ((long)q->lo) >> 31;
        return;
    }
    nneg = n.hi >> 31;
    if(nneg)
        vneg(&n);
    dneg = d.hi >> 31;
```

```
    if(dneg)
        vneg(&d);
    dodiv(n, d, q, 0);
    if(nneg != dneg)
        vneg(q);
}
```

⟨*function* _modv*(arm)* 386a⟩≡ (394d)

```
void
_modv(Vlong *r, Vlong n, Vlong d)
{
    long nneg, dneg;

    if(n.hi == (((long)n.lo)>>31) && d.hi == (((long)d.lo)>>31)) {
        r->lo = (long)n.lo % (long)d.lo;
        r->hi = ((long)r->lo) >> 31;
        return;
    }
    nneg = n.hi >> 31;
    if(nneg)
        vneg(&n);
    dneg = d.hi >> 31;
    if(dneg)
        vneg(&d);
    dodiv(n, d, 0, r);
    if(nneg)
        vneg(r);
}
```

⟨*function* _rshav*(arm)* 386b⟩≡ (394d)

```
void
_rshav(Vlong *r, Vlong a, int b)
{
    long t;

    t = a.hi;
    if(b >= 32) {
        r->hi = t>>31;
        if(b >= 64) {
            /* this is illegal re C standard */
            r->lo = t>>31;
            return;
        }
        r->lo = t >> (b-32);
        return;
    }
    if(b <= 0) {
        r->hi = t;
        r->lo = a.lo;
        return;
    }
    r->hi = t >> b;
    r->lo = (t << (32-b)) | (a.lo >> b);
}
```

⟨*function* _rshlv*(arm)* 386c⟩≡ (394d)

```
void
_rshlv(Vlong *r, Vlong a, int b)
{
    ulong t;
```

```
    t = a.hi;
    if(b >= 32) {
        r->hi = 0;
        if(b >= 64) {
            /* this is illegal re C standard */
            r->lo = 0;
            return;
        }
        r->lo = t >> (b-32);
        return;
    }
    if(b <= 0) {
        r->hi = t;
        r->lo = a.lo;
        return;
    }
    r->hi = t >> b;
    r->lo = (t << (32-b)) | (a.lo >> b);
}
```

⟨*function* _lshv*(arm)* 387a⟩≡ (394d)

```
void
_lshv(Vlong *r, Vlong a, int b)
{
    ulong t;

    t = a.lo;
    if(b >= 32) {
        r->lo = 0;
        if(b >= 64) {
            /* this is illegal re C standard */
            r->hi = 0;
            return;
        }
        r->hi = t << (b-32);
        return;
    }
    if(b <= 0) {
        r->lo = t;
        r->hi = a.hi;
        return;
    }
    r->lo = t << b;
    r->hi = (t >> (32-b)) | (a.hi << b);
}
```

⟨*function* _andv*(arm)* 387b⟩≡ (394d)

```
void
_andv(Vlong *r, Vlong a, Vlong b)
{
    r->hi = a.hi & b.hi;
    r->lo = a.lo & b.lo;
}
```

⟨*function* _orv*(arm)* 387c⟩≡ (394d)

```
void
_orv(Vlong *r, Vlong a, Vlong b)
{
    r->hi = a.hi | b.hi;
```

```
    r->lo = a.lo | b.lo;
}
```

⟨*function* _xorv*(arm)* 388a⟩≡ (394d)

```
void
_xorv(Vlong *r, Vlong a, Vlong b)
{
    r->hi = a.hi ^ b.hi;
    r->lo = a.lo ^ b.lo;
}
```

⟨*function* _vpp*(arm)* 388b⟩≡ (394d)

```
void
_vpp(Vlong *l, Vlong *r)
{

    l->hi = r->hi;
    l->lo = r->lo;
    r->lo++;
    if(r->lo == 0)
        r->hi++;
}
```

⟨*function* _vmm*(arm)* 388c⟩≡ (394d)

```
void
_vmm(Vlong *l, Vlong *r)
{

    l->hi = r->hi;
    l->lo = r->lo;
    if(r->lo == 0)
        r->hi--;
    r->lo--;
}
```

⟨*function* _ppv*(arm)* 388d⟩≡ (394d)

```
void
_ppv(Vlong *l, Vlong *r)
{

    r->lo++;
    if(r->lo == 0)
        r->hi++;
    l->hi = r->hi;
    l->lo = r->lo;
}
```

⟨*function* _mmv*(arm)* 388e⟩≡ (394d)

```
void
_mmv(Vlong *l, Vlong *r)
{

    if(r->lo == 0)
        r->hi--;
    r->lo--;
    l->hi = r->hi;
    l->lo = r->lo;
}
```

⟨*function* _vasop*(arm)* 389⟩≡ (394d)

```
void
_vasop(Vlong *ret, void *lv, void fn(Vlong*, Vlong, Vlong), int type, Vlong rv)
{
    Vlong t, u;

    u = *ret;
    switch(type) {
    default:
        abort();
        break;

    case 1: /* schar */
        t.lo = *(schar*)lv;
        t.hi = t.lo >> 31;
        fn(&u, t, rv);
        *(schar*)lv = u.lo;
        break;

    case 2: /* uchar */
        t.lo = *(uchar*)lv;
        t.hi = 0;
        fn(&u, t, rv);
        *(uchar*)lv = u.lo;
        break;

    case 3: /* short */
        t.lo = *(short*)lv;
        t.hi = t.lo >> 31;
        fn(&u, t, rv);
        *(short*)lv = u.lo;
        break;

    case 4: /* ushort */
        t.lo = *(ushort*)lv;
        t.hi = 0;
        fn(&u, t, rv);
        *(ushort*)lv = u.lo;
        break;

    case 9: /* int */
        t.lo = *(int*)lv;
        t.hi = t.lo >> 31;
        fn(&u, t, rv);
        *(int*)lv = u.lo;
        break;

    case 10:    /* uint */
        t.lo = *(uint*)lv;
        t.hi = 0;
        fn(&u, t, rv);
        *(uint*)lv = u.lo;
        break;

    case 5: /* long */
        t.lo = *(long*)lv;
        t.hi = t.lo >> 31;
        fn(&u, t, rv);
        *(long*)lv = u.lo;
        break;
```

```
    case 6: /* ulong */
        t.lo = *(ulong*)lv;
        t.hi = 0;
        fn(&u, t, rv);
        *(ulong*)lv = u.lo;
        break;

    case 7: /* vlong */
    case 8: /* uvlong */
        fn(&u, *(Vlong*)lv, rv);
        *(Vlong*)lv = u;
        break;
    }
    *ret = u;
}
```

⟨*function* _p2v*(arm)* 390a⟩≡ (394d)

```
void
_p2v(Vlong *ret, void *p)
{
    long t;

    t = (ulong)p;
    ret->lo = t;
    ret->hi = 0;
}
```

⟨*function* _sl2v*(arm)* 390b⟩≡ (394d)

```
void
_sl2v(Vlong *ret, long sl)
{
    long t;

    t = sl;
    ret->lo = t;
    ret->hi = t >> 31;
}
```

⟨*function* _ul2v*(arm)* 390c⟩≡ (394d)

```
void
_ul2v(Vlong *ret, ulong ul)
{
    long t;

    t = ul;
    ret->lo = t;
    ret->hi = 0;
}
```

⟨*function* _si2v*(arm)* 390d⟩≡ (394d)

```
void
_si2v(Vlong *ret, int si)
{
    long t;

    t = si;
    ret->lo = t;
    ret->hi = t >> 31;
}
```

⟨*function* _ui2v*(arm)* 391a⟩≡ (394d)

```
void
_ui2v(Vlong *ret, uint ui)
{
    long t;

    t = ui;
    ret->lo = t;
    ret->hi = 0;
}
```

⟨*function* _sh2v*(arm)* 391b⟩≡ (394d)

```
void
_sh2v(Vlong *ret, long sh)
{
    long t;

    t = (sh << 16) >> 16;
    ret->lo = t;
    ret->hi = t >> 31;
}
```

⟨*function* _uh2v*(arm)* 391c⟩≡ (394d)

```
void
_uh2v(Vlong *ret, ulong ul)
{
    long t;

    t = ul & 0xffff;
    ret->lo = t;
    ret->hi = 0;
}
```

⟨*function* _sc2v*(arm)* 391d⟩≡ (394d)

```
void
_sc2v(Vlong *ret, long uc)
{
    long t;

    t = (uc << 24) >> 24;
    ret->lo = t;
    ret->hi = t >> 31;
}
```

⟨*function* _uc2v*(arm)* 391e⟩≡ (394d)

```
void
_uc2v(Vlong *ret, ulong ul)
{
    long t;

    t = ul & 0xff;
    ret->lo = t;
    ret->hi = 0;
}
```

⟨*function* _v2sc*(arm)* 391f⟩≡ (394d)

```
long
_v2sc(Vlong rv)
{
    long t;
```

```
    t = rv.lo & 0xff;
    return (t << 24) >> 24;
}
```

⟨*function* _v2uc*(arm)* 392a⟩≡ (394d)

```
long
_v2uc(Vlong rv)
{

    return rv.lo & 0xff;
}
```

⟨*function* _v2sh*(arm)* 392b⟩≡ (394d)

```
long
_v2sh(Vlong rv)
{
    long t;

    t = rv.lo & 0xffff;
    return (t << 16) >> 16;
}
```

⟨*function* _v2uh*(arm)* 392c⟩≡ (394d)

```
long
_v2uh(Vlong rv)
{

    return rv.lo & 0xffff;
}
```

⟨*function* _v2sl*(arm)* 392d⟩≡ (394d)

```
long
_v2sl(Vlong rv)
{

    return rv.lo;
}
```

⟨*function* _v2ul*(arm)* 392e⟩≡ (394d)

```
long
_v2ul(Vlong rv)
{

    return rv.lo;
}
```

⟨*function* _v2si*(arm)* 392f⟩≡ (394d)

```
long
_v2si(Vlong rv)
{

    return rv.lo;
}
```

⟨*function* _v2ui*(arm)* 392g⟩≡ (394d)

```
long
_v2ui(Vlong rv)
{

    return rv.lo;
}
```

⟨*function* _testv*(arm)* 393a⟩≡ (394d)

```
int
_testv(Vlong rv)
{
    return rv.lo || rv.hi;
}
```

⟨*function* _eqv*(arm)* 393b⟩≡ (394d)

```
int
_eqv(Vlong lv, Vlong rv)
{
    return lv.lo == rv.lo && lv.hi == rv.hi;
}
```

⟨*function* _nev*(arm)* 393c⟩≡ (394d)

```
int
_nev(Vlong lv, Vlong rv)
{
    return lv.lo != rv.lo || lv.hi != rv.hi;
}
```

⟨*function* _ltv*(arm)* 393d⟩≡ (394d)

```
int
_ltv(Vlong lv, Vlong rv)
{
    return (long)lv.hi < (long)rv.hi ||
        (lv.hi == rv.hi && lv.lo < rv.lo);
}
```

⟨*function* _lev*(arm)* 393e⟩≡ (394d)

```
int
_lev(Vlong lv, Vlong rv)
{
    return (long)lv.hi < (long)rv.hi ||
        (lv.hi == rv.hi && lv.lo <= rv.lo);
}
```

⟨*function* _gtv*(arm)* 393f⟩≡ (394d)

```
int
_gtv(Vlong lv, Vlong rv)
{
    return (long)lv.hi > (long)rv.hi ||
        (lv.hi == rv.hi && lv.lo > rv.lo);
}
```

⟨*function* _gev*(arm)* 393g⟩≡ (394d)

```
int
_gev(Vlong lv, Vlong rv)
{
    return (long)lv.hi > (long)rv.hi ||
        (lv.hi == rv.hi && lv.lo >= rv.lo);
}
```

⟨*function* _lov*(arm)* 393h⟩≡ (394d)

```
int
_lov(Vlong lv, Vlong rv)
{
    return lv.hi < rv.hi ||
        (lv.hi == rv.hi && lv.lo < rv.lo);
}
```

⟨*function* _lsv*(arm)* 394a⟩≡ (394d)

```
int
_lsv(Vlong lv, Vlong rv)
{
    return lv.hi < rv.hi ||
        (lv.hi == rv.hi && lv.lo <= rv.lo);
}
```

⟨*function* _hiv*(arm)* 394b⟩≡ (394d)

```
int
_hiv(Vlong lv, Vlong rv)
{
    return lv.hi > rv.hi ||
        (lv.hi == rv.hi && lv.lo > rv.lo);
}
```

⟨*function* _hsv*(arm)* 394c⟩≡ (394d)

```
int
_hsv(Vlong lv, Vlong rv)
{
    return lv.hi > rv.hi ||
        (lv.hi == rv.hi && lv.lo >= rv.lo);
}
```

⟨libc/arm/vlrt.c 394d⟩≡

```
typedef unsigned long   ulong;
typedef unsigned int    uint;
typedef unsigned short  ushort;
typedef unsigned char   uchar;
typedef signed char schar;

⟨macro SIGN(arm) 382a⟩

typedef struct  Vlong   Vlong;
⟨struct Vlong(arm) 382b⟩

void    abort(void);

/* needed by profiler; can't be profiled */
#pragma profile off

⟨function _addv(arm) 382c⟩

⟨function _subv(arm) 382d⟩

#pragma profile on

⟨function _d2v(arm) 382e⟩

⟨function _f2v(arm) 383a⟩

⟨function _v2d(arm) 383b⟩

⟨function _v2f(arm) 383c⟩

/* too many of these are also needed by profiler; leave them out */
#pragma profile off

⟨function dodiv(arm) 384⟩
```

⟨*function* `_divvu`*(arm)* 385a⟩

⟨*function* `_modvu`*(arm)* 385b⟩

⟨*function* `vneg`*(arm)* 385c⟩

⟨*function* `_divv`*(arm)* 385d⟩

⟨*function* `_modv`*(arm)* 386a⟩

⟨*function* `_rshav`*(arm)* 386b⟩

⟨*function* `_rshlv`*(arm)* 386c⟩

⟨*function* `_lshv`*(arm)* 387a⟩

⟨*function* `_andv`*(arm)* 387b⟩

⟨*function* `_orv`*(arm)* 387c⟩

⟨*function* `_xorv`*(arm)* 388a⟩

⟨*function* `_vpp`*(arm)* 388b⟩

⟨*function* `_vmm`*(arm)* 388c⟩

⟨*function* `_ppv`*(arm)* 388d⟩

⟨*function* `_mmv`*(arm)* 388e⟩

⟨*function* `_vasop`*(arm)* 389⟩

⟨*function* `_p2v`*(arm)* 390a⟩

⟨*function* `_sl2v`*(arm)* 390b⟩

⟨*function* `_ul2v`*(arm)* 390c⟩

⟨*function* `_si2v`*(arm)* 390d⟩

⟨*function* `_ui2v`*(arm)* 391a⟩

⟨*function* `_sh2v`*(arm)* 391b⟩

⟨*function* `_uh2v`*(arm)* 391c⟩

⟨*function* `_sc2v`*(arm)* 391d⟩

⟨*function* `_uc2v`*(arm)* 391e⟩

⟨*function* `_v2sc`*(arm)* 391f⟩

⟨*function* `_v2uc`*(arm)* 392a⟩

⟨*function* `_v2sh`*(arm)* 392b⟩

⟨*function* `_v2uh`*(arm)* 392c⟩

⟨*function* `_v2sl`*(arm)* 392d⟩

⟨*function* _v2ul*(arm)* 392e⟩

⟨*function* _v2si*(arm)* 392f⟩

⟨*function* _v2ui*(arm)* 392g⟩

⟨*function* _testv*(arm)* 393a⟩

⟨*function* _eqv*(arm)* 393b⟩

⟨*function* _nev*(arm)* 393c⟩

⟨*function* _ltv*(arm)* 393d⟩

⟨*function* _lev*(arm)* 393e⟩

⟨*function* _gtv*(arm)* 393f⟩

⟨*function* _gev*(arm)* 393g⟩

⟨*function* _lov*(arm)* 393h⟩

⟨*function* _lsv*(arm)* 394a⟩

⟨*function* _hiv*(arm)* 394b⟩

⟨*function* _hsv*(arm)* 394c⟩

A.6 libstring/

A.7 libflate/

A.8 libregexp/

A.8.1 libregexp/regcomp.h

⟨libregexp/regcomp.h 396⟩≡

```
/*
 *  substitution list
 */
#define NSUBEXP 32
typedef struct Resublist Resublist;
struct Resublist
{
 Resub m[NSUBEXP];
};

/*
 * Actions and Tokens (Reinst types)
 *
 * 02xx are operators, value == precedence
 * 03xx are tokens, i.e. operands for operators
 */
#define RUNE  0177
#define OPERATOR 0200 /* Bitmask of all operators */
```

```
#define START  0200 /* Start, used for marker on stack */
#define RBRA  0201 /* Right bracket, ) */
#define LBRA  0202 /* Left bracket, ( */
#define OR  0203 /* Alternation, | */
#define CAT  0204 /* Concatentation, implicit operator */
#define STAR  0205 /* Closure, * */
#define PLUS  0206 /* a+ == aa* */
#define QUEST  0207 /* a? == a|nothing, i.e. 0 or 1 a's */
#define ANY  0300 /* Any character except newline, . */
#define ANYNL  0301 /* Any character including newline, . */
#define NOP  0302 /* No operation, internal use only */
#define BOL  0303 /* Beginning of line, ^ */
#define EOL  0304 /* End of line, $ */
#define CCLASS  0305 /* Character class, [] */
#define NCCLASS  0306 /* Negated character class, [] */
#define END  0377 /* Terminate: match found */

/*
 *  regexec execution lists
 */
#define LISTSIZE 10
#define BIGLISTSIZE (25*LISTSIZE)
typedef struct Relist Relist;
struct Relist
{
 Reinst*  inst;  /* Reinstruction of the thread */
 Resublist se;  /* matched subexpressions in this thread */
};
typedef struct Reljunk Reljunk;
struct Reljunk
{
 Relist* relist[2];
 Relist* reliste[2];
 int starttype;
 Rune startchar;
 char* starts;
 char* eol;
 Rune* rstarts;
 Rune* reol;
};

extern Relist* _renewthread(Relist*, Reinst*, int, Resublist*);
extern void _renewmatch(Resub*, int, Resublist*);
extern Relist* _renewemptythread(Relist*, Reinst*, int, char*);
extern Relist* _rrenewemptythread(Relist*, Reinst*, int, Rune*);
```

A.8.2 libregexp/regaux.c

⟨*function* _renewmatch 397⟩≡ (399b)

```
/*
 *  save a new match in mp
 */
extern void
_renewmatch(Resub *mp, int ms, Resublist *sp)
{
    int i;

    if(mp==0 || ms<=0)
        return;
```

```
    if(mp[0].s.sp==0 || sp->m[0].s.sp<mp[0].s.sp ||
       (sp->m[0].s.sp==mp[0].s.sp && sp->m[0].e.ep>mp[0].e.ep)){
        for(i=0; i<ms && i<NSUBEXP; i++)
            mp[i] = sp->m[i];
        for(; i<ms; i++)
            mp[i].s.sp = mp[i].e.ep = 0;
    }
}
```

⟨*function* _renewthread 398a⟩≡ (399b)

```
/*
 * Note optimization in _renewthread:
 *  *lp must be pending when _renewthread called; if *l has been looked
 *  at already, the optimization is a bug.
 */
extern Relist*
_renewthread(Relist *lp, /* _relist to add to */
    Reinst *ip,  /* instruction to add */
    int ms,
    Resublist *sep)  /* pointers to subexpressions */
{
    Relist *p;

    for(p=lp; p->inst; p++){
        if(p->inst == ip){
            if(sep->m[0].s.sp < p->se.m[0].s.sp){
                if(ms > 1)
                    p->se = *sep;
                else
                    p->se.m[0] = sep->m[0];
            }
            return 0;
        }
    }
    p->inst = ip;
    if(ms > 1)
        p->se = *sep;
    else
        p->se.m[0] = sep->m[0];
    (++p)->inst = 0;
    return p;
}
```

⟨*function* _renewemptythread 398b⟩≡ (399b)

```
/*
 * same as renewthread, but called with
 * initial empty start pointer.
 */
extern Relist*
_renewemptythread(Relist *lp, /* _relist to add to */
    Reinst *ip,  /* instruction to add */
    int ms,
    char *sp)  /* pointers to subexpressions */
{
    Relist *p;

    for(p=lp; p->inst; p++){
        if(p->inst == ip){
            if(sp < p->se.m[0].s.sp) {
                if(ms > 1)
```

```
                    memset(&p->se, 0, sizeof(p->se));
                p->se.m[0].s.sp = sp;
            }
            return 0;
        }
    }
    p->inst = ip;
    if(ms > 1)
        memset(&p->se, 0, sizeof(p->se));
    p->se.m[0].s.sp = sp;
    (++p)->inst = 0;
    return p;
}
```

⟨*function* _rrenewemptythread 399a⟩≡ (399b)

```
extern Relist*
_rrenewemptythread(Relist *lp, /* _relist to add to */
    Reinst *ip,  /* instruction to add */
    int ms,
    Rune *rsp)  /* pointers to subexpressions */
{
    Relist *p;

    for(p=lp; p->inst; p++){
        if(p->inst == ip){
            if(rsp < p->se.m[0].s.rsp) {
                if(ms > 1)
                    memset(&p->se, 0, sizeof(p->se));
                p->se.m[0].s.rsp = rsp;
            }
            return 0;
        }
    }
    p->inst = ip;
    if(ms > 1)
        memset(&p->se, 0, sizeof(p->se));
    p->se.m[0].s.rsp = rsp;
    (++p)->inst = 0;
    return p;
}
```

⟨libregexp/regaux.c 399b⟩≡

```
#include <u.h>
#include <libc.h>
#include "regexp.h"
#include "regcomp.h"


⟨function _renewmatch 397⟩

⟨function _renewthread 398a⟩

⟨function _renewemptythread 398b⟩

⟨function _rrenewemptythread 399a⟩
```

A.8.3 libregexp/regcomp.c

⟨*constant* TRUE 399c⟩≡ (410c)

```
#define TRUE 1
```

⟨*constant* FALSE 400a⟩≡ (410c)
```
#define FALSE 0
```

⟨*struct* Node 400b⟩≡ (410c)
```
/*
 * Parser Information
 */
typedef
struct Node
{
    Reinst* first;
    Reinst* last;
}Node;
```

⟨*global* reprog 400c⟩≡ (410c)
```
/* max character classes per program is nelem(reprog->class) */
static Reprog *reprog;
```

⟨*constant* NCCRUNE 400d⟩≡ (410c)
```
/* max rune ranges per character class is nelem(classp->spans)/2 */
#define NCCRUNE nelem(classp->spans)
```

⟨*constant* NSTACK 400e⟩≡ (410c)
```
#define NSTACK 20
```

⟨*global* andstack 400f⟩≡ (410c)
```
static Node andstack[NSTACK];
```

⟨*global* andp 400g⟩≡ (410c)
```
static Node *andp;
```

⟨*global* atorstack 400h⟩≡ (410c)
```
static int atorstack[NSTACK];
```

⟨*global* atorp 400i⟩≡ (410c)
```
static int* atorp;
```

⟨*global* cursubid 400j⟩≡ (410c)
```
static int cursubid;  /* id of current subexpression */
```

⟨*global* subidstack 400k⟩≡ (410c)
```
static int subidstack[NSTACK]; /* parallel to atorstack */
```

⟨*global* subidp 400l⟩≡ (410c)
```
static int* subidp;
```

⟨*global* lastwasand 400m⟩≡ (410c)
```
static int lastwasand; /* Last token was operand */
```

⟨*global* nbra 400n⟩≡ (410c)
```
static int nbra;
```

⟨*global* exprp 400o⟩≡ (410c)
```
static char* exprp;  /* pointer to next character in source expression */
```

⟨*global* lexdone 400p⟩≡ (410c)
```
static int lexdone;
```

⟨*global* nclass 400q⟩≡ (410c)
```
static int nclass;
```

⟨*global* classp 401a⟩≡ (410c)
```
static Reclass*classp;
```

⟨*global* freep 401b⟩≡ (410c)
```
static Reinst* freep;
```

⟨*global* errors 401c⟩≡ (410c)
```
static int errors;
```

⟨*global* yyrune 401d⟩≡ (410c)
```
static Rune yyrune;  /* last lex'd rune */
```

⟨*global* yyclassp 401e⟩≡ (410c)
```
static Reclass*yyclassp; /* last lex'd class */
```

⟨*global* regkaboom 401f⟩≡ (410c)
```
static jmp_buf regkaboom;
```

⟨*function* rcerror 401g⟩≡ (410c)
```
static void
rcerror(char *s)
{
    errors++;
    regerror(s);
    longjmp(regkaboom, 1);
}
```

⟨*function* newinst 401h⟩≡ (410c)
```
static Reinst*
newinst(int t)
{
    freep->type = t;
    freep->left = 0;
    freep->right = 0;
    return freep++;
}
```

⟨*function* operand 401i⟩≡ (410c)
```
static void
operand(int t)
{
    Reinst *i;

    if(lastwasand)
        operator(CAT); /* catenate is implicit */
    i = newinst(t);

    if(t == CCLASS || t == NCCLASS)
        i->cp = yyclassp;
    if(t == RUNE)
        i->r = yyrune;

    pushand(i, i);
    lastwasand = TRUE;
}
```

⟨*function* operator 402a⟩≡ (410c)

```
static void
operator(int t)
{
    if(t==RBRA && --nbra<0)
        rcerror("unmatched right paren");
    if(t==LBRA){
        if(++cursubid >= NSUBEXP)
            rcerror ("too many subexpressions");
        nbra++;
        if(lastwasand)
            operator(CAT);
    } else
        evaluntil(t);
    if(t != RBRA)
        pushator(t);
    lastwasand = FALSE;
    if(t==STAR || t==QUEST || t==PLUS || t==RBRA)
        lastwasand = TRUE; /* these look like operands */
}
```

⟨*function* regerr2 402b⟩≡ (410c)

```
static void
regerr2(char *s, int c)
{
    char buf[100];
    char *cp = buf;
    while(*s)
        *cp++ = *s++;
    *cp++ = c;
    *cp = '\0';
    rcerror(buf);
}
```

⟨*function* cant 402c⟩≡ (410c)

```
static void
cant(char *s)
{
    char buf[100];
    strcpy(buf, "can't happen: ");
    strcat(buf, s);
    rcerror(buf);
}
```

⟨*function* pushand 402d⟩≡ (410c)

```
static void
pushand(Reinst *f, Reinst *l)
{
    if(andp >= &andstack[NSTACK])
        cant("operand stack overflow");
    andp->first = f;
    andp->last = l;
    andp++;
}
```

⟨*function* pushator 402e⟩≡ (410c)

```
static void
pushator(int t)
{
    if(atorp >= &atorstack[NSTACK])
```

```
        cant("operator stack overflow");
    *atorp++ = t;
    *subidp++ = cursubid;
}
```

⟨*function* popand 403a⟩≡ (410c)

```
static Node*
popand(int op)
{
    Reinst *inst;

    if(andp <= &andstack[0]){
        regerr2("missing operand for ", op);
        inst = newinst(NOP);
        pushand(inst,inst);
    }
    return --andp;
}
```

⟨*function* popator 403b⟩≡ (410c)

```
static int
popator(void)
{
    if(atorp <= &atorstack[0])
        cant("operator stack underflow");
    --subidp;
    return *--atorp;
}
```

⟨*function* evaluntil 403c⟩≡ (410c)

```
static void
evaluntil(int pri)
{
    Node *op1, *op2;
    Reinst *inst1, *inst2;

    while(pri==RBRA || atorp[-1]>=pri){
        switch(popator()){
        default:
            rcerror("unknown operator in evaluntil");
            break;
        case LBRA:  /* must have been RBRA */
            op1 = popand('(');
            inst2 = newinst(RBRA);
            inst2->subid = *subidp;
            op1->last->next = inst2;
            inst1 = newinst(LBRA);
            inst1->subid = *subidp;
            inst1->next = op1->first;
            pushand(inst1, inst2);
            return;
        case OR:
            op2 = popand('|');
            op1 = popand('|');
            inst2 = newinst(NOP);
            op2->last->next = inst2;
            op1->last->next = inst2;
            inst1 = newinst(OR);
            inst1->right = op1->first;
            inst1->left = op2->first;
```

```
            pushand(inst1, inst2);
            break;
        case CAT:
            op2 = popand(0);
            op1 = popand(0);
            op1->last->next = op2->first;
            pushand(op1->first, op2->last);
            break;
        case STAR:
            op2 = popand('*');
            inst1 = newinst(OR);
            op2->last->next = inst1;
            inst1->right = op2->first;
            pushand(inst1, inst1);
            break;
        case PLUS:
            op2 = popand('+');
            inst1 = newinst(OR);
            op2->last->next = inst1;
            inst1->right = op2->first;
            pushand(op2->first, inst1);
            break;
        case QUEST:
            op2 = popand('?');
            inst1 = newinst(OR);
            inst2 = newinst(NOP);
            inst1->left = inst2;
            inst1->right = op2->first;
            op2->last->next = inst2;
            pushand(inst1, inst2);
            break;
        }
    }
}
```

⟨*function* optimize 404⟩≡ (410c)

```
static Reprog*
optimize(Reprog *pp)
{
    Reinst *inst, *target;
    int size;
    Reprog *npp;
    Reclass *cl;
    int diff;

    /*
     *  get rid of NOOP chains
     */
    for(inst=pp->firstinst; inst->type!=END; inst++){
        target = inst->next;
        while(target->type == NOP)
            target = target->next;
        inst->next = target;
    }

    /*
     *  The original allocation is for an area larger than
     *  necessary.  Reallocate to the actual space used
     *  and then relocate the code.
     */
```

```
    size = sizeof(Reprog) + (freep - pp->firstinst)*sizeof(Reinst);
    npp = realloc(pp, size);
    if(npp==0 || npp==pp)
        return pp;
    diff = (char *)npp - (char *)pp;
    freep = (Reinst *)((char *)freep + diff);
    for(inst=npp->firstinst; inst<freep; inst++){
        switch(inst->type){
        case OR:
        case STAR:
        case PLUS:
        case QUEST:
            *(char **)&inst->right += diff;
            break;
        case CCLASS:
        case NCCLASS:
            *(char **)&inst->right += diff;
            cl = inst->cp;
            *(char **)&cl->end += diff;
            break;
        }
        *(char **)&inst->left += diff;
    }
    *(char **)&npp->startinst += diff;
    return npp;
}
```

⟨*function* dumpstack 405a⟩≡ (410c)

```
static void
dumpstack(void){
    Node *stk;
    int *ip;

    print("operators\n");
    for(ip=atorstack; ip<atorp; ip++)
        print("0%o\n", *ip);
    print("operands\n");
    for(stk=andstack; stk<andp; stk++)
        print("0%o\t0%o\n", stk->first->type, stk->last->type);
}
```

⟨*function* dump 405b⟩≡ (410c)

```
static void
dump(Reprog *pp)
{
    Reinst *l;
    Rune *p;

    l = pp->firstinst;
    do{
        print("%d:\t0%o\t%d\t%d", l-pp->firstinst, l->type,
            l->left-pp->firstinst, l->right-pp->firstinst);
        if(l->type == RUNE)
            print("\t%C\n", l->r);
        else if(l->type == CCLASS || l->type == NCCLASS){
            print("\t[");
            if(l->type == NCCLASS)
                print("^");
            for(p = l->cp->spans; p < l->cp->end; p += 2)
                if(p[0] == p[1])
```

```
                    print("%C", p[0]);
                else
                    print("%C-%C", p[0], p[1]);
            print("]\n");
        } else
            print("\n");
    }while(l++->type);
}
```

⟨*function* newclass 406a⟩≡ (410c)

```
static Reclass*
newclass(void)
{
    if(nclass >= nelem(reprog->class))
        rcerror("too many character classes; increase Reprog.class size");
    return &(classp[nclass++]);
}
```

⟨*function* nextc 406b⟩≡ (410c)

```
static int
nextc(Rune *rp)
{
    if(lexdone){
        *rp = 0;
        return 1;
    }
    exprp += chartorune(rp, exprp);
    if(*rp == L'\\'){
        exprp += chartorune(rp, exprp);
        return 1;
    }
    if(*rp == 0)
        lexdone = 1;
    return 0;
}
```

⟨*function* lex 406c⟩≡ (410c)

```
static int
lex(int literal, int dot_type)
{
    int quoted;

    quoted = nextc(&yyrune);
    if(literal || quoted){
        if(yyrune == 0)
            return END;
        return RUNE;
    }

    switch(yyrune){
    case 0:
        return END;
    case L'*':
        return STAR;
    case L'?':
        return QUEST;
    case L'+':
        return PLUS;
    case L'|':
        return OR;
```

```
    case L'.':
        return dot_type;
    case L'(':
        return LBRA;
    case L')':
        return RBRA;
    case L'^':
        return BOL;
    case L'$':
        return EOL;
    case L'[':
        return bldcclass();
    }
    return RUNE;
}
```

⟨*function* bldcclass 407⟩≡ (410c)

```
static int
bldcclass(void)
{
    int type;
    Rune r[NCCRUNE];
    Rune *p, *ep, *np;
    Rune rune;
    int quoted;

    /* we have already seen the '[' */
    type = CCLASS;
    yyclassp = newclass();

    /* look ahead for negation */
    /* SPECIAL CASE!!! negated classes don't match \n */
    ep = r;
    quoted = nextc(&rune);
    if(!quoted && rune == L'^'){
        type = NCCLASS;
        quoted = nextc(&rune);
        *ep++ = L'\n';
        *ep++ = L'\n';
    }

    /* parse class into a set of spans */
    while(ep < &r[NCCRUNE-1]){
        if(rune == 0){
            rcerror("malformed '[]'");
            return 0;
        }
        if(!quoted && rune == L']')
            break;
        if(!quoted && rune == L'-'){
            if(ep == r){
                rcerror("malformed '[]'");
                return 0;
            }
            quoted = nextc(&rune);
            if((!quoted && rune == L']') || rune == 0){
                rcerror("malformed '[]'");
                return 0;
            }
            *(ep-1) = rune;
```

```
        } else {
            *ep++ = rune;
            *ep++ = rune;
        }
        quoted = nextc(&rune);
    }
    if(ep >= &r[NCCRUNE-1]) {
        rcerror("char class too large; increase Reclass.spans size");
        return 0;
    }

    /* sort on span start */
    for(p = r; p < ep; p += 2){
        for(np = p; np < ep; np += 2)
            if(*np < *p){
                rune = np[0];
                np[0] = p[0];
                p[0] = rune;
                rune = np[1];
                np[1] = p[1];
                p[1] = rune;
            }
    }

    /* merge spans */
    np = yyclassp->spans;
    p = r;
    if(r == ep)
        yyclassp->end = np;
    else {
        np[0] = *p++;
        np[1] = *p++;
        for(; p < ep; p += 2)
            /* overlapping or adjacent ranges? */
            if(p[0] <= np[1] + 1){
                if(p[1] >= np[1])
                    np[1] = p[1]; /* coalesce */
            } else {
                np += 2;
                np[0] = p[0];
                np[1] = p[1];
            }
        yyclassp->end = np+2;
    }

    return type;
}
```

⟨*function* regcomp1 408⟩≡ (410c)

```
static Reprog*
regcomp1(char *s, int literal, int dot_type)
{
    int token;
    Reprog *pp;

    /* get memory for the program */
    pp = malloc(sizeof(Reprog) + 6*sizeof(Reinst)*strlen(s));
    if(pp == 0){
        regerror("out of memory");
        return 0;
```

```
    }
    freep = pp->firstinst;
    classp = pp->class;
    errors = 0;

    if(setjmp(regkaboom))
        goto out;

    /* go compile the sucker */
    lexdone = 0;
    exprp = s;
    nclass = 0;
    nbra = 0;
    atorp = atorstack;
    andp = andstack;
    subidp = subidstack;
    lastwasand = FALSE;
    cursubid = 0;

    /* Start with a low priority operator to prime parser */
    pushator(START-1);
    while((token = lex(literal, dot_type)) != END){
        if((token&0300) == OPERATOR)
            operator(token);
        else
            operand(token);
    }

    /* Close with a low priority operator */
    evaluntil(START);

    /* Force END */
    operand(END);
    evaluntil(START);
#ifdef DEBUG
    dumpstack();
#endif
    if(nbra)
        rcerror("unmatched left paren");
    --andp; /* points to first and only operand */
    pp->startinst = andp->first;
#ifdef DEBUG
    dump(pp);
#endif
    pp = optimize(pp);
#ifdef DEBUG
    print("start: %d\n", andp->first-pp->firstinst);
    dump(pp);
#endif
out:
    if(errors){
        free(pp);
        pp = 0;
    }
    return pp;
}
```

⟨*function* regcomp 409⟩≡ (410c)

```
extern Reprog*
regcomp(char *s)
```

```
{
    return regcomp1(s, 0, ANY);
}
```

⟨*function* regcomplit 410a⟩≡ (410c)

```
extern Reprog*
regcomplit(char *s)
{
    return regcomp1(s, 1, ANY);
}
```

⟨*function* regcompnl 410b⟩≡ (410c)

```
extern Reprog*
regcompnl(char *s)
{
    return regcomp1(s, 0, ANYNL);
}
```

⟨libregexp/regcomp.c 410c⟩≡

```
#include <u.h>
#include <libc.h>
#include "regexp.h"
#include "regcomp.h"

⟨constant TRUE 399c⟩
⟨constant FALSE 400a⟩

⟨struct Node 400b⟩

⟨global reprog 400c⟩

⟨constant NCCRUNE 400d⟩

⟨constant NSTACK 400e⟩
⟨global andstack 400f⟩
⟨global andp 400g⟩
⟨global atorstack 400h⟩
⟨global atorp 400i⟩
⟨global cursubid 400j⟩
⟨global subidstack 400k⟩
⟨global subidp 400l⟩
⟨global lastwasand 400m⟩
⟨global nbra 400n⟩
⟨global exprp 400o⟩
⟨global lexdone 400p⟩
⟨global nclass 400q⟩
⟨global classp 401a⟩
⟨global freep 401b⟩
⟨global errors 401c⟩
⟨global yyrune 401d⟩
⟨global yyclassp 401e⟩

/* predeclared crap */
static void operator(int);
static void pushand(Reinst*, Reinst*);
static void pushator(int);
static void evaluntil(int);
static int bldcclass(void);

⟨global regkaboom 401f⟩
```

⟨*function* rcerror 401g⟩

⟨*function* newinst 401h⟩

⟨*function* operand 401i⟩

⟨*function* operator 402a⟩

⟨*function* regerr2 402b⟩

⟨*function* cant 402c⟩

⟨*function* pushand 402d⟩

⟨*function* pushator 402e⟩

⟨*function* popand 403a⟩

⟨*function* popator 403b⟩

⟨*function* evaluntil 403c⟩

⟨*function* optimize 404⟩

```
#ifdef DEBUG
```
⟨*function* dumpstack 405a⟩

⟨*function* dump 405b⟩
```
#endif
```

⟨*function* newclass 406a⟩

⟨*function* nextc 406b⟩

⟨*function* lex 406c⟩

⟨*function* bldcclass 407⟩

⟨*function* regcomp1 408⟩

⟨*function* regcomp 409⟩

⟨*function* regcomplit 410a⟩

⟨*function* regcompnl 410b⟩

A.8.4 libregexp/regerror.c

⟨*function* regerror 411⟩≡ (412a)

```
void
regerror(char *s)
{
    char buf[132];

    strcpy(buf, "regerror: ");
    strcat(buf, s);
    strcat(buf, "\n");
    write(2, buf, strlen(buf));
```

```
    exits("regerr");
}
```

⟨libregexp/regerror.c 412a⟩≡

```
#include <u.h>
#include <libc.h>
#include "regexp.h"

⟨function regerror 411⟩
```

A.8.5 libregexp/regexec.c

⟨*function* regexec1 412b⟩≡ (416a)

```
/*
 *  return 0 if no match
 *  >0 if a match
 *  <0 if we ran out of _relist space
 */
static int
regexec1(Reprog *progp, /* program to run */
    char *bol, /* string to run machine on */
    Resub *mp, /* subexpression elements */
    int ms,  /* number of elements at mp */
    Reljunk *j
)
{
    int flag=0;
    Reinst *inst;
    Relist *tlp;
    char *s;
    int i, checkstart;
    Rune r, *rp, *ep;
    int n;
    Relist* tl;  /* This list, next list */
    Relist* nl;
    Relist* tle;  /* ends of this and next list */
    Relist* nle;
    int match;
    char *p;

    match = 0;
    checkstart = j->starttype;
    if(mp)
        for(i=0; i<ms; i++) {
            mp[i].sp = 0;
            mp[i].ep = 0;
        }
    j->relist[0][0].inst = 0;
    j->relist[1][0].inst = 0;

    /* Execute machine once for each character, including terminal NUL */
    s = j->starts;
    do{
        /* fast check for first char */
        if(checkstart) {
            switch(j->starttype) {
            case RUNE:
                p = utfrune(s, j->startchar);
                if(p == 0 || s == j->eol)
```

```
            return match;
        s = p;
        break;
    case BOL:
        if(s == bol)
            break;
        p = utfrune(s, '\n');
        if(p == 0 || s == j->eol)
            return match;
        s = p+1;
        break;
    }
}
r = *(uchar*)s;
if(r < Runeself)
    n = 1;
else
    n = chartorune(&r, s);

/* switch run lists */
tl = j->relist[flag];
tle = j->reliste[flag];
nl = j->relist[flag^=1];
nle = j->reliste[flag];
nl->inst = 0;

/* Add first instruction to current list */
if(match == 0)
    _renewemptythread(tl, progp->startinst, ms, s);

/* Execute machine until current list is empty */
for(tlp=tl; tlp->inst; tlp++){ /* assignment = */
    for(inst = tlp->inst; ; inst = inst->next){
        switch(inst->type){
        case RUNE: /* regular character */
            if(inst->r == r){
                if(_renewthread(nl, inst->next, ms, &tlp->se)==nle)
                    return -1;
            }
            break;
        case LBRA:
            tlp->se.m[inst->subid].sp = s;
            continue;
        case RBRA:
            tlp->se.m[inst->subid].ep = s;
            continue;
        case ANY:
            if(r != '\n')
                if(_renewthread(nl, inst->next, ms, &tlp->se)==nle)
                    return -1;
            break;
        case ANYNL:
            if(_renewthread(nl, inst->next, ms, &tlp->se)==nle)
                    return -1;
            break;
        case BOL:
            if(s == bol || *(s-1) == '\n')
                continue;
            break;
        case EOL:
```

```
                    if(s == j->eol || r == 0 || r == '\n')
                        continue;
                    break;
                case CCLASS:
                    ep = inst->cp->end;
                    for(rp = inst->cp->spans; rp < ep; rp += 2)
                        if(r >= rp[0] && r <= rp[1]){
                            if(_renewthread(nl, inst->next, ms, &tlp->se)==nle)
                                return -1;
                            break;
                        }
                    break;
                case NCCLASS:
                    ep = inst->cp->end;
                    for(rp = inst->cp->spans; rp < ep; rp += 2)
                        if(r >= rp[0] && r <= rp[1])
                            break;
                    if(rp == ep)
                        if(_renewthread(nl, inst->next, ms, &tlp->se)==nle)
                            return -1;
                    break;
                case OR:
                    /* evaluate right choice later */
                    if(_renewthread(tlp, inst->right, ms, &tlp->se) == tle)
                        return -1;
                    /* efficiency: advance and re-evaluate */
                    continue;
                case END: /* Match! */
                    match = 1;
                    tlp->se.m[0].ep = s;
                    if(mp != 0)
                        _renewmatch(mp, ms, &tlp->se);
                    break;
                }
                break;
            }
        }
        if(s == j->eol)
            break;
        checkstart = j->starttype && nl->inst==0;
        s += n;
    }while(r);
    return match;
}
```

⟨*function* regexec2 414⟩≡ (416a)

```
static int
regexec2(Reprog *progp, /* program to run */
    char *bol, /* string to run machine on */
    Resub *mp, /* subexpression elements */
    int ms,  /* number of elements at mp */
    Reljunk *j
)
{
    int rv;
    Relist *relist0, *relist1;

    /* mark space */
    relist0 = malloc(BIGLISTSIZE*sizeof(Relist));
    if(relist0 == nil)
```

```
        return -1;
    relist1 = malloc(BIGLISTSIZE*sizeof(Relist));
    if(relist1 == nil){
        free(relist1);
        return -1;
    }
    j->relist[0] = relist0;
    j->relist[1] = relist1;
    j->reliste[0] = relist0 + BIGLISTSIZE - 2;
    j->reliste[1] = relist1 + BIGLISTSIZE - 2;

    rv = regexec1(progp, bol, mp, ms, j);
    free(relist0);
    free(relist1);
    return rv;
}
```

⟨*function* regexec 415⟩≡ (416a)

```
extern int
regexec(Reprog *progp, /* program to run */
    char *bol, /* string to run machine on */
    Resub *mp, /* subexpression elements */
    int ms)  /* number of elements at mp */
{
    Reljunk j;
    Relist relist0[LISTSIZE], relist1[LISTSIZE];
    int rv;

    /*
   *  use user-specified starting/ending location if specified
     */
    j.starts = bol;
    j.eol = 0;
    if(mp && ms>0){
        if(mp->sp)
            j.starts = mp->sp;
        if(mp->ep)
            j.eol = mp->ep;
    }
    j.starttype = 0;
    j.startchar = 0;
    if(progp->startinst->type == RUNE && progp->startinst->r < Runeself) {
        j.starttype = RUNE;
        j.startchar = progp->startinst->r;
    }
    if(progp->startinst->type == BOL)
        j.starttype = BOL;

    /* mark space */
    j.relist[0] = relist0;
    j.relist[1] = relist1;
    j.reliste[0] = relist0 + nelem(relist0) - 2;
    j.reliste[1] = relist1 + nelem(relist1) - 2;

    rv = regexec1(progp, bol, mp, ms, &j);
    if(rv >= 0)
        return rv;
    rv = regexec2(progp, bol, mp, ms, &j);
    if(rv >= 0)
        return rv;
```

```
    return -1;
}
```

⟨libregexp/regexec.c 416a⟩≡

```
#include <u.h>
#include <libc.h>
#include "regexp.h"
#include "regcomp.h"

⟨function regexec1 412b⟩

⟨function regexec2 414⟩

⟨function regexec 415⟩
```

A.8.6 libregexp/regsub.c

⟨function regsub 416b⟩≡ (417a)

```
/* substitute into one string using the matches from the last regexec() */
extern void
regsub(char *sp, /* source string */
    char *dp, /* destination string */
    int dlen,
    Resub *mp, /* subexpression elements */
    int ms)  /* number of elements pointed to by mp */
{
    char *ssp, *ep;
    int i;

    ep = dp+dlen-1;
    while(*sp != '\0'){
        if(*sp == '\\'){
            switch(*++sp){
            case '0':
            case '1':
            case '2':
            case '3':
            case '4':
            case '5':
            case '6':
            case '7':
            case '8':
            case '9':
                i = *sp-'0';
                if(mp!=0 && mp[i].s.sp != 0 && ms>i)
                    for(ssp = mp[i].s.sp;
                         ssp < mp[i].e.ep;
                         ssp++)
                        if(dp < ep)
                            *dp++ = *ssp;
                break;
            case '\\':
                if(dp < ep)
                    *dp++ = '\\';
                break;
            case '\0':
                sp--;
                break;
            default:
```

```
                if(dp < ep)
                    *dp++ = *sp;
                break;
            }
        }else if(*sp == '&'){
            if(mp!=0 && mp[0].s.sp != 0 && ms>0)
                for(ssp = mp[0].s.sp;
                     ssp < mp[0].e.ep; ssp++)
                    if(dp < ep)
                        *dp++ = *ssp;
        }else{
            if(dp < ep)
                *dp++ = *sp;
        }
        sp++;
    }
    *dp = '\0';
}
```

⟨libregexp/regsub.c 417a⟩≡

```
#include <u.h>
#include <libc.h>
#include "regexp.h"

⟨function regsub 416b⟩
```

A.8.7 libregexp/rregexec.c

⟨*function* rregexec1 417b⟩≡ (421a)

```
/*
 *  return 0 if no match
 *  >0 if a match
 *  <0 if we ran out of _relist space
 */
static int
rregexec1(Reprog *progp, /* program to run */
    Rune *bol,  /* string to run machine on */
    Resub *mp,  /* subexpression elements */
    int ms,   /* number of elements at mp */
    Reljunk *j)
{
    int flag=0;
    Reinst *inst;
    Relist *tlp;
    Rune *s;
    int i, checkstart;
    Rune r, *rp, *ep;
    Relist* tl;  /* This list, next list */
    Relist* nl;
    Relist* tle;  /* ends of this and next list */
    Relist* nle;
    int match;
    Rune *p;

    match = 0;
    checkstart = j->startchar;
    if(mp)
        for(i=0; i<ms; i++) {
            mp[i].s.rsp = 0;
```

```
        mp[i].e.rep = 0;
    }
j->relist[0][0].inst = 0;
j->relist[1][0].inst = 0;

/* Execute machine once for each character, including terminal NUL */
s = j->rstarts;
do{
    /* fast check for first char */
    if(checkstart) {
        switch(j->starttype) {
        case RUNE:
            p = runestrchr(s, j->startchar);
            if(p == 0 || s == j->reol)
                return match;
            s = p;
            break;
        case BOL:
            if(s == bol)
                break;
            p = runestrchr(s, '\n');
            if(p == 0 || s == j->reol)
                return match;
            s = p+1;
            break;
        }
    }

    r = *s;

    /* switch run lists */
    tl = j->relist[flag];
    tle = j->reliste[flag];
    nl = j->relist[flag^=1];
    nle = j->reliste[flag];
    nl->inst = 0;

    /* Add first instruction to current list */
    _rrenewemptythread(tl, progp->startinst, ms, s);

    /* Execute machine until current list is empty */
    for(tlp=tl; tlp->inst; tlp++){
        for(inst=tlp->inst; ; inst = inst->next){
            switch(inst->type){
            case RUNE: /* regular character */
                if(inst->r == r)
                    if(_renewthread(nl, inst->next, ms, &tlp->se)==nle)
                        return -1;
                break;
            case LBRA:
                tlp->se.m[inst->subid].s.rsp = s;
                continue;
            case RBRA:
                tlp->se.m[inst->subid].e.rep = s;
                continue;
            case ANY:
                if(r != '\n')
                    if(_renewthread(nl, inst->next, ms, &tlp->se)==nle)
                        return -1;
                break;
```

```
                case ANYNL:
                    if(_renewthread(nl, inst->next, ms, &tlp->se)==nle)
                            return -1;
                    break;
                case BOL:
                    if(s == bol || *(s-1) == '\n')
                        continue;
                    break;
                case EOL:
                    if(s == j->reol || r == 0 || r == '\n')
                        continue;
                    break;
                case CCLASS:
                    ep = inst->cp->end;
                    for(rp = inst->cp->spans; rp < ep; rp += 2)
                        if(r >= rp[0] && r <= rp[1]){
                            if(_renewthread(nl, inst->next, ms, &tlp->se)==nle)
                                return -1;
                            break;
                        }
                    break;
                case NCCLASS:
                    ep = inst->cp->end;
                    for(rp = inst->cp->spans; rp < ep; rp += 2)
                        if(r >= rp[0] && r <= rp[1])
                            break;
                    if(rp == ep)
                        if(_renewthread(nl, inst->next, ms, &tlp->se)==nle)
                            return -1;
                    break;
                case OR:
                    /* evaluate right choice later */
                    if(_renewthread(tlp, inst->right, ms, &tlp->se) == tle)
                        return -1;
                    /* efficiency: advance and re-evaluate */
                    continue;
                case END: /* Match! */
                    match = 1;
                    tlp->se.m[0].e.rep = s;
                    if(mp != 0)
                        _renewmatch(mp, ms, &tlp->se);
                    break;
                }
                break;
            }
        }
        if(s == j->reol)
            break;
        checkstart = j->startchar && nl->inst==0;
        s++;
    }while(r);
    return match;
}
```

⟨*function* rregexec2 419⟩≡ (421a)

```
static int
rregexec2(Reprog *progp, /* program to run */
    Rune *bol, /* string to run machine on */
    Resub *mp, /* subexpression elements */
    int ms,  /* number of elements at mp */
```

```
    Reljunk *j
)
{
    Relist relist0[5*LISTSIZE], relist1[5*LISTSIZE];

    /* mark space */
    j->relist[0] = relist0;
    j->relist[1] = relist1;
    j->reliste[0] = relist0 + nelem(relist0) - 2;
    j->reliste[1] = relist1 + nelem(relist1) - 2;

    return rregexec1(progp, bol, mp, ms, j);
}
```

⟨*function* rregexec 420⟩≡ (421a)

```
extern int
rregexec(Reprog *progp, /* program to run */
    Rune *bol, /* string to run machine on */
    Resub *mp, /* subexpression elements */
    int ms)  /* number of elements at mp */
{
    Reljunk j;
    Relist relist0[LISTSIZE], relist1[LISTSIZE];
    int rv;

    /*
   *  use user-specified starting/ending location if specified
     */
    j.rstarts = bol;
    j.reol = 0;
    if(mp && ms>0){
        if(mp->s.sp)
            j.rstarts = mp->s.rsp;
        if(mp->e.ep)
            j.reol = mp->e.rep;
    }
    j.starttype = 0;
    j.startchar = 0;
    if(progp->startinst->type == RUNE && progp->startinst->r < Runeself) {
        j.starttype = RUNE;
        j.startchar = progp->startinst->r;
    }
    if(progp->startinst->type == BOL)
        j.starttype = BOL;

    /* mark space */
    j.relist[0] = relist0;
    j.relist[1] = relist1;
    j.reliste[0] = relist0 + nelem(relist0) - 2;
    j.reliste[1] = relist1 + nelem(relist1) - 2;

    rv = rregexec1(progp, bol, mp, ms, &j);
    if(rv >= 0)
        return rv;
    rv = rregexec2(progp, bol, mp, ms, &j);
    if(rv >= 0)
        return rv;
    return -1;
}
```

⟨libregexp/rregexec.c 421a⟩≡

```
#include <u.h>
#include <libc.h>
#include "regexp.h"
#include "regcomp.h"

⟨function rregexec1 417b⟩

⟨function rregexec2 419⟩

⟨function rregexec 420⟩
```

A.8.8 libregexp/rregsub.c

⟨*function* rregsub 421b⟩≡ (422a)

```
/* substitute into one string using the matches from the last regexec() */
extern void
rregsub(Rune *sp, /* source string */
    Rune *dp, /* destination string */
    int dlen,
    Resub *mp, /* subexpression elements */
    int ms)  /* number of elements pointed to by mp */
{
    Rune *ssp, *ep;
    int i;

    ep = dp+(dlen/sizeof(Rune))-1;
    while(*sp != '\0'){
        if(*sp == '\\'){
            switch(*++sp){
            case '0':
            case '1':
            case '2':
            case '3':
            case '4':
            case '5':
            case '6':
            case '7':
            case '8':
            case '9':
                i = *sp-'0';
                if(mp[i].s.rsp != 0 && mp!=0 && ms>i)
                    for(ssp = mp[i].s.rsp;
                         ssp < mp[i].e.rep;
                         ssp++)
                        if(dp < ep)
                            *dp++ = *ssp;
                break;
            case '\\':
                if(dp < ep)
                    *dp++ = '\\';
                break;
            case '\0':
                sp--;
                break;
            default:
                if(dp < ep)
                    *dp++ = *sp;
                break;
```

```
            }
        }else if(*sp == '&'){
            if(mp[0].s.rsp != 0 && mp!=0 && ms>0)
            if(mp[0].s.rsp != 0)
                for(ssp = mp[0].s.rsp;
                     ssp < mp[0].e.rep; ssp++)
                    if(dp < ep)
                        *dp++ = *ssp;
        }else{
            if(dp < ep)
                *dp++ = *sp;
        }
        sp++;
    }
    *dp = '\0';
}
```

⟨libregexp/rregsub.c 422a⟩≡

```
#include <u.h>
#include <libc.h>
#include "regexp.h"

⟨function rregsub 421b⟩
```

A.9 lib_core/libthread/arm

⟨libthread/arm.c 422b⟩≡

```
#include <u.h>
#include <libc.h>
#include <thread.h>
#include "threadimpl.h"

/* first argument goes in a register; simplest just to ignore it */
static void
launcherarm(int, void (*f)(void *arg), void *arg)
{
 (*f)(arg);
 threadexits(nil);
}

void
_threadinitstack(Thread *t, void (*f)(void*), void *arg)
{
 ulong *tos;

 tos = (ulong*)&t->stk[t->stksize&~7];
 *--tos = (ulong)arg;
 *--tos = (ulong)f;
 *--tos = 0; /* first arg to launchermips */
 *--tos = 0; /* place to store return PC */

 t->sched[JMPBUFPC] = (ulong)launcherarm+JMPBUFDPC;
 t->sched[JMPBUFSP] = (ulong)tos;
}
```

A.10 lib_core/libthread/x86

A.10.1 libthread/386.c

⟨libthread/386.c 423a⟩≡
```
#include <u.h>
#include <libc.h>
#include <thread.h>

#include "threadimpl.h"

⟨function launcher386 102b⟩

⟨function _threadinitstack 102c⟩
```

A.11 lib_core/libthread/

A.11.1 libthread/ref.c

⟨libthread/ref.c 423b⟩≡
```
#include <u.h>
#include <libc.h>
#include <thread.h>
#include "threadimpl.h"

⟨function incref 94b⟩

⟨function decref 94c⟩
```

A.11.2 libthread/threadimpl.h

⟨enum _anon_ (libthread/threadimpl.h) 423c⟩≡ (423d)
```
enum
{
⟨constant RENDHASH 140c⟩
⟨constant Printsize 98b⟩
⟨constant NPRIV 101a⟩
};
```

⟨libthread/threadimpl.h 423d⟩≡
```
/*
 * Some notes on locking:
 *
 *  All the locking woes come from implementing
 *  threadinterrupt (and threadkill).
 *
 *  _threadgetproc()->thread is always a live pointer.
 *  p->threads, p->ready, and _threadrgrp also contain
 *  live thread pointers.  These may only be consulted
 *  while holding p->lock or _threadrgrp.lock; in procs
 *  other than p, the pointers are only guaranteed to be live
 *  while the lock is still being held.
 *
 *  Thread structures can only be freed by the proc
 *  they belong to.  Threads marked with t->inrendez
 *  need to be extracted from the _threadrgrp before
 *  being freed.
```

```
 *
 *  _threadrgrp.lock cannot be acquired while holding p->lock.
 */

typedef struct Pqueue   Pqueue;
typedef struct Rgrp Rgrp;
typedef struct Tqueue   Tqueue;
typedef struct Thread   Thread;
typedef struct Execargs Execargs;
typedef struct Proc Proc;


/* must match list in sched.c */
⟨enum state 104a⟩
typedef enum state State;

⟨enum chanstate 104b⟩
typedef enum chanstate Chanstate;

⟨enum _anon_ (libthread/threadimpl.h) 423c⟩

⟨struct Rgrp 140d⟩

⟨struct Tqueue 103c⟩

⟨struct Thread 100c⟩

⟨struct Execargs 100b⟩

⟨struct Proc 98a⟩

⟨struct Pqueue 99c⟩

⟨struct Ioproc 119c⟩

void    _freeproc(Proc*);
void    _freethread(Thread*);
Proc*   _newproc(void(*)(void*), void*, uint, char*, int, int);
int _procsplhi(void);
void    _procsplx(int);
void    _sched(void);
int _schedexec(Execargs*);
void    _schedexecwait(void);
void    _schedexit(Proc*);
int _schedfork(Proc*);
void    _schedinit(void*);
void    _systhreadinit(void);
void    _threadassert(char*);
void    _threadbreakrendez(void);
void    _threaddebug(ulong, char*, ...);
void    _threadexitsall(char*);
void    _threadflagrendez(Thread*);
Proc*   _threadgetproc(void);
void    _threadsetproc(Proc*);
void    _threadinitstack(Thread*, void(*)(void*), void*);
void*   _threadmalloc(long, int);
void    _threadnote(void*, char*);
void    _threadready(Thread*);
void*   _threadrendezvous(void*, void*);
void    _threadsignal(void);
```

```
void    _threadsysfatal(char*, va_list);
void**  _workerdata(void);
void    _xinc(long*);
long    _xdec(long*);

extern int      _threaddebuglevel;
extern char*        _threadexitsallstatus;
extern Pqueue       _threadpq;
extern Channel*     _threadwaitchan;
extern Rgrp     _threadrgrp;
```

⟨*constant* DBGAPPL 135d⟩
⟨*constant* DBGSCHED 135e⟩
⟨*constant* DBGCHAN 135f⟩
⟨*constant* DBGREND 135g⟩
⟨*constant* DBGNOTE 135h⟩
⟨*constant* DBGEXEC 136a⟩

⟨*function* ioproc_arg 119b⟩

A.11.3 libthread/xincarm.c

⟨libthread/xincarm.c 425a⟩≡

```
#include "xincport.h"
```

A.11.4 libthread/xincport.h

⟨libthread/xincport.h 425b⟩≡

```
#include <u.h>
#include <libc.h>
#include <thread.h>
```

⟨*global* xincport_lock 94d⟩

⟨*function* _xinc 94e⟩

⟨*function* _xdec 95a⟩

A.11.5 libthread/globals.c

⟨libthread/globals.c 425c⟩≡

```
#include <u.h>
#include <libc.h>
#include <thread.h>
#include "threadimpl.h"
```

⟨*global* _threadpq 99b⟩

⟨*global* procp 98c⟩

⟨*function* _systhreadinit 106a⟩

⟨*function* _threadgetproc 98d⟩

⟨*function* _threadsetproc 99a⟩

A.11.6 libthread/debug.c

⟨libthread/debug.c 426a⟩≡

```
#include <u.h>
#include <libc.h>
#include <thread.h>
#include "threadimpl.h"

⟨global _threaddebuglevel 135c⟩

⟨function _threaddebug 136b⟩

⟨function _threadassert 135b⟩
```

A.11.7 libthread/id.c

⟨libthread/id.c 426b⟩≡

```
#include <u.h>
#include <libc.h>
#include <thread.h>
#include "threadimpl.h"
#include <tos.h>

⟨function threadid 101b⟩

⟨function threadpid 103b⟩

⟨function threadsetgrp 138e⟩

⟨function threadgetgrp 138f⟩

⟨function threadsetname 104c⟩

⟨function threadgetname 104d⟩

⟨function threaddata 138b⟩

⟨function _workerdata 138c⟩

⟨function procdata 138d⟩

⟨global privlock (libthread/id.c) 137b⟩
⟨global privmask 137c⟩

⟨function tprivalloc 137d⟩

⟨function tprivfree 137e⟩

⟨function tprivaddr 138a⟩
```

A.11.8 libthread/exit.c

⟨libthread/exit.c 426c⟩≡

```
#include <u.h>
#include <libc.h>
#include <thread.h>
#include "threadimpl.h"
#include <tos.h>
```

⟨*global* _threadexitsallstatus 124b⟩
⟨*global* _threadwaitchan 125a⟩

⟨*function* threadexits 124c⟩

⟨*function* threadexitsall 124d⟩

⟨*function* threadwaitchan 125b⟩

A.11.9 libthread/lib.c

⟨libthread/lib.c 427a⟩≡

```
#include <u.h>
#include <libc.h>
#include <thread.h>
#include "threadimpl.h"
```

⟨*global* totalmalloc 123f⟩

⟨*function* _threadmalloc 124a⟩

⟨*function* _threadsysfatal 135a⟩

A.11.10 libthread/note.c

⟨libthread/note.c 427b⟩≡

```
#include <u.h>
#include <libc.h>
#include <thread.h>
#include "threadimpl.h"

typedef struct Note Note;
```

⟨*global* _threadnopasser 128c⟩

⟨*constant NFN (libthread/note.c)* 128d⟩
⟨*constant* ERRLEN 128e⟩
⟨*struct* Note 128f⟩

⟨*global* notes 128g⟩
⟨*global* enotes 128h⟩
⟨*global* onnote 128i⟩
⟨*global* onnotepid 128j⟩
⟨*global* onnotelock 128k⟩

⟨*function* threadnotify 129a⟩

⟨*function* delayednotes 129b⟩

⟨*function* _threadnote 130a⟩

⟨*function* _procsplhi 130b⟩

⟨*function* _procsplx 130c⟩

A.11.11 libthread/rendez.c

⟨libthread/rendez.c 428a⟩≡

```
#include <u.h>
#include <libc.h>
#include <thread.h>
#include "threadimpl.h"

⟨global _threadrgrp 140e⟩
⟨global isdirty 141a⟩

⟨function finish 141b⟩

⟨function _threadrendezvous 140f⟩

⟨function _threadflagrendez 141c⟩

⟨function _threadbreakrendez 141d⟩
```

A.11.12 libthread/sched.c

⟨libthread/sched.c 428b⟩≡

```
#include <u.h>
#include <libc.h>
#include <thread.h>
#include "threadimpl.h"
#include <tos.h>

static Thread   *runthread(Proc*);

⟨global _psstate 106d⟩

⟨function psstate 107a⟩

⟨function _schedinit 107b⟩

⟨function needstack 108a⟩

⟨function _sched 108b⟩

⟨function runthread 109a⟩

⟨function _threadready 109b⟩

⟨function yield 110a⟩
```

A.11.13 libthread/create.c

⟨libthread/create.c 428c⟩≡

```
#include <u.h>
#include <libc.h>
#include <thread.h>
#include "threadimpl.h"

⟨function nextID 101c⟩

⟨function newthread 102a⟩
```

⟨*function* threadcreate 101d⟩

⟨*function* _newproc 99f⟩

⟨*function* procrfork 99e⟩

⟨*function* proccreate 99d⟩

⟨*function* _freeproc 100a⟩

⟨*function* _freethread 103a⟩

A.11.14 libthread/channel.c

⟨libthread/channel.c 429⟩≡

```
#include <u.h>
#include <libc.h>
#include <thread.h>
#include "threadimpl.h"

⟨enum _anon_ (libthread/channel.c) 96c⟩

⟨global errcl 113b⟩
⟨global chanlock 95c⟩

static void enqueue(Alt*, Channel**);
static void dequeue(Alt*);
static int canexec(Alt*);
static int altexec(Alt*, int);

⟨constant Closed 96d⟩
⟨constant Intred 128b⟩

⟨function _chanfree 96b⟩

⟨function chanfree 96a⟩

⟨function chaninit 95e⟩

⟨function chancreate 95d⟩

⟨function isopenfor 112⟩

⟨function alt 110c⟩

⟨function chanclose 96e⟩

⟨function chanclosing 97⟩

⟨function runop 116⟩

⟨function recv 117b⟩

⟨function nbrecv 117c⟩

⟨function send 117a⟩

⟨function nbsend 117d⟩
```

⟨*function* channelsize 117e⟩

⟨*function* sendul 117f⟩

⟨*function* recvul 118a⟩

⟨*function* sendp 118b⟩

⟨*function* recvp 118c⟩

⟨*function* nbsendul 118d⟩

⟨*function* nbrecvul 118e⟩

⟨*function* nbsendp 118f⟩

⟨*function* nbrecvp 119a⟩

⟨*function* emptyentry 114a⟩

⟨*function* enqueue 113c⟩

⟨*function* dequeue 113d⟩

⟨*function* canexec 113a⟩

⟨*function* altexecbuffered 115a⟩

⟨*function* altcopy 115b⟩

⟨*function* altexec 114b⟩

A.11.15 libthread/main.c

⟨libthread/main.c 430⟩≡

```
#include <u.h>
#include <libc.h>
#include <thread.h>
#include "threadimpl.h"

typedef struct Mainarg Mainarg;

⟨struct Mainarg 106b⟩

⟨global mainstacksize 105e⟩
⟨global _threadnotefd 136c⟩
⟨global _threadpasserpid 136d⟩

⟨global _mainjmp 105d⟩

static void mainlauncher(void*);
extern void (*_sysfatal)(char*, va_list);
extern void (*__assert)(char*);
extern int (*_dial)(char*, char*, char*, int*);

extern int _threaddial(char*, char*, char*, int*);

⟨global mainp 105c⟩
```

⟨*function* main 105f⟩

⟨*function* mainlauncher 106c⟩

⟨*function skip (libthread/main.c)* 137a⟩

⟨*function* _times 136e⟩

⟨*function* efork 126a⟩

⟨*function* _schedexec 125d⟩

⟨*function* _schedfork 125c⟩

⟨*function* _schedexit 110b⟩

⟨*function* _schedexecwait 126b⟩

A.11.16 libthread/kill.c

⟨libthread/kill.c 431a⟩≡

```
#include <u.h>
#include <libc.h>
#include <thread.h>
#include "threadimpl.h"

static void tinterrupt(Proc*, Thread*);
```

⟨*function* threadxxxgrp 139c⟩

⟨*function* threadxxx 139d⟩

⟨*function* threadkillgrp 139a⟩

⟨*function* threadkill 139b⟩

⟨*function* threadintgrp 140a⟩

⟨*function* threadint 140b⟩

⟨*function* tinterrupt 138g⟩

A.11.17 libthread/ioproc.c

⟨*enum* _anon_ (libthread/ioproc.c) 431b⟩≡ (431c)

```
enum
{
    ⟨constant STACK 119d⟩
};
```

⟨libthread/ioproc.c 431c⟩≡

```
#include <u.h>
#include <libc.h>
#include <thread.h>
#include "threadimpl.h"

⟨enum _anon_ (libthread/ioproc.c) 431b⟩
```

⟨*function* iointerrupt 120b⟩

⟨*function* xioproc 120c⟩

⟨*function* ioproc 119e⟩

⟨*function* closeioproc 120a⟩

A.11.18 libthread/iocall.c

⟨libthread/iocall.c 432a⟩≡

```
#include <u.h>
#include <libc.h>
#include <thread.h>
#include "threadimpl.h"
```

⟨*function* iocall 121a⟩

A.11.19 libthread/chanprint.c

⟨libthread/chanprint.c 432b⟩≡

```
#include <u.h>
#include <libc.h>
#include <thread.h>
```

⟨*function* chanprint 142⟩

A.11.20 libthread/ioclose.c

⟨libthread/ioclose.c 432c⟩≡

```
#include <u.h>
#include <libc.h>
#include <thread.h>
#include "threadimpl.h"
```

⟨*function* _ioclose 122c⟩

⟨*function* ioclose 122b⟩

A.11.21 libthread/iodial.c

⟨libthread/iodial.c 432d⟩≡

```
#include <u.h>
#include <libc.h>
#include <thread.h>
#include "threadimpl.h"
```

⟨*function* _iodial 131a⟩

⟨*function* iodial 131b⟩

A.11.22 libthread/ioopen.c

⟨libthread/ioopen.c 433a⟩≡

```
#include <u.h>
#include <libc.h>
#include <thread.h>
#include "threadimpl.h"

⟨function _ioopen 122a⟩

⟨function ioopen 121b⟩
```

A.11.23 libthread/ioread.c

⟨libthread/ioread.c 433b⟩≡

```
#include <u.h>
#include <libc.h>
#include <thread.h>
#include "threadimpl.h"

⟨function _ioread 122e⟩

⟨function ioread 122d⟩
```

A.11.24 libthread/ioreadn.c

⟨libthread/ioreadn.c 433c⟩≡

```
#include <u.h>
#include <libc.h>
#include <thread.h>
#include "threadimpl.h"

⟨function _ioreadn 123a⟩

⟨function ioreadn 122f⟩
```

A.11.25 libthread/iosleep.c

⟨libthread/iosleep.c 433d⟩≡

```
#include <u.h>
#include <libc.h>
#include <thread.h>
#include "threadimpl.h"

⟨function _iosleep 123e⟩

⟨function iosleep 123d⟩
```

A.11.26 libthread/iowrite.c

⟨libthread/iowrite.c 433e⟩≡

```
#include <u.h>
#include <libc.h>
#include <thread.h>
#include "threadimpl.h"
```

⟨*function* _iowrite 123c⟩

⟨*function* iowrite 123b⟩

A.11.27 libthread/dial.c

⟨libthread/dial.c 434a⟩≡

```
/*
 * old single-process version of dial that libthread can cope with
 */
#include <u.h>
#include <libc.h>

typedef struct DS DS;

static int  call(char*, char*, DS*);
static int  csdial(DS*);
static void _dial_string_parse(char*, DS*);
```

⟨*enum* _anon_ (libthread/dial.c) 131c⟩

⟨*struct* DS*(libthread/dial.c)* 131d⟩

⟨*function* _threaddial 131e⟩

⟨*function* csdial*(libthread/dial.c)* 132⟩

⟨*function* call*(libthread/dial.c)* 133⟩

⟨*function* _dial_string_parse*(libthread/dial.c)* 134⟩

A.11.28 libthread/exec.c

⟨libthread/exec.c 434b⟩≡

```
#include <u.h>
#include <libc.h>
#include <thread.h>
#include "threadimpl.h"
```

⟨*constant* PIPEMNT 127a⟩

⟨*function* procexec 127b⟩

⟨*function* procexecl 128a⟩

Glossary

LOC	Lines Of Code

Indexes

Here is a list of the identifiers used, and where they appear. Underlined entries indicate the place of definition. This index is generated automatically.

Bibliography

[Knu92] Donald E. Knuth. *Literate Programming*. Center for the Study of Language and Information, 1992. For an introduction see http://en.wikipedia.org/wiki/Literate_Program. cited page(s) 13

[Pad09] Yoann Padioleau. Syncweb, literate programming meets unison, 2009. https://github.com/aryx/syncweb. cited page(s) 13